

C++ FROM BEGINNER TO EXPERT

BOOK VII · EXTENDED PROFESSIONAL CURRICULUM

Advanced C++ Architecture and Distributed Systems

Large-Scale Software Architecture,
Distributed Computing, Services, Reliability,
and Production Infrastructure

Dr. Charles Dorner

DESIGN

Contracts, ownership, protocols, and state

IMPLEMENT

C++ services, persistence, and recovery

OPERATE

Security, observability, and release evidence

Make the boundary explicit. Preserve identity. Verify the result.

28 CHAPTERS · WORKED SOURCE · PRACTICE LABS · 2026

Advanced C++ Architecture and Distributed Systems

Large-Scale Software Architecture, Distributed Computing, Services, Reliability, and Production Infrastructure

Dr. Charles Dorner

Charles Dorner

2026

Educational edition. Third-party tools and libraries retain their own licenses.

Advanced C++ Architecture and Distributed Systems

1. About this volume
 1. Audience and prerequisites
 2. Technical baseline
 3. What the companion actually contains
2. How to use this book
 1. Follow the evidence chain
 2. Build and run a chapter
 3. Read code and output honestly
 4. Separate implementation from qualification
 5. Reading paths
 6. Rebuild the books
3. Chapter 1 — Defining Architecture Through Contracts and Failure Boundaries
 1. 1.1 Turning Requirements into Observable Contracts
 1. 1.1.1 Separating Outcomes from Implementation Preferences
 2. 1.1.2 Defining Preconditions, Postconditions, and Invariants
 2. 1.2 Identifying State and Responsibility
 1. 1.2.1 Assigning an Authoritative Owner to Each Fact
 2. 1.2.2 Separating Volatile, Durable, and Reconstructible State
 3. 1.3 Reasoning About Failure Before Choosing Technology
 1. 1.3.1 Distinguishing Safety from Liveness
 2. 1.3.2 Choosing the Smallest Architecture That Preserves the Contract
 4. Worked example — Validate Work Before It Reaches Infrastructure
 1. Requirement and implementation
 2. Trace the state transition
 5. Failure clinic — The Ambiguous Success Message
 6. Engineering case — Turn a promise into an acceptance matrix
 7. Practice and implementation lab
4. Chapter 2 — Evolving Modular Applications into Independent Services
 1. 2.1 Designing Modules Around Cohesive Responsibilities

- 1. 2.1.1 Separating Domain Decisions from Transport and Storage
- 2. 2.1.2 Grouping Changes That Belong Together
- 2. 2.2 Distinguishing Logical and Deployment Boundaries
 - 1. 2.2.1 Understanding the Modular Monolith
 - 2. 2.2.2 Establishing When a Remote Boundary Is Justified
- 3. 2.3 Managing Coupling Across Data and Releases
 - 1. 2.3.1 Avoiding Shared-Database Service Boundaries
 - 2. 2.3.2 Designing a Reversible Migration Path
- 4. Worked example — Substitute a Repository Without Rewriting Policy
 - 1. Requirement and selected design
 - 2. Evidence and interpretation
- 5. Failure clinic — The Distributed Monolith
- 6. Engineering case — Extract one module without exporting its internals
- 7. Practice and implementation lab
- 5. Chapter 3 — Designing Stable C++ APIs and Compatibility Boundaries
 - 1. 3.1 Making Ownership and Lifetime Visible
 - 1. 3.1.1 Distinguishing Owned Values from Borrowed Views
 - 2. 3.1.2 Defining Valid States After Move and Shutdown
 - 2. 3.2 Designing Error and Completion Semantics
 - 1. 3.2.1 Separating Rejection, Failure, and Uncertainty
 - 2. 3.2.2 Specifying Completion and Cancellation
 - 3. 3.3 Managing Source, Binary, Wire, and Behavioral Compatibility
 - 1. 3.3.1 Understanding What Pimpl Hides and What It Does Not
 - 2. 3.3.2 Evolving Contracts Without Surprising Consumers
 - 4. Worked example — Hide Representation While Preserving Lifetime Rules
 - 1. Implementation
 - 2. Evidence and interpretation
 - 5. Failure clinic — The View That Escaped the Call
 - 6. Engineering case — Review a plugin boundary separately from a library boundary
 - 7. Practice and implementation lab
- 6. Chapter 4 — Managing Configuration, Dependencies, and Service Lifecycle

1. 4.1 Treating Configuration as Validated Input
 1. 4.1.1 Parsing Values with Units and Bounds
 2. 4.1.2 Establishing Precedence and Configuration Identity
2. 4.2 Publishing Configuration Without Breaking In-Flight Work
 1. 4.2.1 Using Immutable Snapshots
 2. 4.2.2 Rejecting Invalid Reloads Without Partial Mutation
3. 4.3 Coordinating Startup and Shutdown
 1. 4.3.1 Starting Dependencies Before Advertising Readiness
 2. 4.3.2 Stopping Admission, Draining, and Releasing Resources
4. Worked example — Preserve a Last-Known-Good Snapshot
 1. Implementation and expected behavior
 2. Trace the exception guarantee
5. Failure clinic — Joining the Consumer Before Closing the Queue
6. Engineering case — Reload capacity without discarding accepted work
7. Practice and implementation lab
7. Chapter 5 — Designing Asynchronous C++ Execution and Coroutine Lifetimes
 1. 5.1 Separating Initiation from Completion
 1. 5.1.1 Understanding Operation State and Completion Handlers
 2. 5.1.2 Choosing Execution Contexts and Serialization Rules
 2. 5.2 Understanding What a Coroutine Stores
 1. 5.2.1 Distinguishing Suspension from Parallel Execution
 2. 5.2.2 Preserving Owned Data Across Suspension
 3. 5.3 Coordinating Cancellation and Structured Completion
 1. 5.3.1 Distinguishing Stop Requests from Resource Reclamation
 2. 5.3.2 Keeping Child Work Within an Owned Scope
 4. Worked example — Step Through an Owned Coroutine Frame
 1. Requirement and implementation
 2. Trace the observable sequence
 5. Failure clinic — A Coroutine That Borrows a Dead Buffer
 6. Engineering case — Cancel an operation without destroying a live callback target
 7. Practice and implementation lab

8. Chapter 6 — Building Event-Driven Network Services

1. 6.1 Understanding Readiness and Completion

1. 6.1.1 Interpreting Nonblocking Socket Operations

2. 6.1.2 Comparing Reactor and Completion-Oriented Designs

2. 6.2 Modeling Per-Connection State

1. 6.2.1 Owning Input, Output, and Deadlines

2. 6.2.2 Bounding Admission and Memory

3. 6.3 Completing Connections and Shutting Down Safely

1. 6.3.1 Preserving Partial Writes and Terminal Outcomes

2. 6.3.2 Draining Without Accepting New Work Forever

4. Worked example — Serve a Request Through a Bounded Reactor

1. Implementation

2. Trace the connection lifecycle

5. Failure clinic — Resetting the Budget on Every Read

6. Engineering case — Keep one slow peer from defining the reactor's pace

7. Practice and implementation lab

9. Chapter 7 — Designing Wire Protocols, Serialization, and Schema Evolution

1. 7.1 Separating Messages from Byte Streams

1. 7.1.1 Defining Framing and Size Limits

2. 7.1.2 Keeping Decoding Incremental and Deterministic

2. 7.2 Defining Field Representation and Meaning

1. 7.2.1 Avoiding Raw Object Serialization

2. 7.2.2 Using Schema-Based Serialization Deliberately

3. 7.3 Evolving Protocols Across Mixed Versions

1. 7.3.1 Distinguishing Additive Fields from Semantic Changes

2. 7.3.2 Establishing Version Negotiation and Rejection Rules

4. Worked example — Verify Framing Independently of Read Boundaries

1. Implementation

2. Evidence and interpretation

5. Failure clinic — A Parser That Accepts Only the Prefix

6. Engineering case — Derive a parser test from byte-level boundaries

7. Practice and implementation lab

10. Chapter 8 — Designing RPC Deadlines, Cancellation, and Ambiguous Outcomes

1. 8.1 Defining an RPC Contract

1. 8.1.1 Separating Transport Success from Application Success

2. 8.1.2 Selecting Unary, Streaming, and Asynchronous Operations

2. 8.2 Propagating a Finite Waiting Budget

1. 8.2.1 Using Absolute Local Deadlines

2. 8.2.2 Carrying Budgets Across Hops

3. 8.3 Handling Cancellation and Uncertainty

1. 8.3.1 Treating Cancellation as a Cooperative Signal

2. 8.3.2 Preserving Identity When the Outcome Is Unknown

4. Worked example — Account for Queue Residence in a Shared Budget

1. Deterministic implementation

2. Connect the model to the transport

5. Failure clinic — Three Layers of Retries

6. Engineering case — Account for one deadline across three stages

7. Practice and implementation lab

11. Chapter 9 — Discovering Services and Balancing Work

1. 9.1 Representing Membership as Versioned Information

1. 9.1.1 Distinguishing Names, Endpoints, and Instances

2. 9.1.2 Rejecting Stale Membership Updates

2. 9.2 Separating Health from Routing Policy

1. 9.2.1 Interpreting Liveness, Readiness, and Degradation

2. 9.2.2 Choosing a Load-Balancing Objective

3. 9.3 Coordinating Routing with Failure and State

1. 9.3.1 Retrying Without Violating Placement

2. 9.3.2 Handling Empty and Uncertain Membership

4. Worked example — Preserve the Newest Discovery Snapshot

1. Implementation

2. Evidence and interpretation

5. Failure clinic — Healthy Replicas with Inconsistent Deduplication

6. Engineering case — Recover discovery after a registry generation changes

7. Practice and implementation lab

12. Chapter 10 — Reasoning About Failures, Clocks, and Causality

1. 10.1 Naming the Failures a Design Assumes

1. 10.1.1 Distinguishing Crash, Omission, Delay, and Corruption
2. 10.1.2 Treating Failure Detection as a Decision Under Uncertainty

2. 10.2 Choosing the Right Meaning of Time

1. 10.2.1 Separating Wall Time from Elapsed-Time Budgets
2. 10.2.2 Defining Expiration and Restart Behavior

3. 10.3 Representing Causal Relationships

1. 10.3.1 Using Logical Clocks Without Pretending They Measure Duration
2. 10.3.2 Keeping Ordering Metadata Within a Stable Identity Domain

4. Worked example — Trace a message through two logical clocks

1. Requirement and mechanism
2. Run and interpret
3. Failure clinic — A sorted log tells the wrong story

5. Engineering case — Reconstruct a partial order without inventing a timeline

6. Practice and implementation lab

1. Review and solution reasoning

13. Chapter 11 — Choosing Consistency and Replication Contracts

1. 11.1 Making Read and Write Guarantees Explicit

1. 11.1.1 Distinguishing Linearizable, Sequential, and Eventual Views
2. 11.1.2 Describing Session Guarantees and Read Dependencies

2. 11.2 Understanding What Replication Acknowledges

1. 11.2.1 Separating Receipt, Persistence, and Application
2. 11.2.2 Recovering Lagging Replicas Without Inventing Authority

3. 11.3 Reasoning About Partitions and Availability

1. 11.3.1 Applying CAP to the Failure Case It Describes
2. 11.3.2 Choosing Degradation Rather Than Hiding It

4. Worked example — Expose a stale replica deliberately

1. Requirement and scope
2. Evidence and interpretation
3. Failure clinic — Stale absence is treated as permission

5. Engineering case — Compare two acknowledgement policies under one crash

- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 14. Chapter 12 — Understanding Consensus, Leases, and Fencing
 - 1. 12.1 Identifying the State That Requires Agreement
 - 1. 12.1.1 Distinguishing Replicated Decisions from Broadcast Messages
 - 2. 12.1.2 Separating Safety from the Conditions for Progress
 - 2. 12.2 Interpreting Locks and Leases Correctly
 - 1. 12.2.1 Treating Lease Expiration as an Authority Event
 - 2. 12.2.2 Understanding Why a Distributed Mutex Is Not a Process Stop
 - 3. 12.3 Enforcing Generations at the Resource Boundary
 - 1. 12.3.1 Designing Monotonic Fencing Checks
 - 2. 12.3.2 Preserving Fences Through Recovery and State Transfer
 - 4. Worked example — Reject a resumed stale owner
 - 1. Requirement and boundary
 - 2. Interpret the evidence
 - 3. Failure clinic — Check-then-write leaves a race
 - 5. Engineering case — Restore a value without restoring stale authority
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 15. Chapter 13 — Partitioning Data and Migrating Ownership
 - 1. 13.1 Selecting the Partitioning Boundary
 - 1. 13.1.1 Partitioning by Business Identity Rather Than Convenience
 - 2. 13.1.2 Recognizing Hot Keys and Unequal Work
 - 2. 13.2 Implementing Stable Placement
 - 1. 13.2.1 Comparing Modulo Assignment and Rendezvous Selection
 - 2. 13.2.2 Versioning the Membership and Algorithm Together
 - 3. 13.3 Moving State Before Changing Authority
 - 1. 13.3.1 Defining a Migration State Machine
 - 2. 13.3.2 Avoiding Unsafe Dual Writes and Premature Cleanup
 - 4. Worked example — Measure movement when adding a node
 - 1. Requirement and implementation
 - 2. Interpret the observation

- 3. Failure clinic — A routing rollout outruns a data rollout
- 5. Engineering case — Move one key while preserving a late retry
- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 16. Chapter 14 — Building Transactional Persistence with SQLite
 - 1. 14.1 Designing State Around Invariants
 - 1. 14.1.1 Encoding Keys, Inputs, and Results as a Schema
 - 2. 14.1.2 Placing the Entire Decision in One Transaction
 - 2. 14.2 Owning Native Database Resources in C++
 - 1. 14.2.1 Wrapping Connections, Statements, and Transactions
 - 2. 14.2.2 Distinguishing Busy Conditions from Invalid Data
 - 3. 14.3 Understanding Durability and Recovery Boundaries
 - 1. 14.3.1 Interpreting WAL and Synchronous Settings
 - 2. 14.3.2 Testing Reopen, Rollback, and Concurrent Requests
 - 4. Worked example — Make duplicate acceptance survive a full queue
 - 1. Requirement and implementation
 - 2. Trace the state changes
 - 3. Failure clinic — Success is returned before commit
 - 5. Engineering case — Explain two connections racing on the same key
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 17. Chapter 15 — Migrating Schemas and Supporting Mixed Versions
 - 1. 15.1 Defining Compatibility in Both Directions
 - 1. 15.1.1 Separating Wire, Source, Binary, and Data Compatibility
 - 2. 15.1.2 Preserving Meaning When Fields Are Added
 - 2. 15.2 Planning Expand, Backfill, Switch, and Contract
 - 1. 15.2.1 Expanding Without Requiring Immediate Fleet Replacement
 - 2. 15.2.2 Contracting Only After the Old Dependency Is Gone
 - 3. 15.3 Running Migrations as Controlled Operations
 - 1. 15.3.1 Owning the Migration and Its Transaction Boundary
 - 2. 15.3.2 Verifying Recovery at Every Phase
 - 4. Worked example — Add a defaulted priority field

- 1. Requirement and implementation
- 2. Evidence and interpretation
- 3. Failure clinic — A rolling upgrade changes job meaning
- 5. Engineering case — Roll out an operation-version field without changing old jobs
- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 18. Chapter 16 — Designing Caches with Explicit Freshness
 - 1. 16.1 Deciding What a Cache Is Allowed to Forget
 - 1. 16.1.1 Separating Cached Copies from Authoritative State
 - 2. 16.1.2 Choosing Freshness and Failure Semantics
 - 2. 16.2 Coordinating Versions, Invalidations, and Fills
 - 1. 16.2.1 Rejecting Old Fills After an Invalidation
 - 2. 16.2.2 Bounding the Metadata Needed to Reject Staleness
 - 3. 16.3 Controlling Cache Load and Failure Amplification
 - 1. 16.3.1 Preventing Simultaneous Misses from Overloading the Authority
 - 2. 16.3.2 Measuring Useful Hits Rather Than Only Hit Rate
 - 4. Worked example — Block a stale fill after invalidation
 - 1. Requirement and implementation
 - 2. Evidence and interpretation
 - 3. Failure clinic — Invalidations arrive out of order
 - 5. Engineering case — Reclaim invalidation floors without reopening the race
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 19. Chapter 17 — Designing Messaging and Delivery Semantics
 - 1. 17.1 Separating Commands, Events, and Transport Records
 - 1. 17.1.1 Giving Each Message a Defined Meaning
 - 2. 17.1.2 Choosing Queue or Log Behavior Deliberately
 - 2. 17.2 Interpreting Acknowledgement and Redelivery
 - 1. 17.2.1 Locating the Crash Window Around an Effect
 - 2. 17.2.2 Distinguishing Retryable Failures from Poison Messages
 - 3. 17.3 Preserving the Ordering That the Domain Needs

- 1. 17.3.1 Separating Delivery Order from Completion Order
- 2. 17.3.2 Designing Replay as a Supported Operation
- 4. Worked example — Distinguish three deliveries from two effects
 - 1. Requirement and implementation
 - 2. Trace the result
 - 3. Failure clinic — Acknowledgement races ahead of commit
- 5. Engineering case — Advance a consumer checkpoint without skipping unfinished work
- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 20. Chapter 18 — Implementing Idempotency with a Durable Outbox and Inbox
 - 1. 18.1 Defining Idempotency as an API Contract
 - 1. 18.1.1 Binding One Identity to One Canonical Request
 - 2. 18.1.2 Choosing Receipt Retention and Result Replay
 - 2. 18.2 Making Intent and Effect Locally Atomic
 - 1. 18.2.1 Recording Intent Before Attempting Delivery
 - 2. 18.2.2 Committing the Worker Receipt with Its Effect
 - 3. 18.3 Recovering Every Acknowledgement Gap
 - 1. 18.3.1 Walking the Failure Matrix
 - 2. 18.3.2 Distinguishing Safety from Eventual Completion
 - 4. Worked example — Replay after a lost response
 - 1. Requirement and implementation
 - 2. Trace the evidence
 - 3. Failure clinic — One key is reused for changed work
 - 5. Engineering case — Identify the boundary that cannot be made atomic locally
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 21. Chapter 19 — Containing Failure with Retries, Circuit Breakers, and Bulkheads
 - 1. 19.1 Spending a Retry Budget Deliberately
 - 1. 19.1.1 Distinguishing Attempts from Logical Operations
 - 2. 19.1.2 Combining Backoff, Jitter, and Remaining Time
 - 2. 19.2 Implementing a Circuit Breaker as a State Machine

- 1. 19.2.1 Defining Closed, Open, and Half-Open Behavior
- 2. 19.2.2 Ignoring Completions from an Obsolete Breaker Epoch
- 3. 19.3 Isolating Resource Exhaustion
 - 1. 19.3.1 Building Bulkheads Around Actual Scarce Resources
 - 2. 19.3.2 Recovering Without a Synchronized Surge
- 4. Worked example — Recover through one half-open probe
 - 1. Requirement and implementation
 - 2. Evidence and interpretation
 - 3. Failure clinic — A retry repairs transport but duplicates intent
- 5. Engineering case — Prevent an old success from closing a new breaker
- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 22. Chapter 20 — Engineering Backpressure and Admission Control
 - 1. 20.1 Bounding Every Waiting Location
 - 1. 20.1.1 Distinguishing Connection, Memory, and Durable Backlog Limits
 - 2. 20.1.2 Choosing an Overload Response at the Admission Boundary
 - 2. 20.2 Coordinating Producers and Consumers Safely
 - 1. 20.2.1 Defining Queue Ownership and Close-and-Drain Semantics
 - 2. 20.2.2 Preserving Ownership When Admission Fails
 - 3. 20.3 Regulating Rate and Concurrency
 - 1. 20.3.1 Using Token Buckets Without Confusing Them with Queue Limits
 - 2. 20.3.2 Using Queueing Arithmetic as a Diagnostic, Not a Guarantee
 - 4. Worked example — Bound, close, and drain
 - 1. Requirement and implementation
 - 2. Evidence and interpretation
 - 3. Failure clinic — The queue is bounded but the retry list is not
 - 5. Engineering case — Budget a queue in bytes as well as entries
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 23. Chapter 21 — Coordinating Workflows and Compensating Partial Success
 - 1. 21.1 Modeling the Business Process as State
 - 1. 21.1.1 Naming Intermediate and Terminal Outcomes

- 2. 21.1.2 Separating Orchestration from Choreography
- 2. 21.2 Making Each Step Recoverable
 - 1. 21.2.1 Persisting Commands, Results, and Timers
 - 2. 21.2.2 Deduplicating Events Without Erasing Conflicts
- 3. 21.3 Designing Compensation as New Work
 - 1. 21.3.1 Understanding Why Compensation Is Not Time Travel
 - 2. 21.3.2 Reconciling Stuck and Ambiguous Workflows
- 4. Worked example — Make compensation an explicit transition
 - 1. Requirement and implementation
 - 2. Evidence and interpretation
 - 3. Failure clinic — A late success is discarded during cancellation
- 5. Engineering case — Handle cancellation while a charge outcome is unknown
- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 24. Chapter 22 — Securing Service Identity and Authorization Boundaries
 - 1. 22.1 Establishing Identity Before Making Policy Decisions
 - 1. 22.1.1 Distinguishing Authentication from Authorization
 - 2. 22.1.2 Preserving Identity Across Service Hops
 - 2. 22.2 Protecting Transport Without Losing Application Semantics
 - 1. 22.2.1 Validating Peers, Not Merely Encrypting Bytes
 - 2. 22.2.2 Accounting for Replay, Termination, and Proxy Boundaries
 - 3. 22.3 Limiting Damage from Inputs and Secrets
 - 1. 22.3.1 Validating Resource Use Alongside Syntax
 - 2. 22.3.2 Operating Credentials and Audit Records Safely
 - 4. Worked example — Evaluate tenant-scoped permission
 - 1. Requirement and implementation
 - 2. Evidence and interpretation
 - 3. Failure clinic — Tenant identity is copied from the payload
 - 5. Engineering case — Preserve authorization through an authenticated gateway
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 25. Chapter 23 — Making Distributed Behavior Observable

1. 23.1 Choosing Signals with Defined Meaning
 1. 23.1.1 Measuring Outcomes at the Correct Boundary
 2. 23.1.2 Controlling Label Cardinality
2. 23.2 Connecting Work Across Processes
 1. 23.2.1 Separating Logical Job, Attempt, and Trace Identity
 2. 23.2.2 Placing Spans Around Waiting as Well as Execution
3. 23.3 Designing Diagnostics That Do Not Become the Incident
 1. 23.3.1 Recording Structured, Bounded Logs
 2. 23.3.2 Using Histograms Without Inventing Precision
4. Worked example — Emit a cumulative latency histogram
 1. Requirement and implementation
 2. Interpret the output
 3. Failure clinic — Every retry is counted as another completed job
5. Engineering case — Reconcile attempt counters with durable job state
6. Practice and implementation lab
 1. Review and solution reasoning
26. Chapter 24 — Testing Distributed Behavior with Deterministic Models and Fault Injection
 1. 24.1 Designing Tests Around Invariants
 1. 24.1.1 Separating an Oracle from the Implementation Under Test
 2. 24.1.2 Testing Boundaries and Negative Paths
 2. 24.2 Exploring Histories Deterministically
 1. 24.2.1 Injecting Time, Membership, and Completion Order
 2. 24.2.2 Enumerating Small State Spaces and Recording Seeds
 3. 24.3 Crossing Real Process and Persistence Boundaries
 1. 24.3.1 Placing Failpoints at the Difficult Moments
 2. 24.3.2 Interpreting Sanitizers and Platform Coverage
 4. Worked example — Enumerate duplicate-delivery orders
 1. Requirement and implementation
 2. Evidence and interpretation
 3. Failure clinic — A race test waits and hopes
 5. Engineering case — Test a crash point rather than a convenient outage

6. Practice and implementation lab

1. Review and solution reasoning

27. Chapter 25 — Producing Reproducible Builds and Release Artifacts

1. 25.1 Defining the Build as a Dependency Graph

1. 25.1.1 Expressing Requirements Through Targets

2. 25.1.2 Keeping Generated Outputs Out of the Source Contract

2. 25.2 Recording Provenance and Supply-Chain Decisions

1. 25.2.1 Identifying Source, Toolchain, and Dependency Inputs

2. 25.2.2 Separating Integrity, Authenticity, and Reproducibility

3. 25.3 Packaging the Supported Runtime

1. 25.3.1 Distinguishing a Binary Archive from a Deployable Service

2. 25.3.2 Releasing the Artifact That Was Actually Tested

4. Worked example — Inspect a compile fingerprint

1. Requirement and implementation

2. Evidence and interpretation

3. Failure clinic — The tested binary is replaced during packaging

5. Engineering case — Rebuild a package after its original workspace disappears

6. Practice and implementation lab

1. Review and solution reasoning

28. Chapter 26 — Deploying Services Through Controlled Lifecycle Transitions

1. 26.1 Making Startup and Readiness Explicit

1. 26.1.1 Initializing Before Accepting Work

2. 26.1.2 Separating Liveness, Readiness, and Startup Probes

2. 26.2 Draining Without Losing Ownership

1. 26.2.1 Stopping Admission Before Releasing Resources

2. 26.2.2 Distinguishing Request Cancellation from Service Termination

3. 26.3 Replacing Instances Without Breaking State

1. 26.3.1 Planning Rolling Upgrades Around Compatibility and Capacity

2. 26.3.2 Defining Rollback Conditions Before Deployment

4. Worked example — Model admission and draining

1. Requirement and implementation

2. Evidence and interpretation

- 3. Failure clinic — Liveness depends on every downstream service
- 5. Engineering case — Stop a gateway while accepted work remains pending
- 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 29. Chapter 27 — Operating Reliability, Capacity, and Recovery
 - 1. 27.1 Defining Service Objectives That Match User Outcomes
 - 1. 27.1.1 Choosing Indicators and Denominators
 - 2. 27.1.2 Interpreting Error Budgets Without False Precision
 - 2. 27.2 Planning Capacity and Incident Response
 - 1. 27.2.1 Forecasting the Limiting Resource
 - 2. 27.2.2 Responding with a Hypothesis and a Reversible Action
 - 3. 27.3 Proving Recovery Rather Than Assuming It
 - 1. 27.3.1 Defining Recovery Point and Recovery Time
 - 2. 27.3.2 Closing an Incident with a Causal Review
 - 4. Worked example — Calculate a hypothetical request-count budget
 - 1. Requirement and implementation
 - 2. Evidence and interpretation
 - 3. Failure clinic — A successful backup contains the wrong authority
 - 5. Engineering case — Close an incident only after the backlog is reconciled
 - 6. Practice and implementation lab
 - 1. Review and solution reasoning
- 30. Chapter 28 — Building and Reviewing HarborDispatch
 - 1. 28.1 Defining the Architecture and Its Trust Boundary
 - 1. 28.1.1 Assigning Responsibilities to the Client, Gateway, and Worker
 - 2. 28.1.2 Enumerating Limits and Unsupported Promises
 - 2. 28.2 Following a Job Through Commit and Recovery
 - 1. 28.2.1 Understanding the Submission and Execution Protocol
 - 2. 28.2.2 Recovering the Lost-Reply and Restart Histories
 - 3. 28.3 Building, Exercising, and Reviewing the System
 - 1. 28.3.1 Running the Actual Binaries in a Private Lab
 - 2. 28.3.2 Reviewing the Evidence Before Extending the Design
 - 4. Worked example — Join intent and receipts across two stores

1. Requirement and implementation
2. Interpret the local capstone model
3. Failure clinic — A convenient shortcut changes the authority
5. Engineering case — Choose one extension and preserve the reference contract
6. Practice and implementation lab
 1. Review and solution reasoning
31. Appendix A — HarborDispatch Operations and Diagnostic Workbook
 1. A.1 Establish the Environment Before Starting Processes
 2. A.2 Maintain a Small State Inventory
 3. A.3 Start the Lab and Record Endpoints
 4. A.4 Verify the Contract with a Small Request Set
 5. A.5 Diagnose a Connection Failure by Layer
 6. A.6 Diagnose Pending Jobs Without Erasing Evidence
 7. A.7 Exercise a Lost Reply Deliberately
 8. A.8 Perform a Controlled Restart
 9. A.9 Plan Backup and Restore as an Isolated Drill
 10. A.10 Record an Incident as a Causal Account
32. Appendix B — Complete Core and Application Source
 1. B.1 Read the Ownership Graph Before the Functions
 2. B.2 Domain and Test Assertions
 3. B.3 Wire Representation and Incremental State
 4. B.4 Policy and Model Types
 5. B.5 Queue and Metrics Ownership
 6. B.6 Database Interface and Implementation
 7. B.7 Linux Transport Interface and Reactor
 8. B.8 Worker Process
 9. B.9 Gateway Process
 10. B.10 Client and Build Graph
 11. B.11 How to Modify the Source Without Losing Evidence
33. Appendix C — Verification Atlas and Complete Test Source
 1. C.1 Chapter Executables as Focused Checks
 2. C.2 Core Regression Families

- 3. C.3 File-Backed Store Regression
- 4. C.4 Process Integration Histories
- 5. C.5 Reading Failures Without Broadening the Claim
- 6. C.6 Extending Coverage with an Explicit Hypothesis
- 7. C.7 Configuration Matrix and Release Evidence
- 34. Appendix D — Framework Integration Blueprints
 - 1. D.1 Replace the Transport Without Replacing the Domain
 - 2. D.2 Preserve Deadline and Cancellation Semantics
 - 3. D.3 Add TLS and Identity as a Separate Reviewed Boundary
 - 4. D.4 Introduce a Broker Through the Existing Outbox
 - 5. D.5 Add Observability Without Changing the Service’s Meaning
 - 6. D.6 Prepare an Orchestrated Deployment Only After State Ownership Is Solved
 - 7. D.7 Gate Each Integration with Evidence
- 35. Appendix E — Architecture Review Exercises and Worked Reasoning
 - 1. E.1 A Client Wants to Retry with a New Key
 - 2. E.2 A Team Wants to Delete Receipts After One Hour
 - 3. E.3 A Gateway Should Use Several Workers
 - 4. E.4 A Database Callback Must Leave the Reactor Thread
 - 5. E.5 A Cache Should Serve Every Status Query
 - 6. E.6 A Broker Consumer Wants More Parallelism
 - 7. E.7 A New Schema Changes Units
 - 8. E.8 A Dashboard Reports Perfect Success During Overload
 - 9. E.9 A Backup Restores Only the Gateway
 - 10. E.10 A Circuit Breaker Should Be Shared Across Threads
 - 11. E.11 A Production Review Requests a Performance Number
 - 12. E.12 Write a Filled Architecture Decision Record
- 36. Appendix F — Architecture and Distributed-Systems Glossary
- 37. Closing Perspective — Extend the System Without Losing the Argument
- 38. Technical reference directory
 - 1. R01 — C++ Core Guidelines
 - 2. R02 — Working Draft: Coroutine definitions

3. R03 — Asynchronous operations
4. R04 — Strands: Use Threads Without Explicit Locking
5. R05 — poll(2)
6. R06 — recv(2)
7. R07 — send(2)
8. R08 — RFC 9110: HTTP Semantics
9. R09 — Core concepts, architecture and lifecycle
10. R10 — Deadlines
11. R11 — Cancellation
12. R12 — Retry
13. R13 — Custom Name Resolution
14. R14 — Language Guide: proto3
15. R15 — Time, Clocks, and the Ordering of Events in a Distributed System
16. R16 — In Search of an Understandable Consensus Algorithm: Extended Version
17. R17 — Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services
18. R18 — Transaction
19. R19 — Isolation in SQLite
20. R20 — Write-Ahead Logging
21. R21 — PRAGMA Statements
22. R22 — SQLite Online Backup API
23. R23 — Transaction Isolation
24. R24 — Client-side caching
25. R25 — Consumer Acknowledgements and Publisher Confirms
26. R26 — Design
27. R27 — API guarantees
28. R28 — Authentication
29. R29 — RFC 8446: The Transport Layer Security Protocol Version 1.3
30. R30 — Traces
31. R31 — Metric and label naming
32. R32 — Handling Overload

- 33. R33 — Implementing SLOs
 - 34. R34 — Pod Lifecycle
 - 35. R35 — Configure Liveness, Readiness and Startup Probes
 - 36. R36 — Importing and Exporting Guide
 - 37. R37 — SLSA Levels
 - 38. R38 — Making retries safe with idempotent APIs
 - 39. R39 — FoundationDB: A Distributed Unbundled Transactional Key Value Store
 - 40. R40 — Trace Context
 - 41. R41 — `SSL_set1_host`
 - 42. R42 — C++ Best Practices
 - 43. R43 — Cross-Version Runtime Guarantee
 - 44. R44 — Graceful Shutdown
 - 45. R45 — A Proof for the Queuing Formula: $L = \lambda W$
 - 46. R46 — `close(2)`
 - 47. R47 — `clock_gettime(3)`
 - 48. R48 — Working Draft: Signal handlers
39. New section examples and references
- 1. Title Page
 - 2. Cover
 - 3. Table of Contents

About this volume

Advanced C++ Architecture and Distributed Systems

Large-Scale Software Architecture, Distributed Computing, Services, Reliability, and Production Infrastructure

Dr. Charles Dorner

C++ From Beginner to Expert — Book VII

First edition · 2026

A function returns an error, but the work may already have happened. A process is alive, but it cannot make progress. A second server appears, but it shares the first server's storage and therefore its most important failure mode. A cache answers quickly, but with information old enough to invalidate the user's decision. These are not problems that a more sophisticated pointer type can solve alone. They require an architecture that describes what each participant knows, which participant owns a fact, and what evidence justifies a successful response.

This book extends the original six-volume *C++ From Beginner to Expert* curriculum. It begins where a dependable single-process or heterogeneous application meets independent deployment, remote communication, durable state, and partial failure. Books I-VI remain the original sequence; Book VII is an additional volume, not a renumbering of those books or an assertion that the original curriculum contained seven volumes.

The recurring project is **HarborDispatch**, a fictional internal job-submission system. A client submits bounded work with a stable request key. A gateway records the intent in one database. A dispatcher sends it to a worker with a separate database. The worker records the operation's result, and the gateway reconciles completion. The calculation is intentionally simple: square an integer in the documented range. Keeping the computation inspectable lets the book concentrate on request identity, duplicate delivery, crash recovery, overload, protocol evolution, and operational ownership.

No example describes a particular employer's infrastructure. Hostnames, tenants, requirements, incidents, and workloads are invented teaching material. Numerical examples are calculations under explicit assumptions, not measurements of a customer's systems. Actual local test results are recorded separately in the companion's verification report.

Audience and prerequisites

Readers should be comfortable with C++ classes, RAII, move semantics, containers, exceptions, basic concurrency, CMake, and the distinction between a process and a thread. Book IV's systems material and Book V's testing and build practices are the most direct preparation. GPU programming is not a prerequisite. This volume is about coordinating independently failing components, not about requiring every component to run on a GPU.

An experienced engineer can begin with the chapters on deadlines, consistency, idempotency, or incident response, but should return to the architecture chapters before implementing the capstone. A distributed system’s difficult defects often begin as an unstated local assumption: a callback that outlives its buffer, an error code that implies too much, or a database transaction that accidentally holds a lock while waiting on a remote service.

Technical baseline

The executable companion uses **C++20**, **CMake 3.24 or later**, Linux sockets, the installed SQLite development library, and Python 3 for process-level tests. Linux is an intentional target for the transport exercises. The C++ standard does not define POSIX sockets, `poll`, or SQLite; their behavior must not be mistaken for portable language guarantees. The code does not download dependencies during configuration. See the included build report for the compiler, standard-library, operating-system, and SQLite versions actually exercised.

Framework discussions distinguish the common architectural mechanism from a particular library API. Boost.Asio, gRPC, Protocol Buffers, PostgreSQL, Redis, RabbitMQ, Kafka, OpenTelemetry, Kubernetes, and TLS appear where their documented semantics clarify the design. The default companion does not require those external services. Optional integration material is labeled separately, and its presence is not a claim that those integrations were deployed or tested.

Primary-source references are identified as **R01**, **R02**, and so on, with links in the technical reference directory. Project documentation is version-sensitive. The consulted documentation date is recorded there; installed manuals and the selected release’s documentation remain the authority for actual deployment. Original scenarios, derivations, decision records, and exercises are the teaching narrative, not quotations from those references.

What the companion actually contains

Each of the 28 chapters has a distinct executable demonstration. Some demonstrations are deliberately small policy models; others exercise real sockets or SQLite. The difference is stated in the chapter. A model that shows how a fencing check rejects an old token is not a consensus implementation. A permutation test of a small inbox model is not an exhaustive verification of an arbitrary network. An in-memory database exercises transactions without establishing crash durability; separate process and file-backed tests investigate restart behavior.

The integrated application contains three Linux executables: `harbor_gateway`, `harbor_worker`, and `harbor_client`. The gateway and worker run as separate processes and use separate SQLite database files. The gateway’s pending-job table acts as a durable outbox; the worker’s receipt table acts as an inbox and result record. The tests include lost acknowledgments, duplicate submissions, conflicting identities, fragmented input, an unavailable worker, restart recovery, and transaction rollback after a process is killed.

The supplied transport is IPv4 loopback-only and has no authentication or TLS. It is not a public server, a multiregion database, a complete Raft implementation, or a certified high-availability product. Chapters on security, replication, deployment, and operations explain what must be added and verified for those roles. Do not remove the loopback restriction and treat the teaching program as internet-ready infrastructure.

How to use this book

Follow the evidence chain

Each chapter begins with a requirement or a failure that is easy to misinterpret. Numbered sections establish the underlying concepts; subsections develop the concrete decisions. A worked implementation exposes one mechanism. The failure clinic shows how a plausible shortcut breaks. The practice lab asks for a bounded change, an acceptance condition, and an explanation of the result. Solution notes describe why an answer satisfies the contract instead of merely giving a final number.

Keep four questions beside the editor: **What changed? What became observable? Who may act on that observation? What remains uncertain?** A client receiving a transport timeout has learned that it did not receive a timely answer. It has not necessarily learned that the worker did nothing. A successful COMMIT establishes a database-specific boundary under its durability assumptions. It does not prove that another database contains the same information. Thinking at the right boundary is the central skill of this book.

Build and run a chapter

From the extracted package root:

```
cmake -S source -B build/debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug --parallel 4
ctest --test-dir build/debug --output-on-failure
./build/debug/ch01
```

These commands assume the required compiler, CMake, SQLite development files, and Python are installed. On a system without them, use that distribution's supported installation method; the project deliberately does not run a privileged installer. Paths under build/ are disposable generated files. Source and manuscripts remain outside the build tree.

Run a specific test by its registered name. For example, the fragmented-input process test is selected with `ctest --test-dir build/debug -R '^integration_partial$' --output-on-failure`. Do not describe this as a complete network test: it establishes only the behavior exercised by that scenario. Use the regression suite and then create additional cases for the mechanism you change.

Read code and output honestly

A code listing labeled with a companion path is inserted from that actual file during assembly. Small sketches labeled “illustrative” are explanations, not additional compiled targets. Shell blocks contain commands; configuration blocks contain file contents. No decorative shell prompt is required. Replace a port placeholder only with a port printed by the relevant local server or selected deliberately for the exercise.

C++ tests use explicit checks rather than relying on `assert`, so Release builds do not silently remove their acceptance conditions. The Python integration harness requires ordinary nonoptimized interpreter execution and rejects `-O` or `PYTHONOPTIMIZE` modes that would remove its assertions. A zero test exit status means those conditions passed in that execution. It is not a guarantee of thread scheduling, power-loss durability, TLS policy, or performance on different hardware. Timing values in deterministic models are logical milliseconds supplied by the test, not measurements from a wall clock.

Separate implementation from qualification

There are three levels of work in this volume. **Mechanism laboratories** isolate rules such as partial framing or cache invalidation. **Integration laboratories** run several real processes and persistent stores on a local machine. **Production design exercises** describe deployment, authentication, replication, migration, and operational evidence that cannot be inferred from a local loopback test. Moving between these levels requires additional validation; it is not accomplished by changing a heading from “lab” to “production.”

Never perform fault injection against infrastructure you do not own or administer with explicit authorization. The supplied tests create private temporary directories and use loopback sockets. They do not change firewall rules, terminate unrelated processes, or connect to public endpoints. Database files used in a crash exercise must be disposable. Back up and inspect the target path before adapting an exercise to any non-lab environment.

Reading paths

Chapters 1–4 establish architectural boundaries, APIs, and lifecycle. Chapters 5–9 move from asynchronous C++ execution into protocols, RPC, and discovery. Chapters 10–13 explain failure, consistency, coordination, and placement. Chapters 14–18 connect transactions, evolution, caches, messaging, and idempotency. Chapters 19–24 build resilience, security, observability, and testing. Chapters 25–28 assemble release, deployment, reliability, and the complete application.

A useful study rhythm is one chapter, one changed experiment, and one written argument. Predict the result before execution. Explain a failed prediction rather than editing the test until it passes. Keep reference implementations and previous test cases intact while adding a new mechanism. An engineering notebook that records rejected hypotheses is often more useful than a directory of unexplained successful runs.

Rebuild the books

The package includes the authored Markdown chapters, source files, cover and styles, and the actual assembly and rendering scripts. `python tools/assemble_book.py` resolves the source inclusions and creates the complete manuscript. `python tools/build_books.py` generates the EPUB and PDF from that manuscript. The scripts do not call an AI service, invent missing chapters, or repeat generic paragraphs to increase length. Font files are not redistributed; changing locally installed fonts can change pagination.

The operating rule throughout the volume is: **make the boundary explicit, preserve identity across uncertainty, verify the state transition, and measure the complete service path.**

Chapter 1 — Defining Architecture Through Contracts and Failure Boundaries

HarborDispatch begins as a request: “Let other applications submit work and check the result later.” A first design places a queue behind a network listener and starts several threads. That design may work for a demonstration, but it has not yet answered what happens when the response is lost, when the process stops, or when two clients choose the same request key. Architecture begins with those questions, before a diagram acquires boxes named after technologies.

This chapter turns the request into a contract that can be implemented and tested. The central distinction is between a property the application promises and a mechanism that might help implement it. A queue is a mechanism. “An acknowledged job survives the gateway process restarting” is a property. The latter determines whether an in-memory queue is sufficient.

1.1 Turning Requirements into Observable Contracts

1.1.1 Separating Outcomes from Implementation Preferences

A requirement should describe something an observer can distinguish. “Use asynchronous processing” describes a preference about implementation. “A client receives a stable job key after durable acceptance and can retrieve the eventual result using that key” describes behavior. The second sentence leaves room for different thread models, databases, and transports while making the acceptance point meaningful.

For the teaching service, the input is an identifier and an integer from zero through one million. The result is the integer’s square. The identifier is an ASCII token of at most 64 bytes. These constraints make parsing, storage, and arithmetic bounded. They also make wrong-result detection independent of the worker: the test can compute an expected answer without trusting the remote implementation.

Separate **functional outcomes** from **operational conditions**. Functional outcomes include rejection of malformed input and stable replay of an accepted request. Operational conditions include maximum frame size, admitted backlog, request waiting time, and the behavior of shutdown. A successful arithmetic result does not compensate for an unbounded backlog. Conversely, bounded memory does not compensate for returning the wrong job’s result.

Avoid writing performance targets without a measurement boundary. “Fast” cannot be tested. An illustrative target such as “ninety-nine percent of status queries complete within 100 milliseconds under the named workload” identifies a population, a statistic, a threshold, and a workload. It is still incomplete until the team states whether time begins at the client, the load balancer, or the server’s handler. These

are design questions, not decoration for a benchmark report.

1.1.2 Defining Preconditions, Postconditions, and Invariants

A precondition describes what must hold before an operation is valid. A postcondition describes what the operation establishes when it succeeds. An invariant describes a property preserved across valid operations. For `submit(key, input)`, the precondition includes a valid key and a bounded integer. A successful acceptance establishes that the gateway has a retrievable record. The invariant is that one key cannot silently acquire a different input later.

That invariant has consequences. A repeated key with the same canonical input is a replay; a repeated key with a different input is a conflict. Returning “already exists” without comparing the input loses the distinction. A client that accidentally reuses a key for another job could otherwise receive an unrelated result while believing its new request succeeded.

A C++ type can express part of the contract but not all of it. `std::int64_t` establishes a representation range, not the business limit of one million. A `std::string` can contain spaces, null bytes, and arbitrary lengths; it does not establish an identifier grammar. The boundary must validate these properties and create a domain value only after validation. The C++ Core Guidelines’ interface guidance supports making expectations and ownership visible, but the specific HarborDispatch invariants are the design developed here. (R01)

Write invariants independently of the chosen repository. A memory implementation and a SQLite implementation should both reject changed content under a stable key. Their persistence properties differ, so tests must not treat them as equivalent in every respect. An interface can unify operation names while preserving the need to qualify storage guarantees.

NEW SECTION EXAMPLE

Turn bounded job admission into a testable postcondition

Accept a request only when the input is in range and queue capacity remains; rejection must leave existing work unchanged.

main.cpp

```
#include <array>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> jobs{4};constexpr std::size_t capacity=2;
    auto submit=[&](int x){if(x<0||x>100||jobs.size()==capacity)return
false;jobs.push_back(x);return true;};
    bool a=submit(7),b=submit(101),c=submit(8);
    if(jobs!=std::vector<int>{4,7})return 1;
    std::cout<<std::boolalpha<<"accepted="<<a<<" invalid="<<b<<" full="<<c<<"
count="<<jobs.size()<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The two guards are preconditions for acceptance, not implementation preferences. The successful call appends exactly one value; invalid-input and full-queue calls return false without changing the sequence.

Comparing the entire vector checks the postcondition, including order and lack of partial mutation. Capacity2 is an application requirement in this fixture, independent of vector's allocated capacity.

Expected behavior or output

```
accepted=true invalid=false full=false count=2
```

Concrete experiment

Change the queue limit to 3. The last submission should succeed, but input101 must still be rejected. Update the expected final sequence rather than checking only its size.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

1.2 Identifying State and Responsibility

1.2.1 Assigning an Authoritative Owner to Each Fact

A fact needs an owner that can answer conflicting claims. The gateway owns whether a request has been accepted. The worker owns its receipt and computed result. The client owns whether it has observed an answer. Those facts can legitimately differ at the same moment. The worker may have committed a result while the gateway still records the job as pending because the acknowledgment was lost.

Draw the ownership graph separately from the call graph. In the call graph, the gateway contacts the worker. In the ownership graph, neither process may rewrite the other's database directly. This prevents a shortcut in which the gateway "repairs" a remote result by editing a file it happens to reach. Such a shortcut bypasses the worker's transaction, validation, and audit boundaries.

Derived facts should identify their source and freshness. A dashboard's completed-job count may come from a projection updated after worker receipts. The count can lag without making the authoritative records wrong. A design that labels the dashboard "the database" hides that distinction and invites incorrect incident decisions.

Ownership also has a human dimension. A table, API, or queue without an operational owner becomes an orphaned dependency during an outage. Record who approves changes, who receives alerts, and who can restore the data. A distributed system's boundaries remain incomplete if they describe processes precisely but responsibilities vaguely.

1.2.2 Separating Volatile, Durable, and Reconstructible State

Volatile state disappears when the process ends. Durable state is intended to survive the failures named by its storage contract. Reconstructible state can be rebuilt from another authoritative source. These categories answer different recovery questions and should not be collapsed into “stored somewhere.”

The gateway’s socket read buffer is volatile. Losing it may require a client retry, which is acceptable because request identity is stable. A pending accepted job is not merely a socket buffer; it must remain available after a restart. The dispatcher therefore reads pending work from durable records instead of treating a transient queue as the only copy.

A cache of status responses is reconstructible if the database remains authoritative. Its loss is primarily a performance event. A cache used as the only copy of accepted jobs has become a database regardless of its product name. Its eviction, persistence, replication, and recovery settings must then satisfy the stronger role.

Durability is always conditional. A process restart, operating-system crash, host loss, storage-device failure, and regional outage are different fault scopes. A local database can preserve records across a process restart while failing completely with the disk. Naming the intended fault scope avoids claiming disaster recovery from evidence that only exercised a process restart.

NEW SECTION EXAMPLE

Reconstruct a cache from authoritative records after restart

Keep request counters volatile, committed jobs authoritative, and the ready-result cache reconstructible.

main.cpp

```
#include <map>
#include <string>
#include <iostream>

int main() {
    const std::map<std::string,int> durable{{"a",16},{ "b",25}};
    std::map<std::string,int> cache;unsigned requests=9;
    cache=durable;requests=0; // modeled process restart
    cache.erase("a"); // eviction is allowed
    auto value=[&](const std::string& id){++requests;return durable.at(id);};
    std::cout<<"a="<<value("a")<<" cached="<<cache.size()<<"
requests-since-restart="<<requests<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

durable represents a supplied committed snapshot; this program does not persist it to disk. Evicting a cached copy cannot remove the authoritative result. The request counter intentionally resets at the modeled restart.

The three variables encode different lifetimes. Calling all of them “state” without naming the owner would make it unclear which losses require recovery and which are normal cache/counter behavior.

Expected behavior or output

```
a=16 cached=1 requests-since-restart=1
```

Concrete experiment

Clear the entire cache and serveb again. The authoritative answer must remain25. Then removeb from the authoritative snapshot and handle missing explicitly rather than manufacturing a cached answer.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

1.3 Reasoning About Failure Before Choosing Technology

1.3.1 Distinguishing Safety from Liveness

A safety property forbids a bad event. “A key is never associated with two different inputs” is a safety property. A liveness property requires progress. “Every accepted job eventually completes when dependencies recover and resources remain available” is a conditional liveness property. A system can preserve safety by refusing work forever, but that is not a useful service.

The distinction makes failure handling more precise. When the worker is unavailable, the gateway can preserve accepted intent and stop claiming completion. It may continue accepting jobs until the documented pending limit is reached. Once that limit is reached, rejecting additional new jobs preserves bounded resource use. The service loses some availability, but it does not invent success or erase accepted work.

A liveness claim needs assumptions. Eventual completion cannot be guaranteed if the worker remains absent forever, if the input is permanently invalid, or if the storage is irreparably lost. State the recovery assumptions rather than replacing them with the word “eventual.” Later chapters explore fairness and poison messages, which can prevent progress even when the network is functioning.

Tests should name the property under investigation. A duplicate-delivery test checks a safety boundary. A worker-restart test checks progress under a specific recovery schedule. Combining both in one scenario is useful, but neither result establishes all possible schedules.

1.3.2 Choosing the Smallest Architecture That Preserves the Contract

Begin with a design whose state transitions can be inspected. A modular application with one authoritative store may satisfy the workload more clearly than many independent services. Distribution becomes justified when a requirement needs independent deployment, failure containment, scaling, access control, or ownership. Each new remote boundary adds serialization, latency, ambiguity, and operational work.

For this book, a separate worker is an intentional teaching choice: it creates a real acknowledgment gap and an independently restartable participant. The example does not claim that squaring integers needs a distributed service. The deliberately trivial calculation makes it harder to hide protocol defects behind complicated business logic.

List the costs of a proposed boundary alongside its benefit. A remote worker offers an independent process lifecycle. It also needs a transport contract, a timeout policy, an idempotent execution rule, credentials in a real deployment, and a way to recover pending work. A box that appears “stateless” may still rely on a remote database whose capacity and availability become shared constraints.

An architecture decision record should capture the alternatives considered, assumptions used, and conditions that would justify revisiting the decision. “We chose one worker endpoint for the lab because multiworker discovery is taught separately” is an honest record. “This architecture scales indefinitely” is not a decision record; it is an unsupported prediction.

NEW SECTION EXAMPLE

Allow waiting without violating at-most-one decision

Use an explicit unavailable outcome when acknowledgements are insufficient; lack of progress must not overwrite an already decided value.

main.cpp

```
#include <optional>
#include <string>
#include <iostream>

int main() {
    std::optional<std::string> decided;
    auto decide=[&](std::string value,unsigned votes){
        if(decided)return *decided==value;
        if(votes<2)return false;
        decided=std::move(value);return true;
    };
    bool waiting=decide("A",1),committed=decide("A",2),conflict=decide("B",3);
    std::cout<<std::boolalpha<<waiting<<' '<<committed<<' '<<conflict<<"
    decision="<<*decided<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

One acknowledgement produces no decision. Two permit the local model to choose A, and later attempts cannot change it to B. The safety property is the immutable decision; waiting is a liveness limitation.

This is a decision-state illustration, not a consensus implementation: vote identity, epochs, persistence, and message handling are absent. It demonstrates why “return something anyway” is not an acceptable recovery for missing authority.

Expected behavior or output

```
false true false decision=A
```

Concrete experiment

Call `decide(A,0)` after commitment. Returning the existing decision is safe even without new votes; creating a new decision under the same conditions would not be.

Validation of this new example: gcc: passed

Technical references: CPP20, RAFT. See the added references and revision index.

Worked example — Validate Work Before It Reaches Infrastructure

Requirement and implementation

The parser must reject a malformed key, trailing numeric text, negative values, and values above the allowed maximum. The computational function must also defend its public boundary, because callers other than the parser may construct a Request. This is deliberate defense at two interfaces, not a substitute for type design.

The chapter executable calls the domain API directly. There is no socket, thread, or database in the test path. If this test fails, the investigation begins with parsing and domain rules rather than with network configuration.

Source — source/examples/ch01.cpp

```
#include "harbor/check.hpp"
#include "harbor/domain.hpp"
#include <iostream>
int main() {
    auto parsed=harbor::parse_request("job-17","250");
    auto* r=std::get_if<harbor::Request>(&parsed);
    harbor::check(r && harbor::evaluate(*r)==62500,"domain result");
    for (auto bad : {"-1","2 trailing","1000001",""})
        harbor::check(std::holds_alternative<harbor::Error>(
            harbor::parse_request("job-17",bad)), "invalid input escaped boundary");
    std::cout<<"A transport-independent contract rejects invalid work.\n";
}
```

Run `./build/debug/ch01`. The successful output means the named valid request produced 62,500 and the listed invalid inputs were rejected. The implementation's integer bound also keeps the square within `std::int64_t`: one million multiplied by one million is one trillion, far below the signed 64-bit maximum. This is a derivation from the chosen input contract, not permission to multiply arbitrary signed integers without checking overflow.

Trace the state transition

`parse_request` consumes the complete numeric token using `from_chars`. Merely parsing an initial prefix would allow 250garbage to masquerade as 250. The end pointer must equal the end of the input. The function then returns either a domain request or an error value. Infrastructure adapters can translate that error into a wire response without exposing parser implementation details.

No durable acceptance occurs here. Constructing a valid request only establishes eligibility for submission. The repository decides whether the key conflicts, whether backlog capacity exists, and whether the intent commits. Keeping these stages separate prevents a transport adapter from returning acceptance after validation but before persistence.

Failure clinic — The Ambiguous Success Message

Suppose the gateway prints “job accepted” immediately after pushing into a local `std::queue`. It then crashes before writing the database. The client's success observation and the durable state disagree. Adding a retry to the dispatcher cannot fix the missing job because the dispatcher no longer knows it existed.

The repair is not simply “use a faster database.” Move the externally meaningful acknowledgment after the transaction that records intent. A client may still lose that acknowledgment, so preserve the request key and define replay behavior. This changes the ambiguity from possible lost accepted work into a recoverable question about whether the client has observed an already recorded outcome.

Engineering case — Turn a promise into an acceptance matrix

HarborWorks proposes three client messages: “received,” “accepted,” and “completed.” Before implementation, the team assigns each message an observable prerequisite. Received means a complete valid frame has reached the handler. Accepted means the gateway's transaction has committed a specific key and canonical input. Completed means the gateway has committed a result that matches the worker response and the domain check.

Now examine an operation with key `review1` and input 4. A parser can receive it and then reject it because pending capacity is exhausted. Received therefore does not imply accepted. The gateway can accept it while the worker is offline. Accepted therefore does not imply completed. The worker can complete its effect while the gateway has not recorded the response. Worker completion therefore does not immediately imply gateway completion.

This matrix suggests tests that ordinary function coverage would miss:

Observation	State that must exist	State not yet implied
Valid request parsed	Bounded canonical request object	Durable obligation
ACCEPTED review1 sent	Matching committed gateway row	Worker receipt
Worker RESULT created	Matching committed receipt	Client observation
DONE review1 16 returned	Matching committed gateway result	Global replicated availability

The table also prevents an API designer from assigning success solely according to a function’s return type. A C++ `bool` named `ok` cannot carry these distinctions without surrounding context. A domain result type should identify the operation stage, while transport status should remain separate.

A client that disconnects after sending the frame may have observed none of these responses. Its uncertainty does not erase whichever committed state exists. The recovery action is to retain `review1`, query status, or repeat the same canonical submission. A new key represents a different intent and must not be generated merely to make the user interface stop displaying an error.

For a design review, ask another engineer to derive the matrix from the public interface alone. If they cannot tell whether acceptance is durable, the contract is underspecified. If they assume completion because the response uses the word “success,” change the vocabulary before adding more implementation detail.

This exercise is transferable to file uploads, provisioning requests, and report generation. The exact stages differ, but each externally visible promise should have one concrete prerequisite and a statement of what it does not imply.

Practice and implementation lab

Add a maximum key length test and a test containing an embedded null byte. Then write a one-page contract for a different calculation, such as the sum of a bounded vector. State the element limit, numeric range, overflow policy, request identity rule, and meaning of acceptance. Do not choose a queue or database until those properties are written.

Solution reasoning. A null byte inside a key must be rejected by this grammar even though `std::string` can store it. A 65-byte key must fail before storage. For a vector sum, either establish a bound that proves the accumulator cannot overflow or implement checked addition with an explicit error outcome. Using a wider integer without deriving its limit does not complete the argument.

Chapter takeaway. Architecture becomes testable when success, ownership, identity, and uncertainty are specified as observable contracts.

Chapter 2 — Evolving Modular Applications into Independent Services

A team notices that several components are difficult to change together and proposes making each one a service. The proposal may create independent deployment, but it may also turn every former function call into a synchronous network dependency. The result can be a distributed monolith: separate processes that still require coordinated releases and fail together.

HarborDispatch needs boundaries that remain useful both inside a process and across a network. This chapter develops those boundaries without treating process count as a measure of architectural quality.

2.1 Designing Modules Around Cohesive Responsibilities

2.1.1 Separating Domain Decisions from Transport and Storage

The domain layer decides whether work is valid and how a valid request is interpreted. A transport adapter turns bytes into a request and turns a result into bytes. A storage adapter maintains records according to persistence and concurrency rules. Mixing these responsibilities makes a change in one mechanism leak into every test.

For example, a function that parses a socket buffer, issues SQL, computes the result, prints a log message, and terminates the process on failure has several independent reasons to change. Testing its arithmetic requires constructing a connection and a database even though neither is mathematically relevant. A narrower design lets a unit test call the domain function directly and lets integration tests exercise the adapters separately.

The boundary does not require a large inheritance hierarchy. A free function taking values can express a domain operation well. An interface is useful where behavior must vary independently, such as replacing a repository in a test or switching transports. Avoid adding virtual dispatch merely because a design diagram has an arrow.

Use dependencies in a deliberate direction. The domain should not need to include the network library to represent a request. The adapter may depend on the domain's types. This makes reuse possible without allowing infrastructure to define the business vocabulary accidentally.

2.1.2 Grouping Changes That Belong Together

Cohesion is about the relationship among responsibilities, not file size alone. Validation of a job's input and its canonical representation often belong together because changing one affects the other's correctness. A terminal formatter and a durable transaction usually do not belong together merely because both are short functions.

Ask what changes together during a feature. If adding a new job state always requires edits to five services, the boundary may have split one invariant across too many owners. If changing a metrics backend forces a change to the request domain, an implementation dependency may have become part of the public contract unnecessarily.

A useful module exports a small set of operations and hides details that callers do not need to coordinate. Its tests should describe those operations, not inspect private data structures. A test that reaches into a private queue layout can make a harmless implementation change appear to violate a public contract.

Cohesion is not an argument for unlimited growth. A module that handles unrelated tenants, data types, and lifecycle policies may need to divide. The important question is where independent invariants and independent change ownership already exist, rather than how many lines a style rule permits.

NEW SECTION EXAMPLE

Keep domain validation independent of transport text

Parse a request at the adapter boundary, then call a domain function that accepts an integer rather than socket or database objects.

main.cpp

```
#include <charconv>
#include <string_view>
#include <stdexcept>
#include <iostream>
#include <cstdint>

int main() {
    auto square=[](std::int64_t x){if(x<0||x>1000000)throw std::invalid_argument("domain
range");return x*x;};
    auto handle=[&](std::string_view text){std::int64_t
x=0;auto[p,e]=std::from_chars(text.data(),text.data()+text.size(),x);
    if(e!=std::errc{}||p!=text.data()+text.size())throw
std::invalid_argument("syntax");return square(x);};
    std::cout<<"result="<<handle("12")<<"\n";
    try{(void)handle("12x");return 1;}catch(const
std::invalid_argument&e){std::cout<<e.what()<<"\n";}
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

handle owns syntax and requires complete consumption. square owns the business range and overflow-safe bound. Neither function knows how a connection is accepted or how a result is committed.

A CLI, HTTP adapter, and test can reuse the same domain rule. Moving range checks into each adapter would create multiple authorities that can drift during future changes.

Expected behavior or output

```
result=144
syntax
```

Concrete experiment

Call square(-1) directly and verify domain rejection. Bypassing text parsing must not bypass the domain invariant.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

2.2 Distinguishing Logical and Deployment Boundaries

2.2.1 Understanding the Modular Monolith

A modular monolith is one deployment unit with meaningful internal boundaries. It can contain separate domain libraries, transport adapters, persistence components, and tests without requiring those components to communicate over the network. Calls can use ordinary values and exceptions under one process's lifetime model.

The main advantage for early design is reversibility. A module boundary can be adjusted while a function call remains local. Once many independently deployed clients depend on a wire protocol, changing it requires compatibility across a rollout window. Starting locally can therefore reduce the cost of discovering the right abstraction.

The design still needs discipline. Global mutable state, shared implementation headers, and unrestricted access to every table can erase the boundaries despite a neat directory tree. CMake targets can make include visibility and dependencies explicit, but they cannot by themselves enforce semantic ownership of data. Reviews and tests must check both.

A single process is not automatically a single failure concern. Different thread pools, queues, and resource budgets can provide partial isolation within it. Conversely, several containers on the same overloaded host can share a failure domain. Treat deployment as a concrete resource arrangement, not an architectural virtue assigned by name.

2.2.2 Establishing When a Remote Boundary Is Justified

A remote boundary becomes useful when its benefit outweighs the new coordination cost. An expensive worker may need a different hardware pool. A sensitive data owner may need a separate access boundary. Two teams may require independent release schedules. A service may need to continue accepting durable intent while a downstream worker is restarted.

Each case creates a testable rationale. “The worker can restart without losing acknowledged gateway requests” can be exercised. “The system uses microservices” does not identify a desired outcome. Write the failure and rollout scenarios that the separation is supposed to improve.

Remote boundaries also constrain interfaces. A pointer cannot be transferred as a meaningful cross-process object identity. A reference to a caller’s vector cannot remain valid on another host. Errors can occur after the remote operation has committed, and the caller may not know which outcome occurred. These differences should be visible in the API rather than hidden behind a stub that suggests ordinary local-call semantics.

When extracting a module, first establish stable request and result types, explicit ownership, and tests independent of the original process. Then introduce the transport adapter. The extraction is safer when the network is the last new mechanism rather than the first method used to discover the contract.

NEW SECTION EXAMPLE

Test a module through local and serialized adapters

Use one request contract through two deployment-shaped adapters while keeping transport failures distinct.

main.cpp

```
#include <functional>
#include <string>
#include <iostream>
#include <stdexcept>

int main() {
    using Service=std::function<int(int)>;
    Service local=[](int x){return x*x;};
    Service serialized=[&](int x){std::string wire=std::to_string(x);return
local(std::stoi(wire));};
    for(const auto* service:{&local,&serialized})if((*service)(9)!=81)return 1;
    std::cout<<"local and serialized adapters preserve81\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```


Explanation

Both adapters implement the same logical square operation. The second performs an explicit encoding/decoding boundary, showing a cost and representation obligation that an in-process call does not have.

No network is used here. Turning this adapter into a remote service also introduces deadlines, partial failure, authentication, and deployment ownership; serialization equality alone cannot justify that move.

Expected behavior or output

local and serialized adapters preserve81

Concrete experiment

Inject a decoding error into the serialized adapter and give it a structured failure outcome. Keep the local domain function unchanged.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

2.3 Managing Coupling Across Data and Releases

2.3.1 Avoiding Shared-Database Service Boundaries

Two services that write each other's tables are often coupled more tightly than their network diagram suggests. A schema change by one owner can break another's queries. A transaction from either process can hold locks needed by the other. A repair script can bypass domain validation that only exists in one service's code.

For HarborDispatch, the gateway and worker have separate database files. The gateway sends a command; it does not insert a receipt directly into the worker's table. This intentionally creates an acknowledgment gap that the protocol must handle. Shared access might appear simpler, but it would teach the wrong authority model for independent services.

Sometimes a shared database is a deliberate transitional design. Document it as such. Identify table ownership, permitted readers and writers, migration coordination, and the conditions for removing the shared dependency. Pretending the database is private while several services bypass the owner is worse than acknowledging the coupling honestly.

Reporting queries deserve particular attention. A read-only join across tables may still lock in another service's internal schema. An exported event, snapshot, or reporting view can create an explicit contract instead. That contract has freshness and recovery costs of its own, which later chapters examine.

2.3.2 Designing a Reversible Migration Path

A migration should identify the point at which returning to the old design becomes difficult. Adding an adapter is usually reversible. Changing every client's request format is less so. Deleting the original data after a migration can make rollback impossible without a restore. Name these transitions in the plan.

A common sequence is to establish a stable internal API, build the remote implementation behind it, validate both using the same contract tests, and then move a bounded subset of traffic. If shadow execution is used, ensure the shadow path cannot duplicate side effects. Comparing read-only calculations is different from sending two charge commands.

Do not confuse a feature flag with a complete rollback. If the new service writes a schema the old application cannot read, flipping routing back does not restore compatibility. The rollback needs data and protocol compatibility for the entire window in which both versions may run.

Define success in terms of user-visible behavior, not just a successful deployment command. A service extraction succeeds when the new boundary preserves the invariant, satisfies the workload, and can be operated by its owner. Extra network hops and dashboards are not proof of improvement.

NEW SECTION EXAMPLE

Migrate reads through a compatibility bridge before switching authority

Keep old and new representations side by side long enough to compare results rather than blindly dual-writing two authorities.

main.cpp

```
#include <map>
#include <string>
#include <iostream>

int main() {
    std::map<int,int> old{{1,100},{2,250}};
    std::map<int,std::string> newer;
    for(auto[id,cents]:old)newer[id]=std::to_string(cents);
    bool equivalent=true;for(auto[id,cents]:old)equivalent &= std::stoi(newer.at(id))==cents;
    if(!equivalent)return 1;
    bool use_new=true;
    auto read=[&](int id){return use_new?std::stoi(newer.at(id)):old.at(id)};
    std::cout<<"new-read="<<read(2);use_new=false;std::cout<<" rollback-read="<<read(2)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The backfill derives the new view from the old owner and validates every transferred value before switching reads. The boolean controls one reversible read boundary rather than requiring all callers to understand both formats.

This read-only fixture omits concurrent writes deliberately. A live migration needs a change-capture/fencing protocol; writing independently to both maps would not make them transactional across failures.

Expected behavior or output

```
new-read=250 rollback-read=250
```

Concrete experiment

Corrupt newer[2] before validation. The cutover must be blocked rather than letting the flag hide inconsistent data.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Substitute a Repository Without Rewriting Policy

Requirement and selected design

The application must submit a validated request through a repository abstraction without including database-specific headers in the policy function. The demonstration uses a memory repository so that the dependency direction is easy to inspect. It intentionally does not claim durable acceptance or conflict handling; those are added by the actual store and tested later.

Source — source/examples/ch02.cpp

```

#include "harbor/check.hpp"
#include "harbor/domain.hpp"
#include <iostream>
#include <map>
struct Repository {
    virtual ~Repository()=default;
    virtual void put(harbor::Request)=0;
    virtual std::size_t size() const=0;
};
class MemoryRepository final : public Repository {
    std::map<std::string,harbor::Request> requests_;
public:
    void put(harbor::Request r) override { requests_.insert_or_assign(r.key,r); }
    std::size_t size() const override { return requests_.size(); }
};
void submit(Repository& repo, const harbor::Request& request) {
    (void)harbor::evaluate(request); repo.put(request);
}
int main() {
    MemoryRepository repository;
    submit(repository,{"a",3});
    harbor::check(repository.size()==1,"application policy");
    std::cout<<"Boundary is an interface, not necessarily a network hop.\n";
}

```

submit depends on the Repository operation rather than the concrete map. MemoryRepository owns its container. The caller passes a request by const reference, and the repository takes an owned value. This makes the lifetime transfer visible: the stored object cannot depend on a temporary request remaining alive after submit returns.

The use of insert_or_assign is acceptable only for this narrow substitution demonstration. It would be wrong for HarborDispatch's stable-key contract if used without comparing the previous input. A real adapter must not claim the full domain contract merely because it implements a method with the same name. Interface conformance is syntactic; behavioral substitutability needs tests.

Evidence and interpretation

Run ./build/debug/ch02 and verify that the repository contains one request. Then replace the memory implementation with a recording fake that captures the argument but performs no persistence. The application policy should compile unchanged. This establishes that the dependency is replaceable at the source level.

It does not establish ABI compatibility, network transparency, or transaction equivalence. A database adapter can fail while acquiring a lock or committing. A remote adapter can return an ambiguous result. The abstraction should expose those outcomes when they become part of the contract rather than forcing every implementation into a misleading void return forever.

Failure clinic — The Distributed Monolith

Imagine splitting validation, job acceptance, computation, formatting, and status lookup into five services. Every submission synchronously contacts the next service, and all services require one coordinated release because they share a request structure by convention. A failure in the formatter prevents job acceptance, even though formatting is not necessary to preserve the job.

The design has paid for remote communication without gaining independent responsibility. To repair it, identify the invariant boundary. Validation and durable acceptance can remain together. Computation can be asynchronous after acceptance. Status formatting can operate from the stored state without participating in the acceptance transaction. Independent release follows from a versioned contract and separately owned data, not from assigning a port number to each function.

A shared C++ “common” library can recreate the same coupling. When every service consumes its newest headers and changes together, the library is effectively a global release boundary. Prefer stable, narrow schemas and explicit compatibility windows. Share mechanisms where appropriate, but do not use a shared build as a substitute for a protocol contract.

Engineering case — Extract one module without exporting its internals

Assume a monolithic report application contains parsing, validation, persistence, computation, and presentation. A proposed service extraction moves the computation into another process. The first draft of the interface sends a serialized copy of an internal C++ object containing a database row handle, a vector, and a pointer to configuration.

The interface is not actually a service contract. The row handle belongs to one database session, the pointer belongs to one address space, and the vector’s in-memory layout belongs to one implementation. The extraction must instead define durable input values: operation version, key, bounded integer, and any required domain options.

The next question is failure ownership. In the monolith, a thrown exception might unwind one call stack. Across the process boundary, the caller can time out while the callee continues. The adapter needs a transport result and a domain result, and the application needs a decision for uncertain completion. Those requirements are architectural changes introduced by extraction, not accidental complexity to hide in a helper function.

Draw the dependency direction before writing the network adapter. Domain validation should remain usable without opening a socket. The store should not depend on terminal formatting. The adapter should translate wire values into the same domain request that local tests use. This allows the monolithic path and the service path to be compared against the same semantics during migration.

A staged extraction can retain the local implementation behind an interface while the remote adapter is tested. For pure computations, shadow execution can compare results without changing authoritative state. For mutations, shadowing must not perform a second external effect. A safe shadow path may validate or simulate rather than execute.

Measure the new path's complete cost. Even a fast remote computation adds serialization, queueing, connection behavior, and failure recovery. A benchmark of the computation alone cannot justify extraction for latency. Other benefits, such as independent deployment or isolation, may still justify it, but they should be named honestly.

The decision record should end with a rollback condition. If the remote adapter fails, can the local path resume without losing accepted identities or interpreting new persisted records differently? A fallback is useful only if it preserves the same authority and result contract. Starting both mutation paths independently is not a safe fallback simply because one is local.

Practice and implementation lab

Write a repository contract with outcomes for accepted, duplicate, conflict, and unavailable. Implement a memory adapter that enforces the first three outcomes. Keep the application policy independent of the container type. Then describe how the unavailable outcome changes when the adapter becomes remote: which failures prove rejection, and which leave the caller uncertain?

Solution reasoning. A map lookup followed by payload comparison can distinguish replay from conflict inside a single-threaded model. Concurrent access requires synchronization or a transactional uniqueness mechanism. A local allocation failure before insertion establishes that the insertion did not occur in this implementation. A remote timeout does not establish the same fact. The revised interface should not label both as “not inserted” without evidence.

As a design exercise, propose a service extraction for a system you understand using only fictional or public details. State one benefit, two new failure modes, the data owner, the compatibility window, and the rollback boundary. If the benefit cannot be expressed as a changed operational or development outcome, retain the module locally until the requirement becomes clearer.

Chapter takeaway. A good service boundary begins as a good responsibility boundary; network separation alone does not create one.

Chapter 3 — Designing Stable C++ APIs and Compatibility Boundaries

An internal helper becomes a library. Another team links it into a service. A month later, a harmless-looking member change requires every consumer to rebuild, while an asynchronous callback occasionally reads a string that no longer exists. Neither problem is solved by documenting only the function names. A usable API must define ownership, lifetime, errors, and compatibility as carefully as it defines parameters.

This chapter treats an API as a long-lived agreement between separately changing pieces of software. Source compatibility, binary compatibility, wire compatibility, and behavioral compatibility are distinct agreements.

3.1 Making Ownership and Lifetime Visible

3.1.1 Distinguishing Owned Values from Borrowed Views

An owned value carries responsibility for its storage. A borrowed view identifies storage owned elsewhere. `std::string` commonly expresses ownership; `std::string_view` expresses a nonowning view. Replacing a string with a view can remove a copy, but it can also move a lifetime obligation onto the caller. That is an API change even when ordinary examples continue to work.

A synchronous parser can accept a view and finish before the caller’s buffer disappears. A queued operation cannot retain that same view safely unless the caller guarantees the storage outlives completion. For `HarborDispatch`, validated requests own their keys because the request can outlive the network read buffer. The small copy is justified by a concrete lifetime boundary rather than rejected automatically as overhead.

State whether an operation retains arguments. “The function reads the buffer before returning” and “the function may read the buffer until its completion callback runs” require different caller behavior. A `const` reference prevents mutation through that reference; it does not extend the object’s lifetime or make the underlying object safe against concurrent mutation elsewhere.

Use types to make likely misuse harder. An immutable request value is easier to move between queues than a bundle of raw pointers. A move-only connection handle communicates exclusive ownership. The Core Guidelines discuss these distinctions in their resource and interface rules; the asynchronous retention policy remains the responsibility of the particular API. (R01)

3.1.2 Defining Valid States After Move and Shutdown

A moved-from object still exists. Its permitted operations depend on its contract. Many standard-library types promise a valid but unspecified state after a move, but that does not imply every user-defined type should allow every observer. A library should specify which operations remain meaningful rather than relying on callers to guess from an implementation detail.

The `ClientOptions` example in this chapter allows destruction and reassignment after a move. Calling `endpoint()` on the empty moved-from object throws a documented logic error. Another sensible design could return an empty endpoint. The important point is that the implementation and its tests agree on the chosen policy.

Shutdown creates similar state questions. After `close()`, may a client submit more work? May existing operations still complete? Can `close()` be called twice? Does destruction wait for callbacks? These questions belong in the public contract because they affect the caller's own lifetime ordering.

Avoid interfaces that leave an object half-usable without naming the state. A transport that has stopped reading but still accepts writes may be valid for a protocol half-close, but the distinction must be explicit. An object with an accidental null implementation pointer is not automatically a meaningful "disconnected" state. Model states intentionally and test transitions among them.

NEW SECTION EXAMPLE

Move an owner while preserving an explicit empty state

Expose owned payloads as short-lived borrowed views and define operations on a moved-from owner.

main.cpp

```
#include <memory>
#include <string>
#include <string_view>
#include <iostream>
#include <utility>

int main() {
    struct Payload{std::unique_ptr<std::string> data;
        explicit Payload(std::string s):data(std::make_unique<std::string>(std::move(s))){}
        std::string_view view()const{return
data?std::string_view(*data):std::string_view{}};};
    Payload a("job7");Payload b(std::move(a));
    std::cout<<"source-empty="<<std::boolalpha<<a.view().empty()<<"
destination="<<b.view()<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

`unique_ptr` makes payload ownership exclusive and transfers it during the implicit move. `view()` explicitly returns an empty view for a moved-from owner instead of dereferencing a null pointer.

A returned `string_view` borrows the string; it does not prolong lifetime. Store an owned string when asynchronous work must outlive the payload. Moving the owner here does not copy the underlying text.

Expected behavior or output

```
source-empty=true destination=job7
```

Concrete experiment

Create a view from `b`, then copy it into `std::string` before destroying `b`. Verify the owned copy survives; do not dereference the old view after destruction.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

3.2 Designing Error and Completion Semantics

3.2.1 Separating Rejection, Failure, and Uncertainty

A rejected request was not accepted because a known rule failed. A failed operation encountered a problem while attempting work. An uncertain outcome means the observer cannot determine whether the intended state transition occurred.

Collapsing these into one Boolean forces callers to invent retry behavior without sufficient information.

For an in-process parser, an invalid value can be a structured error. For a repository, duplicate and conflict are normal domain outcomes rather than exceptional crashes. For an RPC, connection refusal before any request leaves the process may differ from a timeout after the peer committed. Even “before any request leaves” must be established by the transport implementation, not inferred from an arbitrary exception string.

A C++20 interface can use `std::variant<ResultValue, Error>` or a project-specific result type. C++23’s `std::expected` is another design option when the selected compiler and library support it, but this companion keeps the C++20 baseline. A result type helps represent alternatives; it does not define their meaning by itself.

Document retry classification separately from human-readable diagnostics. Stable error categories support policy. A diagnostic string supports investigation and may change between releases. Clients should not decide whether to retry by searching for words such as “busy” in an exception message. A later chapter develops bounded retry policy using explicit categories and remaining budget.

3.2.2 Specifying Completion and Cancellation

A completion callback is an event with lifetime consequences. Specify whether it occurs exactly once for each successfully initiated operation, on which execution context it runs, and whether it may run inline before the initiating call returns. These choices affect reentrancy and ownership even when the actual computation is trivial.

Cancellation is usually a request to stop future or ongoing work, not a rollback of completed side effects. A cancelled caller may no longer want a result, while the remote system still needs to reconcile a committed job. A useful interface distinguishes caller interest, operation execution, and durable state rather than assigning them one flag.

Consider an operation object held by a `shared_ptr`. Capturing it in a completion handler can keep the state alive, but capturing the handler inside the object can also create an ownership cycle. Conversely, capturing this without retaining the owner can produce a use-after-free during shutdown. The interface should establish who owns completion state and when that ownership ends.

Completion order is another contract. A stream may preserve message order while several independent requests complete out of order. An API that exposes futures in submission order does not necessarily serialize remote effects. State the ordering scope and avoid promising more than the implementation enforces.

NEW SECTION EXAMPLE

Represent rejection, failure, and uncertain completion separately

Use typed outcomes so a lost reply cannot be mistaken for a confirmed rejection.

main.cpp

```
#include <variant>
#include <string>
#include <iostream>

int main() {
    struct Rejected{std::string reason;}; struct Failed{std::string reason;};
    struct Unknown{std::string operation;}; struct Completed{int result;};
    using Outcome=std::variant<Rejected,Failed,Unknown,Completed>;
    Outcome result=Unknown{"op42"};
    if(auto* u=std::get_if<Unknown>(&result))std::cout<<"reconcile="<<u->operation<<"
do-not-create-new-id\n";
    result=Completed{144};std::cout<<"confirmed="<<std::get<Completed>(result).result<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Unknown carries the stable operation identity needed to query or retry the same logical operation. Returning generic false after a timeout would erase the distinction between “never accepted” and “may have committed.”

The later Completed value represents reconciliation, not a rollback of the external action. A production API also needs documented cancellation semantics: asking to stop does not prove that no effect occurred.

Expected behavior or output

```
reconcile=op42 do-not-create-new-id
confirmed=144
```

Concrete experiment

Add a handler for Rejected that reports its reason without retrying. Keep Unknown on the reconciliation path; these outcomes must not share an unconditional retry branch.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

3.3 Managing Source, Binary, Wire, and Behavioral Compatibility

3.3.1 Understanding What Pimpl Hides and What It Does Not

The pointer-to-implementation idiom, or Pimpl, keeps private data layout out of a public header. A class can hold a pointer to a forward-declared implementation and define its destructor in a source file where the implementation is complete. Changing private fields can then avoid changing the visible class layout.

This does not create a universal C++ ABI. Compiler choices, standard-library ABI, exception settings, runtime linkage, calling conventions, and exported symbols still matter. Exposing `std::string` across a boundary requires compatible expectations on both sides. Pimpl reduces one source of coupling; it does not erase the rest.

For a stable plugin or cross-language boundary, a narrow C ABI with opaque handles, explicit lengths, version queries, and designated allocation/free operations can be easier to control. That choice introduces its own manual error and lifetime rules. Do not export a C-looking function that returns an allocation the other side frees with an incompatible allocator.

Treat binary compatibility as a tested support matrix, not a property inferred from successful compilation. Build an old consumer against the supported public interface, run it against the new library in an isolated test, and inspect the symbols and runtime dependencies that matter for the chosen platform. A header-only change can still alter semantics even when no shared library exists.

3.3.2 Evolving Contracts Without Surprising Consumers

Source compatibility means existing source still compiles. Behavioral compatibility means it still observes the promised semantics. Adding an overload can preserve source syntax while changing overload resolution. Tightening a timeout can preserve types while breaking callers whose workloads previously fit. Returning results in a different order can break a client that incorrectly or correctly depended on the old ordering contract.

Wire compatibility is independent of the C++ ABI. A stable protobuf field number may survive several compiler versions while a shared-library interface does not. Conversely, two programs built with identical compilers can disagree on the meaning of a field. Protocol Buffers documents rules for evolving binary messages, but application meaning and rollout strategy remain separate decisions. (R14)

Use explicit versioned capability where an assumption affects correctness. A peer that does not support idempotency must not be treated as if it does because it accepts the same `Submit` method name. A schema can decode successfully while lacking a field needed to enforce the newer rule. Compatibility tests must therefore exercise behavior, not merely deserialization.

Deprecation should name the supported window, replacement operation, and evidence needed before removal. Record which deployed clients still use the old contract. Removing a method based only on current source searches can miss old binaries, batch jobs, or disaster-recovery images that will return later.

NEW SECTION EXAMPLE

Keep an old client object working across a private Pimpl change

Compile a public client once, then link it with two internal representations under the same local toolchain.

api.hpp

```
#pragma once
#include <memory>
class Count{public:Count();~Count();int value()const;private:struct Impl;std::unique_ptr<Impl>
p;};
```

client.cpp

```
#include "api.hpp"
#include <iostream>
int main(){Count c;std::cout<<c.value()<<'\\n';}
```

v1.cpp

```
#include "api.hpp"
struct Count::Impl{int
x=7;};Count::Count():p(std::make_unique<Impl>()){}Count::~Count()=default;int
Count::value()const{return p->x;}
```

v2.cpp

```
#include "api.hpp"
#include <vector>
struct Count::Impl{std::vector<int>
x{3,4};};Count::Count():p(std::make_unique<Impl>()){}Count::~Count()=default;int
Count::value()const{return p->x[0]+p->x[1];}
```

build.sh

```
set -eu
g++ -std=c++20 -c client.cpp -o client.o
g++ -std=c++20 client.o v1.cpp -o one
g++ -std=c++20 client.o v2.cpp -o two
test "$(/one)" = 7
test "$(/two)" = 7
printf 'unchanged client object: v1=7 v2=7\\n'
```

Run or inspect

```
sh build.sh
```

Explanation

Only the pointer-sized ownership representation is visible in the header; the `int` and `vector` internals are compiled in their respective implementation files. An out-of-line destructor sees the complete `Impl` type.

This demonstrates one controlled compatibility case, not a universal C++ ABI guarantee. Changing public virtual functions, compiler ABI, standard-library runtime,

ownership rules, or behavior can still break consumers despite Pimpl.

Expected behavior or output

unchanged client object: v1=7 v2=7

Concrete experiment

Change `v2.value()` to 8 without changing signatures. Linking still succeeds, but the behavioral contract test fails: binary link success is weaker than compatibility.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CPP20, ITANIUM-ABI. See the added references and revision index.

Worked example — Hide Representation While Preserving Lifetime Rules

Implementation

The demonstration defines `ClientOptions` with an incomplete `Impl`, exclusive ownership, out-of-line special members, and an explicit moved-from observer policy. The implementation is kept in one file for readability; a library would split the public declaration and private definition into separate files.

Source — `source/examples/ch03.cpp`

```
#include "harbor/check.hpp"
#include <memory>
#include <string>
#include <iostream>
class ClientOptions {
    struct Impl;
    std::unique_ptr<Impl> impl_;
public:
    explicit ClientOptions(std::string endpoint);
    ~ClientOptions();
    ClientOptions(ClientOptions&&) noexcept;
    ClientOptions& operator=(ClientOptions&&) noexcept;
    std::string endpoint() const;
};
struct ClientOptions::Impl { std::string endpoint; };
ClientOptions::ClientOptions(std::string e):impl_(std::make_unique<Impl>(Impl{std::move(e)}))
{}
ClientOptions::~~ClientOptions()=default;
ClientOptions::ClientOptions(ClientOptions&&) noexcept=default;
ClientOptions& ClientOptions::operator=(ClientOptions&&) noexcept=default;
std::string ClientOptions::endpoint() const {
    if (!impl_) throw std::logic_error("moved-from options");
    return impl_->endpoint;
}
int main() {
    ClientOptions a("loopback"); ClientOptions b(std::move(a));
    harbor::check(b.endpoint()=="loopback","pimpl move");
    harbor::rejects([&]{ (void)a.endpoint(); }, "moved-from contract");
    std::cout<<"Representation is hidden; ABI policy still needs a toolchain boundary.\n";
}
```

The destructor is defined after `Impl` is complete. The move operations transfer ownership instead of copying the implementation. The observer returns an owned string, so callers do not receive a view into an object that may soon be destroyed. That copy is appropriate for a small configuration value; a high-frequency API might choose another contract deliberately.

Evidence and interpretation

Run `./build/debug/ch03`. The test establishes that the destination retains the endpoint and that observing the moved-from object produces the specified error. It does not establish ABI compatibility across two compilers because both sides of this test are one compilation environment. The distinction is part of interpreting the evidence rather than a defect in the demonstration.

Extend the test by adding a private retry configuration to `Impl`. The public declaration should remain unchanged. Then add a new public observer and decide whether returning a reference is safe. Its lifetime would be tied to the `ClientOptions` object and could be invalidated by replacement. Returning a value avoids that dependency at a known cost.

Failure clinic — The View That Escaped the Call

A client method accepts `std::string_view` payload and queues a lambda that captures the view by value. The caller passes a temporary string. The lambda owns a copy of the view, not a copy of the string. Once the temporary is destroyed, the queued operation refers to invalid storage.

A superficial fix changes the capture to `[&]`, which makes the problem worse by also borrowing the view variable itself. The correct repair is to choose an ownership contract: copy or move the payload into operation state, or require a caller-owned buffer whose lifetime is explicitly tied to completion. The first is usually easier for a general-purpose submission API; the second can be appropriate for specialized zero-copy interfaces with strong usage constraints.

A sanitizer may detect the defect in a particular execution, but a test that happens to run the lambda immediately may not. The API review should identify retention before relying on timing-sensitive detection. Later asynchronous examples return to this same distinction using coroutine frames and completion handlers.

Engineering case — Review a plugin boundary separately from a library boundary

A team wants third parties to load plugins compiled independently from the host. Its initial interface returns `std::vector<std::string>` across a shared-library boundary and allows exceptions to escape. The team also uses `Pimpl` inside the host and assumes this makes the entire plugin interface ABI-stable.

`Pimpl` hides selected implementation details of a class. It does not automatically standardize the compiler ABI, standard-library representation, exception runtime, allocator, or ownership rules used by independently built plugins. The plugin

boundary requires a compatibility policy of its own.

One possible design uses a narrow C-compatible function table with an explicit version and size. Opaque handles identify resources owned by the provider, and paired creation/destruction functions ensure that the same component releases its allocations. Status codes and caller-supplied output buffers can avoid exceptions and container layouts crossing the boundary. This is an architectural option, not an implemented plugin system in the companion.

Even that design needs careful rules. Who owns a callback context? Can callbacks occur after a close function returns? Are calls thread-safe? May a plugin retain a borrowed input pointer? Does a returned byte sequence use UTF-8 or another defined encoding? An opaque pointer hides representation, but it does not answer lifetime questions.

Version negotiation should reject unsupported combinations before invoking arbitrary function pointers. A size field can support additive extension only when both sides obey the interpretation rules. Reading a field beyond the provided structure size is a memory defect, not a graceful compatibility fallback.

Write a boundary test plan with two independently compiled components. Include an old provider with a new host, a new provider with an old host where supported, callback shutdown, allocation release, invalid handles, and provider failure. A unit test that compiles both sides into one executable does not reproduce every shared-library condition.

For HarborDispatch, the service protocol is a separate compatibility boundary from the C++ source API. Network values are explicitly encoded, so they do not depend on vector layout. The cost is that the protocol must handle parsing, versioning, and partial failure. Different boundaries trade different kinds of coupling; architecture is the process of choosing and documenting them rather than declaring one abstraction universally free.

Practice and implementation lab

Design a `SubmitResult` type with accepted, duplicate, conflict, rejected, and uncertain outcomes. Write down which fields each outcome carries. An accepted response should include stable identity; a conflict should not overwrite the original request; an uncertain result should preserve enough information for lookup or safe replay.

Then revise an asynchronous interface that takes a borrowed buffer. Provide two variants: an owned convenience operation and a borrowed advanced operation. Specify completion context, cancellation behavior, and the last point at which the buffer may be read. Avoid calling the borrowed operation “zero cost”; its validation, lifetime coordination, and transport still have costs.

Solution reasoning. An uncertain result should not manufacture a new key for the caller. Doing so can turn a retry into a second logical operation. The owned variant should retain its own buffer until completion. The borrowed variant must establish a completion event even after cancellation so the caller knows when reuse is safe. A statement that “cancellation usually stops it quickly” does not provide that lifetime

guarantee.

Chapter takeaway. A stable API makes ownership, completion, uncertainty, and compatibility explicit enough that independent callers can act correctly.

Chapter 4 — Managing Configuration, Dependencies, and Service Lifecycle

A service starts successfully with one configuration, then fails after a reload changes the queue limit while existing requests still use the old worker count. Another instance reports ready before opening its database. A third hangs during shutdown because a consumer waits on a queue whose producer has already been destroyed. These failures arise from lifecycle design, not from an exotic distributed algorithm.

HarborDispatch keeps startup, request lifetime, reload policy, and shutdown visible. This chapter establishes those responsibilities before asynchronous work makes them harder to inspect.

4.1 Treating Configuration as Validated Input

4.1.1 Parsing Values with Units and Bounds

Configuration is an input language. It needs syntax, types, ranges, and error messages. A field named `timeout` is ambiguous unless its unit is defined. A queue capacity of zero may mean disabled, unlimited, or invalid in different libraries. The application must select one meaning rather than inherit an accidental default.

Parse into a candidate object, validate the entire object, and publish it only when all required relationships hold. A worker count between one and 64 and a queue capacity between one and 4,096 are example lab constraints, not universal tuning recommendations. Cross-field constraints can matter more than individual bounds: a maximum payload multiplied by queue capacity determines a memory budget that neither field expresses alone.

Avoid permissive numeric conversion that silently accepts trailing characters. `100ms` should either be a supported unit-bearing value or a rejected token, not an accidental parse of `100`. Likewise, distinguish an absent field from an explicitly empty value. Those differences can carry operational meaning in credentials, endpoints, and feature flags.

Configuration diagnostics must not expose secrets. Report the offending field and reason without printing a private key or complete connection string. A diagnostic that solves a parsing problem by copying credentials into a shared log has created a second security problem.

4.1.2 Establishing Precedence and Configuration Identity

Many applications combine defaults, files, environment variables, and command-line options. Define the precedence once and expose the effective nonsecret configuration for inspection. Otherwise an operator may edit a file that is silently overridden by an environment variable and conclude that the application ignored the change.

A useful configuration identity is a version or digest of the normalized nonsecret policy, plus separate identifiers for referenced secret versions. Do not calculate a public digest by indiscriminately concatenating all secret values into a diagnostic string. The goal is to correlate behavior with configuration, not to create a new credential archive.

Record startup arguments and effective policy in a structured, bounded form. A support engineer should be able to distinguish two instances running the same binary with different queue limits. Build identity and configuration identity answer different questions; both belong in an incident timeline.

Defaults should be safe for the stated role. The companion's transport binds loopback by construction rather than relying on an operator to remember an address option. A production wrapper would need explicit endpoint and security configuration, but changing the bind address alone is not the same as supplying that wrapper.

NEW SECTION EXAMPLE

Parse duration units and enforce configuration precedence

Use a file default overridden by a supplied environment-shaped value, rejecting strings without the required ms suffix.

main.cpp

```
#include <charconv>
#include <string_view>
#include <optional>
#include <iostream>
#include <stdexcept>

int main() {
    auto duration=[](std::string_view s){if(!s.ends_with("ms"))throw
std::invalid_argument("unit");s.remove_suffix(2);int n=0;
    auto[p,e]=std::from_chars(s.data(),s.data()+s.size(),n);if(e!=std::errc{}||p!=s.data()+s.size())||n<1||n>5000)throw std::invalid_argument("range");return n;};
    std::string_view file="500ms";std::optional<std::string_view> override="250ms";
    std::cout<<"timeout_ms="<<duration(override.value_or(file))<<" source=override\n";
    try{(void)duration("250");return 1;}catch(const
std::invalid_argument&){std::cout<<"missing unit rejected\n";}
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The parser removes only the requiredms suffix, then requires a complete integer in1...5000. Unit validation prevents seconds and milliseconds from being silently interchanged.

value_or makes precedence explicit. The example supplies the environment-shaped value directly so tests do not depend on the current process environment; a real adapter should record which source won.

Expected behavior or output

```
timeout_ms=250 source=override
missing unit rejected
```

Concrete experiment

Clear the override and expect500ms. Then use6000ms and verify rejection rather than clamping a mistyped value silently.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

4.2 Publishing Configuration Without Breaking In-Flight Work

4.2.1 Using Immutable Snapshots

An immutable configuration snapshot lets a request observe one coherent policy throughout its lifetime. The application builds and validates a new snapshot, then replaces the current reference. Existing requests retain the old snapshot until they finish. This avoids a request seeing a new timeout but an old retry count halfway through processing.

The chapter demonstration uses `std::shared_ptr<const Settings>` in one thread to make the lifetime visible. A multithreaded implementation must also synchronize publication, for example through an atomic shared-pointer operation or a mutex. `shared_ptr` reference counting does not make unsynchronized reads and writes of the same shared-pointer variable safe.

Snapshots do not solve every reload problem. A reduced connection limit may prevent new admission while existing connections drain. Replacing an endpoint may require new pools while old requests retain their original connections. Changing a database path may be too consequential to support as a live reload at all. Classify each field as startup-only, replaceable policy, or migration-requiring state.

Use a generation number to identify which snapshot a request used. During a rollout, a small population of old-generation requests may be expected. That observation is more informative than assuming every operation instantly adopted the latest configuration as soon as a file changed.

4.2.2 Rejecting Invalid Reloads Without Partial Mutation

A reload should have a commit point. Parse and validate a candidate outside the active configuration. If validation fails, preserve the previous snapshot and report the failure. Mutating active fields one by one can leave a mixed state when a later field is invalid.

The same principle applies to dependent resources. If a new endpoint requires a connection pool, construct and validate that pool before swapping it into service where practical. If construction fails, retain the old pool. If the resource cannot coexist with the old one, the operation is a disruptive transition and should be

designed as such rather than hidden inside a “reload” function.

Do not treat a successful parse as proof of operational validity. A certificate file can be syntactically readable but expired. An endpoint can be well formed but unreachable. Some checks belong at startup; others are ongoing dependency health. Separate those states so that a temporary remote failure does not corrupt an otherwise valid configuration record.

Record the attempted generation, result, and reason. Retrying a rejected configuration blindly at high frequency can flood logs while obscuring the fact that no policy change occurred. A bounded notification and a retained last-known-good state are usually more useful than repeated stack traces.

NEW SECTION EXAMPLE

Publish an immutable configuration snapshot atomically

Keep an in-flight request on its original settings while a validated replacement becomes visible to new requests.

main.cpp

```
#include <atomic>
#include <memory>
#include <iostream>

int main() {
    struct Config{int timeout_ms;unsigned generation;};
    std::atomic<std::shared_ptr<const Config>> current{std::make_shared<const
Config>(Config{500,1})};
    auto in_flight=current.load();
    auto reload=[&](int timeout){if(timeout<1||timeout>5000)return false;auto
old=current.load();
        current.store(std::make_shared<const
Config>(Config{timeout,old->generation+1}));return true;};
    bool bad=reload(-1);bool good=reload(250);auto fresh=current.load();
    std::cout<<std::boolalpha<<"invalid="<<bad<<" valid="<<good<<"
old="<<in_flight->timeout_ms<<" new="<<fresh->timeout_ms<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Validation completes before publication. A rejected reload never changes `current`. Existing requests retain a `shared_ptr` to an immutable `Config`, so a reload cannot mutate settings halfway through their work.

Atomic `shared_ptr` makes publication safe, not necessarily lock-free. This fixture assumes one configuration writer; concurrent writers need generation/CAS coordination to avoid overwriting each other’s accepted revisions.

Expected behavior or output

```
invalid=false valid=true old=500 new=250
```

Concrete experiment

Retain two snapshots across two valid reloads and print all generations. Old snapshots must remain unchanged until their owners release them.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

4.3 Coordinating Startup and Shutdown

4.3.1 Starting Dependencies Before Advertising Readiness

Startup should follow dependency order. Construct validated configuration, open required stores, verify supported schema versions, initialize execution resources, bind the listener, and only then advertise the appropriate readiness state. The exact sequence depends on the service, but readiness must correspond to a defined useful capability.

The gateway opens its API and dispatcher database connections before starting the dispatcher thread and printing its listening port. This reduces the chance that a “ready” process immediately discovers an unusable database. The printed READY line is a lab synchronization signal, not a production readiness endpoint or proof that the remote worker is available.

A gateway that can durably accept work during a worker outage may reasonably report submission readiness while reporting delivery degradation separately. A service that cannot safely accept any work without the dependency should not report that same readiness. Readiness is a contract for a particular operation, not a universal property inferred from process existence.

Partial startup must unwind safely. RAII releases resources that were successfully acquired. A thread that starts before a later constructor fails must be stopped and joined before captured state is destroyed. Resource declaration order and exception paths deserve the same review as the happy path.

4.3.2 Stopping Admission, Draining, and Releasing Resources

A useful shutdown sequence first stops new admission, then decides what happens to already accepted work, then waits or cancels within a bound, and finally releases resources in dependency order. “Destroy everything” is not a sequence because callbacks and threads may still refer to the objects being destroyed.

The teaching server stops accepting new connections and lets existing connections proceed until their individual deadlines. The gateway then requests dispatcher stop and joins its thread. A request whose durable intent committed can remain pending across shutdown; it does not need to finish its remote calculation before the gateway exits. This is one benefit of separating durable acceptance from completion.

A stop token is cooperative. It does not forcibly interrupt arbitrary blocking code. Every blocking operation in the shutdown path needs a strategy: a deadline, a wakeup mechanism, a close operation with defined semantics, or a documented

external termination policy. Joining a thread that waits forever on an unclosed queue can make graceful shutdown impossible.

Signal handlers are especially constrained. The companion's handler only stores to an atomic stop flag whose lock-free property is checked at compile time. It does not allocate, log through iostreams, lock a mutex, or close random application objects. The ordinary control flow observes the flag and performs the actual shutdown. This keeps asynchronous signal context separate from normal resource management. A relaxed atomic is sufficient for this notification because it publishes no other data; it also permits safe observation when a signal is delivered on a different thread. The handler relies on the specific allowance for plain lock-free atomic operations, not on a claim that every atomic or library call is signal-safe. (R48)

NEW SECTION EXAMPLE

Make readiness depend on initialization and drain ownership

Show an explicit lifecycle that stops admission before draining accepted work.

main.cpp

```
#include <deque>
#include <iostream>

int main() {
    enum class State{starting,ready,draining,stopped};State
    state=State::starting;std::deque<int> jobs;
    auto admit=[&](int id){if(state!=State::ready)return false;jobs.push_back(id);return
    true;};
    bool early=admit(1);bool storage_initialized=true;
    if(storage_initialized)state=State::ready;
    bool accepted=admit(2);state=State::draining;bool late=admit(3);
    unsigned
    completed=0;while(!jobs.empty()){jobs.pop_front();++completed;}state=State::stopped;
    std::cout<<std::boolalpha<<early<<' '<<accepted<<' '<<late<<" drained="<<completed<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The readiness transition occurs only after the dependency flag is established. Once draining begins, admission rejects new work but the queue remains owned until the accepted item finishes.

This deterministic model has no real sockets or threads. In an implementation, protect these transitions with one owner or synchronization and put a finite deadline on draining; a readiness flag alone does not close existing connections.

Expected behavior or output

```
false true false drained=1
```

Concrete experiment

Add a drain budget of zero and report the remaining job as aborted rather than completed. Preserve separate accounting for accepted, completed, and abandoned work.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Preserve a Last-Known-Good Snapshot

Implementation and expected behavior

The candidate validator rejects zero workers and out-of-range capacity before creating a new immutable snapshot. An existing request retains its snapshot while a newer one becomes current.

Source — source/examples/ch04.cpp

```
#include "harbor/check.hpp"
#include <memory>
#include <iostream>
struct Settings { unsigned workers; unsigned queue; };
std::shared_ptr<const Settings> validate(Settings candidate) {
    if (!candidate.workers || candidate.workers>64 || !candidate.queue ||
        candidate.queue>4096)
        throw std::invalid_argument("configuration bounds");
    return std::make_shared<const Settings>(candidate);
}
int main() {
    auto current=validate({2,16});
    auto request_snapshot=current;
    harbor::rejects([&]{ current=validate({0,16}); }, "invalid reload");
    harbor::check(current->workers==2, "failed reload changed config");
    current=validate({4,32});
    harbor::check(request_snapshot->workers==2 && current->workers==4, "snapshot lifetime");
    std::cout<<"Validated snapshot replaced; existing request retained its policy.\n";
}
```

Run `./build/debug/ch04`. A failed reload leaves the worker count at two. A successful reload changes the current count to four while the captured request snapshot still reports two. These observations establish coherent snapshot lifetime in the single-threaded demonstration.

To extend it to several threads, protect the shared current snapshot's publication. Readers should load one snapshot at request start and pass it through the operation rather than repeatedly loading changing policy at every step. If a particular setting must take effect immediately for safety, such as a revoked privilege, it may require a separate live authorization mechanism rather than ordinary request configuration caching.

Trace the exception guarantee

The assignment `current = validate(candidate)` does not run until validation returns a new pointer. If validation throws, the existing shared pointer remains unchanged. The operation therefore has a strong guarantee for the published configuration: either the new complete snapshot replaces the old one, or the old one remains.

That guarantee does not automatically extend to arbitrary side effects inside `validate`. If validation sends an external notification or mutates a shared pool before failing, those actions are not rolled back by the pointer assignment. Keep candidate construction isolated or explicitly account for the side effects.

Failure clinic — Joining the Consumer Before Closing the Queue

A service owns a queue, a producer, and a consumer. During shutdown it joins the producer, then joins the consumer, then closes the queue. If the consumer is waiting for another item, the second join never completes. The close operation that would wake it lies after the blocked join.

The fix is to define terminal queue state and place the close before waiting for consumers to finish. If accepted items must drain, `pop()` returns queued values after close and returns no value only once the queue is empty. If cancellation discards items, report that separate policy explicitly. Do not call both behaviors “close” without explaining which one applies.

Another subtle defect appears when a startup constructor launches a consumer before the producer exists. If producer creation fails, the consumer may wait forever during stack unwinding. A startup guard must close or cancel the queue before its thread owner joins. The same lifecycle dependency exists even before the service becomes ready.

Engineering case — Reload capacity without discarding accepted work

A running service has a queue limit of 100 and currently holds 80 accepted tasks. An operator publishes a new configuration with a limit of 40. A naive reload resizes the queue immediately and drops the last 40 tasks. The configuration parser accepted a valid integer, but the lifecycle policy violated accepted obligations.

Treat configuration publication and state migration separately. The new limit can govern future admission while existing work drains below it. During that interval, new tasks are rejected or deferred according to policy. The service should expose both the configured target and current occupancy so operators understand why admission remains restricted.

The same distinction applies when changing worker count. Reducing a pool should not destroy a worker while it owns a task. A controlled transition stops assigning new work to selected workers, lets them finish or cancel under a defined contract, and joins them before releasing their state. Raising the count can fail partway through

thread creation, so the previous valid configuration needs a safe fallback.

Configuration snapshots help keep one request internally consistent. If a request begins under generation 7, it can retain that immutable snapshot even after generation 8 is published. That does not mean every setting should be frozen forever: credential revocation or emergency admission shutdown may require an independently checked authority boundary. Classify settings by whether they are request-stable, dynamically enforced, or restart-only.

The chapter's `shared_ptr` example preserves object lifetime, not automatic concurrent publication safety. A production reload path must synchronize access to the shared pointer itself, for example through an appropriate atomic shared-pointer facility or a mutex. The pointee's immutability does not remove the data race on an unsynchronized pointer variable.

A reload test should therefore cover malformed configuration, valid-but-incompatible transitions, partial resource acquisition, concurrent readers, and rollback to the previous snapshot. The expected result is not simply "the new file was read." It is that the service either publishes a complete valid generation or continues with the previous valid one, while preserving already accepted work.

This reasoning explains why a configuration file is part of the operational API. Changing it can be as consequential as changing a function signature, and it deserves validation, versioning, and a migration policy.

Practice and implementation lab

Add a `request_timeout_ms` field to Settings and reject zero or excessively large values. Derive one cross-field memory bound using queue capacity and maximum payload size. Then create a reload function that returns a structured rejection without modifying the current snapshot.

Write a lifecycle table for the gateway with states starting, accepting, draining, and stopped. For each state, specify whether a new submission is permitted, whether existing status requests may complete, and which resources must still exist. Add a test that requests shutdown while one client has sent only half a frame; a second client should not be admitted indefinitely after draining begins.

Solution reasoning. In-flight requests should retain a coherent snapshot unless a documented live policy requires otherwise. A pending job can survive shutdown because its authoritative intent is in the database, not because a thread is kept alive forever. A half-frame connection must eventually reach its deadline or be cancelled, so that a malformed or stalled client cannot control the service's shutdown duration without bound.

Chapter takeaway. Configuration changes and lifecycle transitions need commit points, ownership, and completion rules just as database operations do.

Chapter 5 — Designing Asynchronous C++ Execution and Coroutine Lifetimes

A synchronous function's stack often makes lifetime look simple: the caller waits while the callee uses its arguments. An asynchronous operation separates initiation from completion. The initiating function may return while a buffer is still being read, a callback is queued, or a coroutine is suspended. The algorithm may be correct while the ownership around it is not.

This chapter explains asynchronous execution without treating a particular syntax as a complete runtime. A callback, future, or coroutine is a way to express control flow. Scheduling, cancellation, resource limits, and completion ownership still require design.

5.1 Separating Initiation from Completion

5.1.1 Understanding Operation State and Completion Handlers

An asynchronous operation needs state that survives initiation. That state may include a socket, a buffer, a deadline, a decoder position, a callback, and an error result. If any of these are local variables in a function that returns before completion, the operation must either own them elsewhere or retain a valid reference under a documented lifetime rule.

Think of initiation as creating a pending obligation. Completion settles that obligation with a result, error, or cancellation outcome. The caller needs to know whether initiation itself can fail before the obligation exists. Otherwise it may wait for a callback that will never arrive or destroy state while a callback remains possible.

A useful interface states whether a successfully initiated operation invokes its completion exactly once and whether completion may occur inline. Inline completion can reenter caller code before the initiating function finishes updating its own state. Deferring every handler can simplify some reasoning but adds queueing and does not automatically remove races.

Boost.Asio's asynchronous model distinguishes initiating functions and completion handlers, with execution governed by its associated executor machinery. Use the selected release's documented requirements rather than assuming every library follows the same handler lifetime and invocation rules. The book's small coroutine demonstration is independent of Asio and does not implement its execution model. (R03)

5.1.2 Choosing Execution Contexts and Serialization Rules

An executor determines where work is submitted; an execution context supplies machinery that runs it. These terms do not imply one thread per operation. A single event loop can interleave many waiting operations, while a thread pool can run ready handlers concurrently. The correct choice depends on work duration, blocking behavior, and ownership of shared state.

Serialize operations that mutate one protocol state machine unless that state machine is explicitly designed for concurrent access. A connection may require that only one handler updates its decoder or output offset at a time. That is different from requiring every connection in the process to run serially.

A strand in Boost.Asio provides a serialization guarantee for associated handlers; it does not necessarily pin them to one operating-system thread. Thread-local state and thread affinity therefore need separate reasoning. A component can be race-free under a strand while still being wrong if it assumes all calls occur on one specific thread. (R04)

Do not block an event-loop thread on work that belongs elsewhere. A database call, compression operation, or long CPU calculation can delay unrelated connections even though socket reads are asynchronous. Offloading it creates another queue and another lifetime boundary, so the design must also bound that queue and deliver completion back to the correct owner.

NEW SECTION EXAMPLE

Separate asynchronous initiation from result retrieval

Launch work with owned input, retain the future, and observe the completed result throughget.

main.cpp

```
#include <future>
#include <string>
#include <iostream>
#include <thread>

int main() {
    std::promise<std::size_t> promise; auto result=promise.get_future();
    std::jthread worker([input=std::string("owned
request"),p=std::move(promise)]()mutable{p.set_value(input.size());});
    std::cout<<"submitted\n";
    auto length=result.get();worker.join();std::cout<<"completed bytes="<<length<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The lambda captures both request text and promise byvalue/move, so neither borrows a caller's temporary. Initiation creates the worker and returns control before the result is requested.

`Future.get` establishes completion and can propagate a stored exception. The thread join separately ends execution ownership. No assumption is made about whether the worker actually ran before or after the submitted line.

Expected behavior or output

```
submitted
completed bytes=13
```

Concrete experiment

Use `set_exception` for a deliberate failure and catch it at `get`. Do not report completion merely because thread creation succeeded.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

5.2 Understanding What a Coroutine Stores

5.2.1 Distinguishing Suspension from Parallel Execution

A coroutine can suspend and later resume while preserving execution state. Suspension does not itself create a thread, schedule I/O, or guarantee parallel execution. The awaiter and surrounding runtime determine when and where resumption occurs. A program using `co_await` can still block, deadlock, or execute entirely on one thread.

The coroutine frame stores state needed across suspension, including relevant parameters and local variables. The promise object participates in constructing the return object, suspension policy, result handling, and exception behavior. These language mechanisms provide building blocks rather than a universal task type. The standard's coroutine rules define the transformation; library abstractions define the higher-level execution contract. (R02)

A useful mental model is a state machine whose program counter is retained between steps. At each suspension point, identify which resources remain live and which external event is allowed to resume the operation. If two independent actors can resume or destroy the same frame without coordination, the syntax has not prevented a lifetime race.

Do not infer performance from source brevity. Coroutines can remove callback nesting and permit efficient state management, but frame allocation, scheduler overhead, and retained resources depend on the implementation. Measure the actual workload after the ownership and completion contract is correct.

5.2.2 Preserving Owned Data Across Suspension

Passing an object by value into a coroutine can place an owned copy or moved value in the frame. Passing a reference stores a reference-like relationship; it does not extend the referred object's lifetime. A coroutine parameter of `std::string_view` still borrows storage even though the coroutine itself remains alive.

This distinction catches a common defect. A helper receives a temporary string by const reference, returns a suspended coroutine, and the temporary dies before the coroutine resumes. The frame exists, but the referenced string does not. Moving an owned string into the operation is one straightforward repair when the payload size and API permit it.

Capturing coroutine lambdas need additional care because the lambda closure's lifetime and the coroutine frame's lifetime are not automatically the same. A temporary closure can disappear while suspended code still relies on captured members. A named coroutine function with explicit owned parameters often makes the lifetime easier to inspect.

The chapter's `StepTask` owns a string parameter and exposes manual stepping. Its return object exclusively owns the coroutine handle and destroys the frame. The task is not copyable, preventing two ordinary owners from both destroying the frame. Its move constructor clears the source handle. These choices make ownership visible without pretending the example is a production asynchronous task library.

NEW SECTION EXAMPLE

Keep an owned string in a suspended coroutine frame

Construct a coroutine whose by-value parameter survives suspension and whose frame is explicitly destroyed.

main.cpp

```

#include <coroutine>
#include <string>
#include <iostream>
#include <utility>
#include <exception>

struct Task{
    struct promise_type{
        Task get_return_object(){return
Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        std::suspend_always initial_suspend()noexcept{return{}};
        std::suspend_always final_suspend()noexcept{return{}};
        void return_void(){}void unhandled_exception(){std::terminate();}
    };
    std::coroutine_handle<promise_type> h;
    explicit Task(std::coroutine_handle<promise_type>x):h(x){}
    Task(const Task&)=delete;Task&operator=(const Task&)=delete;
    Task(Task&&x)noexcept:h(std::exchange(x.h,{})){}
    ~Task(){if(h)h.destroy();}
    void resume(){if(h&&!h.done())h.resume();}
};
Task make_task(std::string text){std::cout<<"body owns "<<text<<'\n';co_await
std::suspend_always{};std::cout<<"resumed with "<<text<<'\n';}

int main() {
    auto task=make_task(std::string("retained"));
    std::cout<<"created without executing body\n";
    task.resume();
    std::cout<<"first resume reached suspension\n";
    task.resume();
}

```

Run or inspect

```

g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example

```

Explanation

`initial_suspend` prevents the body from running during construction. The string parameter is stored in the coroutine frame, unlike a reference to a caller-owned temporary. The `firstresume` prints it and suspends at `co_await`.

The `secondresume` continues after suspension. A coroutine is not inherently another thread; both resumes execute on the caller here. `Task`'s destructor destroys the suspended/completed frame exactly once, and copying the handle owner is forbidden.

Expected behavior or output

```

created without executing body
body owns retained
first resume reached suspension
resumed with retained

```

Concrete experiment

Destroy the task after the `firstresume`. The frame must still clean up without executing the final print. Do not change the parameter to a reference to a temporary.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

5.3 Coordinating Cancellation and Structured Completion

5.3.1 Distinguishing Stop Requests from Resource Reclamation

A cancellation request says that continuing the operation is no longer desired under the relevant policy. It does not prove that execution has stopped at that instant. A handler may already be running, a kernel operation may be pending, or a remote service may have committed. Resource reclamation must wait for the point that the actual API defines as completion or safe cancellation.

For a borrowed send buffer, this is especially important. The caller cannot reuse the memory merely because it requested cancellation. It needs an event that establishes the transport will no longer read it. A correct cancellation API therefore participates in the same completion accounting as normal success and failure.

C++ stop tokens provide cooperative notification. A function that never checks the token or cannot interrupt its blocking dependency may not respond promptly. Do not assign a hard shutdown bound to `request_stop()` alone. Associate blocking operations with deadlines, wakeups, or documented interruption mechanisms.

Remote cancellation is also not compensation. If a worker has recorded an effect, ending the client's wait does not erase that record. The workflow may need a separate cancel command with its own stable identity and state transition. Chapters 8 and 21 distinguish request cancellation from business-level reversal.

5.3.2 Keeping Child Work Within an Owned Scope

Structured concurrency is the design goal that spawned work has an explicit owning scope and does not escape silently. Before the scope ends, its child operations are completed, cancelled and joined, or transferred to another named owner. This makes cleanup and error propagation easier to reason about than detached tasks with implicit lifetimes.

A `std::jthread` helps with a thread's ownership by requesting stop and joining on destruction, but it does not automatically make the thread body cooperative or prevent it from borrowing destroyed state. Declare resources and thread owners in an order that preserves captured dependencies through the join. Review exceptional startup paths as well as ordinary shutdown.

For request fan-out, collect child outcomes under one operation object. A parent's deadline should constrain the children. When one required child fails, decide whether the others are still useful, can be cancelled, or must finish to preserve consistency. Detaching them because the parent is returning can leak both work and observability.

A durable background job is not necessarily a structured-concurrency violation. It can intentionally outlive the initiating RPC because ownership transfers to a persisted job record and a dispatcher. The transfer must be explicit and acknowledged at the right point. The distinction is between named durable ownership and forgotten in-process

work.

NEW SECTION EXAMPLE

Request cancellation, then join before reclaiming captured state

Use cooperative stop signalling with a join to distinguish cancellation intent from completed cleanup.

main.cpp

```
#include <atomic>
#include <thread>
#include <iostream>

int main() {
    std::atomic<bool> entered=false;int owned_result=0;
    std::jthread child([&](std::stop_token
token){entered.store(true,std::memory_order_release);
    while(!token.stop_requested())std::this_thread::yield();owned_result=7;});
    while(!entered.load(std::memory_order_acquire))std::this_thread::yield();
    child.request_stop();child.join();
    std::cout<<"joined cleanup-result="<<owned_result<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

request_stop only changes a cooperative signal. The child observes it and writes its cleanup result before terminating. The parent reads the ordinary integer only afterjoin, which establishes the necessary completion ordering.

The entered flag avoids making the exercise depend on whether cancellation happens before the child starts. A real child blocked on I/O also needs a cancellation-aware wait or bounded timeout; polling a stop token cannot interrupt every operation.

Expected behavior or output

```
joined cleanup-result=7
```

Concrete experiment

Replace the polling loop with a stop-aware condition-variable wait. Keep the join as the lifetime boundary and demonstrate cancellation both before and after the child begins waiting.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Step Through an Owned Coroutine Frame

Requirement and implementation

The demonstration must preserve a temporary caller string across two suspensions and destroy the frame exactly once. Manual stepping is used so that execution order is deterministic. No network runtime or thread scheduler is implied.

Source — source/examples/ch05.cpp

```
#include "harbor/check.hpp"
#include <coroutine>
#include <exception>
#include <string>
#include <utility>
#include <iostream>
class StepTask {
public:
    struct promise_type {
        std::string value;
        std::exception_ptr error;
        StepTask get_return_object() { return StepTask{Handle::from_promise(*this)}; }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(std::string s) { value=std::move(s); return {}; }
        void return_void() noexcept {}
        void unhandled_exception() { error=std::current_exception(); }
    };
    using Handle=std::coroutine_handle<promise_type>;
private:
    Handle handle_;
public:
    explicit StepTask(Handle h):handle_(h) {}
    ~StepTask() { if (handle_) handle_.destroy(); }
    StepTask(const StepTask&)=delete;
    StepTask(StepTask&& other) noexcept:handle_(std::exchange(other.handle_,{})) {}
    bool next() {
        if (!handle_ || handle_.done()) return false;
        handle_.resume();
        if (handle_.promise().error) std::rethrow_exception(handle_.promise().error);
        return !handle_.done();
    }
    const auto& value() const { return handle_.promise().value; }
};
StepTask stages(std::string owned_key) {

    co_yield "validated:"+owned_key;
    co_yield "scheduled:"+owned_key;
}
int main() {
    auto task=stages(std::string("job-1"));
    harbor::check(task.next() && task.value()=="validated:job-1","first suspension");
    harbor::check(task.next() && task.value()=="scheduled:job-1","owned across suspension");
    harbor::check(!task.next(),"completion");
    std::cout<<"Coroutine lifetime demonstrated; no I/O scheduler is implied.\n";
}
```

`initial_suspend` prevents the body from running until the owner calls `next()`. Each `co_yield` stores a stage description and suspends again. `final_suspend` retains the frame until the owning `StepTask` destroys it. `unhandled_exception` stores an exception so that the next caller-facing step can rethrow it rather than silently lose the failure.

The string parameter is passed by value. The temporary supplied by `main` becomes owned coroutine state instead of a borrowed reference to a dead object. The handle is move-only, and destruction checks for an empty moved-from handle before calling `destroy()`.

Trace the observable sequence

The first `next()` resumes to `validated:job-1`. The second resumes to `scheduled:job-1`. The third reaches completion and returns `false`. The test checks those exact observations. If the task is destroyed after the first step, its remaining frame state is cleaned up without pretending that the second stage executed.

This wrapper intentionally has a narrow usage contract: one owner, no concurrent calls, no external resumption, and no access to `value()` before a successful step. A production task abstraction needs additional rules for awaiting, continuations, cancellation, executors, and concurrent destruction. Understanding the small ownership model is preparation for using those libraries correctly, not a claim that a few dozen lines replace them.

Failure clinic — A Coroutine That Borrows a Dead Buffer

Change `stages(std::string owned_key)` to `stages(const std::string& borrowed_key)` while leaving the temporary call unchanged. The task may still appear to work in one execution because the old memory has not yet been reused. That is not a valid result. The lifetime requirement has been violated before the first manual resume.

Do not include this undefined-behavior variant in ordinary test execution merely to hope a sanitizer catches it. Review the lifetime graph first, and use a dedicated diagnostic exercise only in a disposable test build. The safe variants are to pass ownership into the frame or to retain a caller-owned object until the task is destroyed or completes.

Another defect uses a mutex guard across suspension. The coroutine may resume on a different thread, while the lock type assumes unlock occurs according to the mutex's ownership rules. Even on one thread, holding the lock while waiting for an event that needs the same lock can deadlock. Keep critical sections short and end them before suspension unless the synchronization primitive explicitly supports the intended asynchronous pattern.

Engineering case — Cancel an operation without destroying a live callback target

Consider a coroutine that awaits a network result. The awaiter registers a callback containing a coroutine handle. The caller requests cancellation and destroys the task. Later, the network library invokes the callback. If it resumes the destroyed handle, the program has crossed a lifetime boundary incorrectly.

Cancellation must establish what happens to the callback registration and its target. One design keeps shared operation state alive until the transport confirms that no

callback can occur. Another removes the registration under the transport's supported cancellation protocol and waits for completion ownership to settle. The correct choice depends on the executor and library contract, not on the presence of `co_await` syntax.

An operation state machine might distinguish registered, completing, cancelled-awaiting-transport, and released. Exactly one path owns the final transition that releases resources. A cancellation flag can influence the result delivered to the caller, but the transport still owns its pending callback until the protocol says otherwise.

Do not solve the problem by making every object globally shared without a lifetime plan. Shared ownership can prevent early destruction while creating cycles or retaining a large request graph indefinitely. Use explicit ownership for the operation state, and break callback links when the completion protocol permits it.

Executor affinity is another consideration. A callback may arrive on an I/O thread while the awaiting coroutine expects to resume on a serialized application executor. Resuming directly can violate invariants even when the handle remains alive. Scheduling and lifetime are separate responsibilities that an asynchronous abstraction must compose.

The chapter's `StepTask` is intentionally a manual, single-threaded coroutine demonstration. It owns its frame and advances through explicit resumes. It does not claim to implement the cancellation protocol just described. That narrow scope lets a reader inspect frame lifetime before introducing an external callback system.

To test a real adapter, provide a deterministic fake transport that can delay completion until after cancellation. Verify that the callback either observes a still-valid operation state or is provably deregistered, and that no continuation runs twice. Then test the same contract with the actual library. A happy-path coroutine example cannot supply this evidence because it never crosses the cancellation race.

Practice and implementation lab

Add a third stage that owns a vector of input values and computes a bounded sum. Destroy one task after the first stage and allow another to finish. Instrument a small RAII object inside the coroutine to verify cleanup in both cases. Keep the trace deterministic and avoid measuring timing as a substitute for checking destruction.

Next, design an asynchronous submission interface on paper. State who owns the payload, whether completion can run inline, where it runs, how cancellation is requested, and which event permits buffer reuse. Describe what happens if initiation throws before the operation is registered.

Solution reasoning. Both early destruction and normal completion must release frame-owned objects once. An initiation failure before ownership transfer leaves cleanup with the caller. A successfully initiated borrowed operation must settle its lifetime obligation even when cancellation is requested. A coroutine's existence is not proof that a borrowed object is alive, and a stop request is not proof that a handler has stopped using it.

Chapter takeaway. Asynchronous syntax changes control flow; explicit ownership and completion rules make that control flow safe.

Chapter 6 — Building Event-Driven Network Services

A server that waits for one client to finish sending a request can become unavailable to everyone else when that client sends only two bytes and stops. Creating a thread for every connection avoids that specific blockage but can exhaust threads and memory under a different workload. Event-driven I/O offers another model: retain connection state and make progress only when an operation is ready.

This chapter builds on the framing and lifetime concepts using a small Linux reactor. Its purpose is to expose readiness, partial transfer, fairness, and bounded connection state. It is not a replacement for a mature networking framework in a public service.

6.1 Understanding Readiness and Completion

6.1.1 Interpreting Nonblocking Socket Operations

A nonblocking operation returns instead of waiting indefinitely when it cannot currently make progress. For a socket read, `EAGAIN` or `EWOULDBLOCK` means no data is presently available under that operation's rules. It is not end-of-stream and not a reason to discard a partially decoded message. A zero-length result from a stream receive has a different meaning: the peer has closed its sending direction after available data has been consumed. (R06)

A write can also transfer fewer bytes than requested. The application must retain the unsent suffix and try again when the socket can make progress. Restarting from byte zero would duplicate part of the frame. Treating the short write as complete would truncate it. The output offset is therefore protocol state, not a temporary local convenience.

A readiness notification does not reserve the resource permanently. Between notification and the attempted operation, state can change. The operation's actual return value remains authoritative. Code that assumes readiness guarantees a successful full read can fail under legitimate scheduling and peer behavior.

Error handling needs to distinguish interrupted system calls, temporary inability to progress, terminal connection conditions, and invalid descriptors. The companion's transport keeps these categories local to the socket layer and reports a bounded error to the caller rather than exposing arbitrary operating-system text as an application protocol.

6.1.2 Comparing Reactor and Completion-Oriented Designs

A reactor waits for readiness and then asks the application to perform nonblocking operations. A completion-oriented design submits operations and later receives their completion results. Both need owned buffers, cancellation rules, and limits on outstanding work; the difference is where the pending operation state is represented.

The Linux `poll` interface returns readiness flags for a set of descriptors. The application still calls `recv` or `send` and interprets their results. Flags such as hangup or error can occur alongside readable data, so the connection state machine must not blindly equate every non-read flag with immediate discard. (R05)

A mature library can abstract platform differences and provide composed operations such as reading an entire bounded message. That does not eliminate the caller's duty to retain buffers and respect completion lifetime. The abstraction moves mechanics into the library; it does not repeal the network's partial-failure behavior.

Choose a model that the team can test and operate. A custom event loop may be appropriate for a teaching lab or a tightly constrained subsystem. A production service may benefit from a library with established cancellation, timers, TLS integration, and platform support. Benchmarking a tiny echo loop alone is not enough to decide that tradeoff.

NEW SECTION EXAMPLE

Observe **EAGAIN** after draining a nonblocking socket

Use a local socketpair to separate readiness from a guarantee that every future read succeeds.

Linux/POSIX local sockets; no external service is contacted.

main.cpp

```
#include <sys/socket.h>
#include <fcntl.h>
#include <unistd.h>
#include <cerrno>
#include <poll.h>
#include <iostream>

int main() {
    int fd[2]; if(socketpair(AF_UNIX, SOCK_STREAM, 0, fd) != 0) return 1;
    int flags = fcntl(fd[1], F_GETFL); if(flags < 0 || fcntl(fd[1], F_SETFL, flags | O_NONBLOCK) < 0) return
2;
    char c = 0; bool empty = recv(fd[1], &c, 1, 0) == -1 && (errno == EAGAIN || errno == EWOULDBLOCK);
    if(send(fd[0], "x", 1, 0) != 1) return 3;
    pollfd p{fd[1], POLLIN, 0}; if(poll(&p, 1, 1000) != 1 || recv(fd[1], &c, 1, 0) != 1) return 4;
    bool drained = recv(fd[1], &c, 1, 0) == -1 && (errno == EAGAIN || errno == EWOULDBLOCK);
    close(fd[0]); close(fd[1]); std::cout << std::boolalpha << "empty=" << empty << " byte=" << c << "
drained=" << drained << '\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

`O_NONBLOCK` changes the empty-read outcome from waiting to `EAGAIN/EWOULDBLOCK`. `poll` reports that reading may make progress; the subsequent `recv` performs the actual transfer.

After consuming `x`, the socket is empty again. Reactor handlers must tolerate `EAGAIN` and partial progress rather than assuming a readiness event is a completed message. A completion-oriented API reports completed operations instead of this readiness hint.

Expected behavior or output

```
empty=true byte=x drained=true
```

Concrete experiment

Send two bytes and read one at a time. The first event can correspond to multiple successful reads followed by `EAGAIN`; it is not a one-event-per-byte protocol.

Validation of this new example: gcc: passed

Technical references: `LINUX-SOCKET`, `LINUX-RECV`. See the added references and revision index.

6.2 Modeling Per-Connection State

6.2.1 Owning Input, Output, and Deadlines

Each connection in the companion owns a descriptor, an incremental decoder, an optional output frame, an output offset, a read/write phase, and an absolute local deadline. This state survives across event-loop iterations. No view into a temporary stack buffer is retained after the receive callback returns.

The decoder copies received bytes into bounded owned storage. Once a complete request exists, the synchronous handler reads a view valid for the duration of that call. The handler must not retain the view asynchronously. If the architecture later offloads processing, it must transfer an owned request or otherwise extend the input storage's lifetime.

The connection deadline begins at acceptance and covers frame reception, handler work, and response transmission as checked by the loop. It is not reset every time a client sends one byte. Resetting on each byte can allow a slow sender to retain a connection indefinitely while staying just inside an inactivity timeout.

The deadline is enforced cooperatively by ordinary scheduling. A blocking handler can delay the next deadline check. The SQLite busy timeout bounds one common wait in the teaching application, but the event loop cannot preempt arbitrary callback code. A hard timing guarantee would require a stronger architecture and qualification than this lab provides.

6.2.2 Bounding Admission and Memory

The server admits at most 64 active connections and accepts at most 16 new sockets per loop iteration. Those are explicit teaching limits. A connection that cannot be admitted is closed before allocating a full application operation. This does not make rejection free: the kernel and accept path still consume resources, and a public deployment would require additional front-door controls.

The maximum application frame is 4,096 payload bytes plus a four-byte length header. A connection may also hold an output frame, container overhead, and kernel

socket buffers. Multiplying only the payload limit by connection count therefore underestimates total memory. Resource accounting should include both application-owned and kernel-owned costs.

The accept batch limit protects fairness. Without it, a continuous stream of newly arriving connections can keep the loop accepting while existing clients wait for reads or writes. Similarly, a handler that drains one busy socket without a work limit can starve other connections. Fairness policies decide how much progress one participant may make before the loop returns to others.

For large numbers of descriptors, a different polling facility or library may be more appropriate. That is a scalability decision based on measured workload and platform semantics. The correctness rules for partial I/O and ownership still apply even when the event notification mechanism changes.

NEW SECTION EXAMPLE

Bind receive capacity and a deadline to each connection

Keep per-connection input and state separate so one oversized peer does not overwrite another peer's request.

main.cpp

```
#include <array>
#include <string_view>
#include <iostream>
#include <algorithm>

int main() {
    struct Connection{std::array<char,8> input{};std::size_t used=0;int deadline_ms;
        bool append(std::string_view bytes,int
now){if(now>=deadline_ms||bytes.size()>input.size()-used)return false;
        std::copy(bytes.begin(),bytes.end(),input.begin()+used);used+=bytes.size();return
true;}};
    Connection a{{},0,100},b{{},0,100};
    bool first=a.append("PING",10),large=a.append("T00-LONG",20),other=b.append("OK",30);
    std::cout<<std::boolalpha<<first<<' '<<large<<' '<<other<<" a-bytes="<<a.used<<"
b-bytes="<<b.used<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The remaining-capacity subtraction is safe because `used` never exceeds the fixed array size. Rejection occurs before copying, so the oversized append does not partially mutate `a`.

Each connection owns its deadline and buffer. A production server also caps the number of connections; bounding each object does not bound total memory if peers can create unlimited objects.

Expected behavior or output

```
true false true a-bytes=4 b-bytes=2
```

Concrete experiment

Append to b atnow=100 and verify expiration. Decide explicitly whether equality is expired, as this implementation specifies.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

6.3 Completing Connections and Shutting Down Safely

6.3.1 Preserving Partial Writes and Terminal Outcomes

The server encodes one response and stores its bytes until all have been sent or the connection becomes terminal. Each successful send advances the offset by the returned count. MSG_NOSIGNAL is used on Linux to avoid a broken peer turning a write into an unexpected process-wide signal; the send result still has to be checked. (R07)

A successful send establishes that bytes were accepted by the local networking stack, not that the remote application committed an operation. The application protocol's acknowledgment is a different event. Likewise, a complete response received by the client can arrive after the business operation committed much earlier. These boundaries become crucial when retries are introduced.

On a malformed frame, the decoder marks the connection unusable and the server closes it. Attempting to resynchronize arbitrary malformed bytes can create ambiguous interpretations and additional attack surface. A versioned protocol could define resynchronization deliberately, but this small one-request-per-connection protocol does not.

The deliberate lost-reply failpoint in the worker returns no response after a successful database operation. The transport closes the socket. This is not a random network simulation; it is a controlled way to expose the exact acknowledgment gap the durable retry design must handle.

6.3.2 Draining Without Accepting New Work Forever

Once shutdown is requested, the listener is no longer polled for admission. Existing connections remain subject to their deadlines and can complete. When no peers remain, the event loop returns. This establishes a clear sequence: stop admission, settle bounded connection state, then release the listener and application dependencies.

A request accepted into a durable job table has a different lifetime from its socket. Closing the socket may leave the client uncertain, but the job remains available for lookup or replay. A request that never reached a complete valid frame has not become an accepted job. Keeping these states separate prevents shutdown logic from trying to preserve meaningless partial buffers as durable work.

The gateway's dispatcher is stopped after the network loop exits. It has its own bounded RPC wait and cooperative pause, then its thread owner joins. The order keeps stores and counters alive while the dispatcher may still use them. Reversing member or local declaration order without reviewing destruction can silently break that guarantee.

Shutdown tests should include idle connections, partial frames, a peer that closes during response, and a remote dependency that does not answer. A test that only stops an idle server establishes very little about the difficult lifecycle paths.

NEW SECTION EXAMPLE

Retain an outbound offset across partial sends

Simulate a writer that accepts at most two bytes per attempt and refuses new messages after draining begins.

main.cpp

```
#include <string>
#include <string_view>
#include <algorithm>
#include <iostream>

int main() {
    std::string pending="RESULT", wire; std::size_t offset=0; bool draining=true;
    auto admit=[&](std::string_view text){if(draining)return false; pending+=text; return true;};
    bool extra=admit("NEW"); unsigned attempts=0;
    while(offset<pending.size()){auto n=std::min<std::size_t>(2, pending.size()-offset);
        wire.append(pending, offset, n); offset+=n; ++attempts;}
    std::cout<<"wire="<<wire<<" attempts="<<attempts<<"
    accepted-during-drain="<<std::boolalpha<<extra<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

offset represents bytes already accepted by the transport-shaped writer. Each attempt advances only by actual progress; restarting at zero would duplicate bytes in the peer stream.

Draining closes admission while retaining ownership of the existing response. Real nonblocking writes may return EAGAIN or errors, so add deadline/cancellation exits rather than spinning indefinitely; this deterministic fixture supplies guaranteed two-byte progress.

Expected behavior or output

```
wire=RESULT attempts=3 accepted-during-drain=false
```

Concrete experiment

Make every second write attempt return zero-progress/EAGAIN in a fake transport. Verify the offset remains unchanged on those attempts and that a finite deadline can terminate the drain.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Serve a Request Through a Bounded Reactor

Implementation

The chapter executable creates an ephemeral loopback listener, runs the event loop on an owned `jthread`, exchanges one request, and asks the loop to stop. The transport implementation is in `source/src/net.cpp` and its public API is in `source/include/harbor/net.hpp`.

Source — `source/examples/ch06.cpp`

```
#include "harbor/check.hpp"
#include "harbor/net.hpp"
#include <thread>
#include <iostream>
int main() {
    harbor::Server server(0);
    std::jthread loop([&](std::stop_token stop) {
        server.run([](std::string_view s)->std::optional<std::string> {
            return "echo:"+std::string(s);
        }, [&]{return stop.stop_requested();});
    });
    auto reply=harbor::exchange(server.port(),"alpha",std::chrono::milliseconds(1000));
    harbor::check(reply=="echo:alpha","loopback exchange");
    loop.request_stop();
    std::cout<<"Nonblocking loopback request completed through a bounded frame.\n";
}
```

The handler returns an owned response string. It does not retain the request view. The client uses one deadline across connection establishment, request transmission, and reply reception. This prevents each phase from receiving a fresh full budget that would silently multiply total waiting time.

Trace the connection lifecycle

The operating system chooses a free port because the listener was created with port zero. `server.port()` returns the bound port; the test does not assume that a familiar fixed port is available. The client connects to the loopback address, sends a length-prefixed payload, and waits for a complete response frame.

Inside the loop, the peer moves from reading to writing only after its decoder is complete. A new output frame and offset become connection state. Once all response bytes are sent, the descriptor is closed. The client compares the decoded payload with `echo:alpha`, so a successful TCP connection without a correct response is not enough to pass.

The `jthread` remains alive until after the stop request and then joins during scope exit. It captures the server by reference, but the server's lifetime extends through that join because of declaration order. The example therefore makes its borrowing safe through an explicit owner relationship.

Failure clinic — Resetting the Budget on Every Read

A server grants two seconds to receive a request and resets the timer whenever a byte arrives. A client sends one byte every second. The connection can remain open indefinitely, consuming a slot without completing work. The timer has become an inactivity policy, not a total request budget.

A total deadline starts once and is compared with a monotonic clock. Activity does not extend it unless the protocol explicitly permits a renewal with its own resource policy. An inactivity timeout may still be useful for another purpose, but it must not be confused with the total limit.

A second defect occurs when the server performs a blocking database call in the event-loop thread and assumes socket nonblocking mode makes the entire service nonblocking. During that call, every other connection waits. The repair may be a bounded worker pool and a completion queue, but that adds ownership, backpressure, and cancellation work. The teaching server documents its bounded synchronous handler instead of pretending that this larger design is already implemented.

Engineering case — Keep one slow peer from defining the reactor's pace

A peer sends only two bytes of the four-byte frame header and remains connected. A blocking server that reads until the whole header arrives can stall every later client. A readiness-based server retains the partial header in that peer's state and returns to other ready descriptors.

The companion's integration test creates this stalled peer while another client sends a fragmented valid request. The second request must still progress. This is stronger evidence than testing a fragmented request alone, because the failure of interest is interference between peers.

Fairness also applies to new connections. If the accept loop drains an endlessly replenished backlog before returning to established peers, connection churn can starve response handling. The reactor caps the number of accepts in one iteration. That cap is a policy choice; it should be reviewed with expected workload and listener backlog behavior.

Each connection owns decoder state, output bytes, output offset, and an elapsed deadline. Readiness does not imply a whole request or whole response can be transferred in one call. On a partial write, only the sent prefix is released; the remaining bytes and offset stay owned until another writable event or terminal failure.

A peer can close after sending a complete request. Depending on transport state, the server may still compute or commit an effect before discovering that the response cannot be delivered. This is not repaired by a better poll loop. The domain's idempotency and status protocol handles the resulting uncertainty.

Now consider a different slow path: the SQLite handler waits on a lock. The reactor's socket operations remain nonblocking, but the callback occupies the event-loop thread until it returns. The peer-isolation argument therefore has a limit. A production design requiring tighter latency isolation might dispatch database work to a bounded executor and retain an asynchronous completion state.

That change would introduce new questions: who owns the request while the database task runs, which executor updates the peer state, what happens if the peer expires first, and how does shutdown join outstanding tasks? Performance isolation cannot be added by moving a function to another thread without carrying its ownership and cancellation contract along.

Practice and implementation lab

Run the `integration_partial` test. It opens a connection that sends only part of the header, then completes another request. Explain why the second request can progress. Next, reduce the connection limit in a private branch and create more idle clients than the limit. Confirm that memory growth and active peer count remain bounded and that accepted clients still receive deadline enforcement.

Add a test in which a client closes after sending a complete request but before reading the response. The server must remain alive and serve a later request. Do not assert that the application operation was rolled back; the handler may already have committed durable state.

Solution reasoning. A reactor retains separate state for each peer and waits only on readiness, so one incomplete frame does not block every other peer. Admission bounds application state, while a fairness limit prevents a busy listener from monopolizing the loop. A disconnected client changes response delivery, not necessarily the operation's durable outcome. The correct acceptance condition is continued service plus the documented state of the request, not a blanket assumption of cancellation.

Chapter takeaway. Event-driven I/O is a state-management discipline: readiness, partial progress, ownership, fairness, and deadlines must all be explicit.

Chapter 7 — Designing Wire Protocols, Serialization, and Schema Evolution

A TCP connection delivers bytes in order, but it does not deliver the application messages that the programmer imagined when calling `send`. One write can arrive through several reads, and several writes can be observed together. A receiving program must define message boundaries, validate lengths, and interpret fields independently of its in-memory object layout.

HarborDispatch uses a deliberately small protocol: one bounded length-prefixed request and one response per connection. This chapter explains why that design is sufficient for a lab, where it stops being sufficient, and how a schema-based protocol changes the engineering work.

7.1 Separating Messages from Byte Streams

7.1.1 Defining Framing and Size Limits

Framing determines where one message ends and the next begins. Common choices include a fixed size, a delimiter, a length prefix, or an established protocol with its own framing. Each choice needs rules for malformed input and resource limits. A delimiter is simple only if escaping, binary data, and maximum line length are also defined.

The companion uses a four-byte unsigned big-endian payload length followed by at most 4,096 payload bytes. The header is parsed before allocating unbounded storage. A declared length above the limit makes the connection invalid immediately. The decoder rejects extra bytes supplied beyond one complete frame. The server dispatches as soon as one frame is complete and closes after its response; it does not wait for the peer to declare EOF. Consequently, bytes arriving only after dispatch are not evidence that an earlier effect was rolled back. The protocol does not support pipelined requests.

A length prefix is not a trust boundary by itself. An attacker or defective peer can declare a huge length, send a partial header, or close before the payload completes. The decoder must distinguish incomplete input from invalid input, and the connection must impose a waiting deadline. Without both a byte bound and a time bound, a small-looking protocol can still consume resources indefinitely.

Avoid integer overflow in buffer accounting. Before appending a chunk, check whether its size exceeds the remaining permitted capacity. Comparing `current + incoming` with a limit can overflow when the types or values are uncontrolled. The companion maintains a small established maximum and subtracts the current size from that maximum before admitting more bytes.

7.1.2 Keeping Decoding Incremental and Deterministic

An incremental decoder retains state across input chunks. Its behavior should depend on the byte sequence, not on how the operating system happened to partition that sequence into reads. This is a property that can be tested without a network by feeding every split of a known message.

The companion's Decoder owns the bytes it has accepted. Once four header bytes exist, it calculates the expected payload size. It reports readiness only when the complete declared frame exists. Calling `finish()` before readiness indicates truncation. After a decoding error, the connection is not reused for another interpretation.

A useful decoder API separates “need more input” from “invalid input.” Throwing on every partial header would make ordinary network fragmentation look like corruption. Treating an invalid length as merely incomplete would let a peer hold resources while waiting for a message the application must never accept.

Golden-byte tests are important. An encoder and decoder that share the same endian bug can agree with each other while disagreeing with every other implementation. Compare at least a few encoded messages with independently specified bytes, and then test round trips and fragmentation. Agreement between two functions written from the same mistaken assumption is not an independent oracle.

NEW SECTION EXAMPLE

Decode two length-prefixed frames from byte-at-a-time input

An incremental parser retains incomplete bytes and reports an oversized length as an error distinct from “need more.”

main.cpp

```
#include <cstdint>
#include <vector>
#include <string>
#include <stdexcept>
#include <iostream>

int main() {
    std::vector<std::uint8_t> buffer; std::vector<std::string> messages;
    auto feed=[&](std::uint8_t byte){buffer.push_back(byte);
        while(buffer.size()>=2){std::size_t n=(std::size_t(buffer[0])<<8)|buffer[1];
            if(n>8)throw std::length_error("frame limit");if(buffer.size()<n+2)return;
            messages.emplace_back(buffer.begin()+2,buffer.begin()+2+n);buffer.erase(buffer.beg
in(),buffer.begin()+2+n);}};
    for(auto byte:std::vector<std::uint8_t>{0,2,'0','K',0,1,'!'})feed(byte);
    if(!buffer.empty()||messages!=std::vector<std::string>{"OK","!"})return 1;
    std::cout<<"decoded=OK,! residual=0\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The first two bytes form an unsigned big-endian length, checked against 8 before awaiting the payload. A partial payload is retained; completing one frame removes exactly its header and body, leaving any following bytes available.

Feeding individual bytes tests segmentation independence. This deliberately small vector implementation prioritizes parser visibility over hot-path efficiency; a larger implementation needs bounded receive accumulation and offset-based compaction.

Expected behavior or output

```
decoded=OK,! residual=0
```

Concrete experiment

Feed the entire byte vector in varying chunks using a span-based adapter. Every segmentation must yield the same messages; a header length 9 must fail before a ninth payload byte is accepted.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

7.2 Defining Field Representation and Meaning

7.2.1 Avoiding Raw Object Serialization

A C++ object's memory representation is not a portable wire format. Padding, alignment, byte order, pointer values, enum representation, and compiler layout choices can differ. Even a trivially copyable type is not automatically a cross-platform protocol record. Copying its bytes may be valid for a narrow in-process operation while remaining unsuitable for persistent or remote interchange.

Define fields explicitly. State integer widths or ranges, byte order, string encoding, optionality, and limits. A pointer must become an application identity or serialized content, not a numerical address that is meaningless in another process. A timestamp needs a unit and time basis. A decimal amount needs a scale rather than an assumption that every language will interpret a floating-point approximation the same way.

The teaching payload is whitespace-separated command text. Keys are restricted to ASCII identifier bytes; the numeric parser consumes a complete decimal integer within a fixed range. This avoids an unbounded general-purpose text parser while keeping the protocol inspectable. It does not attempt to represent arbitrary Unicode names or binary job payloads.

Canonicalization matters for identity. The strings 3 and 03 can represent the same accepted integer under this parser. The store compares the canonical numeric value, not the original spelling. For richer payloads, decide whether equality is byte identity, canonical serialized identity, or domain equivalence before storing a request fingerprint.

7.2.2 Using Schema-Based Serialization Deliberately

A schema language can define field numbers, types, messages, and services independently of C++ classes. Protocol Buffers generates language bindings from such definitions. Its binary evolution rules depend on preserving field identities and respecting compatible changes; removed field numbers should not be reused for unrelated meanings. Unknown-field behavior and JSON conversion have their own constraints. (R14)

A schema does not automatically define business validity. A decoded integer may still be outside the domain range. An absent field may carry a default that is ambiguous for the application. A message that decodes successfully can still violate authorization or idempotency requirements. Keep validation after decoding and before effectful processing.

The optional `dispatch.proto` in the companion illustrates a typed contract for submission and status. It is not used by the default length-prefixed service. That separation keeps the core lab dependency-light and avoids pretending that an unexecuted framework integration has been validated by the socket tests.

Generated code and runtime compatibility are also build concerns. Protocol Buffers documents specific cross-version rules, with particularly strict C++ requirements. Do not assume any installed generator can safely be paired with any runtime library merely because the `.proto` file is unchanged. Record and test the selected toolchain pair. (R43)

NEW SECTION EXAMPLE

Encode typed fields without copying object padding

Serialize a16-bit version and32-bit count into a defined six-byte wire representation.

main.cpp

```
#include <array>
#include <cstdint>
#include <iomanip>
#include <iostream>

int main() {
    const std::uint16_t version=1;const std::uint32_t count=0x01020304;
    std::array<std::uint8_t,6> wire{std::uint8_t(version>>8),std::uint8_t(version),
    std::uint8_t(count>>24),std::uint8_t(count>>16),std::uint8_t(count>>8),std::uint8_t(count)};
    std::uint32_t decoded=0;for(std::size_t i=2;i<6;++i)decoded=(decoded<<8)|wire[i];
    if(decoded!=count)return 1;
    std::cout<<std::hex<<std::setfill('0');for(auto
b:wire)std::cout<<std::setw(2)<<unsigned(b);std::cout<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The shifts define `networkbyteorder` and exact widths independently of host endianness and struct alignment. Six bytes are emitted because the schema specifies `2+4`, not because `sizeof` a C++ struct happens to equal 6.

A schema library automates encoding and evolution but still needs field meaning, ranges, and limits. Pointers, allocator state, and padding are never meaningful wire fields.

Expected behavior or output

```
000101020304
```

Concrete experiment

Set `count` to `0xffffffff` and verify all four `count` bytes are `ff`. Add an input-range check if the business domain forbids that otherwise valid wire value.

Validation of this new example: `gcc: passed`

Technical references: CPP20. See the added references and revision index.

7.3 Evolving Protocols Across Mixed Versions

7.3.1 Distinguishing Additive Fields from Semantic Changes

Adding an optional field can be wire-compatible while changing behavior in a way old peers cannot honor. Suppose a new request includes a “do not execute twice” option but an old worker ignores it. The message may decode successfully, yet the caller’s safety requirement is not met. Compatibility must include meaning, not just parser acceptance.

A new field should have a defined behavior when absent and when ignored by an old reader. Avoid using a default value to mean both “not provided” and a meaningful explicit choice unless that ambiguity is intentional. Presence tracking or an explicit enumeration can make the distinction visible.

Removing a field also requires a rollout plan. Old binaries may continue sending it, and old persisted messages may reappear during replay. Reserved field identifiers prevent accidental reinterpretation, but the application must still decide how to handle old content. A queue with a long retention period can make the compatibility window much longer than the normal deployment window.

Test combinations of old writer/new reader and new writer/old reader where support is promised. Include defaults, unknown fields, rejected values, and changed semantics. A test that only compiles the new generated code does not exercise the mixed-version population that exists during an actual rollout.

7.3.2 Establishing Version Negotiation and Rejection Rules

A protocol should fail clearly when an essential capability is missing. This can be implemented through versioned endpoints, explicit capability exchange, or message fields whose unsupported value is rejected. Silent fallback is appropriate only when it preserves the promised semantics.

The simple lab protocol has one closed version and no negotiation header. That is an intentional limit. A production extension should introduce versioning before several independent clients depend on undocumented command variations. Retrofitting an ambiguous interpretation after deployment is harder than reserving a clear protocol boundary early.

Keep version checks separate from authentication. A peer that says it supports version two has not proved its identity. Likewise, a signed request can be authentic but unsupported. Validation order should avoid expensive work on malformed input while still preserving the security requirements of the real transport.

Define whether unsupported input closes the connection, returns an error, or can be skipped. The companion closes malformed binary framing and returns small textual errors for recognized framing with invalid commands. That distinction is part of the protocol. A future multiplexed protocol would need stream-level error handling rather than copying this one-connection policy blindly.

NEW SECTION EXAMPLE

Negotiate an overlapping protocol range before accepting a feature

Choose the highest shared version and reject a new feature when only an older common version is available.

main.cpp

```
#include <algorithm>
#include <optional>
#include <iostream>

int main() {
    auto negotiate=[](int alo,int ahi,int blo,int bhi)->std::optional<int>{int
lo=std::max(alo,blo),hi=std::min(ahi,bhi);if(lo>hi)return{};return hi;};
    auto mixed=negotiate(1,2,1,1),none=negotiate(2,2,1,1);
    std::cout<<"mixed-version="<<*mixed<<" new-feature="<<std::boolalpha<<(*mixed>=2)<<"
incompatible="<<!none<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Version ranges intersect before either side sends version-specific semantics. The mixed fleet agrees on 1; it cannot use a feature whose contract begins at 2 merely because the newer client knows that field.

Adding an optional field can be compatible when old readers safely ignore it. Changing units or meaning under an existing fieldnumber is different and needs a new semantic contract. The fixture assumes valid nonempty local ranges.

Expected behavior or output

```
mixed-version=1 new-feature=false incompatible=true
```

Concrete experiment

Give both peers range1...3 and expect3. Then add explicit validation for reversed local ranges before negotiation.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Verify Framing Independently of Read Boundaries

Implementation

The executable defines the expected bytes for the payload `abc`, then feeds every possible two-fragment partition to the decoder. This tests a property of the protocol rather than relying on the operating system to happen to split a write in a particular way.

Source — `source/examples/ch07.cpp`

```
#include "harbor/check.hpp"
#include "harbor/wire.hpp"
#include <iostream>
int main() {
    const std::string expected("\0\0\0\3abc",7);
    harbor::check(harbor::frame("abc")==expected,"independent golden bytes");
    for (std::size_t split=0;split<=expected.size();++split) {
        harbor::Decoder d;
        d.feed(std::string_view(expected).substr(0,split));
        d.feed(std::string_view(expected).substr(split));
        d.finish(); harbor::check(d.payload()=="abc","fragmentation");
    }
    std::cout<<"Every two-fragment partition preserves the message.\n";
}
```

The encoded message contains four length bytes followed by three payload bytes. The independent literal establishes the endian convention. The loop includes an empty first fragment and an empty second fragment, which are useful boundary cases for the feed interface even though an actual zero-length socket read has a different end-of-stream meaning.

Evidence and interpretation

Run `./build/debug/ch07` and the broader `core_codec` test. The regression test exercises several payload sizes, embedded null bytes, maximum-size frames, oversize declarations, truncation, and trailing bytes. The input partition tests establish that the decoder's result is invariant under those partitions.

They do not establish robustness against every possible byte sequence or every allocation failure. A fuzzing campaign and additional property-based tests can extend the evidence. The bounded state and explicit grammar make such testing tractable: the fuzzer should never be permitted to turn a declared length into an unbounded allocation.

Failure clinic — A Parser That Accepts Only the Prefix

A receiver reads the input token 12oops and uses a conversion function that returns 12 without checking where parsing stopped. The request enters the system with a meaning the sender did not validly specify. If another implementation rejects the same token, clients observe inconsistent behavior depending on routing.

The correction is to require full-token consumption and a defined canonical result. The same principle applies to binary formats: trailing bytes, duplicate fields, unknown versions, and incomplete payloads need explicit rules. “The first part looked valid” is not a complete protocol decision.

Another defect serializes a native struct containing a length and pointer. The receiver interprets the pointer bits as data location. Even on identical architectures, that address belongs to another process. Replace address identity with an application key and transmit the owned content or a separately authorized resource reference.

Engineering case — Derive a parser test from byte-level boundaries

For a three-byte payload `abc`, the frame is the four-byte big-endian length followed by the three payload bytes. The golden sequence is `00 00 00 03 61 62 63` in hexadecimal. This fixture is derived from the protocol, not generated by the encoder under test.

There are eight positions at which the seven-byte sequence can be split into two chunks, including an empty first or second chunk. A correct incremental decoder must produce the same payload for every split. Splitting inside the length prefix is particularly important because an implementation that parses only when the first call contains four bytes can fail there.

A second family of tests handles declared length rather than actual length. A prefix announcing 4097 bytes must be rejected under the 4096-byte contract without waiting for or allocating the entire payload. A prefix announcing three bytes followed by only two must remain incomplete and fail when end-of-input is declared. When extra bytes are fed to the decoder, a complete frame followed by another byte violates its one-frame contract and must be rejected. At the server boundary, dispatch can precede a later arrival of unsolicited bytes; closing that connection cannot undo an effect already committed. Decoder validation and transactional rollback are different boundaries.

Zero-length payloads are valid at the framing layer but may be invalid at the command layer. Keeping those layers separate improves reuse and diagnosis. The decoder can correctly deliver an empty payload, after which the command parser

returns `INVALID`. A frame parser should not need to understand `SUBMIT` to enforce byte bounds.

The decoder also has a failure state. After an oversized or malformed sequence, continuing to feed bytes should not accidentally recover into a valid command under an undefined interpretation. Closing the connection after framing failure is a simple, explicit policy for this protocol.

These tests expose why a round trip alone is weak. An encoder and decoder can agree on the same incorrect little-endian prefix and pass every round-trip case. Golden bytes and the independent Python framing path reduce that shared-error risk.

A production protocol with multiplexing or multiple frames per connection would need a different trailing-data rule and a bounded way to retain unconsumed bytes. Do not remove the current rejection merely to make concatenated frames pass; first define the next-frame state machine, per-connection work budget, and response correlation.

Practice and implementation lab

Extend the decoder tests with every three-chunk partition of a small frame and with a header delivered one byte at a time. Then design, but do not silently deploy, a version-two header containing a magic value and protocol version. Specify how a version-one peer fails when it receives the new format.

For the schema exercise, add a priority field to `dispatch.proto`. Define the absent-value behavior, the supported range, and what an old worker does when it does not understand priority. Decide whether that old behavior is acceptable for every request or whether some clients require explicit capability rejection.

Solution reasoning. Three-chunk testing exercises transitions where the header and payload boundaries occur in different calls. A version field should be checked before interpreting version-specific payload meaning. A priority field can be safely ignored only when the caller's contract permits ordinary scheduling; it cannot be ignored if the request requires a different safety or timing guarantee. Wire compatibility is necessary for mixed-version operation, but semantic compatibility is the actual goal.

Chapter takeaway. A protocol defines bytes, limits, identity, and evolution rules; a successful decode is only the beginning of validation.

Chapter 8 — Designing RPC

Deadlines, Cancellation, and Ambiguous Outcomes

A client sends a request, waits one second, and receives a timeout. The natural question is whether to try again. The dangerous assumption is that timeout means nothing happened. The server may have committed the operation and lost the response, or it may still be running after the caller stopped waiting.

This chapter treats RPC as a distributed protocol rather than an ordinary function call with a slower implementation. Deadlines control resource use, cancellation communicates interest, and stable operation identity makes uncertainty manageable.

8.1 Defining an RPC Contract

8.1.1 Separating Transport Success from Application Success

Transport success means the communication mechanism completed a particular exchange under its rules. Application success means the domain operation established its promised state. These can diverge. A valid response may contain a domain conflict; a broken connection may follow a successful database commit.

Represent transport failures separately from normal application outcomes. A status of “not found” can be a successful lookup operation. A status of “unavailable” may indicate a dependency problem. An “uncertain” submission outcome should preserve the original request identity so the caller can query or replay safely.

An RPC framework supplies useful machinery for service definitions, serialization, status, and call lifecycle. gRPC, for example, supports unary and streaming methods with generated client and server interfaces. The framework’s call abstraction does not remove partial failure or roll back business effects when a call is cancelled. (R09)

Avoid using an exception type as the only durable API contract. A C++ exception can carry local diagnostics, but another language or remote version needs a stable protocol category. Translate at the boundary and retain enough internal context for investigation without exposing sensitive implementation details to the caller.

8.1.2 Selecting Unary, Streaming, and Asynchronous Operations

A unary request/response operation is appropriate when one bounded request produces one bounded result. A streaming operation can carry multiple messages over one call, but it adds flow control, ordering, cancellation, and long-lived resource ownership. Streaming is not automatically faster or more reliable for every workload.

HarborDispatch separates submission from completion. `SUBMIT` returns durable acceptance, while `STATUS` retrieves the current state. This avoids requiring a single connection to survive for the entire job lifetime. It also creates a stable resource that can be observed after a client restart.

A completion stream could improve notification latency, but it should supplement rather than erase durable lookup. Streams can disconnect, and a client needs a cursor or replay mechanism to determine what it missed. Without that mechanism, reconnecting may produce either lost notifications or duplicates with no way to reconcile them.

For large payloads, distinguish the control operation from bulk data transfer. A job request may refer to an authorized immutable object rather than embedding all bytes in an RPC. That design needs object integrity, access control, and lifecycle rules. It is a deliberate new boundary, not a free way to avoid message limits.

NEW SECTION EXAMPLE

Decode transport success separately from application rejection

A received response can be structurally valid while saying the operation was rejected.

main.cpp

```
#include <variant>
#include <string>
#include <iostream>

int main() {
    struct TransportFailure{std::string reason;}; struct Reply{bool accepted;std::string
    detail;};
    std::variant<TransportFailure,Reply> outcome=Reply{false,"quota"};
    if(auto*r=std::get_if<Reply>(&outcome))std::cout<<"transport=received
    application="<<(r->accepted?"accepted":"rejected")<<" reason="<<r->detail<<'\n';
    else return 1;
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The variant separates inability to receive a reply from a parsed application reply. The boolean inside Reply belongs to the service contract, not the TCP connection or HTTP exchange.

A unary call has one result, whereas a stream needs per-item and terminal status semantics. This example is explicitly unary; merely receiving bytes is never promoted to successful work.

Expected behavior or output

```
transport=received application=rejected reason=quota
```

Concrete experiment

Replace the outcome with `TransportFailure{timeout}`. The caller should report uncertain transport state rather than a confirmed quota rejection.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

8.2 Propagating a Finite Waiting Budget

8.2.1 Using Absolute Local Deadlines

A timeout is a duration; a deadline is a point beyond which the caller is no longer willing to wait. Within one process, convert a duration into an absolute deadline using a monotonic clock and compute remaining time before each phase. This keeps connection establishment, queue residence, transfer, and processing within one caller budget rather than giving each phase a fresh full allowance.

The local Budget model uses explicit logical milliseconds so tests can reason without sleeping. If a request expires at tick 200 and the current tick is 140, only 60 milliseconds remain. At tick 200, the budget is expired. The equality rule matters for deterministic boundary tests.

Real systems must account for timer granularity and scheduling delay. A deadline check does not preempt an uncooperative function at the exact nanosecond. The goal is to prevent knowingly starting or waiting on work beyond the budget and to provide a bounded strategy for blocking operations under the platform's behavior.

Do not use wall-clock jumps to measure elapsed work. Calendar time is useful for logs and certificates, but elapsed waiting should use a monotonic source appropriate to the platform. A timestamp that is meaningful on one host cannot simply be compared with another host's monotonic timestamp.

8.2.2 Carrying Budgets Across Hops

Across machines, propagate a remaining allowance or use the framework's documented deadline mechanism. Sending an absolute wall-clock timestamp assumes a relationship among clocks that must be justified. Sending a raw local steady-clock value is invalid because the clocks need not share an origin.

gRPC's deadline guidance recommends explicit deadlines and describes propagation that accounts for elapsed time rather than assuming perfectly synchronized clocks. The server is still responsible for stopping application work it initiated when cancellation is observed. Framework handling of a call does not automatically stop every external task. (R10)

A parent should reserve time for its own completion work. If the full remaining budget is assigned to a child, the child can consume all of it and leave no time for response encoding or delivery. The exact reservation is workload-dependent, but the principle is visible in a simple equation: child allowance is at most parent remaining time minus the parent's necessary completion margin.

Fan-out makes budgeting more complex. Parallel required children can share a parent deadline, while sequential children consume it one after another. Retrying every child independently can multiply load and make the parent miss its budget even when each local retry policy appears reasonable. Name the layer that owns the retry decision.

NEW SECTION EXAMPLE

Subtract elapsed local time from a propagated RPC budget

Pass remaining duration across a hop instead of sending a local monotonic-clock timestamp to another host.

main.cpp

```

#include <chrono>
#include <iostream>
#include <algorithm>

int main() {
    using namespace std::chrono_literals;
    const auto initial_budget=100ms, local_elapsed=35ms, reserve=5ms;
    auto forwarded=std::max(0ms, initial_budget-local_elapsed-reserve);
    const auto remote_queue=20ms;
    auto work_budget=std::max(0ms, forwarded-remote_queue);
    std::cout<<"forward_ms="<<forwarded.count()<<" work_ms="<<work_budget.count()<<"\n";
}

```

Run or inspect

```

g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example

```

Explanation

The caller consumes 35ms and reserves 5ms for returning/cleanup, leaving 60ms. A downstream queue consumes another 20ms, leaving 40ms for work. Durations are clamped at zero so budget exhaustion cannot wrap into a large value.

Each host must anchor the received duration to its own monotonic clock. This fixture uses explicit durations, not actual distributed clock synchronization. Network transmission time and retries must also be charged according to the protocol.

Expected behavior or output

```

forward_ms=60 work_ms=40

```

Concrete experiment

Set `remote_queue` to 70ms and verify `work_ms` becomes zero. The server should reject before expensive work rather than start a fresh 100ms budget.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

8.3 Handling Cancellation and Uncertainty

8.3.1 Treating Cancellation as a Cooperative Signal

Cancellation communicates that a caller or upstream operation no longer wants ongoing work. It should propagate where doing so saves resources safely. A long computation can periodically check a stop condition; a queued request can be removed before execution; a transport can cancel a pending wait under its API's rules.

A handler that is already committing durable state cannot assume cancellation makes the transaction disappear. The application must choose whether to finish the commit, roll back if still possible, or record a compensating action later. That choice depends on the state transition, not only on the RPC status.

The gRPC cancellation documentation emphasizes coordination between the framework and the application handler. It does not generally interrupt arbitrary application code. A service therefore needs explicit cancellation checks and child-operation ownership where those are required. (R11)

Cancellation can also race with normal completion. A robust operation object settles once according to its synchronization rules and still releases resources correctly. A caller should not destroy a borrowed output buffer until completion establishes that no further access is possible, even if it requested cancellation earlier.

8.3.2 Preserving Identity When the Outcome Is Unknown

An ambiguous outcome is not an error to hide; it is a state to represent. The client retains the same request key and can ask for status or replay the same canonical input. The server compares that identity with durable state. This is the foundation for handling a lost acknowledgment without creating a second logical operation.

Generating a new key on every retry defeats the mechanism. The server sees distinct operations and may legitimately execute each one. Likewise, reusing a key with changed content must produce a conflict rather than silently returning the old result. The key identifies intent, not merely a network attempt.

Consider a timeout before the client knows whether submission committed. A subsequent status lookup may return missing because the first attempt never committed, or because the read is stale if the architecture routes it to an asynchronous replica. Safe reconciliation therefore depends on the consistency of the lookup path as well as on request identity. Later chapters make this dependency explicit.

An API can sometimes prove that an operation did not begin, such as a local validation rejection before any remote call. Preserve that stronger information when available. Do not generalize it to every network exception, and do not classify every failure as uncertain when the implementation has clear evidence of rejection. Precision improves both safety and usability.

NEW SECTION EXAMPLE

Reconcile a cancelled call whose effect already committed

Keep operation identity after a lost reply and retrieve the existing result rather than executing a new operation.

main.cpp

```
#include <map>
#include <string>
#include <iostream>

int main() {
    std::map<std::string,int> receipts;unsigned effects=0;
    auto execute=[&](const std::string&id,int x){auto
it=receipts.find(id);if(it!=receipts.end())return it->second;
    ++effects;return receipts.emplace(id,x*x).first->second;};
    (void)execute("job9",6);bool caller_cancelled=true; // reply discarded after commit
    int recovered=execute("job9",6);
    std::cout<<"cancelled="<<std::boolalpha<<caller_cancelled<<" recovered="<<recovered<<"
effects="<<effects<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The cancellation flag belongs to the caller; it cannot erase a server-side effect that was already recorded. Reusing `job9` makes the second attempt a lookup of the receipt instead of another effect.

This in-memory model omits durable persistence and payload-conflict checks, addressed in the idempotency sections. It demonstrates only the control-flow distinction between cancellation intent and external rollback.

Expected behavior or output

```
cancelled=true recovered=36 effects=1
```

Concrete experiment

Retry with a new `job10` identifier and observe two effects. Then add input binding `sojob9` with a different argument is a conflict, not a replay.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Account for Queue Residence in a Shared Budget

Deterministic implementation

The model starts with a total deadline at tick 200. It examines the remaining allowance after initial work and after queue residence. No real-time sleep is involved, so the test is stable across machines.

Source — source/examples/ch08.cpp

```

#include "harbor/check.hpp"
#include "harbor/policies.hpp"
#include <iostream>
int main() {
    harbor::Budget request{200};
    auto child_allowance=request.remaining(35);
    harbor::check(child_allowance==165,"parent budget");
    auto after_queue=request.remaining(140);
    harbor::check(after_queue==60,"queue residence must count");
    harbor::check(request.expired(200),"inclusive deadline");
    std::cout<<"200 ms total: 35 ms used, then 105 ms queued; only 60 ms remain.\n";
}

```

The result shows that 35 milliseconds of initial work followed by 105 milliseconds in a queue leaves 60 milliseconds, not another 200. A system that starts a new timeout when the worker dequeues the request hides queue delay from the caller's contract.

Connect the model to the transport

The real exchange helper creates one `steady_clock` deadline and passes it through connection establishment and all socket waits. The helper uses completion of the framed response as its local success boundary. It does not interpret a timeout as proof that the peer did nothing.

The gateway's background dispatcher has a different lifetime from the client's submission RPC. Once intent is durably accepted, the job can outlive the client's request budget. Its retry loop is owned by the gateway and persistent pending state. This is an explicit handoff of responsibility, not a violation of the caller deadline: the caller asked for acceptance, not for the calculation to finish within that same socket exchange.

Failure clinic — Three Layers of Retries

A caller retries a gateway request three times. The gateway retries a worker call three times. The worker retries a database operation three times. Under an outage, one logical request can create up to 27 attempts at the deepest layer, before accounting for any proxy retries. That multiplicative behavior is derived from the chosen counts, not a universal property of retry libraries.

The repair is to assign retry ownership and propagate budget. A local database may legitimately retry a transient lock under a short bounded policy, but the outer layer must account for that time. A worker call with a durable idempotency key may be retried by one designated layer. Other layers should not silently duplicate the policy without coordination.

A related defect resets a deadline after a retry. Each attempt looks individually bounded, but total waiting becomes unbounded if attempts continue. Bound both attempt count and total elapsed allowance for interactive operations. Durable background delivery may retry indefinitely under an explicit service policy, but it still needs bounded rate, observability, retention, and a response to permanent failures.

Engineering case — Account for one deadline across three stages

Suppose an interactive operation has a hypothetical 200 ms elapsed-time budget. It waits 45 ms for admission and spends 30 ms establishing a connection. The remaining budget is 125 ms before remote execution begins, not another 200 ms.

If the remote call uses 80 ms, only 45 ms remains for response handling and any permitted recovery action. A retry policy proposing a 60 ms delay cannot fit another attempt. Starting it anyway produces predictable deadline violations while increasing dependency load.

Use one local absolute deadline and compute remaining time before each blocking or waiting operation. When sending a request to another process, translate the relevant remaining duration according to the RPC framework's supported deadline mechanism. Do not send a raw `steady_clock` timestamp and assume another host shares its epoch.

The budget also determines queue policy. A task that reaches the front of a worker queue after its caller's deadline should not consume expensive work automatically. Check whether the operation is cancellable interactive work or a durably accepted job whose completion is intentionally independent of the caller's wait. HarborDispatch chooses the latter after acceptance; a synchronous query might choose the former.

Now add an uncertain outcome. The remote worker commits at 190 ms, but the reply arrives after 200 ms. The client can correctly report an expired wait while the business operation correctly completed. A result type that has only "success" and "nothing happened" cannot represent this history.

A status query using the same logical identity resolves the ambiguity later. Its own budget is a new observation request, not a new execution identity. The interface should make that distinction natural for the caller.

For a test, use a fake clock and explicit stage advances rather than real sleeps. Assert the remaining values 200, 155, 125, and 45, then reject the over-budget retry. A separate real-network test verifies that the transport honors its supported timeout path. The model checks arithmetic; the integration checks the actual boundary. Neither alone proves a production end-to-end latency objective.

Practice and implementation lab

Extend the Budget model with a function that returns a child allowance after reserving a completion margin. Test a margin larger than remaining time, equality at expiration, and saturating arithmetic near the maximum tick value. The function should reject or return zero rather than underflow into a huge allowance.

Then run `integration_lost_reply`. Explain why the worker has one receipt even though the gateway experiences a failed exchange. Identify the point where the worker commits, the point where the response is intentionally dropped, and the later replay that recovers the result.

Solution reasoning. The worker's durable receipt is authoritative for its effect, while the transport failure describes only the gateway's observation. Replaying the same key and input returns the stored result. A new key would not be a replay. The test demonstrates one controlled acknowledgment-loss schedule; it does not prove exactly-once behavior for arbitrary external effects such as sending email or charging an unrelated payment system.

Chapter takeaway. Deadlines bound waiting, cancellation coordinates interest, and stable identity preserves correctness when the caller cannot know the outcome.

Chapter 9 — Discovering Services and Balancing Work

A service moves to another address, but some clients continue sending requests to the old one. A new worker becomes healthy, but it receives every new request because all clients react to the same fresh measurement. A load balancer reports several available workers, yet duplicate execution appears because retries were routed to workers with unrelated receipt databases.

Discovery and load balancing are not merely ways to obtain an IP address. They combine membership, freshness, health, placement, and state ownership. This chapter separates those concerns before choosing a routing algorithm.

9.1 Representing Membership as Versioned Information

9.1.1 Distinguishing Names, Endpoints, and Instances

A service name represents a logical capability. An endpoint represents a route to one serving location. An instance represents a particular process or deployment generation. These identities can change at different rates. A service name may remain stable while endpoints are replaced; a process may restart at the same address while losing all volatile state.

Do not use an address as the only durable identity of a participant. A reused port or recycled cloud address can refer to a different instance. Long-lived clients may need to reconnect, refresh credentials, and reestablish protocol capabilities even when the endpoint text looks unchanged.

DNS can participate in discovery, but caching, refresh behavior, TTL handling, connection reuse, and negative answers affect when clients observe changes. A connection pool can retain an old destination after a name resolution result changes. Discovery therefore includes both obtaining new information and deciding how to retire old connections.

gRPC's name-resolution model allows a resolver to provide endpoint information used by load-balancing policies. The resolver and picker are different responsibilities. Understanding that separation is useful even when the lab uses an explicit loopback port rather than a framework resolver. (R13)

9.1.2 Rejecting Stale Membership Updates

A client that receives membership updates asynchronously can observe an old update after a newer one. Attach a monotonic revision or another defined ordering token to snapshots. Apply a new snapshot only if its revision advances the accepted state. Otherwise a delayed event can resurrect retired endpoints.

The chapter model sorts and deduplicates endpoints before publishing them. This gives a canonical membership representation and prevents accidental weighting

caused by duplicate entries. Whether duplicates should instead represent deliberate weight is a different contract; the model chooses uniqueness explicitly.

A revision number is meaningful only within its authority and generation. Restarting a registry with its counter reset can make every future update look stale to clients that retained a larger number. A real protocol needs a stable revision source or a composite generation and revision scheme with defined replacement rules.

Snapshots and incremental watches also have different recovery paths. If a watch loses events, replaying only the next update may not reconstruct membership. The client may need a full snapshot and a watch position tied to that snapshot. The exact mechanism is registry-specific, but the consistency requirement should be visible in the client design.

NEW SECTION EXAMPLE

Ignore a stale discovery snapshot

Use a membership generation to distinguish a newer endpoint set from delayed older discovery data.

main.cpp

```
#include <vector>
#include <string>
#include <iostream>
#include <utility>

int main() {
    struct Membership{unsigned generation=0;std::vector<std::string> instances;
        bool replace(unsigned g,std::vector<std::string> next){if(g<=generation)return
false;generation=g;instances=std::move(next);return true;}};
    Membership m;m.replace(7,{"worker-a@10.0.0.1:7000"});
    bool stale=m.replace(6,{"retired@10.0.0.9:7000"});
    std::cout<<"generation="<<m.generation<<" stale-accepted="<<std::boolalpha<<stale<<"
instance="<<m.instances.front()<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The generation belongs to the discovery authority, not local receipt order. A delayed generation 6 update cannot undo the already published generation 7 membership.

The string keeps instance identity and endpoint visible together for this fixture. In production store them as separate typed fields: reusing the same IP:port does not imply the same process instance or authority.

Expected behavior or output

```
generation=7 stale-accepted=false instance=worker-a@10.0.0.1:7000
```

Concrete experiment

Publish generation8 with an empty list. Empty-but-current membership is a valid state; do not revive generation7 merely to avoid reporting unavailable.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

9.2 Separating Health from Routing Policy

9.2.1 Interpreting Liveness, Readiness, and Degradation

Liveness asks whether a process should continue running under a particular policy. Readiness asks whether it should receive a particular class of work. A degraded service may be alive and able to answer status queries while unable to accept new jobs. One Boolean “healthy” flag often hides these distinctions.

A health probe should test the capability it claims without causing excessive load or creating a dependency cycle. If a liveness probe fails whenever a downstream database is briefly unavailable, an orchestrator may restart every otherwise healthy frontend and make recovery harder. Conversely, a probe that only checks whether a port accepts connections may route traffic to an instance that cannot perform any useful operation.

For HarborDispatch, PING only confirms that the local handler path returns PONG. It does not establish worker availability, backlog capacity, TLS validity, or disaster recovery. Production readiness would need operation-specific checks and a policy for how dependency degradation affects admission.

Do not let a health observation become an unbounded synchronous dependency. A load balancer that must query a control plane for every request can make control-plane latency part of the data path. Cached membership with bounded staleness and explicit failure behavior is often easier to operate than a supposedly perfect real-time lookup that can stop all traffic.

9.2.2 Choosing a Load-Balancing Objective

Round-robin balances request counts under simple assumptions, not necessarily work. Least-outstanding-request policies approximate load but can be misled by request size and delayed completion observations. Randomized choices reduce some coordination effects but still require a definition of eligible endpoints and resource limits.

Choose the objective before the algorithm. A stateless read service may aim to spread load. A stateful worker system may need affinity so that the same request identity reaches the authority that owns its receipt. A cache service may trade uniform distribution against locality. These are different problems, and one picker cannot satisfy them accidentally.

Client-side measurements are partial and delayed. If every client immediately sends all traffic to the worker with the lowest observed latency, that worker can become

overloaded before the measurements catch up. Add damping, bounded concurrency, and randomized selection where appropriate instead of treating a stale metric as an instantaneous global truth.

The companion does not load-balance the integrated job service across independent worker databases. It uses one configured worker endpoint. Routing the same request key to several unrelated receipt stores would weaken its deduplication boundary. Later partitioning and replication designs explain what additional authority is required before that scaling change is safe.

NEW SECTION EXAMPLE

Route only to ready instances, then minimize in-flight work

Filter readiness before applying the load-balancing objective.

main.cpp

```
#include <array>
#include <optional>
#include <string_view>
#include <iostream>

int main() {
    struct Node{std::string_view id;bool alive,ready;unsigned in_flight;};
    const std::array
nodes{Node{"a",true,false,0},Node{"b",true,true,5},Node{"c",true,true,2}};
    std::optional<std::size_t> choice;
    for(std::size_t i=0;i<nodes.size();++i)if(nodes[i].alive&&nodes[i].ready)
        if(!choice||nodes[i].in_flight<nodes[*choice].in_flight)choice=i;
    if(!choice)return 1;std::cout<<"route="<<nodes[*choice].id<<"
in-flight="<<nodes[*choice].in_flight<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Nodea is alive but not ready, so its attractive zero load does not make it eligible. Amongbandc the policy selects the smaller current in-flight count.

This snapshot-based policy does not guarantee perfect fairness under concurrent routing. Counters may be stale and heterogeneous servers may need weighted capacity; those are routing-policy decisions separate from basic eligibility.

Expected behavior or output

```
route=c in-flight=2
```

Concrete experiment

Markc unready and expectb. Then make every node unready and return an explicit unavailable result rather than dereferencing an empty choice.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

9.3 Coordinating Routing with Failure and State

9.3.1 Retrying Without Violating Placement

A retry is another attempt for the same logical operation, not necessarily permission to choose an unrelated state owner. A stateless read may move freely to another replica under its consistency contract. A stateful mutation may require the same partition or a replicated authority that shares the idempotency record.

Suppose worker A commits a receipt and loses its response. A retry routed to worker B, whose database has no record, can execute the operation again. Local deduplication on both workers does not solve global deduplication. The architecture must define shared or partitioned ownership and preserve it across routing changes.

A stable placement function can help route a key, but membership changes complicate the rule. If adding a worker changes the key's owner, the old receipt may still reside on the previous owner. Moving routing before state creates the same duplicate-execution gap. State transfer and routing epochs must be coordinated.

For the teaching service, keep the fixed endpoint and separate database restart tests until these concepts are understood. Scalability is not achieved safely by replacing a single port with a vector of ports and choosing one at random.

9.3.2 Handling Empty and Uncertain Membership

An empty membership snapshot is a meaningful state. The client may fail fast, wait within the caller's remaining budget, or use a documented fallback. It should not index an empty vector, retain a retired endpoint forever without policy, or silently redirect to an unrelated default service.

Distinguish "the registry says no endpoints exist" from "the registry could not be reached." The first is an authoritative observation under the registry's contract; the second is uncertainty about freshness. A last-known-good snapshot may be usable for a bounded period, but the decision needs explicit risk and staleness limits.

A fallback endpoint can create hidden concentration. During a registry outage, every client may converge on it and overload the very path intended for recovery. Capacity and authorization must be considered for fallback traffic just as for ordinary traffic.

Log membership generation and selection category without creating unbounded telemetry. Recording every endpoint and request key as metric labels can produce a cardinality problem. Use bounded categories for metrics and targeted diagnostic traces for individual routing investigations.

NEW SECTION EXAMPLE

Keep retries within the authoritative shard

Exclude a failed instance without sending a tenant's retry to a healthy but wrong shard.

main.cpp

```

#include <array>
#include <optional>
#include <string_view>
#include <iostream>

int main() {
    struct Node{std::string_view id;unsigned shard;bool ready;};
    const std::array nodes{Node{"a",0,false},Node{"b",1,true},Node{"c",0,true}};
    auto choose=[&](unsigned shard)->std::optional<std::string_view>{for(const
auto&n:nodes)if(n.shard==shard&&n.ready)return n.id;return{}};
    auto retry=choose(0),missing=choose(2);
    std::cout<<"retry="<<*retry<<" absent-shard="<<std::boolalpha<<!missing<<'\n';
}

```

Run or inspect

```

g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example

```

Explanation

The partition constraint is checked before health. Nodeb is ready but does not own shard0, so it is not a legitimate retry target. Nodec is the available owner for this fixture.

An empty eligible set is distinct from a failed lookup of the discovery service. Both require explicit outcomes; neither grants permission to route to an arbitrary live process.

Expected behavior or output

```
retry=c absent-shard=true
```

Concrete experiment

Makec unready and requireunavailable for shard0 even thoughb remains healthy. This is a correctness test for placement, not a load-balancing benchmark.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Preserve the Newest Discovery Snapshot

Implementation

The model applies revision two, rejects a late revision one update, removes duplicate endpoint entries, and chooses a member deterministically for a key.

Source — source/examples/ch09.cpp

```
#include "harbor/check.hpp"
#include "harbor/models.hpp"
#include <iostream>
int main() {
    harbor::Discovery registry;
    harbor::check(registry.update(2,{"worker-b","worker-a","worker-a"}),"new revision");
    harbor::check(!registry.update(1,{"retired"}),"stale watch event");
    harbor::check(registry.endpoints.size()==2,"deduplicated membership");
    auto chosen=harbor::placement("job-8",registry.endpoints);
    harbor::check(chosen=="worker-a" || chosen=="worker-b","member routing");
    std::cout<<"Membership revision 2 survived a late revision 1 update.\n";
}
```

Discovery::update is a single-owner model, not a network registry client. It demonstrates ordering and canonicalization. The placement function is a small deterministic teaching implementation; it does not establish a production load distribution or a durable ownership protocol.

Evidence and interpretation

Run ./build/debug/ch09 and core_routing. The tests verify that reordering an identical member set does not change placement and that adding a member only moves a key to that added member under the modeled highest-score rule. They do not test DNS caches, registry outages, endpoint authentication, or actual balancing under load.

The separation is intentional. Membership ordering is a pure property and can be tested exhaustively for small inputs. Network freshness is an integration property that requires a registry or controlled simulation. Production routing needs both, plus the data ownership rule that determines whether a chosen endpoint is permitted to process the operation.

Failure clinic — Healthy Replicas with Inconsistent Deduplication

A gateway begins randomly distributing retries among three workers. Each worker has a local receipt table and passes its own duplicate tests. During a lost-response event, two workers independently record the same key. Every local test remains green, while the system-level promise has been violated.

The missing invariant is the scope of identity. Either all workers must consult a shared authoritative receipt mechanism, or keys must route to a partition whose replicated state preserves the receipt, or the external effect itself must accept a stable idempotency key. A shared in-memory set inside each process is not enough.

A second problem occurs when readiness is derived only from process liveness. A worker that cannot write its receipt store may still accept TCP connections. Routing new mutations there produces retries and uncertainty. Probe and admission policy should reflect the operation's dependencies without turning temporary downstream degradation into a destructive restart loop.

Engineering case — Recover discovery after a registry generation changes

A client has accepted membership revision 900. The registry is restored from an old backup and begins publishing revision 20. A rule that accepts only larger integers will reject every update until the counter exceeds 900, even if the new snapshot is the intended authority.

The problem is not solved by accepting any smaller revision after a timeout. A delayed old update could then replace current membership. The protocol needs an authority for distinguishing a new registry incarnation from an old message.

One design uses a generation identifier plus an ordered revision within that generation. The generation transition itself must be trusted and explicitly accepted; arbitrary lexical comparison of random identifiers does not establish which authority is newer. Another design uses a persistent monotonic revision service that survives the relevant recovery operations. The correct choice depends on the control plane.

A reconnect protocol can fetch a fresh authoritative snapshot and its watch position together. It then resumes incremental updates from that position. If the watch reports a gap beyond retained history, the client returns to snapshot acquisition instead of pretending the next event reconstructs everything it missed.

Connection pools complicate the transition. Updating the endpoint vector does not immediately remove connections to retired instances. Define whether those connections drain, close immediately, or remain eligible for a bounded class of requests. Mutations with state affinity may need stronger handling than stateless reads.

The companion's Discovery model uses one monotonically increasing number and rejects nonadvancing updates. It teaches stale-update rejection within one authority domain. It deliberately does not implement registry generation replacement, watches, or connection-pool reconciliation.

Extend the model in a separate exercise with explicit events: accept generation A revision 900, authorize replacement by generation B, accept B revision 1, reject a delayed A revision 901. The important assertion is that an old generation cannot regain authority merely by carrying a numerically larger revision.

This case illustrates a recurring distributed-systems lesson: counters are ordered only within a specified domain. An API must define who owns that domain and what happens when its state is restored, replaced, or lost.

Practice and implementation lab

Extend the discovery model with a source generation identifier and define how a new generation replaces old revision numbers. Test a registry restart, an empty snapshot, duplicated endpoints, and a delayed update from the previous generation. Do not compare generation strings lexicographically unless the protocol gives that comparison meaning.

Then write a routing policy for a read-only status cache and a separate policy for mutations. Explain why the two may choose different endpoints. For the mutation policy, state where the authoritative request key lives and how that ownership survives a worker replacement.

Solution reasoning. A fresh registry generation requires an explicit authority transition, not merely a lower counter accepted at random. Read-only cached status may tolerate bounded staleness; mutation deduplication cannot tolerate unrelated authorities for one key. A load balancer distributes attempts, while the architecture determines whether those attempts are semantically safe on the selected workers.

Chapter takeaway. Discovery identifies candidates; health filters capabilities; routing must still respect freshness, workload, and authoritative state ownership.

Chapter 10 — Reasoning About Failures, Clocks, and Causality

A worker has not answered for two seconds. It might have crashed, the network might be delayed, its reply might have been lost, or the client might be paused. These possibilities look similar from one observation but imply different recovery actions. Distributed architecture becomes difficult when a component must act without knowing which explanation is true.

HarborDispatch resolves this uncertainty by retaining a request identity and making repeated delivery safe within a defined database boundary. It does not infer from a timeout that the worker performed no work. This chapter develops the failure and time models that make that distinction precise.

10.1 Naming the Failures a Design Assumes

10.1.1 Distinguishing Crash, Omission, Delay, and Corruption

A crash-stop model assumes a participant stops taking steps and does not resume. A crash-recovery model permits a participant to restart, often retaining durable state but losing memory. An omission means an expected message is not delivered or processed. A delay means it arrives later than expected. Corruption or Byzantine behavior is a different and stronger problem: a participant or communication path can supply incorrect information, perhaps inconsistently to different peers.

These categories are assumptions for reasoning, not labels an observer can always assign accurately. A TCP disconnect establishes a connection event, not the precise state of a business transaction. A process that restarts at the same address can retain its database while losing its connection state and counters. A process can also be alive but unable to make progress because its disk is full.

Specify which failures the implementation is intended to tolerate. The companion tests process restart, interrupted transactions, dropped application replies, unavailable dependencies, malformed frames, and bounded overload. They do not implement Byzantine consensus or verify the behavior of a failing storage controller. Cryptographic transport protection, replicated storage, and operational recovery would add different guarantees, not magically extend every test already performed.

A useful design review names the unit that fails. A thread exception, a process crash, a machine loss, and a regional outage have different state-loss boundaries. “The service recovered” is incomplete until the review explains which state survived and which participants were replaced.

10.1.2 Treating Failure Detection as a Decision Under Uncertainty

A timeout is a policy for how long to wait, not proof of remote death. Its practical value is that it bounds local resource commitment and triggers a defined next action. That action may be to return an uncertain result, retry with the same identity, query status, or move responsibility to a durable recovery workflow.

Suppose the gateway sends EXEC report¹⁷ 6. The worker commits result 36 and the connection closes before a response reaches the gateway. A timeout does not justify submitting a different key. Doing so would transform uncertainty about one operation into a second operation. The stable key lets the gateway ask the worker to recognize the prior effect.

Failure detectors also interact with load. Aggressive timeouts during a temporary slowdown can cause retries, additional connections, and more queueing. The detector can help create the condition it is detecting. A separate admission limit and retry budget are therefore as important as the timeout value.

Document the transition caused by suspicion. A client may stop routing new traffic to an endpoint while allowing existing requests to complete. A leader election may require an authority and quorum protocol. A local timeout cannot safely appoint a new writer by itself because the old writer may still be executing elsewhere.

NEW SECTION EXAMPLE

Turn a missed heartbeat into suspicion rather than proof of death

Exercise a failure detector with supplied times and show that a delayed heartbeat can clear suspicion.

main.cpp

```
#include <iostream>

int main() {
    int last_seen_ms=0;constexpr int timeout=100;
    auto suspected=[&](int now){return now-last_seen_ms>timeout;};
    bool at90=suspected(90),at120=suspected(120);
    last_seen_ms=125;bool after_late=suspected(130);
    std::cout<<std::boolalpha<<"suspect90="<<at90<<" suspect120="<<at120<<"
    after-heartbeat="<<after_late<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The detector observes elapsed silence, not the remote process's actual state. Both a crash and a sufficiently delayed message can produce the 120ms suspicion.

Receiving a heartbeat at 125 clears this detector’s suspicion. It should not automatically restore stale leadership authority: membership epochs and fencing are separate mechanisms. Supplied times make the test deterministic.

Expected behavior or output

```
suspect90=false suspect120=true after-heartbeat=false
```

Concrete experiment

Delay the heartbeat until 200ms and list which actions your service permits while suspicious. Separate “stop routing new work” from “declare the old leader unable to write.”

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

10.2 Choosing the Right Meaning of Time

10.2.1 Separating Wall Time from Elapsed-Time Budgets

Wall time answers questions such as when a log entry occurred relative to a calendar. An elapsed-time clock answers how much time has passed within a process or machine clock domain. Use `std::chrono::steady_clock` for the companion’s local deadlines, because their purpose is to bound elapsed waiting rather than to match a civil timestamp.

Do not serialize a `steady_clock::time_point` and assume another host can interpret it. Its epoch and relationship to another process’s clock are not a cross-machine protocol. An RPC can convey a remaining duration, with the receiver applying its own budget rules, or a wall-clock deadline under an explicitly managed synchronization and uncertainty model. Neither choice removes network transit time automatically.

A practical budget calculation starts at admission. A 500 ms request that spends 350 ms waiting has only 150 ms left for connection establishment, execution, and response. Resetting a fresh 500 ms timeout at each stage silently expands the end-to-end allowance. A deadline carried through local stages is easier to audit than unrelated per-call durations.

Wall-clock timestamps remain useful for operators, but comparisons across hosts have uncertainty. Record timezone offsets, preserve source identity, and avoid using a one-millisecond ordering difference as proof of causality when clock synchronization error could be larger. A log aggregation system can sort records without proving that this is their true causal order.

On Linux, clock APIs distinguish clocks affected by civil-time adjustment from clocks intended for elapsed measurements, and also distinguish whether suspend time counts. The chosen clock must match the requirement. This operating-system distinction does not specify a universal implementation for every C++ standard-library clock. (R47)

10.2.2 Defining Expiration and Restart Behavior

Expiration needs a boundary convention. In this companion, `now >= end` means an elapsed-time budget is expired. A cache entry is not usable at exactly its expiration tick. Tests at equality are valuable because off-by-one assumptions often differ between callers and implementations.

Persisted deadlines require special care. A local monotonic timestamp stored before a reboot may not retain meaning after restart. Persist a business expiration in a supported time representation, or persist enough state to reestablish a policy on recovery. A job's durable acceptance and its caller's interactive waiting budget are different concepts: the client can stop waiting while the accepted job remains pending.

HarborDispatch deliberately keeps background delivery alive after the submitting RPC ends. That is an application decision, not a universal recommendation. A payment authorization, a cache refresh, and a batch report may need different expiration behavior. Each requires a definition of whether an expired job is cancelled, retained for inspection, or still completed.

When timeout arithmetic uses fixed-width integers, define overflow behavior. Saturating addition is useful for a "no sooner than" limit because wrapping to a small timestamp could expire immediately. Saturation is not appropriate for every counter; logical clocks in this chapter instead reject overflow rather than silently changing their ordering meaning.

NEW SECTION EXAMPLE

Do not compare monotonic timestamps from different process lifetimes

Keep a request budget as a duration and use durable expiration as a separately defined wall-time policy.

main.cpp

```
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono_literals;
    const auto started=std::chrono::steady_clock::time_point{100ms};
    const auto now=std::chrono::steady_clock::time_point{135ms};
    auto remaining=100ms-std::chrono::duration_cast<std::chrono::milliseconds>(now-started);
    const std::chrono::seconds expires_epoch{2000},wall_now_epoch{2001};
    std::cout<<"local-budget-ms="<<remaining.count()<<"
persisted-expired="<<std::boolalpha<<(wall_now_epoch>=expires_epoch)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Steady-clock subtraction measures a local elapsed interval unaffected by wall-clock adjustments. Its `time_point` value is not a portable persistent timestamp or a valid timestamp to compare on another host.

The persisted epochseconds illustrate a different expiration contract. Wall-clock corrections and uncertainty need a policy for durable leases/credentials; this simple comparison does not establish a safe distributed lease.

Expected behavior or output

```
local-budget-ms=65 persisted-expired=true
```

Concrete experiment

Simulate a wall-clock rollback to 1999. Local elapsed budget must remain 65ms, while the wall-expiration decision changes; explain why the two clocks cannot be interchanged.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

10.3 Representing Causal Relationships

10.3.1 Using Logical Clocks Without Pretending They Measure Duration

Lamport's happened-before relation captures ordering from local program steps and message communication. A logical clock can assign numbers such that a causally earlier event receives a smaller number. The converse does not hold: smaller logical numbers do not establish that two events are causally related. (R15)

The model increments its counter for a local event. On receiving a message with counter incoming, it sets its counter above both its current value and incoming. This allows a receiver to represent that its receive event follows the sender's event without comparing wall clocks.

Logical-clock differences have no duration unit. A counter difference of 50 is not 50 milliseconds and does not show how much work occurred. A participant with frequent internal events can advance more quickly than another participant that spends a long time blocked. Logical time is an ordering tool, not a latency measurement.

A scalar timestamp plus a participant identifier can define a deterministic total ordering for a particular application, but that ordering adds a tie-break rule; it does not discover causality that the scalar clock did not contain. Vector clocks can represent more information about concurrent histories at the cost of metadata and membership complexity. Select the representation according to what decisions the application actually needs to make.

10.3.2 Keeping Ordering Metadata Within a Stable Identity Domain

A logical clock that resets while retaining the same participant identity can produce values that look older than prior events. A durable participant may need to preserve its counter; a newly created participant may need a new incarnation identifier. The correct choice depends on whether old messages and new messages can coexist.

The companion's `LamportClock` is an in-memory teaching type. Its maximum value causes an exception. It is not a persistent event protocol. This boundary keeps the example small enough to inspect while making clear what a real event stream must additionally solve.

Event identifiers, sequence numbers, revisions, and fencing tokens are related but not interchangeable. An event identifier distinguishes one occurrence. A sequence number orders events in a stream. A revision identifies an accepted version of state. A fencing token can authorize an externally checked generation. Reusing one field for all four purposes makes it difficult to reason about rollover, retention, and authority.

For `HarborDispatch`, the request key identifies a logical job and the canonical input binds its meaning. That identity survives retries. A future ordered event feed would need its own sequence and recovery cursor; the request key alone does not tell a consumer which event follows which.

NEW SECTION EXAMPLE

Advance Lamport clocks across a send and receive

Preserve causal order with logical counters without interpreting their gaps as elapsed duration.

main.cpp

```
#include <algorithm>
#include <cstdint>
#include <iostream>

int main() {
    std::uint64_t a=0,b=0;
    auto local=[](std::uint64_t&c){return ++c;};
    local(a);auto sent=local(a);local(b);
    b=std::max(b,sent)+1;auto reply=local(b);a=std::max(a,reply)+1;
    std::cout<<"send="<<sent<<" receive="<<reply-1<<" reply="<<reply<<" final-a="<<a<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

A local event increments its counter. A receive takes `max(local,message)+1`, making its timestamp greater than both the earlier local state and the causally prior send.

The counter gap says nothing about milliseconds. Equal or ordered Lamport values do not identify every concurrent event; include a stable node/epoch tie-breaker if a total ordering of otherwise unrelated records is required.

Expected behavior or output

send=2 receive=3 reply=4 final-a=5

Concrete experiment

Insert 100 local events onb before receiving. The receive timestamp grows but the causal rule remains identical; no network-duration conclusion follows.

Validation of this new example: gcc: passed

Technical references: CPP20, LAMPORT-CLOCKS. See the added references and revision index.

Worked example — Trace a message through two logical clocks

Requirement and mechanism

Process A emits an event, process B receives it, B performs local work, and A later receives B's reply. The test must establish ordering along those message edges without using sleeps or wall-clock measurements.

Source — source/examples/ch10.cpp

```
#include "harbor/check.hpp"
#include "harbor/models.hpp"
#include <iostream>
int main() {
    harbor::LamportClock a,b;
    auto send=a.tick();
    b.tick(); auto receive=b.receive(send);
    harbor::check(send<receive,"happens-before clock condition");
    auto reply=b.tick(); a.receive(reply);
    harbor::check(a.value()>reply,"causal merge");
    std::cout<<"Logical timestamps order message causality, not elapsed seconds.\n";
}
```

Run and interpret

Build and run ch10. The increasing values reflect the local increment and receive rules. They demonstrate the particular causal chain constructed by the example. They do not prove that arbitrary distributed events are globally synchronized.

Trace the receive operation explicitly. If A holds 3 and receives 8, the new value must be 9, not 4 and not 8. Incrementing only the local counter would fail to represent the receive edge. Copying the incoming value without incrementing would fail to distinguish the receive event from the send event under this clock rule.

The model test also exercises overflow. Rejecting a transition at the maximum counter leaves the application with a failure it must handle; it does not offer continued operation with invalid ordering. A persistent service should stop or change its defined identity domain before a counter can wrap.

Failure clinic — A sorted log tells the wrong story

Imagine that gateway logs show a response at 12:00:00.100 and worker logs show the corresponding commit at 12:00:00.120. It is tempting to conclude that the response preceded the commit. Clock offset, timestamp placement, and buffering can explain the apparent inversion. Correlation identifiers and causal instrumentation are stronger evidence than simply sorting wall timestamps.

Record both the event's meaning and where it is measured: "commit returned," "response send completed locally," and "client received response" are distinct points. Even perfectly synchronized clocks cannot fix misleading event labels.

Engineering case — Reconstruct a partial order without inventing a timeline

Consider four events. A1 is a local event on process A. A2 sends a message from A to B. B1 receives that message. C1 is an unrelated local event on process C. Program order establishes A1 before A2, and communication establishes A2 before B1. Nothing in this description orders C1 relative to the other events.

Logical clocks might assign A1 the value 1, A2 the value 2, B1 the value 3, and C1 the value 1. A deterministic display can sort by (`clock`, `process_id`), but that display adds an ordering convention for C1. It does not prove that C1 happened before or after A2 in physical time.

Now suppose a log collector receives B1 first, C1 second, and A2 third. Collector arrival order is another observation, affected by buffering and transport. It does not reverse the causal edge from A2 to B1. A sound incident analysis retains event identity and causal relationships instead of treating ingestion order as execution order.

Wall timestamps can add useful approximate context. If clocks are synchronized within a known bound, sufficiently separated timestamps can support additional reasoning. But a design must know that bound and its failure behavior before relying on it for correctness. "NTP is installed" is not a quantitative clock uncertainty contract.

Use this case to distinguish three questions: what caused what, what order a display chooses, and how much elapsed time a request consumed. Logical clocks help with the first; a tie-break can help with the second; a suitable elapsed-time measurement helps with the third.

For HarborDispatch, the most useful recovery relation is often state-based rather than timestamp-based: a matching worker receipt proves a particular local effect was committed, and a matching gateway result proves completion was recorded there. The exact ordering of unrelated log lines does not change those durable facts.

A review exercise can ask readers to draw the event graph and list only the orderings it supports. Then add a message from B to C and observe how one new edge changes what can be inferred. This trains a habit of deriving causality from mechanisms rather than from the appearance of a sorted report.

Practice and implementation lab

Add a third logical clock. Construct one message path A to B to C and another event on A that is not communicated to C. Show which orderings follow from the graph and which cannot be inferred. Do not use random sleeps as a correctness test; construct the event sequence directly.

Then write a recovery policy for a participant restart. Specify whether its identity changes, whether its counter is persisted, and how a delayed message from the previous incarnation is interpreted. Include a test where the old message arrives after the restart.

Review and solution reasoning

A logical clock preserves a direction of implication: causal precedence implies increasing logical time. An unrelated pair can also have increasing values. A correct explanation therefore names the communication edges instead of treating every numeric comparison as causality.

A timeout can justify releasing a local connection budget while leaving the remote effect uncertain. Stable operation identity and a status or replay protocol address that uncertainty. They answer a different question from failure detection.

Chapter takeaway. State the failure model, choose clocks for their actual purpose, and preserve the difference between suspicion, ordering, and proof of an effect.

Chapter 11 — Choosing Consistency and Replication Contracts

A service adds a read replica and becomes faster until a client submits a job, immediately checks its status, and receives `MISSING`. Both servers may be behaving according to their local state. The architecture failed to tell the client what a read after a write was allowed to observe.

Replication creates more copies, but the meaning of those copies depends on acknowledgement, ordering, failure recovery, and read routing. This chapter develops a vocabulary for those choices without pretending that a few vector operations implement a replicated database.

11.1 Making Read and Write Guarantees Explicit

11.1.1 Distinguishing Linearizable, Sequential, and Eventual Views

A linearizable operation behaves as if it took effect at one point between its invocation and completion, while respecting real-time precedence between completed operations. Sequential consistency preserves a single order consistent with each participant's program order but does not add the same real-time requirement. Eventual consistency describes convergence under suitable conditions; by itself it leaves many details of intermediate reads and conflicts unspecified.

These definitions should lead to concrete API questions. After `SUBMIT key 8` returns accepted, must a subsequent `STATUS key` on any eligible endpoint find that job? May an old replica report missing? Does the client carry a revision that a replica must reach before answering? A name such as “strong read” is less useful than an observable rule for this history.

Different operations can have different contracts. A public activity counter may tolerate delay while a uniqueness check cannot. A service that offers both modes should make the choice explicit rather than silently changing behavior according to which server happens to answer.

The companion's gateway stores accepted intent in its local SQLite database and serves status from that database. It does not replicate this state. A success observed on one gateway cannot be generalized to a different gateway with a different file. Replication terminology in this chapter is architectural instruction, not a claim that the capstone is a cluster.

11.1.2 Describing Session Guarantees and Read Dependencies

Read-your-writes means a session can observe its own completed writes under the service's stated conditions. Monotonic reads prevent a session from moving backward through versions already observed. These are useful requirements when a fully global ordering is unnecessary, but they still need implementation mechanisms.

One mechanism is to return a committed revision with a write and require later reads to reach at least that revision. A lagging replica can wait within the request budget, redirect, or return an explicit unavailable response. Returning stale data while ignoring the revision defeats the contract even if the replica is otherwise healthy.

Session affinity can reduce stale reads but is not automatically a guarantee. The selected instance can restart, the connection can break, or a load balancer can move the session. If the guarantee depends on one instance, the recovery behavior must say what happens when that instance is unavailable.

A read dependency should be carried as data, not inferred from a client address. Two users behind one proxy can share an address, and one user's requests can arrive through several connections. A revision or session token needs integrity and ownership rules if it affects routing or admission.

NEW SECTION EXAMPLE

Enforce read-your-writes with a minimum applied version

A client carries the committed version it needs; a lagging replica refuses a stale read instead of pretending it meets the session guarantee.

main.cpp

```
#include <optional>
#include <iostream>

int main() {
    struct Replica{unsigned applied;int value;
        std::optional<int> read(unsigned required)const{if(applied<required)return{};return
value;}};
    Replica stale{7,10},fresh{8,20};unsigned client_version=8;
    auto a=stale.read(client_version),b=fresh.read(client_version);
    std::cout<<"stale-rejected="<<std::boolalpha<<!a<<" current="<<*b<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The client's version8 records its dependency on a preceding write. A replica only at7 cannot satisfy that read even if it is healthy and fast.

This implements a local minimum-version gate, not linearizability for a whole service. The system still needs a trustworthy version domain, routing/waiting policy, and an authoritative commit rule.

Expected behavior or output

```
stale-rejected=true current=20
```

Concrete experiment

Advance the stale replica to version8 with value20 and rerun. It should become eligible without changing the client's required version.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

11.2 Understanding What Replication Acknowledges

11.2.1 Separating Receipt, Persistence, and Application

A follower can receive bytes, append them to a log, flush them according to a storage contract, and apply them to its state machine at different moments. “The follower has the write” is ambiguous unless it identifies the stage. A read from that follower can lag even after it has received the log entry.

The leader’s acknowledgement policy defines which failures can lose an acknowledged write. Acknowledging after only local memory differs from acknowledging after durable storage on a quorum. Neither policy is meaningfully evaluated without naming the failure domains: several processes on the same disk are not independent durable copies against loss of that disk.

Replication also copies more than business rows. Idempotency records, sequence positions, schema metadata, and authorization-relevant state may participate in the correctness boundary. If a new leader has the job effect but not its receipt, a retry can produce a duplicate. If it has a receipt whose referenced effect was not retained, it can claim work that disappeared.

A complete replicated state machine therefore defines the atomic relationship among these records. Copying only the visible result column because it seems important can break the semantics that made the original single-node transaction safe.

11.2.2 Recovering Lagging Replicas Without Inventing Authority

A lagging replica may replay a retained log or install a snapshot and then catch up from a corresponding position. The snapshot must represent a consistent state, and the replay position must match it. Combining a snapshot from one point with an unrelated log cursor can omit or repeat changes.

A replica that has not caught up should not necessarily serve all request classes. It might answer health checks, expose progress, or serve explicitly stale analytics. Routing a freshness-sensitive read to it merely because its process is alive turns recovery into a user-visible consistency defect.

Retention limits matter. If the leader deletes history before a slow replica consumes it, that replica needs a fresh snapshot or another recovery path. An operator needs to know whether the lag is a brief delay, an incompatible schema, a disk problem, or a gap beyond retained history.

Membership changes require their own authority protocol. Removing a slow participant and adding a replacement changes the set of machines whose acknowledgements count. A safe membership transition cannot be reduced to editing every process’s list independently and hoping those edits occur simultaneously.

NEW SECTION EXAMPLE

Count acknowledgements at the required durability boundary

Ignore receipt-only acknowledgements when deciding whether a quorum persisted an entry.

main.cpp

```

#include <array>
#include <set>
#include <iostream>

int main() {
    enum class Stage{received,persisted,applied};
    struct Ack{unsigned node,index;Stage stage;};
    std::array acks{Ack{1,9,Stage::persisted},Ack{2,9,Stage::received},Ack{1,9,Stage::applied},
    Ack{3,9,Stage::persisted}};
    std::set<unsigned> durable;
    for(auto a:acks)if(a.index==9&&a.stage!=Stage::received)durable.insert(a.node);
    std::cout<<"distinct-durable="<<durable.size()<<"
    majority-of3="<<std::boolalpha<<(durable.size())>=2)<<'\n';
}

```

Run or inspect

```

g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example

```

Explanation

Node1's applied acknowledgement does not count as a second node: the set deduplicates identities. Node2 only received the entry and is excluded from the persisted quorum.

This arithmetic is a focused acknowledgement check, not a consensus protocol. Epochs, membership configuration, log matching, and recovery rules are also required before a real replicated system can commit safely.

Expected behavior or output

```
distinct-durable=2 majority-of3=true
```

Concrete experiment

Change node3's stage to received and expect no durable majority. Do not count bytes buffered in a replica process as a persisted vote.

Validation of this new example: gcc: passed

Technical references: CPP20, RAFT. See the added references and revision index.

11.3 Reasoning About Partitions and Availability

11.3.1 Applying CAP to the Failure Case It Describes

The CAP result concerns the inability to guarantee both the specified consistency and availability properties in a system that can experience network partitions. Its availability notion is not simply an uptime percentage, and its consistency notion is not a generic synonym for valid data. Treating it as a permanent instruction to “pick any two” obscures the actual partition case. (R17)

During a partition, two groups may be unable to communicate. A design that requires a single ordered authority may reject some operations on the side that cannot establish that authority. A design that accepts updates independently must define how conflicts are represented and reconciled, if reconciliation is possible for the operation.

Some operations combine naturally; others do not. Independent additions to a suitable counter can be easier to merge than two claims on the last seat in a venue. The business invariant determines whether concurrent acceptance is acceptable. A generic merge function cannot undo every externally observed promise.

For HarborDispatch, allowing isolated workers with independent receipt stores to execute the same key would expand availability but weaken the promised effect boundary. Calling the system “eventually consistent” would not explain how a duplicate external action should be repaired.

11.3.2 Choosing Degradation Rather Than Hiding It

A service can degrade selectively. It may continue serving cached reference data while refusing mutations, accept durable intent without claiming completion, or return an explicit freshness limit. This is often more useful than pretending all operations share one availability decision.

The accepted-pending-completed distinction in HarborDispatch is a small example. The gateway can accept bounded intent while the worker is unavailable, but it does not claim completion. Once its pending quota is exhausted, it returns BUSY; unbounded acceptance would merely convert a dependency outage into disk or memory exhaustion.

Degradation must remain observable. Operators need to distinguish “no new jobs were submitted” from “all new jobs were rejected,” and clients need to distinguish “not found” from “cannot obtain a sufficiently current read.” Collapsing these outcomes into one empty response produces misleading dashboards and unsafe retries.

Recovery also needs a policy for queued work. When a dependency returns, replay can overwhelm it. Bound concurrent recovery, preserve fairness where required, and measure the time until backlog clears. Restored connectivity does not instantly restore normal service if hours of work remain pending.

Make partition degradation explicit at a write boundary

Classify a write under majority availability without presenting an unavailable write as a successful local update.

main.cpp

```
#include <array>
#include <string_view>
#include <iostream>

int main() {
    auto write_policy=[](unsigned reachable)->std::string_view{return
    reachable>=2?"may-propose":"unavailable";};
    for(unsigned reachable:std::array{1u,2u,3u})std::cout<<reachable<<" reachable:
"<<write_policy(reachable)<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

A three-member majority requires two participants. The one-node partition returns unavailable instead of silently acknowledging a write that cannot meet the advertised quorum contract.

“May-propose” is intentionally weaker than committed: reachability alone does not prove persistence or correct leadership. CAP reasoning concerns behavior under a partition; this local policy model does not measure normal-case availability.

Expected behavior or output

```
1 reachable: unavailable
2 reachable: may-propose
3 reachable: may-propose
```

Concrete experiment

Add a separately named stale-read mode that includes its version. Do not use the same success label for current quorum-backed state and stale local state.

Validation of this new example: gcc: passed

Technical references: CPP20, RAFT. See the added references and revision index.

Worked example — Expose a stale replica deliberately

Requirement and scope

Construct a local leader/follower model in which an accepted append is visible on the leader before it is copied to the follower. The purpose is to demonstrate a possible observation, not to implement leader election, quorum persistence, or a network protocol.

Source — source/examples/ch11.cpp

```
#include "harbor/check.hpp"
#include <iostream>
#include <vector>
struct Replica { std::vector<int> log; int read() const { return log.empty()?0:log.back(); }
};
int main() {
    Replica leader,follower;
    leader.log.push_back(7); // local commit model, not a consensus implementation
    harbor::check(leader.read()==7 && follower.read()==0,"stale read witness");
    follower.log=leader.log;
    harbor::check(follower.read()==7,"eventual catch-up in this schedule");
    std::cout<<"A copied log converges after delivery; it does not prove linearizability.\n";
}
```

Evidence and interpretation

The initial follower read remains behind. After the explicit copy, it observes the copied state. This trace is useful because it makes the missing mechanism visible: nothing in a local append automatically updates another object, much less another machine.

A production replication system would have to decide when to acknowledge the append, how to identify the log position, how to recover an interrupted transfer, and when the follower becomes eligible for a particular read. The model deliberately supplies none of those guarantees.

Failure clinic — Stale absence is treated as permission

Suppose an application reads a replica, sees that key job81 is absent, and then creates it elsewhere. A stale negative read is not a uniqueness lock. Enforce uniqueness at the authority that serializes the relevant mutation, and let the transaction return an existing or conflicting record. The caller’s preliminary read may be a convenience, but it cannot replace that check.

Engineering case — Compare two acknowledgement policies under one crash

Imagine a three-replica store with a leader L and followers F1 and F2. A new record exists only in L’s memory when L responds success. L then loses both process memory and its local disk. Neither follower has the record. The client observed success under a policy that did not preserve the write against this failure.

Now consider a different policy requiring the relevant durable replication and commitment conditions before success. That policy may survive the same leader loss if the surviving replicas retain the committed state and the protocol safely elects a new leader. The difference is not the number of boxes in the diagram; it is the acknowledgement rule and recovery protocol.

Do not shorten the second policy to “two copies means safe.” The copies may share a failure domain, the follower may only have received bytes without persisting them, or the new leader selection may ignore the committed log requirement. Quorum arithmetic must be connected to actual persistence and protocol rules.

A read adds another stage. F1 might have persisted a log entry but not yet applied it to its query state. Routing a read to F1 without an applied-revision check can still return an older value. Replication receipt, commit, application, and read eligibility remain distinct.

For an idempotent operation, include the receipt in the replicated state. If only the visible result survives, a retry may be indistinguishable from a new operation. If only the receipt survives but its referenced effect is missing, the service can replay a false success. The atomic relationship among these records is part of what replication must preserve.

The companion's replica model deliberately omits all of this machinery. Its purpose is to make lag visible, not to imply a high-availability store. The integrated service uses local SQLite files and states its recovery scope accordingly.

A useful design worksheet names five moments: leader admission, local persistence, follower persistence, commitment, and read application. For each client-visible response, mark the moments that must have occurred. That worksheet reveals ambiguous success language before a live failover test turns it into a data-loss incident.

Practice and implementation lab

Extend the model with an applied revision. Have the client request a minimum revision after a write. Return one of three outcomes: a value meeting the requirement, an explicit wait-needed result, or an unavailable result after a simulated budget expires. Do not silently return an older value.

Write a separate history for two partitioned groups accepting the same unique reservation. Identify the exact business promise each group made and whether any merge can satisfy both. This exercise should show why conflict resolution depends on domain semantics.

Review and solution reasoning

A correct revision-aware read checks its required revision before selecting data. Copying the latest value available locally is insufficient if the contract names a newer committed state. A test should include both positive and negative reads because stale absence can be especially dangerous.

A replica's existence does not establish independent durability, read freshness, or safe failover. Each property requires a protocol and a failure-domain statement. These distinctions prepare the next chapter's discussion of consensus and fencing.

Chapter takeaway. Replicate a defined state and acknowledgement contract; do not mistake additional copies for an automatic consistency or availability guarantee.

Chapter 12 — Understanding Consensus, Leases, and Fencing

An old leader pauses for several seconds. A coordinator declares it unhealthy and appoints a replacement. The old process resumes and writes to storage. The replacement is also writing. Both processes can be locally convinced that their actions are legitimate.

This failure does not require malicious behavior. Scheduling delay, a long pause, or a partition can produce it. A safe design must distinguish choosing an authority from enforcing that authority at the resource being changed.

12.1 Identifying the State That Requires Agreement

12.1.1 Distinguishing Replicated Decisions from Broadcast Messages

Broadcasting a message to several machines does not establish that they agree on one committed order. Recipients can miss different messages, restart at different times, and disagree about which sender was authoritative. Consensus addresses agreement under a defined failure model, with safety and progress conditions that depend on the protocol's assumptions.

Raft organizes a replicated log around terms, leader election, log replication, and commitment rules. Its paper explains the constraints that let committed entries survive leadership changes. The rules matter as a whole; adopting only the idea of a numbered leader term is not an implementation of Raft. (R16)

For a replicated HarborDispatch, the agreed state would include the job meaning and receipt semantics, not just a list of workers. A leader's decision to accept a key and input must be preserved if future leaders are expected to recognize retries. The state machine must apply the same committed operation consistently on each replica.

Use a maintained, appropriate coordination or storage system rather than creating a production consensus protocol from a tutorial example. The book supplies local models for understanding authority checks. It does not supply a Raft library, a replicated database, or a proof of distributed consensus.

12.1.2 Separating Safety from the Conditions for Progress

Safety asks whether conflicting committed decisions can occur. Progress asks whether an operation eventually completes when the required conditions hold. A protocol can preserve safety by declining to proceed when it cannot obtain sufficient authority. That behavior may be operationally inconvenient but is different from returning an unsafe success.

A majority-based quorum relies on intersection properties among the relevant sets. In a fixed five-member cluster, a majority has three members. Two such majorities overlap, but that arithmetic alone does not implement a safe log protocol. Members

must enforce voting and log rules, and membership changes need a transition mechanism that preserves the required relationships.

A partition with only two of five members cannot manufacture the missing authority by reducing its local timeout. A shorter timeout can change when it suspects failure, not how many independent votes it possesses. Reconfiguring membership during an incident is therefore a safety-sensitive operation, not a routine way to silence an availability alert.

Operational runbooks should state which failures the cluster can tolerate, how to inspect membership and commitment progress, and when to stop rather than force recovery. An emergency procedure that discards committed history may be a disaster-recovery decision requiring explicit data-loss acknowledgement, not ordinary failover.

NEW SECTION EXAMPLE

Apply only the committed prefix of a replicated decision log

Keep received-but-uncommitted operations from changing application state.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    struct Entry{unsigned index;int delta;};const std::array
log{Entry{1,5},Entry{2,-2},Entry{3,100}};
    unsigned committed=2,applied=0;int value=0;
    for(auto e:log)if(e.index<=committed&&e.index>applied){value+=e.delta;applied=e.index;}
    std::cout<<"received="<<log.size()<<" applied="<<applied<<" value="<<value<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Only indices up to committed are applied. The received 100-delta at index 3 is not yet an agreed application decision and must not appear in the visible state.

Advancing committed requires a real protocol, not this loop. The example isolates ordered application of an already established prefix; deterministic state machines must interpret the same committed commands identically.

Expected behavior or output

```
received=3 applied=2 value=3
```

Concrete experiment

Advance committed to 3 and apply only unapplied entries. The result should be 103, not 106; already applied entries must not execute twice.

Validation of this new example: gcc: passed

Technical references: CPP20, RAFT. See the added references and revision index.

12.2 Interpreting Locks and Leases Correctly

12.2.1 Treating Lease Expiration as an Authority Event

A lease grants authority for a bounded interval under the coordinator’s rules. Its expiration can let another participant obtain authority. It does not forcibly stop the previous holder’s CPU, revoke every buffered I/O operation, or erase a delayed request already in the network.

Suppose worker A holds a lease, pauses, and misses renewal. Worker B obtains the next lease. A later resumes with stale local memory indicating ownership. Checking a local Boolean before writing is not enough. Even checking the lease service just before writing leaves a gap between the check and the external action unless the resource can enforce the decision atomically.

Lease duration also trades failure detection speed against false suspicion and renewal overhead. An interval shorter than ordinary scheduling or communication delays can create unnecessary turnover. A long interval can delay recovery. Neither setting substitutes for a resource-side authority check.

The semantics of a real coordination system should be read precisely. For example, etcd documents guarantees for its API operations and identifies distinctions among read modes and other behaviors. Do not assume that every watch notification or local cache has the same authority as a completed operation with the required consistency. (R27)

12.2.2 Understanding Why a Distributed Mutex Is Not a Process Stop

A mutex in one address space controls participating threads that obey its protocol. A distributed lock similarly controls participants only through its defined interfaces. It cannot prevent an old process from writing directly to an unrelated storage API that ignores the lock.

Ask where enforcement occurs. If a coordinator grants a lock but the target storage accepts any request with network access, the lock is advisory from the storage’s perspective. Correctness depends on every writer cooperating and on the timing assumptions of that cooperation.

A safer design can attach a monotonically increasing fencing token to each ownership grant and require the protected resource to reject older tokens. The grant system provides ordered authority; the resource performs the decisive comparison. The two responsibilities are complementary.

Not every external service supports fencing. An email provider or physical device may expose an API without a generation check. In that case, the architecture needs a different authority boundary, an idempotency facility, a serializing adapter, or a carefully stated limitation. Renaming an advisory lock “exclusive” does not create enforcement.

NEW SECTION EXAMPLE

Reject a late lease holder at the resource boundary

An expired worker can still be running; versioned authority, not its cooperation, protects the resource.

main.cpp

```
#include <cstdint>
#include <string>
#include <iostream>

int main() {
    struct Resource{std::uint64_t fence=0;std::string value;
        bool write(std::uint64_t token,std::string next){if(token<fence)return
false;fence=token;value=std::move(next);return true;}};
    Resource r;r.write(10,"old");r.write(11,"new-owner");bool
late=r.write(10,"late-old-write");
    std::cout<<"late-accepted="<<std::boolalpha<<late<<" value="<<r.value<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The resource remembers highest authority11 after the new owner writes. The old worker's delayedtoken10 request is rejected even though that process did not stop when its lease expired.

Fencing values must come from a trusted monotonic authority, and the resource must persist its fence as required by recovery. This in-memory fixture illustrates enforcement; it is not a lease service.

Expected behavior or output

```
late-accepted=false value=new-owner
```

Concrete experiment

Delete the resource-side comparison and observe the late overwrite. Increasing a client-side lease timeout would not repair that missing enforcement.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

12.3 Enforcing Generations at the Resource Boundary

12.3.1 Designing Monotonic Fencing Checks

The local model keeps the highest accepted token. A write with a smaller token is rejected. A write with an equal token is allowed because the current holder can perform multiple operations; a larger token advances the accepted generation. The comparison and state update belong in one atomic resource operation in a real implementation.

Imagine writes with tokens 7, then 8, then a delayed 7. The last write must not overwrite the value accepted from generation 8. This is exactly the scenario the chapter's test constructs. It avoids relying on timing by presenting the delayed write explicitly.

The token issuer is outside the model. A client must not be allowed to invent an enormous token and permanently exclude legitimate owners. Authentication, authorization, grant ordering, and token durability are necessary parts of an actual system. An unsigned integer by itself has no trustworthy authority.

Counter wraparound is also an authority problem. A restarted issuer that reuses smaller tokens can make valid new holders look stale. Preserve the ordering domain or adopt a formally defined new generation mechanism. Do not reset the counter merely because a configuration file was recreated.

12.3.2 Preserving Fences Through Recovery and State Transfer

If the protected resource restarts and forgets its highest accepted token, a delayed old write can become acceptable again. Persist the fence with the protected state or recover it from an authority before accepting mutations. The same rule applies when moving data to another shard or restoring a backup.

A backup can contain an older fence than clients currently hold. Recovery must account for outstanding writers and tokens. Isolating the restored service, revoking or advancing authority, and reconciling external state may be required before traffic resumes. Data restoration and writer authorization are not independent checkboxes.

Replicating only the value while omitting the fence also weakens the guarantee. The resource state includes the information used to reject stale authority. Treat that metadata as part of the correctness model, not as an expendable cache.

For HarborDispatch, the worker receipt table prevents repeat application of a stable key within one database. It does not implement lease ownership or fencing across independent worker databases. Adding the model's token check to one process would not turn the application into a safe multi-leader service. The complete authority and persistence design must come first.

Restore a fencing watermark together with the protected data

Persisted state transfer must include the highest accepted generation, not only the last value.

main.cpp

```
#include <string>
#include <iostream>
#include <cstdint>

int main() {
    struct Snapshot{std::uint64_t fence;std::string value;};
    Snapshot durable{44,"committed"};Snapshot restored=durable;
    auto write=[&](std::uint64_t f,std::string v){if(f<restored.fence)return
false;restored={f,std::move(v)};return true;};
    bool stale=write(43,"obsolete"),current=write(45,"replacement");
    std::cout<<std::boolalpha<<"stale="<<stale<<" current="<<current<<"
fence="<<restored.fence<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The snapshot includes both value and fence. Restoring only the string while resetting fence to zero would let obsolete writers regain authority after restart.

This is a state-transfer model with an already trusted snapshot. Real durability requires atomic publication of data and watermark, and equal-generation replay needs operation-level idempotency when repeated writes can have effects.

Expected behavior or output

```
stale=false current=true fence=45
```

Concrete experiment

Create a deliberately defective restore that initializes fence=0 and demonstrate the stale43 write is accepted. Keep that regression test beside the corrected restore.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Reject a resumed stale owner

Requirement and boundary

Accept generation 7, then generation 8. Present another write from generation 7 and verify that it cannot replace generation 8's value. This is a deterministic local model of the resource-side rule only.

Source — source/examples/ch12.cpp

```
#include "harbor/check.hpp"
#include "harbor/models.hpp"
#include <iostream>
int main() {
    harbor::FencedRegister storage;
    harbor::check(storage.write(7,"old owner"),"initial term");
    harbor::check(storage.write(8,"new owner"),"new term");
    harbor::check(!storage.write(7,"late stale write"),"fence stale writer");
    harbor::check(storage.value()=="new owner","value preserved");
    std::cout<<"Storage rejected an old epoch; token issuance still requires authority.\n";
}
```

Interpret the evidence

The successful test shows that the comparator preserves the highest generation in this object for the supplied history. It does not show that tokens were issued safely, authenticated, replicated, or persisted. Those are separate proof and test obligations.

An equal-token write remains acceptable by design. A fencing token orders ownership generations, not individual operations within one holder's lifetime. If those operations also require order or deduplication, include an operation sequence or identity under a separate rule.

Failure clinic — Check-then-write leaves a race

A client asks the coordinator whether it still holds the lease, receives yes, and then pauses. The lease expires before the client writes. A second client obtains a newer lease and writes first. The old client's eventual write remains unsafe if storage ignores generation. This trace explains why a successful authority check at time T does not authorize every future action indefinitely.

The repair is not to make the interval between check and write "very small." Move the relevant authority comparison into the atomic resource operation or use an interface whose semantics already provide the necessary protection.

Engineering case — Restore a value without restoring stale authority

A protected register has accepted generation 12 and value B. A backup from earlier contains generation 9 and value A. Restoring both fields accurately reproduces the old snapshot, but it does not automatically make that snapshot safe to expose while writers from later generations still exist.

Suppose an old client with token 10 sends a delayed write after restore. The restored register sees 10 greater than 9 and accepts it, even though generation 12 had previously superseded that authority. The backup is internally consistent yet stale relative to the external writer population.

Recovery must therefore coordinate state and authority. One approach isolates the restored resource, establishes a new authority generation above all relevant prior grants through the actual coordinator, and requires writers to reacquire permission. Another uses a recovery protocol that preserves the highest fence independently. The

correct mechanism depends on the system's failure model and token issuer.

A client-provided maximum token cannot safely solve this problem because an untrusted client could claim any number. The resource must trust the grant authority, and the authority must have its own persistence and recovery semantics. Fencing transfers trust to a controlled issuer; it does not eliminate trust.

This case also explains why resetting a counter during a deployment is not a harmless cleanup. Numbers that order authority are part of durable protocol state. They need a defined exhaustion and incarnation strategy, just like database schema versions and log positions.

The local FencedRegister model can demonstrate rejection of smaller tokens during one lifetime. Extending it to backup recovery requires modeling the external population of outstanding grants, not merely serializing two fields. A unit test that restores value and token should be accompanied by a history containing an already issued later token.

For the capstone, receipt recovery has a related but distinct issue: an old worker backup can forget effects that callers already observed. The remedy is not a fencing token pasted onto the request; it is an authoritative recovery and reconciliation procedure for the complete state. Similar-looking metadata can solve different problems, and a design must preserve that distinction.

Practice and implementation lab

Extend the register model with persisted snapshots represented by a pair of value and token. Restore both fields, then replay an old write and verify rejection. Construct a deliberately incomplete snapshot that restores only the value, and explain the violated assumption without deploying it anywhere.

Next specify an issuer API. Describe who can request a grant, how grant generations survive restart, and how a resource validates the association between holder and token. Identify which of these requirements a local unit test can examine and which require integration with an actual authority service.

Review and solution reasoning

A lease and a fence answer different questions. A lease decides who should act now under a coordinator's model. A fence lets a resource reject an actor from an older accepted generation. Neither guarantees that a current actor's business operation is valid; authorization and domain validation still apply.

A consensus library, a database transaction, and a distributed lock are not interchangeable products. Choose the primitive whose guarantees match the state transition, and keep its assumptions visible in the service contract.

Chapter takeaway. Agreement must establish authority, and the protected resource must enforce it even when an old process resumes.

Chapter 13 — Partitioning Data and Migrating Ownership

The worker database has reached its practical capacity, so a team adds a second worker and routes each key by hash. New jobs spread out. A retry for an old job now reaches the new worker, which has never seen its receipt. A change intended to improve capacity has changed the meaning of request identity.

Partitioning is a placement decision and an ownership protocol. This chapter develops both, with a small rendezvous-hashing model that demonstrates routing properties without pretending to move durable state.

13.1 Selecting the Partitioning Boundary

13.1.1 Partitioning by Business Identity Rather Than Convenience

A partition key determines which operations share an authority. Choosing job ID spreads independent jobs, while choosing tenant ID keeps a tenant's jobs together. The first may improve distribution; the second may simplify tenant-level transactions and quotas. Neither is universally correct.

Start with the invariant that must be checked atomically. If two records participate in one uniqueness or balance constraint, placing them on different independent stores changes how that constraint can be enforced. The application may need a coordinator, a redesigned invariant, or a workflow that tolerates intermediate states.

For HarborDispatch, the idempotency key and its receipt must have one defined authority. A partition key based on the canonical job key is a possible extension, but all retries must use the same placement domain. Changing normalization between clients can route equivalent jobs differently. A byte-level key contract belongs in the wire protocol and the placement specification.

Do not include volatile values such as current time, connection ID, or attempt count in a placement key for stable logical work. Those fields vary precisely when a retry needs to find the previous effect. Include an explicit tenant namespace if keys are not globally unique, and bind that namespace to an authenticated identity in a production design.

13.1.2 Recognizing Hot Keys and Unequal Work

Uniform key counts do not imply uniform resource demand. One tenant can produce most of the traffic; one job can require far more computation than another. A partition with few keys can be the bottleneck if those keys are hot or their records are large.

Measure work at the partition boundary: request rate, service time, bytes, lock contention, backlog, and retained state. A histogram of key counts answers a different question from a histogram of CPU demand. Avoid changing the placement

algorithm before determining which resource is actually uneven.

A single hot key cannot always be split without weakening its ordering or atomicity contract. A counter might support partial aggregation, but a sequential workflow with one authoritative state may not. Scaling a hot aggregate sometimes requires changing the business model rather than merely adding workers.

Per-tenant admission can limit one tenant's impact, but it does not create more capacity. A quota should make the overload visible and predictable. It should not silently drop accepted work or treat one tenant's duplicate retry as a new quota-consuming business operation without a deliberate rule.

NEW SECTION EXAMPLE

Partition related records by tenant identity

Use the business owner as the partition key and separately count the work distribution.

main.cpp

```
#include <array>
#include <map>
#include <string>
#include <iostream>

int main() {
    struct Order{int tenant,order;};const std::array
orders{Order{10,1},Order{10,2},Order{10,3},Order{11,4}};
    std::map<int,unsigned> load;
    for(auto o:orders){int partition=o.tenant%2;++load[partition];}
    std::cout<<"tenant10 partition="<<10%2<<" work="<<load[0]<<" tenant11
work="<<load[1]<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

All three orders for tenant10 share a partition, preserving a potential tenant-scoped transaction boundary. Hashing individual order IDs instead could scatter state that must be updated together.

The uneven3-to1 count demonstrates a hot-tenant risk despite a simple two-way key distribution. Balanced key counts and balanced processing costs are different requirements.

Expected behavior or output

```
tenant10 partition=0 work=3 tenant11 work=1
```

Concrete experiment

Give tenant10 a thousand orders. Explain why adding partitions does not split one indivisible hot key unless the business ownership model also changes.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

13.2 Implementing Stable Placement

13.2.1 Comparing Modulo Assignment and Rendezvous Selection

A simple placement function can compute $\text{hash}(\text{key}) \% N$. Changing N often remaps many keys. That may be acceptable for stateless work, but it creates substantial movement when keys own durable data. Stable placement techniques aim to limit changes when membership changes.

The companion's rendezvous model scores each key-node pair and selects the highest score, with a deterministic lexical tie-break. Adding a node can move a key to the new node only when that node outranks the previous winner; a key should not jump from old node A to old node B merely because a new candidate was added.

This movement property follows from the selection rule: scores among existing candidates do not change. It does not establish ideal distribution for every key set or hash function. The supplied stable hash is a small noncryptographic teaching implementation. It must not be used for signatures, credential storage, or adversarially robust partition assignment.

A real placement specification also defines node identifiers, byte encoding, algorithm version, and tie-breaking. `std::hash` is not a portable persisted-routing specification across every implementation. Use a deliberately selected stable algorithm and test golden key-to-node vectors across supported clients.

13.2.2 Versioning the Membership and Algorithm Together

A placement result depends on both the algorithm and the membership snapshot. Two clients with different snapshots can choose different owners for the same key even when they run identical code. Attach a placement epoch or revision to the routing decision and define how servers handle outdated epochs.

A server can reject a stale routing epoch and provide an authenticated redirect under a controlled protocol. It can also forward to the current owner, but forwarding introduces loops, additional budgets, and trust questions. An unbounded chain of redirects is not a recovery strategy.

Algorithm changes deserve the same treatment as membership changes. Replacing a hash implementation can remap the entire keyspace while the node list remains unchanged. Record the algorithm version in migration plans and deployment compatibility tests.

The chapter executable tests placement independently of network connections. It verifies membership-order independence and the added-node movement property for a finite key set. It does not claim that clients in a live cluster all possess the same membership revision.

NEW SECTION EXAMPLE

Use rendezvous scores to minimize movement on node removal

Choose the highest stable key/node score and verify that removing a non-winning node leaves placement unchanged.

main.cpp

```
#include <array>
#include <cstdint>
#include <string_view>
#include <vector>
#include <algorithm>
#include <iostream>

int main() {
    auto score=[](std::string_view key,char node){std::uint64_t h=14695981039346656037ull;
        for(unsigned char
c:key){h^=c;h*=1099511628211ull;}h^=unsigned(node);h*=1099511628211ull;return h;};
    auto owner=[&](std::string_view key,const std::vector<char>&nodes){return
*std::max_element(nodes.begin(),nodes.end(),[&](char a,char b){return
score(key,a)<score(key,b);}});};
    std::vector<char> nodes{'a','b','c'};char first=owner("tenant42",nodes);
    char removed=first=='a'?'b':'a';nodes.erase(std::find(nodes.begin(),nodes.end(),removed));
    char second=owner("tenant42",nodes);if(first!=second)return 1;
    std::cout<<"non-owner removal preserves placement\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The score depends on the complete key and each node identity, not the node's vector position. Removing a node that did not win cannot change the maximum among the remaining scores.

The explicit unsigned hash arithmetic is stable for this ASCII fixture; `std::hash` would not be a durable cross-version placement specification. Real deployments must version encode, score algorithm, node IDs, and membership, including tie rules and empty membership.

Expected behavior or output

```
non-owner removal preserves placement
```

Concrete experiment

Remove the winning node instead and verify exactly this key moves to the next-highest scorer. Test empty membership before calling owner.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

13.3 Moving State Before Changing Authority

13.3.1 Defining a Migration State Machine

A migration needs named phases, such as prepare, copy, catch up, transfer authority, verify, and retire. The exact protocol depends on the storage system, but the sequence must explain which owner accepts writes at each point and how writes occurring during the copy are retained.

A read-only copy taken while writes continue becomes stale immediately. A snapshot plus change log can close that gap if the cursor and snapshot are consistent. Alternatively, a small system may pause writes during a controlled move. That is an availability tradeoff, not a technical embarrassment; it can be safer than inventing an untested online migration protocol.

Move all state that participates in semantics: business records, receipts, version counters, tombstones, and fencing information. Omitting deduplication history can make the new owner repeat an already applied operation. Omitting deletions can resurrect removed records.

For the lab application, the simplest safe manual relocation is to stop the relevant processes, preserve the database and its consistent state, move it to a controlled local destination, then restart with the same authority and retained receipts. This does not demonstrate online sharding. It is a deliberately narrower operation that can be understood and tested.

13.3.2 Avoiding Unsafe Dual Writes and Premature Cleanup

Writing to both old and new stores seems like an easy migration strategy until one write succeeds and the other fails. The caller must then decide which store is authoritative, how to repair divergence, and whether to return success. Without a protocol, dual writes merely add another distributed transaction problem.

A safer architecture might use one authoritative log or outbox to drive the secondary copy, with a tracked position and reconciliation. Cutover occurs only when the new owner has reached the required state and authority is transferred under a defined rule. Even then, delayed old requests must be fenced or redirected safely.

Keep the previous state long enough to investigate and recover, but do not mistake a stale backup for a safe rollback target. After the new owner accepts writes, reverting to the old copy can lose those writes. Rollback may require reverse catch-up or a forward repair rather than a simple address change.

Deletion of old receipts is especially sensitive. A long-delayed retry may arrive after migration. Define the deduplication retention window and how requests outside it are handled. Storage reclamation changes the temporal boundary of idempotency; it is not just housekeeping.

Fence writes before handing a shard to a new owner

Represent migration as copy, catch-up, freeze, and cutover phases rather than changing a routing label immediately.

main.cpp

```
#include <string>
#include <iostream>

int main() {
    enum class Phase{copying,caught_up,frozen,cutover};Phase phase=Phase::copying;
    int source=5,destination=source;std::string owner="old";
    source+=2;destination=source;phase=Phase::caught_up;
    phase=Phase::frozen;bool late_write=phase!=Phase::frozen;
    if(destination==source){owner="new";phase=Phase::cutover;}
    std::cout<<"source="<<source<<" destination="<<destination<<"
    late-write="<<std::boolalpha<<late_write<<" owner="<<owner<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The first copy is not sufficient because the source can change afterward. Catch-up brings the destination to 7 before the freeze prevents another old-owner write during cutover.

The enum makes the ordering explicit but does not implement a distributed fence or live replication. A real migration must prove the destination has the required committed position and ensure stale routes cannot bypass the frozen owner.

Expected behavior or output

```
source=7 destination=7 late-write=false owner=new
```

Concrete experiment

Skip catch-up and require the cutover guard to fail while destination remains 5. Never label the new owner authoritative before that equality/version condition is met.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Measure movement when adding a node

Requirement and implementation

Route a finite set of keys over three nodes, then add a fourth. Verify that any moved key selects the new node and that reordering the membership vector does not change a result.

Source — source/examples/ch13.cpp

```
#include "harbor/check.hpp"
#include "harbor/models.hpp"
#include <iostream>
int main() {
    std::vector<std::string> before{"a","b","c"},after=before; after.push_back("d");
    unsigned moved=0;
    for (int i=0;i<200;++i) {
        auto key="job-"+std::to_string(i);
        auto old=harbor::placement(key,before),next=harbor::placement(key,after);
        if (old!=next) { ++moved; harbor::check(next=="d","unrelated reassignment"); }
    }
    std::cout<<"moved<<" of 200 teaching keys moved to the added member.\n";
}
```

Interpret the observation

The executable prints a moved-key count for its particular keys and teaching hash. Treat that count as an observation, not a universal percentage or performance benchmark. The structural property being checked is more important than obtaining an aesthetically pleasing distribution.

The model does not copy any SQLite rows. A successful routing test therefore does not authorize replacing the integrated worker endpoint with this function. Durable receipt ownership and migration must be added before that change preserves the application's semantics.

Failure clinic — A routing rollout outruns a data rollout

Client version A sends key K to worker 1. Client version B sends K to worker 2 after a membership update. Worker 1 already holds the receipt, but worker 2 does not. Both clients can receive locally valid responses while the global business effect is duplicated.

The defect is not in either client's hash arithmetic. The rollout failed to coordinate placement with state authority. A migration test should deliberately run old and new routing views together and verify the protocol's stale-epoch behavior.

Engineering case — Move one key while preserving a late retry

Key route7 belongs to shard A and has a committed receipt. A membership change assigns it to shard B. A client still holding the old map sends a retry to A, while another client with the new map sends the same retry to B.

A safe migration must define both paths. Before authority transfer, A remains the writer and B is a copy in progress. After transfer, B owns complete required state, and A rejects or forwards stale-epoch requests under a bounded protocol. There must not be an interval in which both independent stores treat the key as new.

A copy phase needs a consistent starting point and a way to account for writes during copying. If A commits an additional result after the snapshot, B must catch it up before becoming authoritative. For immutable completed receipts the copy is simpler, but pending state and other mutable metadata still require a rule.

Record a cutover revision and verify B has reached it. Then transfer authority and update routing. Delayed clients do not disappear at cutover, so A's retained redirect or rejection state needs a lifetime compatible with client cache and request retention. Deleting A immediately may turn a recoverable stale route into an unexplained outage.

Rollback after B accepts new work is not simply routing back to A. A may now be stale. A safe rollback requires reverse synchronization or another authority transition that preserves B's accepted state. Calling the old node a "backup" does not make it current.

The rendezvous-hash test supplies only the placement component of this story. It can identify which keys would move, but it cannot establish that their receipts arrived or that old writers were fenced. A migration plan should keep those columns separate: selected destination, copied state, caught-up revision, transferred authority, and verified retry behavior.

For a small system, stopping writes during a controlled move can avoid the complexity of online catch-up. That is a legitimate availability tradeoff when documented and tested. The unsafe choice is to claim an online migration while using a file copy that races with ongoing writes and has no cutover protocol.

Practice and implementation lab

Add an explicit placement revision to a request model. Make a simulated owner reject requests from an obsolete revision. Implement a bounded redirect count and test a two-node redirect loop. The expected result is an explicit failure, not an infinite retry.

Then design a migration worksheet for ten job keys. List the original owner, new owner, copied receipt, copied result, routing epoch, and final verification. Include one job committed during the copy and explain how the chosen protocol captures it.

Review and solution reasoning

A correct answer separates mapping from migration. A deterministic mapping is necessary for consistent routing but cannot prove that the selected node owns complete state. The authority transfer is the point where a new participant becomes allowed to mutate that state.

A hot key is a workload property. Splitting it safely may require a domain redesign; randomizing retries across replicas is not a substitute. Preserve the invariant first, then choose how much parallelism it permits.

Chapter takeaway. Placement tells you where a key should go; a migration protocol makes that destination a legitimate owner.

Chapter 14 — Building Transactional Persistence with SQLite

The gateway accepts a job and then crashes. After restart, the caller asks whether the job exists. If acceptance was only a message in memory, the service has lost an externally visible promise. If acceptance was committed in durable state, the process can recover its obligation even though its threads, sockets, and counters disappeared.

SQLite provides a concrete transactional boundary for this book's local service. The goal is not to turn SQLite into a distributed database. It is to implement a small, inspectable persistence layer and understand exactly which guarantees come from the transaction and which remain outside it.

14.1 Designing State Around Invariants

14.1.1 Encoding Keys, Inputs, and Results as a Schema

The gateway's jobs table contains a primary key, a bounded integer input, and an optional result. A null result means pending. A nonnull result means the gateway has recorded a verified worker result. The worker's receipts table binds the same key to its canonical input and result.

The schema reinforces the domain: keys are unique, inputs remain in the accepted range, and stored results cannot be negative. Application validation still matters because a database constraint rarely expresses every semantic rule. The store verifies that a completed result equals the domain calculation before updating a job.

Choose representations deliberately. The input is parsed into an integer before it becomes the operation's meaning. Text 03 and text 3 therefore represent the same canonical input. An input string with trailing nonnumeric content is rejected. Idempotency comparisons operate on canonical meaning, not arbitrary formatting differences.

Do not store raw C++ object bytes as a database record. Padding, endianness, compiler layout, and object ownership are not a durable schema. Bind defined scalar values and strings through the database API. A row should remain interpretable independently of one process's memory layout.

14.1.2 Placing the Entire Decision in One Transaction

A submission must distinguish new work, a duplicate of the same work, a conflict under the same key, and exhausted pending capacity. If the application checks these conditions outside a transaction and inserts later, concurrent submissions can invalidate the observation.

The store starts an immediate transaction, checks an existing key first, checks capacity only for genuinely new work, and then inserts. This ordering matters when the queue is full: an existing identical request remains a duplicate rather than being

rejected as new capacity consumption.

The uniqueness constraint is a final safeguard, but it does not by itself produce the desired API result. The application must compare the existing input and return duplicate or conflict appropriately. Catching every constraint failure and reporting “already done” would hide changed request meaning.

SQLite documents how explicit transactions begin, commit, and roll back, including distinctions among deferred, immediate, and exclusive starts. The companion uses `BEGIN IMMEDIATE` to acquire the write transaction before its read-check-write sequence. (R18)

NEW SECTION EXAMPLE

Commit bounded jobs under database constraints

Enforce a unique key and bounded input in SQLite, then roll back a transaction when a second row violates the range.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
```

```
~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database db;
    db.exec("CREATE TABLE jobs(id TEXT PRIMARY KEY,input INTEGER NOT NULL CHECK(input BETWEEN
0 AND 1000000),result INTEGER)");
    {Transaction tx(db);db.exec("INSERT INTO jobs VALUES('good',12,144)");tx.commit();}
    try{Transaction tx(db);db.exec("INSERT INTO jobs VALUES('temporary',3,9)");db.exec("INSERT
INTO jobs VALUES('bad',-1,NULL)");tx.commit();return 1;}
    catch(const std::runtime_error&){}
    auto count=scalar(db,"SELECT count(*) FROM jobs");
    if(count!=1)return 2;std::cout<<"committed rows="<<count<<" result="<<scalar(db,"SELECT
result FROM jobs WHERE id='good'")<<"\n";
    }catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

TheCHECK constraint protects the persisted representation even if a caller bypasses application validation. Both writes in the failing transaction are rolled back, including the otherwise validtemporary row.

The transaction destructor performs nonthrowing best-effortROLLBACK unlesscommit completed. Domain-specific result correctness still requires application checks; this schema alone does not prove result=input².

Expected behavior or output

```
committed rows=1 result=144
```

Concrete experiment

Attempt a duplicategoodkey and verify it fails without changing the existing result. Add a result constraint only after defining pendingNULL semantics precisely.

Validation of this new example: gcc: passed

Technical references: SQLITE-C, SQLITE-TRANSACTION. See the added references and revision index.

14.2 Owning Native Database Resources in C++

14.2.1 Wrapping Connections, Statements, and Transactions

A connection owns the database handle. A prepared statement owns its compiled statement handle. A transaction object owns the responsibility to roll back unless commit succeeds. These are separate lifetimes and should appear as separate RAI types or equally explicit scopes.

The implementation prepares parameterized statements and binds values. It does not assemble SQL by concatenating a user key. Binding protects the SQL grammar

boundary, while domain validation protects the business boundary. Both are necessary; a safely bound million-character key can still be an unacceptable resource request.

A transaction destructor must not throw during stack unwinding. It attempts cleanup without converting a prior error into termination. Successful commit is explicit, and a failed commit remains an error rather than being relabeled success because the object is about to be destroyed.

Connection ownership also clarifies threading. The gateway handler and dispatcher use separate connections to the same database file. Each connection has a specific owning execution path. A library's internal threading mode does not make arbitrary application-level sequences atomic across calls.

14.2.2 Distinguishing Busy Conditions from Invalid Data

A write can fail because another connection holds the relevant transaction, because storage is unavailable, because a constraint is violated, or because the statement itself is invalid. These causes should not all trigger the same retry behavior.

The companion configures a bounded busy timeout of 100 ms. This allows some local contention to settle, but it also means a supposedly short callback can spend time waiting in SQLite. The network reactor is nonblocking with respect to socket readiness; its synchronous database handler is not a hard-real-time or fully asynchronous storage engine.

A production adapter should categorize database errors and expose enough context for diagnosis without leaking SQL secrets or raw credentials. A transient lock conflict may be retryable within a budget. A schema mismatch or invalid invariant should fail visibly rather than be retried forever.

Do not hold a transaction while making a remote RPC. Doing so couples database locks to network delay and can exhaust local concurrency during a dependency outage. HarborDispatch commits pending intent first, performs delivery outside the transaction, and records completion in a later transaction. Chapter 18 explains the resulting recovery protocol.

NEW SECTION EXAMPLE

Bind text safely and classify lock contention

Use RAII statements for a quote-containing key, then hold an immediate transaction on one connection while another requests a write.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database first("busy.db");first.exec("CREATE TABLE IF NOT EXISTS jobs(id TEXT PRIMARY
KEY,value INTEGER)");
    {Statement insert(first,"INSERT OR REPLACE INTO jobs
VALUES(?,?)");insert.text(1,"a'quoted");insert.integer(2,7);if(insert.row())return 1;}
    Database second("busy.db");Transaction locked(first);
    int result=sqlite3_exec(second.p,"BEGIN IMMEDIATE",nullptr,nullptr,nullptr);
    if(result!=SQLITE_BUSY)return 2;
    std::cout<<"bound rows="<<scalar(first,"SELECT count(*) FROM jobs")<<"
second-writer=BUSY\n";
}catch(const std::exception&e){std::cerr<<e.what()<<\n';return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

Binding preserves the SQL grammar boundary: the apostrophe in the key is data, not a query delimiter. `SQLITE_TRANSIENT` makes SQLite own a safe copy for statement execution.

`BEGIN IMMEDIATE` reserves the writer on first, so second receives `SQLITE_BUSY` rather than a schema or input error. Retry policy can treat contention differently, but must remain bounded. All statements are destroyed before their connections close.

Expected behavior or output

```
bound rows=1 second-writer=BUSY
```

Concrete experiment

End the first transaction before the second begins and verify the second writer can proceed. Keep an invalid `CHECK` input as a separate nonretryable error case.

Validation of this new example: gcc: passed

Technical references: `SQLITE-C`, `SQLITE-TRANSACTION`. See the added references and revision index.

14.3 Understanding Durability and Recovery Boundaries

14.3.1 Interpreting WAL and Synchronous Settings

Write-ahead logging changes how updates and readers interact. Readers can observe a consistent snapshot while committed changes are represented in the WAL, and checkpointing later transfers appropriate pages into the main database. It does not make SQLite a multi-writer distributed storage system. (R19, R20)

The companion requests WAL mode and `synchronous=FULL`. These settings are part of its intended local durability policy, but actual durability still depends on the filesystem, storage stack, and correct platform behavior. A process-restart test cannot certify arbitrary power-loss behavior or hardware that misreports completed writes. (R21)

A live database may include committed state in files beyond the main `.db` file. Copying that one file while the application is active is not a general backup procedure. Use a supported SQLite backup mechanism or a carefully controlled consistent snapshot. The online backup API exists for this purpose. (R22)

Checkpoint behavior and retained WAL size are operational concerns. Long-running readers can affect checkpoint progress. Monitor storage growth and transaction behavior rather than assuming that a small logical table always occupies a small fixed set of bytes.

14.3.2 Testing Reopen, Rollback, and Concurrent Requests

An in-memory database is useful for fast logic tests, but it disappears with its connection and does not test process recovery. The companion therefore also creates private file-backed databases, closes and reopens them, and verifies retained rows and receipt counts.

The storage tests exercise two connections submitting the same worker operation. The required outcome is one receipt and consistent results for both callers. They also attempt an incorrect completion value and verify that the pending job remains unchanged.

A separate process test begins a transaction, inserts a row, and is killed before commit. Reopening the database must not expose that uncommitted row. This is evidence about interrupted-process transaction recovery in the tested environment. It is deliberately not called a power-failure qualification.

The database files contain accepted work and deduplication history. Losing the gateway file can lose pending obligations; losing the worker file can lose knowledge of completed effects. Back up and recover both authorities according to the business contract, and reconcile them before reopening traffic after a partial restore.

NEW SECTION EXAMPLE

Verify rollback and reopen under WAL configuration

Create a disposable on-disk SQLite database, commit one value, roll back another, and reopen it.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    {Database db("reopen.db");db.exec("PRAGMA journal_mode=WAL; PRAGMA synchronous=FULL;
CREATE TABLE state(value INTEGER)");
        {Transaction tx(db);db.exec("INSERT INTO state VALUES(11)");tx.commit();}
        {Transaction tx(db);db.exec("UPDATE state SET value=99");}
    }
    Database reopened("reopen.db");auto value=scalar(reopened,"SELECT value FROM state");
    if(value!=11)return 1;std::cout<<"reopened committed value="<<value<<"\n";
}catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

WAL and synchronous=FULL are requested before transactions. The first scope commits; the second transaction never commits, so its destructor rolls back before the connection closes.

Reopening proves the committed value survives a clean process-shaped close/reopen and that rollback worked. It does not qualify power-loss behavior, controller caches, or copying a live.db without its WAL. Use SQLite backup APIs for live backup.

Expected behavior or output

reopened committed value=11

Concrete experiment

Move the update into an explicitly committed transaction and expect after reopen. Then add an abrupt-process-exit fixture rather than claiming this clean close simulates a crash.

Validation of this new example: gcc: passed

Technical references: [SQLITE-C](#), [SQLITE-WAL](#), [SQLITE-TRANSACTION](#). See the added references and revision index.

Worked example — Make duplicate acceptance survive a full queue

Requirement and implementation

With one pending slot, accept a new key, recognize its identical duplicate, reject changed input under that key, and reject an additional new key until capacity is released. All decisions occur under the store's transaction rules.

Source — source/examples/ch14.cpp

```
#include "harbor/check.hpp"
#include "harbor/store.hpp"
#include <iostream>
int main() {
    harbor::Store db(":memory:");
    harbor::Request r{"job-1", 12};
    harbor::check(db.submit(r)==harbor::Store::Submit::accepted, "insert");
    harbor::check(db.submit(r)==harbor::Store::Submit::duplicate, "duplicate");
    harbor::check(db.submit({"job-1", 13})==harbor::Store::Submit::conflict, "changed payload");
    harbor::check(db.pending().size()==1, "one committed intent");
    std::cout<<"Unique request identity and payload comparison share a transaction.\n";
}
```

Trace the state changes

The first successful insert creates one pending row. A duplicate is an observation of the existing canonical request and does not create another row. A conflict returns without replacing the original input. Capacity rejection leaves the original row intact.

The example uses `:memory:` to isolate these decisions. The file-backed and process-level tests supply different evidence; the chapter does not treat one as a

substitute for the other. Run `store_transactions` through `CTest` to exercise the persistent store tests.

Failure clinic — Success is returned before commit

If the handler sends `ACCEPTED` before transaction commit, a later commit failure can erase the promise the client already received. Reordering the response after commit closes that particular gap. It does not guarantee the client receives the response; a lost reply remains possible after a successful commit.

The correct response to that second gap is stable identity and lookup or replay, not moving the response before commit again. Distributed uncertainty often cannot be eliminated, but it can be made recoverable.

Engineering case — Explain two connections racing on the same key

Two worker connections receive `EXEC same42 9` nearly together. Each intends to create a receipt for the same canonical request. The required outcome is one retained receipt and the same result returned to both callers, not two independent records or an unexplained error for the second caller.

The store begins an immediate transaction before checking the receipt. One writer obtains the transaction first. It sees no row, inserts the receipt, and commits. The other writer waits according to the configured bounded busy policy, then examines the now-existing row and returns the matching saved result. If contention exceeds that policy, it can fail visibly and be retried safely later with the same identity.

Now change the second request to input 10. It must not receive the saved result for input 9 as if its new work succeeded. The existing row is compared with canonical meaning, and the conflict is reported. The unique key constraint prevents duplication; the application comparison prevents meaning from being overwritten or confused.

A check outside the transaction would not provide the same reasoning. Both connections could observe absence and then race to insert. The constraint would still prevent two rows, but the losing path would need careful error interpretation and a fresh lookup. Keeping the read-check-write decision inside the transaction makes the intended result easier to follow.

The busy timeout is a bound on one database waiting mechanism, not a guarantee that the entire request completes within that many milliseconds. Scheduling, I/O, statement execution, and response transfer add other costs. The reactor's handler can block while the database waits, which is explicitly documented rather than hidden behind nonblocking socket terminology.

The test uses two connections against a private file-backed database and verifies receipt count and returned results. It provides stronger evidence for this local contention path than an in-memory single-connection test. It still does not establish behavior for every filesystem, storage device, or distributed database engine.

The general lesson is to make the serializing authority explicit. In this lab it is SQLite's transaction and uniqueness machinery over one file. Replacing that file with two independent copies removes the authority even if both copies retain exactly the same C++ wrapper code.

Practice and implementation lab

Add a read-only inspection command that reports pending and completed counts without returning every key. Use a transaction or query semantics appropriate to the consistency required. Explain whether two separately executed counts must describe the same snapshot.

Then add a deliberate constraint violation test. Verify the exception path, unchanged state, and ability to perform a later valid operation on the same connection. A failed statement must not leave the application unknowingly inside an abandoned transaction.

Review and solution reasoning

A strong solution identifies the atomic decision, the commit point, and the response point separately. The response cannot cause the transaction to become durable retroactively. Likewise, local durability cannot prove that a remote client observed the result.

WAL, RAIL, parameter binding, and uniqueness constraints address different layers. Their composition produces a useful store, but each has limits. The database remains one local authority with a tested recovery path, not a replicated service merely because two processes can open it.

Chapter takeaway. Commit the whole local invariant, own every native resource, and test the recovery boundary you actually intend to depend on.

Chapter 15 — Migrating Schemas and Supporting Mixed Versions

Version 2 adds a required field. The new server works in isolation, but half the clients still send version 1 requests. A database rollback restores the old executable while leaving a new schema behind. A worker replays an older pending job and interprets it under new rules.

An upgrade changes a live compatibility graph: producers, consumers, stored records, generated code, and operational scripts may all run different versions for a time. This chapter treats migration as a staged state transition rather than a single deployment command.

15.1 Defining Compatibility in Both Directions

15.1.1 Separating Wire, Source, Binary, and Data Compatibility

A source-compatible library can still change its binary layout. A wire-compatible message can still change business meaning. A database column can be readable while its new units make old calculations wrong. “Backward compatible” must name the artifact and direction being evaluated.

For a request interface, test an old client against a new server and a new client against an old server where both combinations are supported. For persisted data, test new code reading old rows and determine whether old code may read rows written by the new version. These are independent questions.

Generated serialization code adds another compatibility layer. The schema, generator version, generated sources, and runtime library must form a supported combination. Protobuf documents cross-version runtime guarantees and limits; compatibility should be checked against that policy rather than assumed from an unchanged message name. (R43)

The capstone’s simple text protocol is deliberately version 1. It does not implement negotiation. Its optional .proto design is an integration exercise, not evidence that the shipped loopback binaries already speak gRPC. Keep these two artifacts clearly separate during experimentation.

15.1.2 Preserving Meaning When Fields Are Added

An optional field with a default can support an additive change, but the default is part of the contract. If a new priority field defaults to zero, specify whether zero means ordinary priority, unknown priority, or highest urgency. Different interpretations can produce inconsistent scheduling even though parsing succeeds everywhere.

Avoid reusing removed field identifiers for unrelated meanings in a schema format that uses numeric tags. Preserve reserved identifiers as appropriate, and test old serialized fixtures. A message parser accepting bytes is only the first step; the resulting domain object must also have the intended meaning. (R14)

Canonicalization rules can be breaking changes too. If an idempotency key was case-sensitive and becomes case-insensitive, previously distinct jobs can collide. If input units change from milliseconds to seconds without a new field or version, replayed old records can produce wrong behavior.

Write down the interpretation of absent, empty, zero, and invalid values. They are not automatically the same. A migration test should include these cases explicitly rather than relying only on ordinary populated records.

NEW SECTION EXAMPLE

Preserve old field meaning when a new field appears

An old reader consumes `amount_cents` and ignores a new memo, while a new reader accepts both schema versions.

main.cpp

```
#include <map>
#include <string>
#include <iostream>

int main() {
    using Row=std::map<std::string, std::string>;
    auto old_reader=[](const Row&r){return std::stoi(r.at("amount_cents"));};
    auto new_reader=[&](const Row&r){int cents=old_reader(r);auto it=r.find("memo");return
std::pair{cents, it==r.end()?std::string{}:it->second}};
    Row old{{"amount_cents", "1250"}}, newer{{"amount_cents", "1250"}, {"memo", "invoice"}};
    if(old_reader(newer)!=1250||new_reader(old).first!=1250)return 1;
    std::cout<<"old-on-new=1250
new-on-old-memo-empty="<<std::boolalpha<<new_reader(old).second.empty()<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The additive memo field has a defined empty default and does not change the units or interpretation of `amount_cents`. Both mixed-version directions are tested explicitly.

Renaming the amount or switching it to dollars would be a semantic/data compatibility break even if map parsing still succeeded. The example does not claim binary ABI compatibility; it tests a logical record contract.

Expected behavior or output

```
old-on-new=1250 new-on-old-memo-empty=true
```

Concrete experiment

Change the new amount field to 12.50 dollars while keeping its old name. The old reader's result exposes the semantic break; create a new field and migration instead.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

15.2 Planning Expand, Backfill, Switch, and Contract

15.2.1 Expanding Without Requiring Immediate Fleet Replacement

An expansion introduces a representation that old and new participants can coexist with. For a database, this might be an added nullable column or a column with a safe default. For a service, it might be a new optional response field ignored by older readers.

Deploy readers that can understand both representations before requiring writers to emit only the new one. This ordering reduces the interval in which valid new data is unreadable by still-running consumers. It does not remove the need to know which consumers exist; background jobs and support tools are part of the compatibility graph.

A backfill transforms existing data in bounded batches. Track progress with a stable cursor and make repeated batches safe. A single giant transaction can hold locks or consume resources for too long. An untracked script can stop halfway and leave operators unable to distinguish completed from skipped rows.

Validate both counts and semantics. Equal row counts do not prove that every converted timestamp used the correct units or that every key preserved its normalization. Use representative old fixtures, boundary values, and independent checks of the transformation.

15.2.2 Contracting Only After the Old Dependency Is Gone

The contraction removes old fields, compatibility paths, or storage. It should occur only after the deployment and retention window no longer requires them. A recently upgraded server can still hold old pending messages, old snapshots, or clients that reconnect infrequently.

Telemetry can help establish usage, but absence in sampled traffic is not proof that a code path is unused. Combine operational observations with inventory, explicit version policy, and retained-data analysis. A dormant disaster-recovery tool can become important precisely when normal systems are unavailable.

Rollback becomes more complicated after contraction. An old binary may no longer understand the new schema, and a data conversion may be lossy. Define whether rollback means reverting code, applying a reverse data migration, or deploying a forward fix. Do not promise that replacing one executable reverses every state change.

For HarborDispatch, the default store accepts schema version 1 and rejects a database claiming a newer unsupported version. Failing closed is intentional: silently opening unknown state could corrupt the meaning of accepted work. A future version must supply a tested migration before changing this rule.

Expand and backfill before switching a read query

Keep the originalname column readable while introducing display_name and backfilling existing rows.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```


main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database db;db.exec("CREATE TABLE users(id INTEGER PRIMARY KEY,name TEXT NOT NULL); INSERT
    INTO users VALUES(1,'Ada')");
    db.exec("ALTER TABLE users ADD COLUMN display_name TEXT");
    {Transaction tx(db);db.exec("UPDATE users SET display_name=name WHERE display_name IS
    NULL");tx.commit();}
    Statement old(db,"SELECT name FROM users WHERE id=1"),newer(db,"SELECT
    COALESCE(display_name,name) FROM users WHERE id=1");
    if(!old.row()||!newer.row())return 1;
    std::string a=reinterpret_cast<const char*>(sqlite3_column_text(old.p,0));
    std::string b=reinterpret_cast<const char*>(sqlite3_column_text(newer.p,0));
    if(a!="Ada"||a!=b)return 2;std::cout<<"old=Ada new=Ada old-column-retained\n";
    }catch(const std::exception&e){std::cerr<<e.what()<<'\n';return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

ALTER adds a nullable field without requiring every existing row to be supplied immediately. Backfill copies the old authoritative value; COALESCE lets new code read rows during the transition.

The old column is deliberately retained, so an old binary can still issue its originalSELECT. Contracting the schema requires evidence that old writers/readers are gone and a policy for maintaining both fields during the overlap.

Expected behavior or output

```
old=Ada new=Ada old-column-retained
```

Concrete experiment

Insert a row between expansion and backfill using onlyname. The new query must still return the originalname until backfill completes.

Validation of this new example: gcc: passed

Technical references: SQLITE-C, SQLITE-TRANSACTION. See the added references and revision index.

15.3 Running Migrations as Controlled Operations

15.3.1 Owning the Migration and Its Transaction Boundary

Choose who runs the migration. If every replica automatically attempts schema changes on startup, simultaneous deployment can create contention and confusing failures. A dedicated migration step or an explicitly serialized startup protocol can make ownership clearer.

Some database changes can be transactional under the selected engine and version; others require different operational handling. Consult the actual database documentation rather than assuming that every ALTER operation has identical locking,

rewrite, or rollback behavior across products. PostgreSQL’s transaction isolation documentation illustrates why concurrent transaction behavior must be understood at the chosen engine’s level. (R23)

The chapter’s SQLite example performs a small additive migration in a controlled, in-memory database. It is not a performance claim about altering a large production table. Real migrations need a preflight that checks free storage, backup validity, expected lock duration, client compatibility, and abort conditions.

A migration identifier should be immutable once applied. Editing the text of an already recorded migration makes different environments claim the same version while holding different schemas. Add a new migration and preserve the history, including its source revision or checksum.

15.3.2 Verifying Recovery at Every Phase

Test interruption before a migration starts, during a backfill batch, after schema expansion, after new writers begin, and before contraction. Each phase can require a different restart or rollback action. “The migration script is idempotent” is not enough unless repeated execution preserves the intended data at these points.

A backup restore test should use the application version intended to read the restored data. Restoring bytes successfully while discovering that no available binary understands them is not a complete recovery path. Keep compatible binaries, migration tools, and schema documentation under the same release retention policy.

Mixed-version tests should exercise actual interfaces. If version 1 ignores a new field, verify that its reserialization does not accidentally erase information needed downstream. If a service forwards messages it does not interpret, unknown-field preservation may matter. If it reconstructs a new domain object, that behavior may be different.

For durable jobs, record which semantics were accepted. A pending operation should not silently change its meaning because a new worker version changed a calculation rule. Either preserve the original operation version or perform an explicit, authorized migration of the pending intent.

NEW SECTION EXAMPLE

Record a migration identifier in the same transaction as its schema change

Apply an additive migration once and prove a retry does not duplicate the change.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database db;db.exec("CREATE TABLE schema_migrations(version INTEGER PRIMARY KEY); CREATE
TABLE jobs(id INTEGER PRIMARY KEY)");
    auto migrate=[&]() {Transaction tx(db);if(scalar(db,"SELECT count(*) FROM schema_migrations
WHERE version=2")){tx.commit();return false;}
        db.exec("ALTER TABLE jobs ADD COLUMN note TEXT; INSERT INTO schema_migrations
VALUES(2)");tx.commit();return true;};
    bool first=migrate(),second=migrate();
    std::cout<<"first="<<std::boolalpha<<first<<" repeated="<<second<<"
versions="<<scalar(db,"SELECT count(*) FROM schema_migrations")<<"\n";
    }catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

BEGIN IMMEDIATE serializes the decision with other writers. The schema change and version row commit together, so a successful retry sees version2 and does not repeat ALTER.

A migration identifier must name immutable migration content. This small SQLite ALTER is transactional; other database engines and operations have different locking and rollback behavior. Record target-specific assumptions instead of generalizing this test.

Expected behavior or output

```
first=true repeated=false versions=1
```

Concrete experiment

Inject an invalid SQL statement before inserting version2. After rollback, inspect PRAGMA table_info and the migration table to confirm neither half of the change survived.

Validation of this new example: gcc: passed

Technical references: SQLITE-C, SQLITE-TRANSACTION. See the added references and revision index.

Worked example — Add a defaulted priority field

Requirement and implementation

Create a version 1 table, insert an old-style row, add a priority column with an explicit default, and verify that an old-style insert still succeeds under the expanded schema. Record the new schema version only with the successful migration.

Source — source/examples/ch15.cpp

```
#include "harbor/check.hpp"
#include <iostream>
#include <sqlite3.h>
int main() {
    sqlite3* raw=nullptr;
    harbor::check(sqlite3_open(":memory:",&raw)==SQLITE_OK,"open");
    struct Owner { sqlite3* db; ~Owner(){sqlite3_close(db);} } owner{raw};
    auto sql=[&](const char* text) { return sqlite3_exec(raw,text,nullptr,nullptr,nullptr); };
    harbor::check(sql("CREATE TABLE jobs(id TEXT PRIMARY KEY); PRAGMA
user_version=1;")==SQLITE_OK,"version one");
    harbor::check(sql("BEGIN; ALTER TABLE jobs ADD COLUMN priority INTEGER NOT NULL DEFAULT 0;
PRAGMA user_version=2; COMMIT;")==SQLITE_OK,"expand");
    harbor::check(sql("INSERT INTO jobs(id) VALUES('old-client');")==SQLITE_OK,"old writer
remains valid");
    std::cout<<"Additive migration preserves this old INSERT contract.\n";
}
```

Evidence and interpretation

The example demonstrates one additive change supported by the selected SQLite operations. The old insert continues to omit the new column, and the default supplies its defined value. This is useful evidence for that schema transition, not a general proof that every field addition is compatible.

The capstone’s store is not automatically upgraded to this demonstration schema. It remains version 1. Keeping the migration exercise separate prevents readers from confusing an illustrative table with the application’s supported on-disk contract.

Failure clinic — A rolling upgrade changes job meaning

An old worker interprets input 10 as ten units, while a new worker interprets it as ten thousand units. Both accept the same wire syntax. A retry routed across versions can now conflict or produce a different result under the same idempotency key.

The correction is to version the operation semantics, preserve the accepted meaning in durable state, and ensure that duplicate comparisons include the relevant version. Merely adding a new release number to the log does not repair the business ambiguity.

Engineering case — Roll out an operation-version field without changing old jobs

Version 1 computes the square of an input. A proposed version 2 introduces a different calculation. Reusing the same input field and changing the worker implementation would make old pending jobs execute under new semantics. It would also make a retry conflict with a result produced before deployment.

Expand the stored and wire representation with an explicit operation version. Existing rows receive the documented version 1 interpretation. New readers understand both versions before new writers are allowed to emit version 2. The worker’s duplicate comparison includes the operation version as part of canonical meaning.

A compatibility matrix now has useful entries. An old worker can process version 1 and must reject version 2. A new worker can process both. A new gateway must not route version 2 to an old worker without capability selection. An old gateway must continue interpreting version 1 results correctly during the mixed interval.

Backfilling the version field should be repeatable and should not recalculate existing results under the new formula. The migration assigns meaning to historical rows; it does not reinterpret them. A test fixture containing a completed version 1 receipt is therefore essential.

Only after old producers, readers, retained messages, and recovery tools no longer require the old representation can contraction be considered. Even then, retaining the ability to interpret historical receipts may remain necessary for audit and replay. “No current client sends version 1” is not the same as “no persisted version 1 data exists.”

Rollback must also preserve this graph. If version 2 jobs have been accepted, restoring an old worker that rejects them can leave durable obligations pending. That may be an acceptable temporary degraded mode under a plan, but it is not a transparent rollback. A forward-compatible repair may be safer than attempting to erase the new jobs.

The shipped capstone intentionally supports only its original operation. This case is a design exercise for extending that contract. It demonstrates why semantic versioning of work matters more than merely incrementing a package version or adding a release label to logs.

Practice and implementation lab

Create old-format fixtures containing absent, zero, maximum, and malformed values. Write a migration that adds an explicit operation-version field and rejects unsupported future versions. Test reexecution after a completed migration and after a simulated interrupted batch.

Prepare a compatibility matrix with old and new clients, servers, and stored rows. Mark combinations supported, rejected deliberately, or untested. Then write the exact order in which readers, writers, and cleanup are deployed.

Review and solution reasoning

An additive change is safe only when its default and interpretation preserve old behavior where promised. A deployment plan should therefore state semantic compatibility, not merely parser compatibility. Retained messages and backups extend the lifetime of older representations.

A reversible code change can accompany an irreversible data change. Recovery planning must follow the state, not just the repository commit. This is why migration evidence belongs beside release artifacts rather than in a forgotten one-off script.

Chapter takeaway. Upgrade the compatibility graph in stages, preserve accepted meaning, and test the state transitions that a rollback would actually encounter.

Chapter 16 — Designing Caches with Explicit Freshness

A cache invalidation arrives, removes a value, and appears to work. A slow read that began earlier then finishes and inserts the old value again. The cache is stale even though every invalidation message was delivered.

Caching is a concurrency and consistency problem as well as a performance technique. This chapter examines versions, expiration, fill coordination, and bounded resource use. The local model focuses on one race: preventing an older fill from crossing a known invalidation boundary.

16.1 Deciding What a Cache Is Allowed to Forget

16.1.1 Separating Cached Copies from Authoritative State

A cache stores information that can be reconstructed under an explicit policy. If losing an entry loses the only record of a completed effect, that entry is authoritative state regardless of whether the code calls it a cache. HarborDispatch’s worker receipts are not expendable cache entries; they define how delayed retries are interpreted.

Determine the consequence of a miss before introducing caching. A miss may trigger an authoritative read, a computation, or an unavailable response. It should not cause an application to invent a default that violates the original contract. An absent authorization entry, for example, must not automatically mean permission is granted.

Read-through, cache-aside, and write-through arrangements place responsibility in different components. In cache-aside, the application commonly reads the cache, fetches on a miss, and inserts the result. That sequence is not atomic with concurrent writes unless additional mechanisms coordinate them.

Cache a domain result whose interpretation is stable. Caching an entire transport response can inadvertently retain an expired credential, a caller-specific authorization decision, or a short-lived redirect. Define the key’s dimensions and the value’s validity independently of the convenience of serializing one object.

16.1.2 Choosing Freshness and Failure Semantics

Freshness is a requirement, not a consequence of a fast lookup. Specify whether a value may be stale for a bounded duration, must meet a minimum revision, or is safe only for advisory display. Different fields on one page can have different requirements.

A time-to-live limits how long an entry is locally eligible under the chosen clock. It does not guarantee that the entry was current when inserted. An old fill that starts late or reads a lagging replica can be stale immediately while receiving a fresh TTL.

Define the behavior when the authority is unavailable. Serving a known stale value with its age can be acceptable for some dashboards, but not for every decision.

Return a distinct stale or unavailable outcome rather than silently changing a strict read into an approximate one.

Negative caching also needs care. Caching MISSING can hide a newly created job until expiration. For a submit-then-status workflow, the application may bypass a negative cache or require a revision barrier. A cache hit should not override a caller's stronger dependency requirement.

NEW SECTION EXAMPLE

Treat expiry as a cache miss, not deletion of the source of truth

Cache an authoritative value until an explicit time and refresh it after expiry.

main.cpp

```
#include <optional>
#include <iostream>

int main() {
    struct Entry{int value;int expires_ms;};std::optional<Entry> cache=Entry{10,100};int
    authoritative=20;
    auto read=[&](int now){if(cache&&now<cache->expires_ms)return
    cache->value;cache=Entry{authoritative,now+50};return authoritative;};
    std::cout<<"before="<<read(99)<<" at-expiry="<<read(100)<<"
    refreshed-expiry="<<cache->expires_ms<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The cache permits a value10 before its definedexpiry100, even though the authority is now20. At equality the value is stale and a refresh obtains20; expiration semantics are explicit.

The source of truth remains separate from the disposable cached copy. This fixture assumes a successful authority read; a production stale-on-error policy needs a separately named result and age rather than silently extending freshness.

Expected behavior or output

```
before=10 at-expiry=20 refreshed-expiry=150
```

Concrete experiment

Make the authority unavailable at100 and choose either an error or explicitly marked stale result. Do not call the cached10fresh after changing the policy.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

16.2 Coordinating Versions, Invalidations, and Fills

16.2.1 Rejecting Old Fills After an Invalidation

The model records a minimum acceptable version, called a floor, for each invalidated key. Invalidation at version 5 evicts an older cached value and prevents a later fill of version 4 from being inserted. A fill at version 5 or later remains eligible under the model's rule.

This closes the specific sequence: start read of version 4, commit version 5, deliver invalidation 5, finish old read, attempt to cache version 4. Merely deleting the entry at invalidation would not close it. The floor remembers information about the invalidation after the old value is gone.

Versions must belong to a defined authority and key domain. Comparing unrelated counters from two replicas can reject valid data or admit stale data. A version should describe the state being cached, not the client process's local attempt number.

Distributed cache products provide their own invalidation and client-tracking mechanisms. Redis documents server-assisted client-side caching and invalidation behavior; using such a facility still requires understanding connection lifecycle, missed information, and application key semantics. The local model is not an implementation of Redis tracking. (R24)

16.2.2 Bounding the Metadata Needed to Reject Staleness

Remembering every invalidation forever would itself become unbounded storage. The teaching model keeps maps of entries and floors and deliberately does not implement eviction or floor garbage collection. Its purpose is to expose the correctness requirement before optimizing memory use.

A production design needs a safe point after which an old fill cannot arrive or cannot be accepted. This may involve bounded fill lifetimes, generation changes, reconnect invalidation, or a stronger authoritative revision check. Deleting a floor while an old fill remains in flight can reopen the race.

Entry eviction and invalidation-floor eviction are different operations. Removing a cached value can be harmless because it causes a refetch. Removing version knowledge can affect which refetch results are accepted. Apply retention reasoning to both.

The same principle appears in deduplication. Deleting a receipt reopens the possibility of treating an old request as new. Metadata that seems auxiliary can encode a correctness boundary over time. Capacity planning must include that metadata rather than only payload bytes.

NEW SECTION EXAMPLE

Reject an old fill that completes after invalidation

Capture a cache generation when starting a fetch and compare it again before publication.

main.cpp

```
#include <optional>
#include <iostream>

int main() {
    unsigned generation=5;std::optional<int> cached;
    unsigned fetch_generation=generation;int fetched_value=10;
    ++generation;cached.reset(); // invalidation arrives while fetch is in flight
    bool accepted=fetch_generation==generation;
    if(accepted)cached=fetched_value;
    std::cout<<"old-fill-accepted="<<std::boolalpha<<accepted<<" cache-empty="<<!cached<<"
    generation="<<generation<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The invalidation increments a watermark rather than only deleting the current value. The older fetch still carries generation 5, so it cannot repopulate the cache after generation 6 becomes authoritative.

Generation metadata also needs a lifetime and capacity policy. Dropping a watermark while an older fetch can still return reintroduces the race. In a real multithreaded cache, check-and-publish must be atomic under a lock or equivalent ownership rule.

Expected behavior or output

```
old-fill-accepted=false cache-empty=true generation=6
```

Concrete experiment

Start another fetch at generation 6 and allow it to publish. Then inject another invalidation before publication and require rejection again.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

16.3 Controlling Cache Load and Failure Amplification

16.3.1 Preventing Simultaneous Misses from Overloading the Authority

A popular entry expires and thousands of requests miss together. If each independently fetches from the database, the cache has concentrated load into a burst. Fill coalescing can allow one in-flight fetch per key while other callers wait or receive a defined stale result.

Coalescing introduces ownership and cancellation questions. One caller timing out should not necessarily cancel a shared fetch still needed by others. Conversely, a fetch with no remaining consumers should not retain resources forever. Give the shared operation its own bounded lifetime and define how waiters detach.

Avoid holding a global cache mutex during remote I/O. Protect the state transition that installs or finds an in-flight fill, then release the lock before waiting. Completion must verify that it still belongs to the accepted generation before publishing its result.

A single hot key can also accumulate an unbounded waiter list. Limit waiters, impose deadlines, and choose an overload response. Coalescing reduces duplicate authority reads, but it does not eliminate the memory cost of everyone waiting for one slow operation.

16.3.2 Measuring Useful Hits Rather Than Only Hit Rate

A hit rate can improve while user outcomes worsen. Large cheap objects might dominate hits while the expensive miss path remains unchanged. A stale hit can be counted as success even when it causes an incorrect decision. Measure latency, freshness violations, authority load, and memory consumption alongside hits and misses.

Separate cold-start behavior from steady state. A rollout that replaces every instance simultaneously can erase local caches and overload the database. A gradual rollout, prewarming under a budget, or a shared cache can change that behavior, but each adds its own operational costs.

Do not infer performance from the model example. It uses ordered maps for clarity and does not benchmark an eviction policy. Real cache selection depends on workload sizes, contention, object lifetime, and failure semantics. Optimize only after a representative measurement establishes a need.

For the integrated HarborDispatch service, no cache is inserted into authoritative job lookup. The database is small and the semantic path remains inspectable. The cache chapter demonstrates an extension mechanism without pretending that every service requires caching to be professional.

NEW SECTION EXAMPLE

Coalesce concurrent cache misses into one owned fill

Let two callers share one future for a missing key instead of independently querying the authority.

main.cpp

```
#include <future>
#include <optional>
#include <iostream>

int main() {
    std::promise<int> producer; auto shared=producer.get_future().share(); unsigned
    authority_calls=0;
    std::optional<std::shared_future<int>> in_flight;
    auto request=[&]() { if(!in_flight) { ++authority_calls; in_flight=shared; } return *in_flight; };
    auto first=request(), second=request(); producer.set_value(42);
    std::cout<<"first="<<first.get()<<" second="<<second.get()<<"
    authority_calls="<<authority_calls<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The first miss creates the in-flight entry; subsequent misses observe the same `shared_future`. Completing the producer makes one result available to every waiter without repeating the authority operation.

This is a deterministic single-owner model. A concurrent map needs a synchronized install-if-absent operation, bounded waiters, and removal of failed/completed fills. A high hit rate alone would not reveal a stampede on a rarely accessed but expensive key.

Expected behavior or output

```
first=42 second=42 authority-calls=1
```

Concrete experiment

Complete the promise with an exception and verify both callers observe failure. Remove the failed in-flight entry so a later request can make a bounded new attempt.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Block a stale fill after invalidation

Requirement and implementation

Store version 4, invalidate at version 5, reject a delayed fill of version 4, accept version 5, and enforce expiration at the exact boundary.

Source — source/examples/ch16.cpp

```
#include "harbor/check.hpp"
#include "harbor/models.hpp"
#include <iostream>
int main() {
    harbor::VersionedCache cache;
    cache.put("job", "pending", 4, 100);
    cache.invalidate("job", 5);
    harbor::check(!cache.put("job", "pending", 4, 200), "late stale fill");
    harbor::check(cache.put("job", "done", 5, 200), "current fill");
    harbor::check(cache.get("job", 199) == "done" && !cache.get("job", 200), "expiry boundary");
    std::cout << "An invalidation floor stopped an old in-flight read from resurrecting\n";
}
```

Evidence and interpretation

The test's success demonstrates the version-floor rule for a finite sequence. It does not demonstrate thread safety, distributed notification delivery, bounded memory, or a production cache replacement policy. The model is single-owner and uses explicit ticks so no sleep is needed.

Observe that get checks eligibility without mutating the maps. An expired value can remain allocated until a separate policy removes it. Logical expiration and memory reclamation are intentionally separated; a real implementation must handle both.

Failure clinic — Invalidations arrive out of order

Suppose invalidation 9 arrives before invalidation 7. A naive assignment of the floor to the latest-arriving number would move it backward to 7. The model takes the maximum, preserving the strongest known boundary. The test should therefore include reordered invalidations as well as reordered fills.

A larger number is meaningful only within the same version domain. An attacker-controlled “version” cannot be trusted to establish freshness, and a new authority generation requires a defined comparison rule.

Engineering case — Reclaim invalidation floors without reopening the race

The versioned cache stores a floor of 20 for key K. A read of version 19 began before invalidation and remains in flight. An eviction policy removes K’s floor to reclaim memory. The old read then completes and inserts version 19 because the cache no longer remembers that version 20 was required.

A bounded implementation needs more than a maximum map size. It needs a rule for when outstanding old fills can no longer publish. One approach assigns a cache generation to each fill. When a generation is retired, its fills may complete for their callers under a defined policy but cannot repopulate the new generation’s cache.

Another approach bounds fill lifetime and retains invalidation metadata until every fill that could predate it has completed or been invalidated. This requires tracking actual operation lifetime, not assuming that a timeout stopped a remote callback. The cancellation lessons from the coroutine chapter apply here too.

A generation reset can safely cause more misses, but it must not let untrusted or stale callbacks bypass the publication check. The callback should carry the generation captured at admission and compare it at publication under the same synchronization that protects cache state.

The model’s put method accepts equal versions and replaces the value. That assumes a version uniquely identifies the authoritative state for the key. If two different values can legitimately carry the same version, the protocol is underspecified. A production implementation may need to reject equal-version conflicts or compare an authoritative content identifier.

Test reclamation with an explicit delayed fill. Invalidate, retire the relevant generation, reclaim metadata, then deliver the old completion. The expected result is rejection of cache publication, not necessarily cancellation of every caller’s operation. These are separate outcomes.

This case reveals why cache metadata is not always disposable in the same way as cached payload. It can encode a temporal safety condition. A capacity optimization

must preserve that condition or replace it with another mechanism whose assumptions are equally clear.

Practice and implementation lab

Add a simulated in-flight fill with a captured version and completion event. Enumerate the histories where invalidation occurs before the fill starts, during the fill, and after publication. State which values may be returned in each history under your chosen contract.

Then design a bounded floor-retention policy. Identify the assumption that makes reclamation safe, such as a maximum fill lifetime plus a generation barrier. Include a test in which a fill exceeds that limit and is rejected rather than published.

Review and solution reasoning

A valid solution treats TTL as local eligibility, not as proof of current authority state. It also recognizes that coalescing, invalidation, and eviction solve different problems. Combining them without explicit states can create races that no individual mechanism has in isolation.

When caching is unnecessary, leaving it out preserves a simpler consistency story. Architecture includes choosing which complexity not to introduce.

Chapter takeaway. Cache only reconstructible information, define freshness explicitly, and retain enough version knowledge to reject late stale work.

Chapter 17 — Designing Messaging and Delivery Semantics

A consumer updates its database and crashes before acknowledging a message. The broker sends the message again. If the consumer instead acknowledges first, a crash between acknowledgement and update can lose the business effect. The difficult part is the boundary between message delivery and application state.

Messaging systems provide useful storage, routing, and consumer coordination. They do not automatically make every downstream side effect occur once. This chapter defines acknowledgements, ordering, replay, and consumer identity before the next chapter joins them with local transactions.

17.1 Separating Commands, Events, and Transport Records

17.1.1 Giving Each Message a Defined Meaning

A command requests an action from an authority. An event records that something occurred under an identified producer's contract. A transport record is the broker's stored representation. Treating these as synonyms can create feedback loops and unclear ownership.

A command such as “create report” needs an operation identity and a response or status path. An event such as “report completed” should identify the completed operation and the state transition it describes. Neither should rely solely on the broker's delivery tag as a durable business identity; tags and offsets have transport-specific scopes.

Include the minimum metadata needed to interpret and recover the message: message identity, operation identity where distinct, schema version, entity or partition key, and relevant state revision. Avoid copying credentials or unnecessary personal data into every event. Long retention turns message payloads into a long-lived data store.

HarborDispatch sends an EXEC command whose key is stable across attempts. Its worker response reports the saved result. The integrated lab does not run a message broker, but the same identity requirement appears when the transport is a durable queue rather than a direct loopback connection.

17.1.2 Choosing Queue or Log Behavior Deliberately

A work queue commonly distributes jobs among consumers, with acknowledgement controlling when work is considered handled under the broker's rules. An append-only log retains records by position and allows consumers to track their own progress. Retention, replay, ordering, and consumer ownership differ between these models.

RabbitMQ distinguishes publisher confirmations from consumer acknowledgements. A publisher confirmation concerns the publish side; it is not proof that a consumer committed a business effect. A consumer acknowledgement concerns delivery handling under the consumer protocol; it is not a receipt for an unrelated external action unless the application establishes that relationship. (R25)

Kafka describes delivery and transaction semantics in relation to its log, producers, consumers, and transaction-aware processing. Guarantees have a scope. A transactional operation within that scope does not automatically make an arbitrary external database update or email send part of the same atomic boundary. (R26)

Select the transport after defining replay and ownership needs. Adding a broker because the architecture diagram looks more distributed can create operational state without resolving the actual business ambiguity.

NEW SECTION EXAMPLE

Distinguish a command from the event it produces

Apply a validated intent to stock, then emit a fact only after the state transition succeeds.

main.cpp

```
#include <optional>
#include <iostream>

int main() {
    struct Reserve{int quantity;};struct Reserved{int quantity;int remaining;};int stock=5;
    auto handle=[&](Reserve c)->std::optional<Reserved>{if(c.quantity<1||c.quantity>stock)retu
rn{};stock-=c.quantity;return Reserved{c.quantity,stock};};
    auto event=handle({3}),rejected=handle({4});
    std::cout<<"reserved="<<event->quantity<<" remaining="<<event->remaining<<"
second-event="<<std::boolalpha<<bool(rejected)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Reserve is a request that may be rejected. Reserved describes a completed state change. Emitting the event before validation would falsely announce an effect that never happened.

A queue might distribute commands among workers; an eventlog might retain facts for multiple readers. Those transport choices do not erase the semantic distinction. Persistence/atomic publication is intentionally deferred to the outbox examples.

Expected behavior or output

```
reserved=3 remaining=2 second-event=false
```


Concrete experiment

ReplayReserved into a separate read model, not back into handle(Reserve). Otherwise replaying historical facts could repeat the business effect.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

17.2 Interpreting Acknowledgement and Redelivery

17.2.1 Locating the Crash Window Around an Effect

Consider three events: receive message, commit effect, acknowledge delivery. A crash before commit should leave the message eligible for later processing. A crash after commit but before acknowledgement can cause redelivery. The consumer needs to recognize that the effect is already present.

An at-most-once approach can avoid repeated processing by accepting the possibility of loss. An at-least-once approach can preserve another delivery attempt under its retention and recovery assumptions but requires duplicate handling. Neither label states the full application's correctness contract by itself.

The phrase "exactly once" should always be followed by the boundary: exactly one committed row transition in this database, exactly one visible output record under this transaction protocol, or some other precisely defined effect. It should not be used as an unexplained property of an entire chain of unrelated systems.

In the chapter's in-memory inbox, an event ID is associated with its payload. Repeating the same pair does not add the value again; reusing the ID with a changed payload is an error. The map disappears on process exit, so this example explicitly does not solve crash recovery.

17.2.2 Distinguishing Retryable Failures from Poison Messages

A temporary database outage may justify redelivery. A malformed schema version, violated domain invariant, or permanently unauthorized action may not improve with another identical attempt. Repeatedly delivering a poison message at the front of a queue can block otherwise valid work.

Define terminal handling: quarantine the message with reason and provenance, move it to a controlled dead-letter path, reject it under policy, or require manual repair. A dead-letter queue is not a disposal bin. Someone must own retention, inspection, redrive authorization, and the risk that an old message's meaning has changed.

Bound retries by count, time, or operational policy appropriate to the work. Interactive RPCs often have a caller deadline. Durable workflows can have longer recovery horizons, but they still need a terminal or escalation policy. "Retry forever" can hide a permanent defect for months while accumulating backlog.

The simple HarborDispatch dispatcher retains pending work and retries its fixed worker. It has no persistent quarantine state and processes bounded batches in row order. A permanently failing early operation can therefore impair later progress. This

is a documented extension requirement, not silently presented as a full broker scheduler.

NEW SECTION EXAMPLE

Make the effect-acknowledgement gap observable

Simulate redelivery after an effect completed but its acknowledgement was lost, with receipt-based duplicate suppression.

main.cpp

```
#include <set>
#include <string>
#include <iostream>

int main() {
    std::set<std::string> receipts; int balance=0; unsigned deliveries=0;
    auto deliver=[&](const
std::string&id){++deliveries; if(receipts.contains(id)) return; balance+=5; receipts.insert(id);};
    deliver("m1"); bool ack_lost=true; if(ack_lost) deliver("m1");
    std::cout<<"deliveries="<<deliveries<<" effect="<<balance<<"
receipts="<<receipts.size()<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Two deliveries correspond to one logical message. The receipt prevents the repeated callback from adding 5 again after the acknowledgement gap.

In-memory receipt insertion and balance modification are not crash-atomic here. Production code must commit them together; otherwise a crash between effect and receipt can still duplicate work. A malformed poison message needs rejection/dead-letter policy rather than endless transient retries.

Expected behavior or output

```
deliveries=2 effect=5 receipts=1
```

Concrete experiment

Inject a crash-model reset between `balance+=5` and receipt insertion to expose the remaining hole. Then implement the pair in one SQLite transaction.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

17.3 Preserving the Ordering That the Domain Needs

17.3.1 Separating Delivery Order from Completion Order

Two messages can be delivered in order and complete out of order when processed concurrently. If event 2 depends on event 1's committed effect, a consumer pool must preserve that dependency or reject/defer the second transition. A FIFO input queue alone does not guarantee FIFO effects.

Partitioning by entity can make per-entity ordering easier while allowing parallelism across entities. The partition key must match the state that needs sequencing. Routing related events under different keys can erase the intended ordering boundary.

A version check can make stale updates harmless or explicit. For example, an entity expecting revision 6 should not blindly apply revision 8 as if revision 7 had been seen. The policy may fetch missing state, buffer within a bound, or treat events as snapshots rather than deltas. Those choices produce different recovery behavior.

Avoid asserting a global total order unless the application requires and implements one. It can become an expensive coordination point. Many systems need only per-account or per-job ordering plus explicit dependencies between a smaller set of operations.

17.3.2 Designing Replay as a Supported Operation

Replay can rebuild a projection, repair a bug, or test a new consumer. It can also repeat an external side effect if the consumer confuses reconstruction with live execution. Separate pure projection updates from actions such as charging, sending, or provisioning.

A replay should identify its source range, consumer version, checkpoint, and effect policy. If it writes to a new projection, isolate that destination until validation completes. If it updates a live projection, define how concurrent new events are ordered relative to replayed history.

Retained schemas matter. A consumer deployed today may need to understand events written years earlier. Preserve readers or migration adapters, and test old fixtures. Deleting schema support because no current producer emits the old version can break recovery from retained history.

A replay cursor is not a business idempotency key. The same event can appear through a different transport path or after export and import. Preserve domain event identity when the effect contract depends on recognizing it independently of delivery position.

Apply completed messages in sequence rather than completion order

Buffer an out-of-order completion until its predecessor is available.

main.cpp

```
#include <map>
#include <string>
#include <vector>
#include <iostream>

int main() {
    std::map<unsigned,std::string> completed;unsigned next=1;std::vector<std::string> applied;
    auto finish=[&](unsigned seq,std::string value){completed.emplace(seq,std::move(value));
        while(completed.contains(next)){applied.push_back(completed.at(next));completed.erase(
next++);}};
    finish(2,"second");finish(1,"first");finish(3,"third");
    std::cout<<"application order:";for(auto&s:applied)std::cout<<' '<<s;std::cout<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Task2 finishes first, but the application waits for sequence1. Once1 arrives, the contiguous prefix1,2 can be applied before3.

The reorder buffer needs a capacity and gap-recovery policy in production; a permanently missing predecessor cannot justify indefinite growth. Ordering is defined per stream here, not globally across unrelated partitions.

Expected behavior or output

```
application order: first second third
```

Concrete experiment

Never deliver sequence1 and cap the buffer at two entries. Specify whether excess completions are rejected, persisted elsewhere, or trigger recovery.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Distinguish three deliveries from two effects

Requirement and implementation

Process events e1, e1, and e2, adding only the first occurrence of each stable identity. Verify that an existing identity always retains the same payload.

Source — source/examples/ch17.cpp

```

#include "harbor/check.hpp"
#include <iostream>
#include <map>
#include <vector>
int main() {
    std::map<std::string,int> inbox; int total=0;
    for (const auto&
event:std::vector<std::pair<std::string,int>>{{"e1",5},{ "e1",5},{ "e2",3}}) {
        auto [it,inserted]=inbox.emplace(event);
        harbor::check(it->second==event.second,"identity conflict");
        if (inserted) total+=event.second;
    }
    harbor::check(total==8,"duplicate delivery effect");
    std::cout<<"Three deliveries caused two effects; this in-memory inbox is not crash
durable.\n";
}

```

Trace the result

The total is eight because the value five is applied once and the value three is applied once. The duplicate delivery remains observable as a repeated input, but it does not create another effect in the model. This is stronger than merely discarding every repeated string without checking its meaning.

The test prints its limitation: the inbox is volatile. A process restart would remove the seen-ID map. Chapter 18 changes the mechanism by placing identity and effect in one persistent transaction.

Failure clinic — Acknowledgement races ahead of commit

A consumer acknowledges immediately after parsing, then starts the database transaction. If the process stops before commit, the broker may reasonably consider the message handled while the application has no effect. The repair is to connect acknowledgement to the actual application durability point under the chosen delivery protocol.

Moving the acknowledgement later creates the duplicate window instead of the loss window. That is expected. Pair it with durable duplicate handling rather than assuming that timing can eliminate every crash boundary.

Engineering case — Advance a consumer checkpoint without skipping unfinished work

A log consumer receives offsets 40 and 41 and processes them concurrently. Offset 41 finishes first. The consumer records its next position as 42 and then crashes while offset 40 remains unfinished. On restart, beginning at 42 skips work that never committed.

A checkpoint describing a contiguous completed prefix must not advance past a gap. The consumer can track completed offsets and move the checkpoint only when every earlier required offset has completed under its effect contract. Alternatively, it can serialize processing where the throughput and ordering requirements permit.

The effect transaction also matters. If processing offset 40 updates an external database and the checkpoint is stored independently, a crash between those operations still creates a duplicate or loss window. A durable inbox keyed by event identity can make replay safe, while a supported transactional log-processing facility may atomically coordinate a narrower set of outputs and offsets.

Do not confuse offset 40 with the event's universal identity. Exporting and replaying the event through another topic can assign it a different position. If duplicate effects must be recognized across that path, preserve a domain event identifier rather than relying only on the original transport coordinate.

Partition ownership can change during consumer rebalance. An old consumer may still have in-flight work when a new consumer begins. The application needs the broker client's supported ownership protocol and, where necessary, effect-side deduplication or fencing. A local thread stop request is not proof that every external action has ceased.

A deterministic test can represent offsets 40 and 41 as two task states. Complete 41 first and verify that the contiguous checkpoint remains at 40. Then complete 40 and advance to 42. Add a restart between those transitions and verify replay of the unfinished work without duplicating already retained effects.

The companion does not run this broker scenario. Its direct dispatcher has one sequential delivery owner, which avoids this particular offset gap while limiting concurrency. The case shows what new state would become necessary when increasing parallelism in a log-based adapter.

Practice and implementation lab

Extend the model with an explicit delivery attempt and separate event ID. Reorder duplicate deliveries and verify that the final total remains stable. Inject an identity conflict and require a visible error rather than applying the new payload.

Design a broker integration test plan without claiming it has run. Include consumer crash before commit, crash after commit before acknowledgement, poison-message quarantine, partition reassignment, and replay of an old schema fixture. State which assertions belong to the broker and which belong to the application database.

Review and solution reasoning

A publisher confirmation, a delivery acknowledgement, a consumer checkpoint, and a committed business receipt are four different observations. A sound explanation follows the chain and identifies the gap between each adjacent pair.

Ordering is also scoped. A transport's sequence is useful only if the application preserves or checks the corresponding state dependency through concurrent processing and recovery.

Chapter takeaway. Describe delivery, acknowledgement, effect, and replay separately; then join the necessary boundaries with explicit state and transactions.

Chapter 18 — Implementing Idempotency with a Durable Outbox and Inbox

The gateway commits a job, the worker commits its result, and the worker's reply disappears. This is not an exotic corner case. It is the ordinary ambiguity created whenever an effect and its acknowledgement occur in different processes.

HarborDispatch does not try to make the network deliver exactly once. It retains intent at the gateway, binds a stable identity to one meaning at the worker, and retries until the gateway can record a matching result. This chapter explains the transaction boundaries that make that design useful and the limits that keep its claims honest.

18.1 Defining Idempotency as an API Contract

18.1.1 Binding One Identity to One Canonical Request

An idempotency key is meaningful only with a defined scope and payload association. In the lab, keys are global within a worker database, one to 64 characters, and restricted to ASCII letters, digits, underscore, and hyphen. The canonical input is a nonnegative integer no greater than one million.

The same key with the same integer means another attempt at the same job. The same key with a different integer is a conflict. Treating every existing key as unconditional success would allow a client to unknowingly receive the result of different work.

Real APIs may scope keys by tenant, operation type, and version. Include all fields that change meaning in the comparison or in a stable canonical fingerprint. A fingerprint must have a specified encoding and collision policy; calling `std::hash` on a transient object is not a durable request identity scheme.

AWS's discussion of idempotent APIs emphasizes caller-provided request identity and the treatment of retries as the same intended operation. The book applies that principle to a small local transaction model rather than copying a cloud service's implementation or guarantees. (R38)

18.1.2 Choosing Receipt Retention and Result Replay

The receipt stores the input and result, not merely a Boolean "seen." This allows a duplicate request to receive the original result and allows changed meaning to be detected. A system that keeps only a hash or seen flag may be unable to reconstruct the promised response.

Receipt retention defines how far into the future a retry can be recognized. The teaching service retains receipts indefinitely and does not implement pruning. This preserves a simple correctness story but is not a bounded production retention policy. Operators must account for database growth.

A production policy could define a deduplication window, archive receipts, or reject keys whose valid replay period has ended. It must not silently delete receipts and continue claiming that any arbitrarily old retry is safe. Cleanup changes semantics and belongs in the API documentation.

Returning the stored result is not a promise that every external dependency remains unchanged. A result may describe a historical action. Distinguish “this request was completed with result X” from “the world currently has state X.” Status and current-state queries can be different endpoints.

NEW SECTION EXAMPLE

Bind an idempotency key to canonical input and replayed output

Use a receipt table to distinguish same-key/same-input retry from a same-key/different-input conflict.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```


main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database db;db.exec("CREATE TABLE receipts(id TEXT PRIMARY KEY,input INTEGER NOT
    NULL,result INTEGER NOT NULL)");
    auto execute=[&](const std::string&id,int x){Transaction tx(db);Statement
    existing(db,"SELECT input,result FROM receipts WHERE id=?1");existing.text(1,id);
    if(existing.row()){if(sqlite3_column_int(existing.p,0)!=x)throw
    std::runtime_error("conflict");int result=sqlite3_column_int(existing.p,1);tx.commit();return
    result;}
    Statement put(db,"INSERT INTO receipts VALUES(?1,?2,?3)");put.text(1,id);put.integer(2
    ,x);put.integer(3,x*x);put.row();tx.commit();return x*x;};
    int first=execute("a",12),again=execute("a",12);bool conflict=false;
    try{(void)execute("a",13);}catch(const std::runtime_error&){conflict=true;}
    std::cout<<first<<' '<<again<<" conflict="<<std::boolalpha<<conflict<<"
    receipts="<<scalar(db,"SELECT count(*) FROM receipts")<<'\n';
    }catch(const std::exception&e){std::cerr<<e.what()<<'\n';return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

The primary key identifies the operation, while `input` binds its meaning. An identical retry returns the stored result; a changed `input` throws and rolls back rather than returning somebody else's result.

The fixture uses small integers, not an unbounded general square API. Real receipts need tenant/operation scope, canonical encoding, retention, and the actual effect in the same transaction where possible.

Expected behavior or output

```
144 144 conflict=true receipts=1
```

Concrete experiment

Parse 03 and 3 into integers before calling `execute`. They should bind the same canonical `input`, while 3x should fail before the receipt transaction.

Validation of this new example: gcc: passed

Technical references: [SQLITE-C](#), [SQLITE-TRANSACTION](#). See the added references and revision index.

18.2 Making Intent and Effect Locally Atomic

18.2.1 Recording Intent Before Attempting Delivery

The gateway's pending row is its outbox entry. Acceptance commits the key and input with a null result. The network dispatcher later selects bounded pending batches and sends EXEC to the worker. The user-facing response is sent only after acceptance has committed.

If the gateway crashes before delivery, the pending row survives and is found after restart. If delivery succeeds but recording completion fails, the row remains pending

and is retried. These two cases share the same recovery action because worker-side deduplication makes replay meaningful.

The outbox need not always be a separate table. It can be represented by a pending state in the same authoritative record when that state is sufficient to drive delivery. What matters is that the business intent and the delivery obligation are committed together, not the table's name.

Do not put a remote send inside the transaction and assume that makes it atomic. The database cannot roll back bytes already observed by another process. Keeping the transaction short and persisting the obligation makes the gap explicit and recoverable.

18.2.2 Committing the Worker Receipt with Its Effect

The worker starts an immediate transaction, checks whether the key already exists, compares its canonical input, and either returns the saved result or inserts a new receipt. The recorded square result is the lab's durable effect. The receipt and effect are represented together in one row.

For a more complex local business update, place that update and the inbox receipt in the same database transaction. If they are committed independently, a crash between them can leave either a duplicate window or a false receipt. A unique key alone does not fix a separate transaction boundary.

An external side effect cannot automatically join this local atomicity. Sending email, charging through another provider, or controlling hardware needs the external system's own idempotency or transactional facility, a serializing adapter, or a compensation and reconciliation design. The worker's SQLite commit cannot roll back those external effects.

The pure square calculation may be executed more than once before or during retries; the claim is not that the CPU multiplies exactly once. The relevant invariant is one recorded effect for a given key and input in the retained worker database. Precise wording prevents implementation details from being mistaken for a universal exactly-once guarantee.

NEW SECTION EXAMPLE

Commit business state and outbox intent together

Roll back both a balance update and its pending message when the transaction fails before commit.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database db;db.exec("CREATE TABLE balance(value INTEGER);INSERT INTO balance
VALUES(0);CREATE TABLE outbox(id TEXT PRIMARY KEY,delta INTEGER)");
    auto accept=[&](const char*id,int delta,bool fail){Transaction tx(db);
        Statement update(db,"UPDATE balance SET
value=value+?1");update.integer(1,delta);update.row();
        Statement insert(db,"INSERT INTO outbox
VALUES(?1,?2)");insert.text(1,id);insert.integer(2,delta);insert.row();
        if(fail)throw std::runtime_error("injected before commit");tx.commit();};
    try{accept("lost",5,true);}catch(const std::runtime_error&){
        accept("kept",7,false);
        std::cout<<"balance="<<scalar(db,"SELECT value FROM balance")<<"
pending="<<scalar(db,"SELECT count(*) FROM outbox")<<'\n';
    }catch(const std::exception&e){std::cerr<<e.what()<<'\n';return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

The balance change and outbox row share one SQLite transaction. The injected failure leaves neither the 5-unit effect nor its delivery obligation behind; the successful 7-unit transaction commits both.

No network send is inside this transaction. A dispatcher later delivers the outbox entry, and the receiver must commit an inbox receipt with its local effect. Two local transactions do not become one distributed atomic commit merely because both use SQL.

Expected behavior or output

```
balance=7 pending=1
```

Concrete experiment

Add an inbox table on a second database and redeliver kept twice. Require one receiver effect while the sender can retry until its completion is recorded.

Validation of this new example: gcc: passed

Technical references: [SQLITE-C](#), [SQLITE-TRANSACTION](#). See the added references and revision index.

18.3 Recovering Every Acknowledgement Gap

18.3.1 Walking the Failure Matrix

Before the gateway commit, no accepted obligation exists. After gateway commit but before worker commit, the gateway retains pending work. After worker commit but before a received reply, the worker retains a replayable result. After a received reply but before gateway completion commit, the gateway may retry and obtain that same result. After gateway completion commit, status can return done.

This sequence is the central recovery argument. Each gap has durable state on at least the relevant side and a defined action. It does not require both databases to commit simultaneously, but it does require their independent state not to be casually discarded.

A partial disaster restore can violate the argument. Restoring an old worker database may remove receipts for effects already reported by the gateway. Restoring an old gateway may remove pending jobs or completed status records. Recovery must reconcile authorities and any real external effects before reopening ordinary traffic.

The integration suite includes a worker mode that commits its first valid operation and then drops the reply deliberately. The gateway must retry, complete the job, and leave exactly one receipt. The failpoint is placed after commit specifically to test the ambiguous outcome, not merely an easy failure before work begins.

18.3.2 Distinguishing Safety from Eventual Completion

Durable duplicate handling makes retries safe within the defined boundary; it does not guarantee timely completion under every outage. If the worker never returns, pending work remains pending until policy or an operator resolves it. If storage is full, progress can stop. If a request permanently conflicts with external state, identical retries do not repair it.

The dispatcher uses bounded batches, a fixed endpoint, transport deadlines, and delayed retries. It does not implement priority scheduling, multiple independent worker authorities, a persistent dead-letter queue, or per-job terminal failure states. Those are deliberate extension points described in the operations chapters.

A pending quota of 128 prevents unlimited new backlog in the lab. It does not bound historical completed rows or receipt retention. Memory limits, pending limits, and durable-storage limits are separate. A production admission policy should consider all three.

A caller's interactive deadline can expire while the accepted job continues. The client should preserve the key and query status or repeat the same submission. Creating a new key because the user interface timed out can create another logical job even when the first one is progressing correctly.

NEW SECTION EXAMPLE

Enumerate acknowledgement-loss points and replay one stable operation

Check all four communication gaps in a small outbox/inbox state model.

main.cpp

```
#include <array>
#include <optional>
#include <iostream>

int main() {
    for(int gap=0;gap<4;++gap){bool pending=true,done=false;std::optional<int> receipt;int
effects=0;
        auto worker=[&]() {if(!receipt){++effects;receipt=49;}return *receipt;};
        if(gap>=1)(void)worker(); // remote commit may already exist
        if(gap>=3){done=true;pending=false;}
        if(pending){int result=worker();if(result==49){done=true;pending=false;}}
        if(!done||pending||effects!=1)return 1;
    }
    std::cout<<"four acknowledgement-gap histories: one effect each\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The gaps range from no workercommit through a workerreceipt whose reply/completion record is missing, to an already completed sender. Recovery repeats the same operation and uses the receipt to avoid another effect.

This is a deterministic state model assuming both sides retain committed state. It does not prove real disk durability or external side-effect exactly-once behavior. Those assumptions must be tested across actual process/database boundaries as well.

Expected behavior or output

four acknowledgement-gap histories: one effect each

Concrete experiment

Delete the workerreceipt after its effect while keeping the effect count. Recovery should now expose duplication, demonstrating why an inconsistent disaster restore needs reconciliation.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Replay after a lost response

Requirement and implementation

Use separate gateway and worker stores. Accept a request, apply it at the worker, simulate losing the first result, apply it again with the same identity, and record the matching completion at the gateway.

Source — source/examples/ch18.cpp

```
#include "harbor/check.hpp"
#include "harbor/store.hpp"
#include <iostream>
int main() {
    harbor::Store gateway(":memory:"), worker(":memory:");
    harbor::Request r{"job-1", 9}; gateway.submit(r);
    auto ignored_reply = worker.apply(r); (void) ignored_reply; // acknowledgment lost
    auto retried_reply = worker.apply(r);
    gateway.complete(r, retried_reply);
    harbor::check(worker.receipt_count() == 1, "one durable effect record");
    harbor::check(gateway.lookup(r.key) -> result == 81, "reconciled response");
    std::cout << "Lost acknowledgment caused a retry, not a second worker receipt.\n";
}
```

Trace the evidence

The worker receipt count remains one, and the gateway ends with the expected result. These stores are in-memory for the chapter demonstration; process restart and file-backed recovery are covered by separate tests. The distinction matters because a map or an in-memory database cannot establish survival after process loss.

The gateway verifies the worker result against the teaching domain before marking complete. For a real expensive calculation, independent verification might use checksums, a reference model, domain constraints, or a different validation strategy.

The square function makes the comparison exact and inexpensive without requiring floating-point tolerances.

Failure clinic — One key is reused for changed work

A client submits key batch9 with input 12, then retries batch9 with input 13 after editing a form. This is not an idempotent retry. The service returns conflict rather than replacing the accepted job or pretending the new input was executed.

The user interface should distinguish resubmission of the same operation from a new operation. A new business intent receives a new identity; an uncertain result retains its original identity. That distinction belongs above the transport layer.

Engineering case — Identify the boundary that cannot be made atomic locally

Replace the worker's square calculation with an email send. The worker starts a SQLite transaction, sends the email, and then inserts a receipt. The process crashes after the provider accepts the email but before the receipt commits. A retry sees no receipt and sends again.

Reversing the order is also unsafe for a different reason. If the worker commits the receipt before sending and then crashes, a retry can return a saved success even though no email was sent. The local database cannot atomically commit an unrelated provider's effect simply by moving statements around.

Possible solutions depend on the external interface. A provider-supported idempotency key can bind retries to one accepted send under that provider's documented scope and retention. A transactional resource manager may support a stronger coordinated protocol. A workflow can record uncertain status and reconcile with the provider. Some domains accept possible duplicates with explicit user-facing semantics. None should be claimed without examining the actual service contract.

An outbox still helps: it preserves the obligation to attempt delivery. An inbox still helps: it records what the worker knows. But the gap between the external effect and local knowledge remains unless another mechanism closes or manages it. This is the distinction between a recoverable workflow and an imaginary universal transaction.

The lab chooses a result row as its durable effect precisely so that receipt and effect can be represented atomically in one SQLite transaction. The arithmetic is pure and repeatable. That makes the recovery argument inspectable without suggesting that all real-world side effects share the same convenience.

For an extension review, draw a box around the atomic resource. Put every state transition inside or outside that box. Then list crash points between external transitions. A diagram that places an email provider visually beside a database does not put them in the same transaction.

The acceptance criterion should be phrased at the actual boundary: one provider-accepted operation under a documented key contract, or eventual reconciliation of uncertain sends, for example. Avoid advertising exactly-once effects

merely because a local table contains a unique key.

Practice and implementation lab

Draw the five commit and acknowledgement intervals in the failure matrix. For each interval, identify which database contains the authoritative recovery information and which request is safe to repeat. Include a response that arrives after the caller has already stopped waiting.

Run the `integration_lost_reply`, `integration_restart`, and `integration_worker_restart` cases through CTest. Inspect the test source to see where the failure is injected. Do not infer hardware power-loss guarantees from process termination tests.

Review and solution reasoning

A durable outbox preserves an obligation; a durable inbox ties repeated delivery to a retained effect. Neither is a synonym for a broker, and neither can atomically control an unrelated external system without an additional protocol.

The strongest useful claim is bounded and specific: under the tested local transaction and retained-database assumptions, repeated delivery of the same key and canonical input returns one saved effect. That claim is actionable precisely because its limits are visible.

Chapter takeaway. Persist intent, bind identity to meaning, commit local effects with receipts, and make every acknowledgement gap recoverable.

Chapter 19 — Containing Failure with Retries, Circuit Breakers, and Bulkheads

A dependency slows down. Every caller retries immediately. A circuit breaker opens, then every instance probes recovery at the same instant. Meanwhile, a shared thread pool fills with waiting calls and unrelated requests stop progressing.

Resilience mechanisms can amplify an incident when their state, scope, and resource costs are unclear. This chapter treats retries as additional load, breakers as state machines, and bulkheads as explicit resource boundaries.

19.1 Spending a Retry Budget Deliberately

19.1.1 Distinguishing Attempts from Logical Operations

An attempt is one transmission or execution try. A logical operation can include several attempts under one stable identity. Track both. A service completing 100 logical jobs after 400 attempts has a different dependency burden from one completing them after 100 attempts.

A retry policy must define whether its maximum counts retries after the first attempt or all attempts including the first. The companion's policy names `max_attempts` and includes the first attempt. This prevents a common off-by-one mismatch between configuration and tests.

Retry only failures whose meaning permits it. A temporary connection failure before any request bytes were sent may be straightforward. A timeout after possible remote commit requires the idempotency contract from Chapter 18. Invalid input and authorization failures usually need correction rather than another identical attempt.

Choose one retry owner per boundary. If an SDK retries three times, an adapter retries three times, and an application retries three times, the dependency can receive up to 27 attempts for one logical action under that composition. Each layer may appear locally conservative while the combined policy is aggressive.

19.1.2 Combining Backoff, Jitter, and Remaining Time

Exponential backoff increases the delay ceiling as failures continue, up to a cap. Jitter spreads retry times so many clients do not remain synchronized. The companion uses a uniform delay between zero and the current capped ceiling for its teaching retry policy.

Backoff must fit within the operation's remaining budget. If only 20 ms remains and the next delay alone is 100 ms, sleeping and then starting a doomed attempt wastes resources. Return the appropriate timeout or pending state instead. A background durable workflow can have a different recovery horizon, but it still needs a defined policy.

Avoid integer overflow while doubling a delay. A wrapped value can become tiny and create a retry storm after a long outage. The model uses bounded arithmetic and tests the cap. Random generators are injected or explicitly seeded in tests so the range properties can be checked reproducibly.

Jitter is not a correctness proof and does not guarantee a maximum total attempt rate across a fleet. Combine it with concurrency limits, rate limits, and admission policy. A million clients each retrying infrequently can still overwhelm a small dependency.

NEW SECTION EXAMPLE

Spend one retry budget across all attempts

Charge work and backoff against a single 100ms deadline rather than restarting the budget per attempt.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    int remaining=100; unsigned attempts=0;
    for(int delay:std::array{10,20,40}){
        constexpr int work=25; if(remaining<work) break;
        remaining-=work; ++attempts;
        if(remaining<delay) break; remaining-=delay;
    }
    std::cout<<"attempts="<<attempts<<" remaining-ms="<<remaining<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Attempt 1 consumes 25ms then 10ms backoff, leaving 65. Attempt 2 consumes 25+20, leaving 20, so a third 25ms attempt is not admitted.

The delays are deterministic stand-ins for bounded backoff/jitter. Real retries must preserve the logical operation ID and classify retryable errors. A new 100ms timeout per attempt would violate the caller's total budget.

Expected behavior or output

```
attempts=2 remaining-ms=20
```

Concrete experiment

Increase total budget to 150 and trace each subtraction. Add seeded jitter that never exceeds the remaining time after reserving the next attempt's cost.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

19.2 Implementing a Circuit Breaker as a State Machine

19.2.1 Defining Closed, Open, and Half-Open Behavior

A closed breaker admits calls and records relevant outcomes. An open breaker rejects or redirects new calls according to policy. After a cooldown, a half-open state admits a bounded probe to determine whether ordinary traffic should resume. These names are useful only when transitions and counted outcomes are specified.

The model counts consecutive failures until a threshold opens the breaker. It admits one half-open probe at a time. A successful probe closes the breaker; a failed probe reopens it. This is a deliberately small single-owner policy, not a thread-safe distributed health system.

Define which failures affect the breaker. A client-side validation error may say nothing about dependency health. A timeout caused by local queueing may require local capacity action rather than declaring the remote service broken. Counting every exception indiscriminately can open a breaker for reasons outside its intended scope.

The open-state response must preserve semantics. Returning a fabricated success to avoid errors is not resilience. A read may use a documented stale fallback; a mutation may return unavailable or remain durably pending. The fallback needs its own capacity and correctness review.

19.2.2 Ignoring Completions from an Obsolete Breaker Epoch

Calls admitted before a breaker opens can complete afterward. If their successes blindly reset the breaker, an old result can close a newly opened state. The model attaches an epoch ticket to each admitted call and ignores completions from an older epoch.

This is an example of a broader asynchronous pattern: completion must be associated with the state generation that admitted it. A timer, connection attempt, or configuration reload can have the same stale-completion problem.

The epoch protects state transitions, not business effects. Ignoring an old breaker completion does not undo the underlying remote operation. The application's request identity and result handling still determine what that operation means.

Tests should construct the stale completion directly. Admit two calls, use failures to advance the state, then complete an old ticket successfully. Verify that it cannot erase the new open state. Sleep-based tests that hope to observe this race are less precise than explicit state-machine inputs.

NEW SECTION EXAMPLE

Ignore a stale success after a circuit-breaker epoch changes

An earlier request's completion must not close a breaker that opened due to newer failures.

main.cpp

```
#include <iostream>

int main() {
    enum class Mode{closed,open,half_open};Mode mode=Mode::closed;unsigned epoch=1;
    unsigned old_request_epoch=epoch;
    mode=Mode::open;++epoch;
    auto success=[&](unsigned request_epoch){if(request_epoch!=epoch)return
false;mode=Mode::closed;return true;};
    bool stale=success(old_request_epoch);mode=Mode::half_open;bool probe=success(epoch);
    std::cout<<"stale-closed="<<std::boolalpha<<stale<<" probe-closed="<<probe<<"
final-closed="<<(mode==Mode::closed)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Opening the breaker advances an epoch. A delayed success from epoch 1 is obsolete and cannot change epoch 2 state, even though its individual request succeeded.

The half-open probe carries the current epoch and may restore closed state on success. Production code must limit concurrent probes and define time-based transitions; this fixture isolates stale-completion handling.

Expected behavior or output

```
stale-closed=false probe-closed=true final-closed=true
```

Concrete experiment

Open the breaker again before the probe returns. Its success must now be ignored too, demonstrating why an enum alone is insufficient.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

19.3 Isolating Resource Exhaustion

19.3.1 Building Bulkheads Around Actual Scarce Resources

A bulkhead limits how much one class of work can consume so another class retains capacity. Separate worker pools, connection limits, queue capacities, or database budgets can implement it. The correct boundary depends on the scarce resource, not merely on service names.

Two separate thread pools can still share one saturated database connection pool. Two service processes can still share a full disk. A design review should trace shared resources below the visible architecture boxes and identify which failures remain coupled.

Reserve enough capacity for recovery and control operations. If status queries and shutdown commands must wait behind unlimited expensive jobs, operators may lose the ability to understand or stop overload. The teaching service exposes a small

handler path, but production control-plane isolation would require its own explicit design and tests.

Bulkheads also create utilization tradeoffs. Unused capacity in one partition may coexist with rejected work in another. Dynamic borrowing can improve efficiency but complicates guarantees. Begin with a simple bounded policy whose protection can be measured before introducing adaptive sharing.

19.3.2 Recovering Without a Synchronized Surge

When a dependency recovers, queued and retried work can exceed normal arrival rates. Closing every breaker and releasing all backlog at once may cause another outage. Ramp admitted concurrency, apply jitter, and preserve ordinary traffic fairness where required.

A recovery metric should include backlog age and clearance time, not only whether a health probe succeeds. A worker can answer PING while still having hours of delayed jobs to process. User-visible recovery depends on the business service, not one control response.

HarborDispatch's dispatcher sleeps after delivery failure and checks its stop request during short sleep segments. This makes shutdown cooperative without waiting for a single long backoff interval. It still depends on bounded RPC and database operations returning; it is not an instantaneous cancellation facility.

The integrated dispatcher does not use the model breaker or token bucket automatically. The examples teach separate mechanisms and their tests. Combining them is an exercise that requires choosing one state owner and defining how durable pending work behaves while the breaker is open.

NEW SECTION EXAMPLE

Protect separate resource pools with independent bulkheads

Exhausting a reporting pool must not consume the capacity reserved for interactive requests.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    struct Pool{unsigned used=0,limit;bool acquire(){if(used==limit)return false;++used;return true;}void release(){if(used)--used;}};
    Pool report{0,1},interactive{0,2};
    bool first=report.acquire(),overflow=report.acquire(),user=interactive.acquire();
    std::cout<<std::boolalpha<<"report="<<first<<" report-overflow="<<overflow<<"
    interactive="<<user<<'\n';
    report.release();interactive.release();
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Each pool owns its own permit count. The second report request is rejected at its scarce-resource boundary, but an interactive permit remains available.

The model is single-threaded; real permits need synchronized acquisition and RAI release. Separate queues are not a bulkhead if they still compete for the same unbounded connection or memory pool underneath.

Expected behavior or output

```
report=true report-overflow=false interactive=true
```

Concrete experiment

Give both pools a shared total memory budget and enforce both limits. Verify that isolation promises match the resource actually being protected.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Recover through one half-open probe

Requirement and implementation

Open a breaker after the configured failures, reject calls during cooldown, admit only one recovery probe, and close only after that probe succeeds under the current epoch.

Source — source/examples/ch19.cpp

```
#include "harbor/check.hpp"
#include "harbor/policies.hpp"
#include <iostream>
int main() {
    harbor::Breaker circuit(2,100);
    circuit.complete(*circuit.acquire(0),false,0);
    circuit.complete(*circuit.acquire(1),false,1);
    harbor::check(!circuit.acquire(100),"open cooldown");
    auto probe=circuit.acquire(101); harbor::check(probe.has_value(),"probe available");
    harbor::check(!circuit.acquire(101),"only one recovery probe");
    circuit.complete(*probe,true,102);
    harbor::check(circuit.state()==harbor::Breaker::State::closed,"closed after proof");
    std::cout<<"Two failures opened the circuit; one later probe established recovery.\n";
}
```

Evidence and interpretation

The test advances an explicit tick value. It verifies state transitions without waiting for real seconds. That makes exact equality boundaries and stale epochs repeatable. It does not measure live dependency behavior or prove that the chosen threshold is operationally appropriate.

The policy tests also inspect jitter limits and capped delays. A random sample within the allowed range is not a promise of a particular delay distribution in production, but checking that no sample exceeds the cap catches arithmetic and range errors.

Failure clinic — A retry repairs transport but duplicates intent

An application catches every timeout and creates a new job key. The retry then succeeds, and the first attempt later completes too. The resilience layer improved the chance of a response while breaking the one-operation intent.

The repair is to carry the same logical identity across attempts and distinguish uncertain completion from new business work. Retry code should receive the identity as immutable request data rather than generate it inside the attempt loop.

Engineering case — Prevent an old success from closing a new breaker

At tick 0, a closed breaker admits calls A and B under epoch 3. A fails, and enough failures open the breaker and advance its epoch. B succeeds later. If the completion handler simply resets the failure count and closes the breaker, B's stale result can undo the newer state transition.

The epoch ticket solves this local race by tying completion to the generation that admitted it. B's result can still be recorded as a successful attempt for observability, but it cannot mutate the current breaker's state when its ticket is obsolete. The business response and policy update are separate decisions.

After cooldown, one half-open probe is admitted under the current epoch. Other callers are rejected or use the documented fallback until that probe resolves. Without a single-probe rule, every waiting caller can become a recovery probe and recreate the original overload.

A per-process single probe does not mean one probe across the fleet. If 500 instances share the same cooldown, 500 probes can still arrive together. Jitter, staggered recovery, and a dependency capacity budget address that larger scope. A local unit test cannot establish fleet-wide behavior without a corresponding model or integration.

Also consider what counts as failure. A caller cancelling before a request is sent may not indicate dependency health. A permission denial can show a configuration problem rather than a transient service outage. A classification policy should be testable and should avoid treating every exception as evidence for the same breaker.

The model tests explicit events rather than relying on real timing. To extend it, record an event history containing acquire, success, failure, cooldown expiry, and stale completion. Verify both state and admission decisions after each event. If the policy changes from consecutive failures to a rolling window, update the model and evidence rather than retaining tests for a different algorithm.

This case illustrates a common pattern in asynchronous C++: an operation's result may remain meaningful while its right to change a newer owner state has expired. Generation-aware completion handling makes that distinction explicit.

Practice and implementation lab

Add a simulated dependency with outcomes failure, timeout-after-effect, and success. Combine a finite caller budget with the retry policy and verify that no attempt begins after expiry. Preserve the request key across all attempts.

Then add a second work class with an independent concurrency budget. Saturate the first class and verify that the second can still make progress. Document any remaining shared resource that could defeat the isolation.

Review and solution reasoning

A correct design assigns every policy a scope: logical operation, dependency, tenant, process, or fleet. It also identifies which component owns state transitions. A single-owner breaker can be correct locally while independent fleet-wide probes still create a surge.

Retries, breakers, and bulkheads solve different parts of failure containment. Their composition is useful only when it preserves the business contract and keeps total resource demand bounded.

Chapter takeaway. Count attempts as load, reject stale policy completions, and isolate the resources whose exhaustion would otherwise spread failure.

Chapter 20 — Engineering Backpressure and Admission Control

A producer is faster than its consumer. The queue grows, latency rises, and memory eventually runs out. Adding more memory delays the failure without changing the relationship between arrival rate and service capacity.

Backpressure makes that relationship explicit. Admission decides which work becomes the system's obligation; buffering decides how admitted work waits; scheduling decides how it progresses. This chapter treats these as separate policies with separate measurements.

20.1 Bounding Every Waiting Location

20.1.1 Distinguishing Connection, Memory, and Durable Backlog Limits

A service can queue work in a socket receive buffer, a connection object, an executor, a database table, and a downstream broker. Limiting only one location does not bound the entire path. A request rejected by one queue may already occupy memory elsewhere.

HarborDispatch defines concrete teaching limits: at most 64 live peer connections per server, a payload no larger than 4096 bytes, a two-second cooperative connection deadline, and at most 128 new pending gateway jobs. These numbers are lab design choices, not universal sizing recommendations.

Completed jobs and worker receipts remain retained. The pending limit therefore does not bound total database size. Similarly, a fixed connection count does not guarantee a fixed process resident set because libraries, allocator behavior, database caches, and thread stacks contribute additional memory.

Create a resource budget table before capacity testing. For each waiting location, record its maximum entries, maximum item size, ownership, timeout or retention rule, and overload response. This often reveals an unbounded hidden list before a stress test finds it through an outage.

20.1.2 Choosing an Overload Response at the Admission Boundary

A rejected new request is different from an accepted request that later fails. Return that distinction clearly. If the system has not committed an obligation, it can tell the caller to retry according to a bounded policy. Once it has accepted durable work, dropping it silently violates the contract.

The gateway checks an existing key before pending capacity. An identical duplicate does not consume a new slot and can still receive `ACCEPTED` when the backlog is full. A changed input conflicts. A genuinely new key receives `BUSY` if the pending limit is reached.

That ordering is important for recovery. A caller with an uncertain response must be able to discover an already accepted operation even during overload. Treating all requests as new capacity demand can prevent the very retries needed to resolve ambiguity.

Google's SRE discussion of overload emphasizes protecting services from demand beyond their capacity rather than relying on unlimited queueing. The chapter applies that operational idea to explicit local limits and testable outcomes. (R32)

NEW SECTION EXAMPLE

Admit only when connection and byte budgets both allow it

Bound aggregate buffering independently from the number of requests.

main.cpp

```
#include <cstdint>
#include <iostream>

int main() {
    unsigned connections=0;std::size_t bytes=0;
    auto admit=[&](std::size_t size){if(connections==3||size>100-bytes)return
false;++connections;bytes+=size;return true;};
    bool a=admit(60),b=admit(50),c=admit(30);
    std::cout<<std::boolalpha<<a<<' '<<b<<' '<<c<<" connections="<<connections<<"
bytes="<<bytes<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Two independent limits prevent a small number of huge requests from exhausting memory. The subtraction $100 - \text{bytes}$ is safe because every accepted transition maintains $\text{bytes} \leq 100$.

Rejecting the 50-byte request leaves both counters unchanged, so a later 30-byte request still fits. This model counts committed reservations; asynchronous implementations must release them exactly once on every terminal path.

Expected behavior or output

```
true false true connections=2 bytes=90
```

Concrete experiment

Add `release(size)` with checked accounting and test double-release rejection. Do not use an unchecked unsigned subtraction that wraps the memory ledger.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

20.2 Coordinating Producers and Consumers Safely

20.2.1 Defining Queue Ownership and Close-and-Drain Semantics

The bounded queue owns accepted values. `try_push` returns false when the queue is full or closed. `pop` waits until a value is available or closure makes further values impossible. Closing the queue wakes waiters and allows already accepted values to drain.

The predicate is the shared state, not the notification itself. A condition-variable wake can be spurious, and a notification can occur before a waiter begins waiting. The implementation checks the predicate under the same mutex that protects the queue and closed flag.

The queue must outlive every thread using it. A destructor that destroys the mutex while a consumer is blocked is not made safe by the fact that the container is an RAII object. The owner must close, join, and only then destroy the shared state.

The teaching concurrent test uses bounded producers and consumers, verifies exact accepted-item accounting, and checks the sum after drain. Its producer retry loop yields while the queue is full. That is a finite test strategy, not a recommendation to busy-retry indefinitely in a production service.

20.2.2 Preserving Ownership When Admission Fails

An API taking a task by value may move it into the function before discovering that admission fails. The caller must know whether rejection leaves its original task intact or whether it must reconstruct the request from retained logical data. This matters for move-only buffers and closures.

Design the interface accordingly. A reservation API can acquire capacity before transferring a large resource. A `try_submit` can return the rejected task. A caller can retain an immutable request description and construct the task only after admission. Each choice has different complexity and lifetime costs.

Do not silently drop exceptions inside a worker. A task outcome should become a result, a stored error, or an observable failure path. An exception that terminates a worker can reduce capacity and leave queued work stranded if the supervisor assumes the thread still exists.

Shutdown should stop admission before joining consumers. If a consumer waits forever for a producer that has already been joined or destroyed, cleanup deadlocks. Write the ownership graph and shutdown order explicitly; then test partial startup failure as well as ordinary steady-state termination.

NEW SECTION EXAMPLE

Preserve a caller-owned object when queue admission fails

Accept unique ownership only after a boundedqueue has space, then close and drain already accepted work.

main.cpp

```
#include <deque>
#include <memory>
#include <iostream>

int main() {
    std::deque<std::unique_ptr<int>> q; bool closed=false;
    auto push=[&](std::unique_ptr<int>&item){if(closed||q.size()==1) return
false;q.push_back(std::move(item));return true;};
    auto a=std::make_unique<int>(7),b=std::make_unique<int>(9);bool
accepted=push(a),full=push(b);closed=true;
    int drained=*q.front();q.pop_front();
    std::cout<<std::boolalpha<<"accepted="<<accepted<<" full="<<full<<"
caller-retain="<<bool(b)<<" drained="<<drained<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

push takes the unique_ptr by reference so it can test capacity before moving. After successful admission a is empty; after rejection b still owns 9 and can retry or release it deliberately.

Closing changes admission policy but does not destroy queued objects. The consumer drains 7 before the queue is released. A concurrent version needs one synchronization rule around capacity, close, and ownership transfer.

Expected behavior or output

```
accepted=true full=false caller-retain=true drained=7
```

Concrete experiment

Attempt push(b) after close even though the queue is now empty. It must still fail and leave b owned by the caller.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

20.3 Regulating Rate and Concurrency

20.3.1 Using Token Buckets Without Confusing Them with Queue Limits

A token bucket allows bursts up to a capacity and replenishes tokens at a rate. It controls admission over time. A concurrency limit controls simultaneous in-flight work. A queue limit controls waiting storage. A service may need all three because none implies the others.

The model uses explicit ticks, a finite positive capacity, and a nonnegative replenishment rate. It rejects backward time because silently subtracting elapsed time could create invalid token state. The model is single-owner; concurrent callers require synchronization around the combined refill-and-consume operation.

Choose the cost unit. One token per request assumes roughly equal work. Weighted cost can represent bytes or estimated computation, but estimates can be wrong or adversarial. Combine client-provided declarations with server-side validation and maximum actual resource use.

A per-process bucket multiplies with replica count. Ten instances each configured for 100 requests per second can admit roughly ten times that rate in aggregate under their policies. A fleet-wide quota needs an authority or approximation strategy and a stated behavior during coordination failure.

20.3.2 Using Queueing Arithmetic as a Diagnostic, Not a Guarantee

For a stable system under appropriate averaging conditions, Little's law relates average number in the system L , average arrival rate λ , and average time W : $L = \lambda \times W$. It is a useful consistency check on measurements, not a formula that guarantees tail latency or stability under arbitrary bursts. (R45)

As a hypothetical example, 200 completed requests per second with an average end-to-end time of 0.05 seconds implies about ten requests in the measured system on average. If observed queue occupancy is much larger, verify that the boundaries, units, and averaging intervals match before tuning thread counts.

Near saturation, waiting can dominate service time. Increasing admitted concurrency may reduce throughput if it creates contention, cache pressure, or database lock queues. Measure completed useful work and latency distributions, not only CPU utilization.

Set overload criteria before a load test. The desired result may be stable latency for admitted work plus explicit rejections above capacity. A benchmark that reports higher throughput by accepting work it never completes is measuring admission, not service delivery.

NEW SECTION EXAMPLE

Separate a token bucket from concurrent request capacity

Rate permits accumulate with `virtualtime`, while a separate in-flight limit protects outstanding work.

main.cpp

```
#include <algorithm>
#include <iostream>

int main() {
    double tokens=2,last=0;unsigned in_flight=0;
    auto admit=[&](double now){tokens=std::min(2.0,tokens+(now-last)*1.0);last=now;
        if(tokens<1||in_flight==1)return false;tokens-=1;++in_flight;return true;};
    bool a=admit(0),b=admit(0);--in_flight;bool c=admit(0);--in_flight;bool
    d=admit(0),e=admit(1);
    std::cout<<std::boolalpha<<a<<' '<<b<<' '<<c<<' '<<d<<' '<<e<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The bucket initially permits a burst of two, refilling at one token/second up to capacity two. The in-flight cap of one can reject a request even when a token is available; that rejection does not consume the token.

After two completions at virtual time zero, tokens are exhausted; at one second a new token permits work. A token bucket limits admission rate, not queued memory or request duration.

Expected behavior or output

```
true false true false true
```

Concrete experiment

Change in-flight capacity to two. The second simultaneous request can now run, but sustained admission must still obey the one-token/second refill.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Bound, close, and drain

Requirement and implementation

Accept two items into a queue of capacity two, reject a third, consume accepted items, close the queue, and verify that no new work is admitted. Separately demonstrate a bounded token bucket.

Source — source/examples/ch20.cpp

```
#include "harbor/check.hpp"
#include "harbor/queue.hpp"
#include "harbor/policies.hpp"
#include <iostream>
int main() {
    harbor::BoundedQueue<int> queue(2);
    harbor::check(queue.try_push(1) && queue.try_push(2), "admission");
    harbor::check(!queue.try_push(3), "capacity bound");
    queue.close();
    harbor::check(queue.pop()==1 && queue.pop()==2 && !queue.pop(), "close then drain");
    harbor::TokenBucket quota(2, 10);
    harbor::check(quota.admit(2, 0) && !quota.admit(1, 0) && quota.admit(1, 100), "quota refill");
    std::cout<<"Queue capacity bounds residence; token rate bounds admission frequency.\n";
}
```

Evidence and interpretation

The simple example exposes state transitions in one thread. The `core_queue` test adds multiple producers and consumers and checks exact accounting after closure. Neither demonstrates arbitrary scheduler fairness or a hard upper bound on wall-clock completion time.

The `integration_overload` test stops the worker, submits 128 unique jobs, verifies rejection of the next new key, and checks that an identical duplicate remains accepted. It then inspects persisted pending state. This connects admission policy to the actual network application rather than only a container class.

Failure clinic — The queue is bounded but the retry list is not

A frontend rejects tasks when its queue is full, then appends every rejected task to an unbounded retry vector. The queue's capacity test passes while process memory still grows without limit. Follow rejected work to its final owner and retention policy. A bound is useful only if overflow does not move into an unbounded neighboring structure.

Engineering case — Budget a queue in bytes as well as entries

A queue of 100 requests appears bounded. If each request can own a 10 MiB payload, the queue alone can retain roughly one GiB before accounting for container overhead, in-flight copies, and response buffers. An entry limit is not a byte budget.

For the teaching wire protocol, the 4096-byte frame limit constrains one payload. A 64-peer server therefore has a bounded number of bounded request buffers under its implementation, but the process also owns output buffers, database state, allocator metadata, and library caches. Compute an estimate from the actual ownership graph rather than multiplying one number and calling it a complete memory guarantee.

Admission can reserve both an entry slot and a byte allowance. If either reservation fails, reject before transferring ownership of a large buffer. Release both reservations on every terminal path: normal completion, parsing failure, cancellation, timeout, and shutdown. A leaked reservation can mimic a memory leak by permanently reducing capacity.

Weighted admission creates another issue: a client can understate expected work. Validate declared lengths before allocation and enforce measured limits during processing. For expensive operations, an estimated cost can guide fairness while a hard maximum prevents one inaccurate estimate from consuming unbounded resources.

The durable pending quota also needs a disk perspective. Historical completed rows remain retained, and SQLite indexes and WAL activity contribute bytes beyond the logical input. A process with only ten pending jobs can still run out of disk after retaining millions of completed records.

Test capacity boundaries with exact counts and sizes. Verify that a duplicate submission does not reserve a second durable slot, that rejection leaves prior work intact, and that closing a queue drains accepted values without leaking permits. A stress test should report both admission and completed work so it cannot appear successful by simply buffering more requests.

This case turns “bounded” from an adjective into a measurable contract. Name the resource, unit, owner, release point, and overflow action. Without those five details, a

limit in one class can coexist with unbounded growth elsewhere.

Practice and implementation lab

Draw every waiting location from client to worker. Assign a maximum size and lifetime to each. Include connection establishment, request framing, pending storage, dispatcher batch, worker transaction, and response buffering.

Add a second admission class for status reads and define how it behaves when mutation admission is full. Test that the chosen policy does not let cheap probes starve useful work or let expensive work prevent recovery observations.

Review and solution reasoning

A sound overload policy preserves the distinction among rejected, accepted-pending, and completed work. It also separates rate, concurrency, memory, and disk budgets. A queue length alone cannot summarize those resources.

Boundedness is a safety property about resource growth; progress and fairness require additional assumptions and policies. Tests should report both without claiming that one proves the other.

Chapter takeaway. Make admission an explicit promise, bound every waiting location, and measure completed work rather than the number of requests merely allowed to wait.

Chapter 21 — Coordinating Workflows and Compensating Partial Success

A workflow reserves capacity, charges an account, and then fails to create the report. Rolling back one local database does not undo a completed charge in another system. A distributed workflow must represent partial success rather than pretend it never happened.

This chapter introduces explicit workflow states, compensation, durable orchestration, and event identity. The local saga model is intentionally small: it checks transition rules, not a complete payment or inventory system.

21.1 Modeling the Business Process as State

21.1.1 Naming Intermediate and Terminal Outcomes

A workflow should describe what has actually happened. States such as created, reserved, charged, completed, compensating, and cancelled distinguish obligations that need different recovery actions. A single Boolean success cannot represent a charge that succeeded while final delivery failed.

Define legal transitions and their preconditions. A charge event requires a reservation in the model. A finish event requires the charged state. A cancellation after reservation or charge enters compensating rather than immediately claiming that all prior effects have been undone.

Terminal states should have precise meanings. Cancelled in the model means compensation has been acknowledged under the model's rule. In a real workflow, it may mean a refund was requested, settled, or accepted for later processing; those are different promises. Choose terminology that matches the external provider's contract.

State names also guide authorization. The right to submit a job does not necessarily include the right to refund, force-complete, or replay a workflow. Administrative transitions should be explicit operations with audit evidence, not manual database edits disguised as ordinary traffic.

21.1.2 Separating Orchestration from Choreography

An orchestrator records workflow state and decides which command to issue next. Choreography lets participants react to events without one central step controller. Both can work, but each distributes reasoning differently.

An orchestrator makes the overall state machine easier to inspect, while becoming an authority that needs persistence, recovery, and controlled ownership. Choreography can reduce direct coordination but may make dependencies and failure paths harder to discover when many consumers react independently.

Choose based on the process's complexity, ownership, and recovery needs rather than a blanket rule that one style is more modern. A workflow crossing several teams may benefit from explicit versioned contracts even when one service orchestrates its steps.

HarborDispatch's gateway dispatcher is a simple orchestrator for one remote step. It does not implement the multistep saga model automatically. The distinction prevents a reader from assuming that the capstone can compensate arbitrary external effects merely because this chapter describes the concept.

NEW SECTION EXAMPLE

Represent an order workflow with explicit intermediate states

Authorize only the next legal business transition rather than inferring progress from log messages.

main.cpp

```
#include <iostream>

int main() {
    enum class State{created,reserved,charged,completed,compensating};State s=State::created;
    auto reserve=[&]() {if(s!=State::created)return false;s=State::reserved;return true;};
    auto charge=[&](bool ok){if(s!=State::reserved)return
false;s=ok?State::charged:State::compensating;return true;};
    bool out_of_order=charge(true);reserve();charge(false);
    std::cout<<"early-charge="<<std::boolalpha<<out_of_order<<"
compensation-needed="<<(s==State::compensating)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Charging is illegal before reservation. A failed charge moves into the named compensating state instead of pretending that the workflow returned to its initial state.

An orchestrator can persist these transitions and dispatch the next command. Choreography distributes reactions among event consumers, but still needs explicit state and operation identity to avoid contradictory effects.

Expected behavior or output

```
early-charge=false compensation-needed=true
```

Concrete experiment

Add a release-reservation completion that leaves compensating only after its result arrives. Retrying that command must retain its operation identity.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

21.2 Making Each Step Recoverable

21.2.1 Persisting Commands, Results, and Timers

A workflow engine must survive losing its process memory. Persist the current state, the identity of issued commands, relevant results, and the information needed to resume waiting or retrying. A sleeping thread is not a durable timer.

When a step command and state transition must remain consistent, use a local transaction and outbox pattern. Record that the step is pending and record the outgoing command together. On restart, reissue the same command identity under the downstream idempotency contract.

A result can arrive after a timeout or cancellation decision. The workflow must recognize which step generation it belongs to and decide whether to incorporate it, compensate it, or reject it as obsolete. Dropping all late responses can hide an effect that still needs reversal.

Persisted time requires the clock considerations from Chapter 10. A monotonic in-process deadline cannot simply be resumed after a host reboot by reusing its raw count. Store a defined business deadline or a recoverable timer representation, and specify how clock uncertainty affects execution.

21.2.2 Deduplicating Events Without Erasing Conflicts

The model maps event identity to event kind. Repeating the same identity and kind is a duplicate and causes no new transition. Reusing the same identity for another kind is a conflict. This prevents a replay from changing its meaning silently.

A real event may also carry payload, operation version, and actor identity. The deduplication comparison must include the fields that determine the transition's meaning. Recording only the event name can accept a changed amount or destination under the same identifier.

Duplicate recognition must be atomic with the state transition. If the state changes and the receipt is stored separately, a crash can allow the event to be applied again. The local model has no persistence; a production implementation would put both records in a transaction or a replicated state-machine operation.

Out-of-order events require policy. A finish received before charge might be invalid, delayed, or evidence that an earlier event was missed. The model rejects it. Another application might buffer within a bound and reconcile. Neither policy should be implicit in a container insertion order.

NEW SECTION EXAMPLE

Deduplicate workflow events without hiding conflicting payloads

Bind an event ID to its result and reject a later changed result under the same identity.

main.cpp

```
#include <map>
#include <string>
#include <iostream>

int main() {
    std::map<std::string,int> seen;int applied=0;
    auto event=[&](const std::string&id,int value){auto
it=seen.find(id);if(it!=seen.end())return it->second==value?"duplicate":"conflict";
    seen.emplace(id,value);applied+=value;return "applied";};
    std::cout<<event("reserve7",3)<<' '<<event("reserve7",3)<<' '<<event("reserve7",4)<<"
total="<<applied<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

An identical redelivery is harmless, but the same event ID with value 4 contradicts the previously recorded value 3. Treating both as duplicates would hide a real corruption or producer-bug signal.

This in-memory model loses receipts on restart. A recoverable workflow must persist the effect and deduplication record together and preserve pending timers and commands so recovery does not require guessing the earlier intent.

Expected behavior or output

```
applied duplicate conflict total=3
```

Concrete experiment

Serialize both seen and applied, restore them, then redeliver reserve7. Restoring only the total cannot identify the duplicate.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

21.3 Designing Compensation as New Work

21.3.1 Understanding Why Compensation Is Not Time Travel

Compensation is another business action that aims to restore an acceptable condition. It may not reverse every consequence of the original action. A refund can incur fees, a released reservation may no longer be available, and a sent notification cannot be unsent from a recipient's memory.

Define compensation requirements per step. Some steps are naturally reversible; others require a different remedial action or human review. A workflow should identify the point beyond which cancellation is no longer promised or requires a special outcome.

Compensation can fail too. It needs its own operation identity, retries, deadlines, observability, and terminal escalation. Immediately marking a workflow cancelled

when a refund command is merely sent confuses intent with effect in exactly the way earlier chapters warned against.

The order of compensation depends on dependencies among effects. Reversing steps mechanically in reverse order can be a useful starting point, but domain constraints may require another sequence. Document why each compensation is safe in the state where it is executed.

21.3.2 Reconciling Stuck and Ambiguous Workflows

A reconciliation process compares durable workflow state with authoritative downstream state. It can detect a step whose response was lost, a compensation that remains pending, or an externally completed action not yet reflected locally. Reconciliation is part of the design, not evidence that the normal path was useless.

Manual repair must preserve identity and auditability. Provide commands such as retry step, confirm external receipt, or request compensation with recorded reason and actor. Directly flipping a status column can bypass invariants and leave pending outbox commands that later run unexpectedly.

Define an escalation age for stuck workflows and a responsible owner. A dashboard containing thousands of permanently “in progress” rows is not operational control. Include the last successful transition, last attempt, current authority, and next eligible action in diagnostic state.

For the lab service, a pending job has a simpler recovery path: replay the same worker command until a result can be recorded, subject to its documented limitations. Extending it to a saga requires additional durable states and commands rather than just adding more network calls inside the dispatcher loop.

NEW SECTION EXAMPLE

Treat compensation as a new auditable operation

Refund a completed charge through an idempotent compensating command rather than deleting history.

main.cpp

```
#include <map>
#include <string>
#include <iostream>

int main() {
    std::map<std::string,int> ledger;
    auto apply=[&](std::string id,int
amount){auto[it,inserted]=ledger.emplace(std::move(id),amount);return inserted;};
    apply("charge7",-30);bool
refund=apply("refund-charge7",30),retry=apply("refund-charge7",30);
    int net=0;for(auto&[id,amount]:ledger)net+=amount;
    std::cout<<std::boolalpha<<"refund="<<refund<<" repeated="<<retry<<"
records="<<ledger.size()<<" net="<<net<<"'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The original charge remains visible and the refund has its own identity. A net balance of zero does not mean the charge never occurred: fees, notifications, and authorization histories may still exist.

A repeated refund ID does not add another 30. Production code must also reject reuse of an ID with a different amount and reconcile ambiguous remote outcomes; this model demonstrates compensation rather than time-travel rollback.

Expected behavior or output

```
refund=true repeated=false records=2 net=0
```

Concrete experiment

Add a nonrefundable fee entry of -2. The compensated net is now -2, demonstrating why compensation need not reverse every observable effect.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Make compensation an explicit transition

Requirement and implementation

Reserve, charge, request cancellation, and then acknowledge compensation. Verify that duplicate acknowledgements do not repeat transitions and that a late forward transition is rejected after cancellation.

Source — source/examples/ch21.cpp

```
#include "harbor/check.hpp"
#include "harbor/models.hpp"
#include <iostream>
int main() {
    harbor::Saga saga;
    saga.event("e1", "reserve"); saga.event("e2", "charge");
    saga.event("e3", "cancel"); saga.event("e4", "compensated");
    harbor::check(!saga.event("e4", "compensated"), "duplicate compensation acknowledgment");
    harbor::check(saga.state() == harbor::Saga::State::cancelled, "terminal state");
    harbor::rejects([&]{saga.event("e5", "finish");}, "late forward success");
    std::cout << "Compensation is an explicit transition, not time running backward.\n";
}
```

Evidence and interpretation

The model reaches cancelled only through compensating. This distinguishes a request to undo from an acknowledgement that the model's compensation condition holds. Event identities prevent the same acknowledgement from changing state twice.

The model does not perform an actual reservation, charge, or refund. Its names describe an invented workflow for reasoning. Production use would need provider contracts, durable event storage, authorization, and reconciliation tests. Do not present the executable as a payment integration.

Failure clinic — A late success is discarded during cancellation

A charge request times out, the workflow begins cancellation, and a delayed success arrives. Discarding the response because the state is no longer “charging” can leave a completed charge with no compensation. The state machine should record the evidence and determine whether an additional compensating action is required.

This is another reason to separate logical operation identity from attempt identity. The late response may belong to the original step even when a newer attempt or cancellation command exists.

Engineering case — Handle cancellation while a charge outcome is unknown

A workflow reserves capacity and sends a charge command. The charge response is lost. The user requests cancellation. The workflow cannot safely assume either that no charge occurred or that a charge definitely occurred. Its state must represent uncertainty and preserve the original charge identity.

A reconciliation step can query the payment authority using that identity, or repeat the same idempotent charge request if the provider’s contract makes that safe. If the charge is confirmed, the workflow can issue a separately identified refund or other compensation. If it is confirmed absent and no in-flight action can still create it under the protocol, the workflow can release the reservation without claiming a refund happened.

A late charge success can arrive after cancellation begins. The orchestrator should associate it with the original step and update the recovery obligation, not discard it because the current UI says cancelled. A user-visible cancellation request and completed compensation are different states.

The compensation itself can have an uncertain reply. It needs its own stable identity and status path. Reusing the original charge key for a semantically different refund can violate the provider’s identity contract; generating a new refund identity on every retry can duplicate compensation. Operation type and logical intent must remain explicit.

The local saga model uses a simplified compensated event to acknowledge completion. It does not perform or verify a real refund. A production implementation should refine that event’s meaning according to the external authority and preserve the corresponding receipt or reference.

An operator repair command should not merely overwrite state to cancelled. It should record the evidence used, the actor, the intended reconciliation transition, and any remaining external obligation. Otherwise the normal dispatcher may later issue a command that the manual edit forgot to remove or supersede.

This scenario explains why a saga is not a distributed rollback instruction. It is a stateful recovery process that must account for partial knowledge, repeated messages, and actions whose consequences cannot be erased instantaneously.

Practice and implementation lab

Add a failed-compensation state and a retry command with a stable identity. Construct histories in which compensation succeeds on the first attempt, succeeds after an uncertain reply, and remains unresolved. Define the operator-visible outcome for each.

Then persist the model state and event receipts in one SQLite transaction as a separate exercise. Kill a test process between transaction begin and commit, reopen the database, and verify that the event and state transition appear together or neither appears. Keep this exercise isolated from the capstone's version 1 schema.

Review and solution reasoning

A correct workflow distinguishes accepted commands, observed effects, compensation intent, and compensation completion. It also treats reconciliation as a first-class path with authority and audit rules.

The saga model illustrates legal state transitions; it does not make independent databases atomic. Its usefulness comes from representing partial success honestly and giving recovery a defined place in the architecture.

Chapter takeaway. Model partial success, persist recovery obligations, and treat compensation as accountable new work rather than an imaginary rollback of the world.

Chapter 22 — Securing Service Identity and Authorization Boundaries

A request carries a field named `tenant`, and the service trusts it. A TLS connection succeeds, and the application assumes the caller may perform any operation. A support log records an entire credential-bearing request because that seemed convenient during debugging.

These are boundary errors, not missing encryption algorithms. Security architecture must connect verified identity, permitted action, resource ownership, transport protection, and operational handling. The lab remains loopback-only and deliberately does not claim a production authentication or TLS implementation.

22.1 Establishing Identity Before Making Policy Decisions

22.1.1 Distinguishing Authentication from Authorization

Authentication establishes an identity under a trust mechanism. Authorization decides whether that identity may perform a particular action on a particular resource. A successfully authenticated service is not necessarily allowed to read another tenant's job or force a workflow transition.

Represent a verified principal separately from untrusted request fields. A transport or identity adapter can validate credentials and construct a principal containing approved identity attributes. Domain policy then evaluates the action and resource using that principal. Do not let ordinary request parsing construct trusted identity merely because the input contains the right field names.

The chapter example accepts a principal supplied by its test fixture and checks tenant equality plus a named permission. Its comment makes the trust assumption explicit: a verified adapter must supply this object in a real system. The example does not parse or validate tokens, certificates, passwords, or signatures.

An authorization decision should include both operation and object scope. Permission to submit work does not imply permission to inspect every result. A key lookup should enforce the tenant boundary even when the key is difficult to guess. Unpredictable identifiers are not a substitute for access control.

22.1.2 Preserving Identity Across Service Hops

A gateway can authenticate an external caller and then call a worker under its own service identity. The worker needs to know which claims it may trust and whether it should enforce end-user scope, gateway scope, or both. Blindly forwarding arbitrary user headers can turn the gateway into a confused deputy.

Define delegation explicitly. A signed or otherwise protected assertion can carry a limited audience, action scope, expiration, and correlation with the original request. The downstream service validates it according to the selected identity system rather than trusting a string because it arrived from a familiar network segment.

Do not reuse one omnipotent credential across every service. Narrow identities reduce the consequence of one component being compromised. Operational access, deployment access, and runtime data access should have distinct responsibilities and audit trails.

The integrated HarborDispatch protocol has no principal field or authorization enforcement. It binds only to IPv4 loopback for controlled teaching exercises. A production adapter must be designed and tested before changing the listener to a remotely reachable address. Local binding reduces exposure; it does not protect against every untrusted local process.

NEW SECTION EXAMPLE

Authorize a tenant-scoped operation after identity is established

Use an authenticated principal and explicit tenant/scope checks rather than trusting identity fields in the request body.

main.cpp

```
#include <string>
#include <set>
#include <iostream>

int main() {
    struct Principal{std::string subject,tenant;std::set<std::string> scopes;};
    Principal verified{"worker7","tenantA",{"job.read"}};
    auto allow=[&](const std::string&tenant,const std::string&operation){return
verified.tenant==tenant&&verified.scopes.contains(operation)};
    std::cout<<std::boolalpha<<" read-own="<<allow("tenantA","job.read")<<"
write="<<allow("tenantA","job.write")<<" other-tenant="<<allow("tenantB","job.read")<<"'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The principal is a trusted fixture standing in for completed authentication. Authorization then checks both resource tenant and operation scope. An authenticated identity is not automatically permitted every operation.

Do not construct verified from arbitrary JSON supplied by the caller. At service hops propagate verified claims through a trusted mechanism and distinguish the end user from the calling service identity.

Expected behavior or output

```
read-own=true write=false other-tenant=false
```

Concrete experiment

Add `job.writetothescopescope` and confirm cross-tenantaccess remainsdenied. Scope escalation mustnotbypass resourceownership.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

22.2 Protecting Transport Without Losing Application Semantics

22.2.1 Validating Peers, Not Merely Encrypting Bytes

TLS provides a secure-channel protocol, but an application must configure trust and peer verification correctly. A client that encrypts traffic while accepting any certificate can still communicate with an unintended peer. Certificate chain validation and expected peer identity are related but distinct checks. (R29)

For hostname-based connections, use the library's supported expected-host verification mechanism rather than manually comparing a certificate display string. OpenSSL documents `SSL_set1_host` for configuring the expected peer name in its verification workflow. A complete integration must also configure verification and trust correctly; one function call is not an entire security setup. (R41)

Mutual TLS adds client certificate authentication under the selected trust policy. It does not automatically decide which application operations that client may perform. Map authenticated identities into explicit authorization policy and define rotation, expiration, and revocation handling according to the chosen infrastructure.

gRPC documents credential mechanisms and TLS integration for its APIs. Use those supported facilities when building the optional gRPC adapter rather than wrapping an insecure custom channel and labeling it authenticated. That integration is not compiled or deployed by the default companion. (R28)

22.2.2 Accounting for Replay, Termination, and Proxy Boundaries

Transport security and application idempotency solve different replay problems. A user can repeat a valid authenticated operation intentionally or after an uncertain response. The service still needs operation identity and conflict detection. Encryption does not make retries semantically unique.

TLS 1.3 early data has replay considerations, so mutation handling must follow the protocol's constraints and the application's safety model. Do not enable a latency optimization for non-idempotent operations merely because the handshake is faster. (R29)

When a proxy terminates TLS, identify the remaining hop's protection and the trust boundary for forwarded identity. A header claiming the original caller is trustworthy only when untrusted clients cannot forge it through an alternate path. Network policy, proxy configuration, and application validation must agree.

Health endpoints, metrics endpoints, and administrative commands need their own exposure policy. A diagnostic response can reveal configuration or traffic patterns even when it contains no password. The fact that an endpoint is read-only does not make it harmless to expose without review.

NEW SECTION EXAMPLE

Verify the requested service name against a certificate

Generate a disposable local certificate and use OpenSSL's hostname checker to reject the wrong service identity.

Only local certificate generation and identity matching are exercised; no network service is contacted.

check.cpp

```
#include <openssl/pem.h>
#include <openssl/x509v3.h>
#include <cstdio>
#include <memory>
#include <iostream>
int main(){std::unique_ptr<FILE,decltype(&fclose)>
f(fopen("cert.pem","rb"),fclose);if(!f)return 1;
std::unique_ptr<X509,decltype(&X509_free)>
cert(PEM_read_X509(f.get(),nullptr,nullptr,nullptr),X509_free);if(!cert)return 2;
int good=X509_check_host(cert.get(),"worker.test",0,0,nullptr);
int bad=X509_check_host(cert.get(),"wrong.test",0,0,nullptr);
if(good!=1||bad!=0)return 3;
std::cout<<"worker.test accepted; wrong.test rejected\n";
}
```

run.sh

```
set -eu
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 1 -subj
/CN=worker.test -addext subjectAltName=DNS:worker.test 2>certificate.log
g++ -std=c++20 check.cpp -lssl -lcrypto -o check
./check
```

Run or inspect

```
sh run.sh
```

Explanation

X509_check_host compares the requested service name with certificate identity, including the subject alternative name. Encryption without the correct identity policy could protect a conversation with the wrong peer.

This test verifies hostname matching only. Its self-signed certificate is a private fixture, not a public trust chain, and no TLS handshake occurs. A real client needs chain verification, SSL_set1_host, appropriate SNI, and transport deadlines.

Expected behavior or output

```
worker.test accepted; wrong.test rejected
```

Concrete experiment

Add a second SAN for `api.test`. Both listed names should match, while `wrong.test` still fails; do not disable hostname verification to resolve a production mismatch.

Validation of this new example: `gcc: passed`

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: `OPENSSL-HOST`, `TLS13`. See the added references and revision index.

22.3 Limiting Damage from Inputs and Secrets

22.3.1 Validating Resource Use Alongside Syntax

A syntactically valid request can still be too large, too frequent, or too expensive. Validate payload length before allocating based on a declared size. Validate domain ranges before arithmetic or database insertion. Apply per-operation resource limits even after authentication.

The wire decoder rejects payloads above 4096 bytes and does not allocate an arbitrary declared length. Keys and integers have explicit limits. These checks reduce the attack and failure surface, but they do not constitute a complete denial-of-service defense for a public service.

Prepared SQL parameters protect the SQL grammar boundary. They do not decide whether a caller owns a key or whether a query is permitted. Similarly, a safe C++ container does not prevent an unbounded number of authorized requests from exhausting memory. Compose controls at the relevant layers.

Test negative cases as first-class behavior: malformed frames, trailing command tokens, invalid ranges, cross-tenant access, unknown actions, and unsupported versions. An error response should not include raw secrets, stack dumps, or unrelated database contents.

22.3.2 Operating Credentials and Audit Records Safely

Credentials need storage, rotation, scope, and revocation procedures. Avoid embedding production secrets in source, images, command-line examples, or book fixtures. Use the target platform's approved secret mechanism and ensure the application can reload or restart safely when credentials change.

Command-line arguments and environment variables have different exposure characteristics on different platforms. Do not present either as universally secret. Review process inspection, crash dumps, logs, support bundles, and child-process inheritance in the actual deployment environment.

Audit records should capture who requested which sensitive action, the resource scope, the decision, and a correlation identifier. They should not copy the credential itself. Preserve enough evidence to investigate policy decisions without making the audit store a second credential repository.

Administrative repair requires especially careful authorization. A command that replays jobs, changes receipt retention, restores a database, or advances a fencing generation can affect more than one ordinary request. Separate those capabilities from the regular submission interface and record their use.

NEW SECTION EXAMPLE

Validate resource use before parsing and redact secrets before logging

Bound request size and avoid echoing a bearer token into an error event.

main.cpp

```
#include <string>
#include <string_view>
#include <iostream>

int main() {
    auto handle=[](std::string_view body,std::string_view token){
        if(body.size()>16)return std::string("event=request_rejected reason=size
token=[redacted]");
        return std::string(token.empty()? "event=unauthenticated": "event=request_checked");};
    std::cout<<handle(std::string(17,'x'), "secret-value")<<'\n';
    std::cout<<handle("job7","")<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The byte bound is checked before expensive parsing. The error records a category but never concatenates the token, so malformed input cannot turn authentication material into a log payload.

A real gateway also bounds decoded collections, decompression, concurrent requests, and log retention. This fixture illustrates validation and redaction placement, not complete authentication or credential storage.

Expected behavior or output

```
event=request_rejected reason=size token=[redacted]
event=unauthenticated
```

Concrete experiment

Use a token containing newlines and verify that it never appears in output. Add an escaped, bounded request-ID field instead of logging the raw request.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Evaluate tenant-scoped permission

Requirement and implementation

Allow a verified principal to perform a named action only within its permitted tenant scope. Reject an unrelated tenant and a missing permission. Keep credential verification outside this small domain-policy function.

Source — `source/examples/ch22.cpp`

```
#include "harbor/check.hpp"
#include <iostream>
#include <string>
#include <set>
struct Principal { std::string tenant; std::set<std::string> permissions; };
bool allowed(const Principal& verified, const std::string& resource_tenant,
             const std::string& action) {
    return verified.tenant==resource_tenant && verified.permissions.contains(action);
}
int main() {
    // Principal must come from a verified transport/authentication boundary.
    Principal p{"tenant-a",{"status:read"}};
    harbor::check(allowed(p,"tenant-a","status:read"),"own tenant read");
    harbor::check(!allowed(p,"tenant-b","status:read"),"cross tenant denial");
    harbor::check(!allowed(p,"tenant-a","job:submit"),"action denial");
    std::cout<<"Identity and authorization are different checks.\n";
}
```

Evidence and interpretation

The example verifies a pure authorization predicate for supplied test principals. It is useful because policy can be tested without a network handshake. It is insufficient by itself because a real system must prevent untrusted input from impersonating those principals.

A complete integration test would connect authenticated clients with different identities, exercise the adapter that constructs the principal, and verify both allowed and denied paths. It would also test expired or untrusted credentials under the selected provider and library. Those tests are not claimed for the loopback companion.

Failure clinic — Tenant identity is copied from the payload

An attacker submits a valid token for tenant A and a payload naming tenant B. If the handler simply constructs `Principal{payload.tenant, ...}`, it discards the verified identity and trusts the attacker-controlled field. Bind resource lookup to the authenticated principal and explicitly check any requested tenant scope.

This is a semantic flaw that memory sanitizers will not detect. Security testing needs policy histories and adversarial inputs in addition to native-code defect detection.

Engineering case — Preserve authorization through an authenticated gateway

An external client authenticates as tenant A to a gateway. The gateway calls a worker using its service credential and forwards a payload containing tenant=B. If the worker trusts the payload tenant because the gateway connection is authenticated, the client has crossed an authorization boundary through a trusted intermediary.

The gateway must validate the requested resource scope against the verified principal before delegation. The worker must know whether the gateway is authorized to assert end-user scope and how that assertion is protected. A service credential authenticates the gateway; it does not automatically validate every claim the gateway forwards.

One design uses a narrow internal request constructed only after authorization, with an authenticated delegation mechanism and an explicit audience. Another lets the worker independently validate a delegated token. The choice depends on the identity infrastructure, but both require a clear distinction between untrusted request data and trusted claims.

Test a valid identity with a mismatched tenant, a valid tenant with a missing action permission, an expired delegation, and a request arriving through a path that bypasses the expected gateway. A positive test proving that the ordinary client succeeds does not address these cases.

Logging should preserve the decision category and relevant nonsecret identity reference without copying tokens. A support bundle containing bearer credentials can create a new access path during an investigation. Diagnostic convenience must not expand the trust boundary accidentally.

The supplied principal model tests the final policy predicate only. It deliberately does not manufacture authentication from a string. Likewise, the actual loopback application contains no tenant security layer and must not be exposed publicly on the strength of this example.

This case demonstrates a useful architectural review question: which component is allowed to turn a claim into trusted identity? Trace that conversion explicitly. Every later authorization decision depends on it, and no amount of encryption on an unrelated hop can repair a mistaken answer.

Practice and implementation lab

Extend the policy model with separate submit, inspect-own, and administer permissions. Create a table of allowed and denied combinations, including a valid identity with no permissions. Keep default behavior deny rather than interpreting a missing policy as unrestricted access.

Write a TLS integration checklist for the optional adapter: trust roots, peer name, client identity where required, protocol configuration, credential reload, error reporting, and negative certificate tests. Do not mark any item complete until an actual integration run provides evidence.

Review and solution reasoning

Authentication, authorization, transport protection, idempotency, and resource control address different threats. A strong design names the authority behind each decision and prevents untrusted fields from crossing into trusted state without validation.

The safe boundary for the supplied application is a controlled local lab. Extending exposure requires implementing and testing security mechanisms, not merely changing an address constant.

Chapter takeaway. Verify identity at the boundary, authorize each action and resource explicitly, and keep secrets out of the artifacts used to observe the system.

Chapter 23 — Making Distributed Behavior Observable

A dashboard says requests are successful, but jobs remain pending. A log contains thousands of retries without showing which logical operation they belong to. A trace shows a fast worker span while most latency was spent waiting before the span began.

Observability is useful when signals correspond to the service's contracts. This chapter designs metrics, logs, and traces around accepted work, attempts, durable effects, and user-visible completion rather than around whatever counters were easiest to increment.

23.1 Choosing Signals with Defined Meaning

23.1.1 Measuring Outcomes at the Correct Boundary

Count accepted submissions separately from completed jobs. A gateway returning `ACCEPTED` has committed intent, not necessarily delivered a result. Likewise, a successful TCP exchange may contain an application-level `CONFLICT` or `BUSY` response. Metrics must identify which layer's success they describe.

Useful job-service signals include new accepted jobs, recognized duplicates, conflicts, capacity rejections, pending age, completed jobs, dispatch attempts, and dispatch errors. Their relationships can reveal failure: growing pending age with successful health probes indicates a different condition from failed listeners.

Use units in names or metadata and preserve consistent aggregation. Seconds, bytes, and counts should not share ambiguous labels. Prometheus naming guidance provides conventions for metric names, units, and label use; the companion's small exposition example follows the same emphasis on explicit meaning without running a Prometheus server. (R31)

A process counter commonly resets on restart. It is not a durable lifetime business total. HarborDispatch's gateway counters are atomic in-memory observations, while job and receipt rows are persistent state. A monitoring system should understand counter resets rather than interpret a decrease as negative traffic.

23.1.2 Controlling Label Cardinality

Each distinct label combination can create another time series. A request ID, raw URL, arbitrary key, or user-supplied error string can therefore create unbounded cardinality. Use bounded categories for aggregate metrics and place individual identities in controlled logs or traces when appropriate.

Choose labels that answer operational questions, such as operation class, result category, or a bounded deployment region. A tenant label may be justified in some systems, but its scale and access implications need review. Do not add every available request field because it might someday be useful.

Error categories should be stable. A raw exception message can include paths, IDs, or library wording that changes across versions. Map it to a controlled code for counting while retaining sanitized diagnostic detail separately.

The teaching histogram has no dynamic labels. That keeps its state bounded and its test understandable. A production metrics adapter would also need concurrency, lifecycle, export failure, and registry ownership rules.

NEW SECTION EXAMPLE

Count logical outcomes separately from transport attempts

Count logical outcomes separately from transport attempts.

main.cpp

```
#include <array>
#include <string_view>
#include <map>
#include <iostream>

int main() {
    struct Attempt{std::string_view operation;bool success;};
    const std::array attempts{Attempt{"a",false},Attempt{"a",true},Attempt{"b",true}};
    std::map<std::string_view,bool> completed;unsigned failures=0;
    for(auto a:attempts){if(!a.success)++failures;completed[a.operation]=a.success;}
    unsigned successes=0;for(auto&[id,ok]:completed)successes+=ok;
    std::cout<<"attempts="<<attempts.size()<<" attempt-errors="<<failures<<"
    successful-operations="<<successes<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Operation a has two attempts but one final successful outcome. Counting at the wrong boundary makes the user-operation success rate differ from the transport-attempt success rate.

The map uses operation IDs only inside a bounded test fixture. Do not export those IDs as unbounded metric labels; place them in logs or traces while aggregate metrics use finite outcome and component dimensions.

Expected behavior or output

```
attempts=3 attempt-errors=1 successful-operations=2
```

Concrete experiment

Add three failed attempts for a before its success. The logical success count must remain two while the attempt-error count increases.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

23.2 Connecting Work Across Processes

23.2.1 Separating Logical Job, Attempt, and Trace Identity

A logical job key survives retries. An attempt identifier distinguishes individual delivery tries. A trace identifier connects observations within a tracing system's model. These identities can be related without being identical.

If every retry creates a new trace, retain a way to associate those traces with the same job. If one long trace spans an hours-long workflow, consider storage, sampling, and tooling limitations. Span links or explicit correlation records can represent asynchronous relationships without pretending every operation is one synchronous call stack.

OpenTelemetry describes traces as spans representing operations and their relationships. W3C Trace Context defines interoperable propagation fields. Those standards help carry context, but the application still chooses meaningful span boundaries and must treat incoming context as untrusted metadata rather than authorization. (R30, R40)

The default companion does not link an OpenTelemetry SDK or export traces. Its chapters and optional integration plan describe how to add that adapter. This distinction matters because a trace-shaped string in a log is not a tested telemetry integration.

23.2.2 Placing Spans Around Waiting as Well as Execution

A span around the worker calculation can be very short while end-to-end completion is slow. Include queue residence, connection establishment, retries, database waits, and durable workflow intervals where they matter to the question being investigated.

Avoid double-counting overlapping intervals as if they were sequential. A parent span can include child work, and concurrent children can overlap. Summing all span durations can exceed wall-clock latency without indicating an error. Interpret the graph, not just the numbers.

Cancellation and late completion should remain visible. Marking an attempt timed out does not prove the remote effect failed. A later receipt can resolve the logical job while the original client span remains an unsuccessful wait. Both observations can be correct.

Sampling also limits evidence. A missing trace does not prove that no request occurred. Use aggregate counters and durable state to complement sampled traces, especially when investigating rare recovery paths or reconciling business effects.

NEW SECTION EXAMPLE

Preserve logical job identity across distinct attempt spans

Emit an explicit trace fixture containing queue wait and execution duration for a retry.

main.cpp

```
#include <array>
#include <string_view>
#include <iostream>

int main() {
    struct Span{std::string_view job,attempt,trace;int queued_ms,work_ms;};
    const std::array
    spans{Span{"job7","try1","traceA",20,5},Span{"job7","try2","traceA",10,6}};
    for(auto&s:spans)std::cout<<s.job<<' '<<s.attempt<<' '<<s.trace<<" wait="<<s.queued_ms<<"
    work="<<s.work_ms<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The job ID identifies the business operation, attempt IDs distinguish retries, and trace identity connects related work. Collapsing them into one field loses either retry visibility or operation identity.

The 20 ms queue wait is separate from the 5 ms execution time. A span around only the worker function misses most of the first attempt's elapsed time. These durations are deterministic fixtures, not measured latency.

Expected behavior or output

```
job7 try1 traceA wait=20 work=5
job7 try2 traceA wait=10 work=6
```

Concrete experiment

Add a child span for an inter-service hop and preserve the job ID. Give a retry its own span ID because it represents another execution.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

23.3 Designing Diagnostics That Do Not Become the Incident

23.3.1 Recording Structured, Bounded Logs

A useful log event identifies its event type, time, component, operation or attempt correlation, outcome category, and bounded diagnostic fields. It should make a specific transition observable: intent committed, delivery attempted, receipt found, completion recorded, or shutdown begun.

Do not log entire payloads by default. Large or secret-bearing requests can exhaust storage and expose sensitive data. Bound field lengths, sanitize control characters, and select a retention and access policy appropriate to the data.

Logging can block or allocate. A high-rate synchronous logger in a reactor can distort the very latency being measured. Use a bounded logging path with an explicit drop or fallback policy and a counter for lost diagnostics. An unbounded asynchronous logger merely moves the memory problem to another queue.

During an incident, increase detail narrowly by operation, component, or time window. Turning on global per-byte logging for every connection can make an overloaded service less recoverable. Preserve the ability to turn diagnostic detail back down.

23.3.2 Using Histograms Without Inventing Precision

A histogram records counts within defined buckets and can support aggregate latency analysis. Its resolution depends on bucket boundaries. A reported percentile derived from coarse buckets is an estimate, not an exact stored request duration.

The example uses cumulative buckets at 0.005, 0.02, 0.1, 1.0, and positive infinity seconds. An observation at 0.05 increments the 0.1, 1.0, and infinity cumulative buckets. The infinity count equals the total observation count. Tests verify these relationships.

Do not average percentiles from separate instances and call the result a fleet percentile. Aggregate the underlying distribution representation using a supported method, then estimate the percentile. The same caution applies to mixing different bucket definitions or measurement boundaries.

Histograms should observe finite nonnegative durations. A negative value from a wrong clock or a NaN from bad arithmetic should be rejected or categorized, not silently poison aggregate state. The model validates its observations for that reason.

NEW SECTION EXAMPLE

Use explicit histogram buckets without inventing percentile precision

Aggregate five durations into inclusive upper-bound buckets without inventing percentile precision.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    const std::array bounds{10,50,100};std::array<unsigned,4> buckets{};
    for(int ms:std::array{3,8,20,80,150}){std::size_t
b=0;while(b<bounds.size()&&ms>bounds[b])++b;++buckets[b];}
    std::cout<<"<=10="<<buckets[0]<<" (10,50]="<<buckets[1]<<" (50,100]="<<buckets[2]<<"
>100="<<buckets[3]<<"\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The loop places each observation into exactly one disjoint bucket. The fourth bucket retains overflow instead of discarding long requests.

A bucketed histogram cannot recover an exact p99 unless sufficient resolution or raw observations are retained. Choose boundaries around operational thresholds and document whether exported buckets are disjoint or cumulative.

Expected behavior or output

```
<=10=2 (10,50]=1 (50,100]=1 >100=1
```

Concrete experiment

Add observations of 10, 50, and 100 and verify boundary inclusion. Then produce cumulative exporter counts without counting any original observation twice.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Emit a cumulative latency histogram

Requirement and implementation

Observe three durations, expose cumulative bucket counts, and preserve total count and sum. Keep the implementation single-owner and independent of a network exporter so its arithmetic is directly testable.

Source — source/examples/ch23.cpp

```
#include "harbor/check.hpp"
#include "harbor/metrics.hpp"
#include <iostream>
int main() {
    harbor::Histogram metric;
    metric.observe(0.003); metric.observe(0.05); metric.observe(2.0);
    auto text=metric.text();
    harbor::check(text.find("harbor_request_seconds_count 3")!=std::string::npos,"count");
    harbor::check(text.find("{le=\"+Inf\"} 3")!=std::string::npos,"cumulative terminal
bucket");
    std::cout<<text;
}
```

Interpret the output

The three observations occupy different ranges, and every observation appears in the infinity bucket. The sum is a total of measured seconds, not a percentile. The output is an exposition example; the capstone's METRICS command returns a different small set of counters rather than this histogram automatically.

An integration should select one registry and thread-ownership policy, then test exporter behavior and scraping independently. Merely printing valid-looking metric text does not establish monitoring availability or alert correctness.

Failure clinic — Every retry is counted as another completed job

A dashboard increments `jobs_total` after every successful worker response, including duplicate replays. Lost replies cause retries, and the dashboard reports more completed jobs than distinct receipts. The metric mixes attempts and logical effects.

Count distinct accepted or completed jobs at the relevant durable transition, and count attempts separately. Use receipt-state reconciliation when exact business totals matter; in-memory metrics are not a replacement for the database ledger.

Engineering case — Reconcile attempt counters with durable job state

During a lost-reply test, one logical job produces two dispatcher attempts, one worker receipt, and one gateway completion. A dashboard that counts all worker responses as completed jobs can report two completions. A dashboard that counts only client-observed responses can report zero if the caller stopped waiting before either result was seen.

Neither counter is necessarily wrong; its name and interpretation are wrong if it claims a different boundary. Define `dispatch_attempts` as transport attempts, `worker_receipts` as retained logical effects, and `jobs_completed` as gateway completion transitions. Their expected relationships depend on the recovery protocol.

A restart resets in-memory attempt counters while durable receipts remain. The monitoring system needs to handle that reset and should not subtract the new counter from the old value as if traffic became negative. Exact historical business counts should come from retained authoritative records or a correctly maintained event ledger, not a volatile process metric alone.

Pending count and pending age reveal different problems. Ten recent jobs can be ordinary load; one job pending for a day can indicate a poison operation or lost recovery path. Add an age signal with a defined clock and persistence representation rather than assuming that count alone describes backlog health.

For traces, connect the first attempt's worker commit, lost reply, second attempt's receipt lookup, and gateway completion. The first attempt can be unsuccessful from the client's waiting perspective while the worker effect is successful. A trace viewer should not force those observations into one Boolean.

A useful incident report includes a small reconciliation table: accepted unique keys, completed gateway keys, worker receipt keys, and unresolved differences. Protect the identifiers according to policy, and avoid running an unbounded full-table dump on a busy production system. The lab's small private databases make direct inspection safe for teaching, but scale changes the diagnostic method.

This case gives observability a correctness role. Signals should help explain the invariant rather than obscure it with attractive aggregate numbers. When the counts disagree, investigate the boundary definitions before assuming the database or monitoring system is broken.

Practice and implementation lab

Create a signal dictionary for HarborDispatch. For each metric, state its unit, increment point, reset behavior, labels, and the question it answers. Include at least one measure of backlog age rather than only backlog count.

Design a trace for a lost-reply scenario. Show the first worker commit, failed response delivery, second attempt, receipt replay, and gateway completion. Identify which spans are unsuccessful attempts even though the logical job eventually succeeds.

Review and solution reasoning

A useful signal retains its meaning during retries and restarts. Its name should not promise a stronger outcome than the event being counted. Metrics, logs, traces, and durable records complement one another because each has different retention, sampling, and cost properties.

Observability is part of architecture: it determines whether an operator can distinguish overload, delay, duplicate delivery, and lost state without guessing from one green health indicator.

Chapter takeaway. Instrument the transitions that define service behavior, keep identities and units distinct, and bound the cost of observation itself.

Chapter 24 — Testing Distributed Behavior with Deterministic Models and Fault Injection

A service passes its happy-path test one hundred times and still loses a job during a restart. The repeated test exercised the same successful history. The missing evidence concerns an entirely different sequence of events.

Distributed testing needs both controllable models and real integration boundaries. Models make difficult histories repeatable. Process tests expose actual sockets, persistence, and lifecycle behavior. Neither should be presented as proof of every possible execution.

24.1 Designing Tests Around Invariants

24.1.1 Separating an Oracle from the Implementation Under Test

An oracle describes the expected result independently enough to detect a defect. If an encoder and decoder share the same wrong byte order, a round-trip test can pass. Add golden bytes produced from the protocol specification or an independent implementation.

The wire tests include a known length-prefixed payload and multiple fragment splits. Python integration tests construct frames with `struct.pack("!I", length)`, independently of the C++ encoder. This does not eliminate every shared misunderstanding, but it is stronger than testing only one implementation against itself.

For job effects, the oracle can be exact: a bounded nonnegative integer squared. A receipt count and key/input association provide independent checks of duplicate behavior. For concurrency, verify accepted-item accounting and values, not merely that no exception occurred.

State the invariant before writing the test. Examples include “a changed payload cannot reuse an accepted key,” “a closed queue accepts no new item,” and “a stale fencing token cannot replace newer state.” These statements guide both ordinary examples and injected failures.

24.1.2 Testing Boundaries and Negative Paths

Boundary inputs reveal different mechanisms: zero, maximum accepted size, one beyond maximum, incomplete framing, trailing data, duplicate identity, and changed meaning. A generated random test may never hit an exact equality boundary that a small explicit test covers immediately.

Negative tests should verify the postcondition after rejection. Invalid input must not create a row. An incorrect completion must not change pending state. An oversized frame must not make the server unusable for the next valid connection. An exception should leave owned resources in a usable or deliberately closed state.

Use assertions that remain active in optimized test builds. The companion's check function throws on failure instead of relying on standard assert, which can be removed when NDEBUG is defined. Release testing should retain its oracle.

A test timeout is a guard against an infinite hang, not a timing certification. Passing within a generous test limit says little about a production latency objective unless the test was designed as a controlled performance experiment.

NEW SECTION EXAMPLE

Test a transfer invariant against an independent arithmetic oracle

Check both conservation and rejection behavior against an independent transfer oracle.

main.cpp

```
#include <array>
#include <iostream>

int main() {
    auto transfer=[](int&from,int&to,int amount){if(amount<0||amount>from)return
false;from-=amount;to+=amount;return true;};
    unsigned checked=0;
    for(int amount:std::array{-1,0,3,5,6}){int from=5,to=2;bool ok=transfer(from,to,amount);
        bool expected=amount>=0&&amount<=5;

if(ok!=expected||from+to!=7||from!=(expected?5-amount:5)||to!=(expected?2+amount:2))return
1;checked;}
    std::cout<<"five cases preserve balances and rejection semantics\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The oracle calculates expected balances directly from the original 5 and 2 rather than calling transfer again. Rejected requests must leave both balances unchanged.

Conservation alone is insufficient: moving the wrong amount can still preserve the total. The independent postconditions check each balance and the admission decision.

Expected behavior or output

```
five cases preserve balances and rejection semantics
```

Concrete experiment

Change the implementation to add amount+1 to the destination, then to transfer amount-1 on both sides. Explain why the individual-balance oracle catches the second defect even though conservation passes.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

24.2 Exploring Histories Deterministically

24.2.1 Injecting Time, Membership, and Completion Order

Policy tests use explicit ticks rather than sleeping. Breaker recovery at time T , cache expiration at time T , and token refill can then be checked exactly. An injected clock also makes overflow and backward-time handling practical to test.

Asynchronous state tests can deliver completions in a chosen order. Admit an operation under one epoch, advance the epoch, then deliver its old result. This directly tests stale-completion rejection without hoping an operating-system scheduler produces the right race.

Membership models can compare different orders and revisions. The rendezvous tests verify that reordering the same candidates does not change placement and that adding a candidate changes ownership only according to the algorithm's rule. They do not require launching a registry to test that pure property.

Keep the model's assumptions visible. A single-owner map is not a thread-safe distributed store. A deterministic retry schedule is not a real network. Models are valuable because their scope is small enough to reason about, not because they replace all integration tests.

24.2.2 Enumerating Small State Spaces and Recording Seeds

A small finite history can be enumerated completely. The chapter example permutes a multiset containing one duplicate event and checks that every distinct order produces the same deduplicated set. Its claim is limited to that finite model and those event semantics.

Larger generated tests should record seeds and failing histories so they can be reproduced. Shrinking a failure to a smaller sequence often reveals the actual invariant violation more clearly than a thousand-line random trace.

Simulation has been used in substantial database engineering; FoundationDB's published design discusses deterministic simulation as part of its testing approach. That work motivates the value of controlled failures, but this companion's small models are not equivalent to FoundationDB's infrastructure or coverage. (R39)

Avoid presenting a fixed number of passing schedules as a probability that the system is correct. The schedule distribution may not reflect production, and unmodeled failures remain outside the experiment. Report what was explored and what was excluded.

NEW SECTION EXAMPLE

Enumerate callback orderings with a stale-generation guard

Explore both arrival orders of two completion results whose generations differ.

main.cpp

```
#include <array>
#include <algorithm>
#include <iostream>

int main() {
    std::array<unsigned,2> order{1,2}; unsigned histories=0;
    do{unsigned current=0; int value=0; for(auto
generation:order)if(generation>current){current=generation; value=int(generation)*10;}
        if(current!=2||value!=20)return 1; ++histories;
    }while(std::next_permutation(order.begin(),order.end()));
    std::cout<<"histories="<<histories<<" final-generation=2 value=20\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The test delivers both 1-then-2 and 2-then-1 without wall-clock sleeps. The guard rejects generation 1 after generation 2, making final state independent of these two arrival orders.

This exhausts the two-result fixture, not every thread interleaving or protocol history. Record the finite bound and extend the model when introducing additional states.

Expected behavior or output

```
histories=2 final-generation=2 value=20
```

Concrete experiment

Add generation 3 and enumerate all six permutations. Remove the generation guard and retain the first failing order as a regression fixture.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

24.3 Crossing Real Process and Persistence Boundaries

24.3.1 Placing Failpoints at the Difficult Moments

A useful failpoint names its position relative to state change. “Drop reply after worker commit” tests a different problem from “reject request before parsing.” Both can be useful, but only the first exercises uncertain completion after a durable effect.

The integration suite starts actual gateway and worker processes with temporary file-backed databases and ephemeral loopback ports. It can stop a worker, restart a gateway with the same database, fragment a request, submit malformed frames, and deliberately drop a committed reply.

Process readiness is observed through a READY port line after local startup. That is a test-harness handshake, not the service’s full production readiness contract. It lets

the harness avoid arbitrary startup sleeps while still testing network operations directly.

Temporary directories and loopback binding limit the tests' footprint. Cleanup terminates child processes and removes their private files. Tests should not select arbitrary production ports, mutate a user's database, or require access to a remote service unless the reader explicitly configures a separate integration environment.

24.3.2 Interpreting Sanitizers and Platform Coverage

AddressSanitizer and UndefinedBehaviorSanitizer can detect important classes of native-code defects on executed paths. They do not prove protocol correctness, authorization policy, durable recovery, or absence of every concurrency error. A clean run complements functional evidence rather than replacing it.

Compiler diversity can reveal portability assumptions and diagnostics. The final report records the actual GCC and Clang configurations run for this edition, including the standard library used. Two compilers using the same library are not equivalent to testing every library implementation or operating system.

Performance measurements from instrumented builds should not be compared directly with ordinary Release results as if only the algorithm changed. Instrumentation changes costs. Use sanitizers to investigate correctness and separate controlled optimized runs for performance claims.

The companion does not certify hard-real-time scheduling, arbitrary packet-loss networks, TLS, broker transactions, or Kubernetes deployment. Those require additional environments and tests. Explicitly naming untested boundaries is part of a useful verification report.

NEW SECTION EXAMPLE

Exit a child after commit and reopen its receipt in the parent

Cross a real process boundary at the lost-reply failure point using a disposable SQLite file.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};

struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};

inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}

struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"
#include <unistd.h>
#include <sys/wait.h>
#include <cerrno>

int main(){try{
    {Database setup("crash.db");setup.exec("CREATE TABLE receipts(id TEXT PRIMARY KEY,result
INTEGER");}
    pid_t child=fork();if(child<0)return 1;
    if(child==0){Database worker("crash.db");worker.exec("BEGIN IMMEDIATE;INSERT INTO receipts
VALUES('job7',49);COMMIT");_exit(0);}
    int status=0;while(waitpid(child,&status,0)<0){if(errno!=EINTR)return 2;}
    if(!WIFEXITED(status)||WEXITSTATUS(status)!=0)return 3;
    Database parent("crash.db");auto result=scalar(parent,"SELECT result FROM receipts WHERE
id='job7'");
    std::cout<<"reopened after child exit: result="<<result<<" rows="<<scalar(parent,"SELECT
count(*) FROM receipts")<<"\n";
}catch(const std::exception&e){std::cerr<<e.what()<<"\n";return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```

Explanation

The child commits before `_exit`, deliberately skipping C++ destructors and any application reply. The parent waits for child completion, opens its own database connection, and recovers the committed receipt.

This is an actual process and persistence test, not simulated network loss. It does not certify power-loss durability; a repeated external effect still requires atomic effect/receipt handling. No open database connection is inherited at fork.

Expected behavior or output

```
reopened after child exit: result=49 rows=1
```

Concrete experiment

Move `_exit` before COMMIT and expect no receipt after recovery. Use a fresh database for each case so earlier state cannot mask the failure point.

Validation of this new example: gcc: passed

Technical references: [SQLITE-C](#), [SQLITE-TRANSACTION](#), [LINUX-FORK](#). See the added references and revision index.

Worked example — Enumerate duplicate-delivery orders

Requirement and implementation

Take a finite multiset of event identities with one duplicate. Enumerate its distinct permutations and verify that each history produces the same set of logical effects.

Source — `source/examples/ch24.cpp`

```
#include "harbor/check.hpp"
#include <algorithm>
#include <iostream>
#include <set>
#include <vector>
int main() {
    std::vector<int> schedule{1,1,2,3}; unsigned checked=0;
    do {
        std::set<int> receipts;
        unsigned effects=0;
        for (int id:schedule) if (receipts.insert(id).second) ++effects;
        harbor::check(effects==3,"schedule duplicated effect"); ++checked;
    } while (std::next_permutation(schedule.begin(),schedule.end()));
    harbor::check(checked==12,"distinct schedules");
    std::cout<<"Checked all 12 distinct orderings of a bounded duplicate-delivery model.\n";
}
```


Evidence and interpretation

There are twelve distinct permutations for the chosen four-item multiset. The example counts the histories it actually explores and checks the resulting effect set. This is a small exhaustive model, not an exhaustive distributed-system test.

Change the model to include identity conflicts and the expected behavior changes: some histories should reject the conflict rather than converge silently. A useful model retains the domain contract instead of forcing every possible input into a success result.

Failure clinic — A race test waits and hopes

A test sleeps for ten milliseconds and assumes the worker has committed. On a slow machine it has not; on a fast machine it committed much earlier. Failures become flaky, and the test's causal claim is unclear.

Replace the sleep with an observable synchronization point where possible: a committed row, a specific state transition, or a test-only acknowledgement. Use bounded waiting around that predicate, not a fixed delay as evidence that the event occurred.

Engineering case — Test a crash point rather than a convenient outage

A test stops the worker before the gateway sends a request. The gateway later retries and succeeds. This is useful evidence for dependency unavailability, but it does not test duplicate safety after a committed effect.

Move the failpoint to the interval after the worker's transaction commits and before the reply is delivered. Now the gateway cannot distinguish completed work from failed delivery using the connection alone. The test must verify a later retry returns the retained result and that the receipt count remains one.

The worker's drop-first-reply option creates exactly that history. It does not crash the process, so receipt persistence across process replacement is a separate test. Combining both ideas into one oversized test could make failures harder to diagnose. Separate cases isolate the mechanism under examination while the complete suite covers their composition more broadly.

An independent oracle should inspect the authoritative effect count and result, not merely accept the gateway's final success message. If both the worker and gateway contained the same duplicate-counting bug, a response-only test could miss it. The integration suite queries the worker count and verifies status values.

The test harness uses ephemeral ports and waits for a specific startup line instead of sleeping for an arbitrary duration. It also bounds every wait and cleans up processes on failure. A test that hangs indefinitely or leaks a server into later tests can create misleading failures unrelated to the intended invariant.

Record the first failing history and preserve relevant stderr. Sanitizer output should not disappear because a cleanup helper catches every exception. Conversely, a

normal process signal during an intentional test should not automatically be classified as an application defect. The harness must understand the expected termination path.

The general pattern is to name the failpoint relative to a commit, publication, or ownership transfer. “Make it fail sometimes” is much less useful than “lose the acknowledgement after the durable effect and before local completion.” Precise placement turns failure injection into evidence rather than noise.

Practice and implementation lab

Run one integration case repeatedly while retaining its temporary logs on failure. Then add a deliberate failpoint immediately before gateway completion commit. Verify that restart and replay still produce one worker receipt and a completed gateway row.

Create a coverage table with columns for invariant, model test, native unit test, process integration test, and untested environment. Include at least one row where a passing unit test is insufficient, such as certificate verification or live broker acknowledgement behavior.

Review and solution reasoning

A good test report names the input, history, environment, and oracle. It does not inflate internal checks into independent test cases or claim that a compilation result proves runtime behavior.

Determinism is most useful when it makes a failure explainable. Real integration is most useful when it tests a boundary the model intentionally omitted. Combining them produces stronger evidence than either alone.

Chapter takeaway. Test the failure history and its invariant, use independent evidence where possible, and keep the boundary of every verification claim visible.

Chapter 25 — Producing Reproducible Builds and Release Artifacts

A binary passes its tests on the developer’s workstation but fails on another machine because it loads a different shared library. A package contains the executable but not its schema expectations or recovery instructions. A release is rebuilt from an unrecorded dependency revision and no longer matches the tested artifact.

A production build is a transformation with inputs, tools, outputs, and evidence. This chapter extends the CMake and testing work of earlier volumes into provenance, packaging, dependency policy, and reproducible release review.

25.1 Defining the Build as a Dependency Graph

25.1.1 Expressing Requirements Through Targets

CMake targets should carry their compile features, include directories, definitions, and link dependencies. A consumer of `harbor_store` needs the public interfaces and SQLite linkage declared by that target, not an undocumented global include path copied from another application.

The companion separates domain and policy headers, the SQLite implementation, and Linux transport implementation. Executables link the components they need. This exposes architectural dependencies in the build graph: the pure model examples do not need to pretend they are network servers, while the gateway explicitly depends on persistence and transport.

Use separate build directories for different compilers, build types, and sanitizer settings. Reusing one cache while changing incompatible assumptions can make it difficult to determine which configuration produced a result. The package’s verification script uses named build directories and records separate logs.

CMake’s importing and exporting guidance describes target usage requirements and consumption of installed packages. An exported library interface is part of the product contract, not simply a convenience for one source tree. (R36)

25.1.2 Keeping Generated Outputs Out of the Source Contract

Generated files need provenance. For a protobuf adapter, record the schema and generator/runtime versions. For a configuration header, record the inputs and the command that produces it. Do not edit generated output and then assume regeneration preserves the change.

A build should distinguish source inputs from disposable artifacts. Object files, CMake caches, compiler depfiles, and local test databases should not be mistaken for required source. The book package excludes build trees while retaining the scripts and fixtures needed to create them.

Offline and restricted environments require an explicit dependency strategy. This companion's default build discovers an already installed SQLite development library rather than downloading third-party source during configuration. The README identifies that requirement; the ZIP does not claim to bundle every platform library.

For an organization-specific artifact repository, use reviewed configuration and placeholders in documentation rather than embedding private endpoints or credentials. A reproducible procedure describes the dependency identity and verification method without disclosing confidential infrastructure.

NEW SECTION EXAMPLE

Declare the generated configuration header's producer

Generate a configuration header in the build tree and expose that directory only to its consumer.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(Graph LANGUAGES CXX)
set(CONFIG_GENERATION 7 CACHE STRING "configuration generation")
configure_file(config.hpp.in generated/config.hpp @ONLY)
add_executable(app main.cpp)
target_compile_features(app PRIVATE cxx_std_20)
target_include_directories(app PRIVATE "${CMAKE_CURRENT_BINARY_DIR}/generated")
```

config.hpp.in

```
#pragma once
inline constexpr unsigned config_generation=@CONFIG_GENERATION@;
```

main.cpp

```
#include "config.hpp"
#include <iostream>
int main(){std::cout<<"generation="<<config_generation<<"\n";}
```

run.sh

```
set -eu
cmake -S . -B build >configure.log
cmake --build build >build.log
./build/app
cmake -S . -B build -DCONFIG_GENERATION=8 >>configure.log
cmake --build build >>build.log
./build/app
```

Run or inspect

```
sh run.sh
```

Explanation

`configure_file` declares how the header derives from a template and cache configuration. The include path names the binary tree, preventing a stale generated source-tree file from becoming accidental authority.

The second configuration changes the generation to 8. The incremental build must recompile the dependent translation unit, which checks a declared input change

rather than merely showing that a clean build works.

Expected behavior or output

```
generation=7
generation=8
```

Concrete experiment

Delete build/generated/config.hpp and configure again. Edit the template or cache input, not the generated header.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: CMAKE-CONFIGURE, CMAKE-BUILDSYSTEM. See the added references and revision index.

25.2 Recording Provenance and Supply-Chain Decisions

25.2.1 Identifying Source, Toolchain, and Dependency Inputs

Record source revision or archive checksum, compiler version, C++ language mode, standard library, build type, relevant flags, CMake version, and dependency versions. These facts help another engineer reproduce a defect and distinguish a source change from a toolchain change.

The chapter example prints a small compile fingerprint. It is useful for identifying some build facts, but it is not a signed provenance statement. A malicious or simply mistaken build can print any string. Trust requires a controlled process and evidence tied to the delivered artifact.

A dependency lock or pinned revision reduces accidental drift, but pinning alone does not establish safety. The selected dependency still needs provenance, license review, vulnerability maintenance, and a plan for updates. Reproducibility and security overlap without being identical.

SLSA defines supply-chain security levels and provenance-related expectations. Use a specified version of that framework when evaluating a build process. The companion provides local build records and hashes; it does not claim a SLSA level or a hardened hosted build service. (R37)

25.2.2 Separating Integrity, Authenticity, and Reproducibility

A checksum detects a difference from a known expected byte sequence. It does not establish who supplied that expected checksum. A signature can provide authenticity under a trusted key and verification policy, but the key's custody and the signing workflow matter.

Reproducible builds aim to make a defined set of inputs produce matching outputs under a controlled process. PDF generation, timestamps, compiler paths, and archive metadata can introduce nondeterminism. State whether reproducibility means

identical manuscript content, repeatable compilation, or byte-identical release artifacts.

For this book, the assembler reads current source files into listings and records their hashes. That prevents the manuscript from silently showing an old code string while the tests execute a new file. The publishing pipeline can regenerate the documents from the included manuscript, styles, and tools without fetching missing chapters.

Renderer and font availability can change pagination and output bytes. The package documents its publishing dependencies but does not distribute font files. Rebuilding with different installed fonts may preserve content while changing line breaks. That is a meaningful distinction to record rather than hide.

NEW SECTION EXAMPLE

Record source and toolchain identity without calling a checksum a signature

Record exact source and toolchain identity without treating a checksum as a signature.

main.cpp

```
#include <iostream>
int main(){std::cout<<"artifact7\n";}
```

provenance.py

```
import hashlib,json,pathlib,subprocess
source=pathlib.Path('main.cpp').read_bytes()
record={'source_sha256':hashlib.sha256(source).hexdigest(),
        'compiler':subprocess.check_output(['g++','--version'],text=True).splitlines()[0],
        'flags':['-std=c++20','-O2']}
pathlib.Path('provenance.json').write_text(json.dumps(record,indent=2))
assert
hashlib.sha256(pathlib.Path('main.cpp').read_bytes()).hexdigest()==record['source_sha256']
subprocess.run(['g++',*record['flags'],'main.cpp','-o','app'],check=True)
record['binary_sha256']=hashlib.sha256(pathlib.Path('app').read_bytes()).hexdigest()
pathlib.Path('provenance.json').write_text(json.dumps(record,indent=2))
print('source and binary identities recorded; authenticity not asserted')
```

Run or inspect

```
python3 provenance.py
```

Explanation

The provenance record binds source bytes, compiler description, flags, and resulting binary hash. It permits later comparison without relying on a filename that happens to contain release.

A SHA-256 digest detects changes relative to a trusted reference; it does not identify an author or prove reproducibility. Authenticity needs a trusted signature or attestation, and reproducibility needs an independent controlled rebuild.

Expected behavior or output

```
source and binary identities recorded; authenticity not asserted
```

Concrete experiment

Modify `main.cpp` after recording provenance and verify that its hash no longer matches. Do not update the approved reference until the revision has been reviewed.

Validation of this new example: `gcc: passed`

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: GCC-OUTPUT, REPRO-BUILDS. See the added references and revision index.

25.3 Packaging the Supported Runtime

25.3.1 Distinguishing a Binary Archive from a Deployable Service

An executable archive is not automatically a complete deployment. The service also depends on configuration, writable storage, schema compatibility, runtime libraries, identities, monitoring, and operational ownership. A package should state which of these are included and which must be provided by the target environment.

The companion's CPack configuration can produce a local binary archive. It is a lab artifact, not a portable appliance containing every shared dependency or a production service account configuration. Inspect dynamic linkage on the target platform and validate the runtime environment before relying on it.

Do not copy arbitrary shared libraries from a developer workstation without understanding loader paths, compatibility, licensing, and security update ownership. A container image or package manager can help control a runtime, but each introduces its own base-image and update policy.

Persisted data should live outside replaceable application binaries. A deployment that overwrites the directory containing the only job database can erase accepted work. Define installation paths and state paths separately, with permissions appropriate to the service identity.

25.3.2 Releasing the Artifact That Was Actually Tested

Build once under the release process, test that artifact, and promote the same artifact where the deployment model permits. Recompiling independently in each environment can introduce differences that the original test report did not cover.

Attach evidence to artifact identity: tests, known limitations, schema range, configuration schema, dependency inventory, and rollback conditions. A log saying "all tests passed" is less useful when it cannot be associated with a particular source and binary.

Run a relocation test from an extracted package. Absolute paths accidentally captured during development often appear only when the project moves. The book's tools resolve the project root relative to their own location and accept explicit build destinations rather than relying on a private home directory.

A release review should also test negative startup. An unsupported newer database schema, invalid port, inaccessible state directory, or missing runtime dependency

should fail visibly without partially accepting work. Successful startup on one prepared machine is only one part of release evidence.

NEW SECTION EXAMPLE

Package and execute the same tested binary

Install an application into staging and verify that the package contains the same binary that passed its test.

main.cpp

```
#include <iostream>
int main(){std::cout<<"ready-v1\n";}
```

package.sh

```
set -eu
g++ -std=c++20 -O2 main.cpp -o app
test "$(. /app)" = ready-v1
mkdir -p stage/bin
cp app stage/bin/app
cmp app stage/bin/app
tar -czf release.tar.gz -C stage .
mkdir unpack
tar -xzf release.tar.gz -C unpack
cmp app unpack/bin/app
test "$(unpack/bin/app)" = ready-v1
printf 'tested, packaged, extracted: identical binary runs\n'
```

Run or inspect

```
sh package.sh
```

Explanation

No rebuild occurs between validation and packaging. `cmp` checks staging and extracted artifacts against the tested executable, then the harness executes the extracted path.

This host package is not a universal deployment bundle. Runtime libraries, platform compatibility, configuration, permissions, and licenses remain release inputs; creating an archive alone does not prove startup on another host.

Expected behavior or output

```
tested, packaged, extracted: identical binary runs
```

Concrete experiment

Inspect this trusted local binary with `ldd` or `readelf` and record which runtime libraries the target must provide.

Validation of this new example: `gcc: passed`

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: `GCC-OUTPUT`, `LINUX-LOADER`. See the added references and revision index.

Worked example — Inspect a compile fingerprint

Requirement and implementation

Build a program that checks the requested language baseline and reports selected implementation facts. Explain which facts are compile-time observations and which release properties remain unproven.

Source — `source/examples/ch25.cpp`

```
#include "harbor/check.hpp"
#include <iostream>
int main() {
    static_assert(__cplusplus>=202002L);
    harbor::check(sizeof(void*)>=4, "supported word size");
    std::cout<<"language="<<__cplusplus<<" pointer_bits="<<sizeof(void*)*8<<'\n';
#ifdef __clang__
    std::cout<<"compiler=clang "<<__clang_major__<<'. ' <<__clang_minor__<<'\n';
#elif defined(__GNUC__)
    std::cout<<"compiler=gcc "<<__GNUC__<<'. ' <<__GNUC_MINOR__<<'\n';
#endif
    std::cout<<"Record dependencies and artifact digests separately; a version string is not
    provenance.\n";
}
```

Evidence and interpretation

The example verifies that its compilation mode meets the C++20 baseline used by the companion. Compiler identity and pointer size help characterize the build, but they do not prove source authenticity, complete dependency pinning, or cross-platform compatibility.

Compare output from the actual GCC and Clang configurations recorded in `TEST_REPORT.md`. Different compilers may still use the same standard library. Do not infer library diversity solely from compiler diversity.

Failure clinic — The tested binary is replaced during packaging

A CI job tests a Debug executable, then a packaging step rebuilds Release with different feature flags and ships it without rerunning tests. The release archive is not the tested artifact. The repair is to test the final configuration and preserve the relationship between build inputs, test evidence, and output checksum.

Engineering case — Rebuild a package after its original workspace disappears

A source archive contains correct C++ files but its CMake configuration includes an absolute include directory under the author's home directory. The build succeeds in the original workspace and fails after extraction elsewhere. The archive was not self-contained even though every visible source file was present.

A relocation test creates a fresh directory, extracts the package, and runs its documented commands there. No symlink back to the original workspace should supply missing inputs. Build directories are recreated, and generated files come from included sources and scripts.

The publishing pipeline needs the same test. A book can contain the right manuscript while its rendering script reads an unshipped cover image or a stale source listing from another project. Resolve paths relative to the package root and record which files are consumed. The assembler's include manifest makes source-listing dependencies inspectable.

Environment dependencies remain legitimate when documented. Pandoc, a Python renderer, CMake, compilers, and SQLite development files need not be copied into the archive. But the README must distinguish these installed tools from files the package claims to include. A script that silently downloads a different chapter or dependency revision is not equivalent to a deterministic local rebuild.

Byte-for-byte equality can be a separate target. A PDF may include timestamps or use different installed fonts, while its text and structure remain the same. State the reproducibility level being checked. For this package, manuscript assembly should reproduce identical text from unchanged authoring and source inputs; rendering records its tool versions and may differ across environments.

A checksum manifest helps verify extracted files against the delivered archive. It does not authenticate an archive obtained from an untrusted source unless the expected manifest is itself trusted. Keep integrity checks and provenance claims separate.

The release review should therefore ask two questions: can another engineer recreate the artifact from the supplied inputs, and can they establish that those are the intended trusted inputs? Reproducibility helps the first; a controlled distribution and signing process supports the second.

Practice and implementation lab

Build the companion in separate Debug and Release directories. Run the same functional tests in both. Record compiler and SQLite versions and verify that checks remain active in Release.

Then extract the final source package into a fresh temporary directory, configure from there, and run selected integration tests. Confirm that no build step requires the original /mnt/data path or an unshipped generated file.

Review and solution reasoning

A useful build record distinguishes reproducibility of content from byte identity and separates checksum integrity from trusted provenance. It also names runtime dependencies that the archive does not bundle.

Release engineering closes the loop between code and operations: the artifact must be identifiable, testable, installable, and recoverable by someone other than its original author.

Chapter takeaway. Make the build graph explicit, attach evidence to the actual artifact, and package the runtime contract rather than only an executable filename.

Chapter 26 — Deploying Services Through Controlled Lifecycle Transitions

A process receives a termination signal while clients are still connected and a dispatcher is delivering pending work. Immediately exiting is simple, but it can interrupt responses. Waiting forever is also simple, but it can stall an upgrade indefinitely.

Deployment is a protocol among the service, its clients, its state, and its supervisor. This chapter defines startup, readiness, draining, replacement, and rollback without assuming that an orchestrator can infer application semantics automatically.

26.1 Making Startup and Readiness Explicit

26.1.1 Initializing Before Accepting Work

Construct configuration, storage connections, schema checks, and required internal state before advertising readiness for the corresponding operation. A listener accepting connections while initialization is incomplete can turn startup into a stream of misleading failures.

Partial startup needs cleanup. If the first resource is acquired and the second fails, ownership wrappers must release the first. If a background thread starts before the state it depends on is fully established, a constructor failure can make shutdown unsafe. The gateway constructs both database connections before launching its dispatcher thread.

Different operations can become available at different times. A diagnostic endpoint may answer before mutation admission is enabled. A status endpoint may work while a worker dependency is unavailable. The policy must identify what “ready” means for each traffic class.

The test harness’s `READY` port line means the local process reached its startup point and exposes a selected loopback port. It does not prove downstream health, backlog clearance, authenticated access, or disaster-recovery readiness.

26.1.2 Separating Liveness, Readiness, and Startup Probes

A liveness check influences whether a supervisor restarts a process. A readiness check influences whether it receives traffic. A startup check can allow initialization to complete before ordinary liveness expectations apply. Kubernetes documents these distinctions and their operational effects. (R35)

Choose probe dependencies carefully. Restarting every gateway because one shared worker is temporarily unavailable can destroy useful local state and create synchronized recovery load. A readiness policy might stop new work while preserving status access, whereas a liveness policy might leave the process running.

Probes themselves consume resources. A expensive database scan every second can become part of the incident. Use bounded checks that correspond to the capability claimed, and measure probe failure categories rather than exposing only a generic Boolean.

The companion does not ship a public Kubernetes deployment for its unauthenticated listener. Deployment examples in this chapter are design guidance. A real manifest requires a reviewed security adapter, state placement, service identity, probes, resource limits, and storage policy.

NEW SECTION EXAMPLE

Keep startup, liveness, and readiness probes semantically separate

Model a process that is alive before storage initialization and remains alive while draining.

main.cpp

```
#include <iostream>

int main() {
    struct Status{bool process=true,initialized=false,draining=false;
        bool startup()const{return initialized;}bool live()const{return process;}bool
ready()const{return process&&initialized&&!draining;}}s;
    std::cout<<std::boolalpha<<"boot live="<<s.live()<<" ready="<<s.ready()<<'\\n';
    s.initialized=true;std::cout<<"initialized startup="<<s.startup()<<"
ready="<<s.ready()<<'\\n';
    s.draining=true;std::cout<<"drain live="<<s.live()<<" ready="<<s.ready()<<'\\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Readiness depends on initialization and admission state. Liveness reports whether the process remains running in this model, while startup records whether initialization has completed.

A draining instance should stop receiving new traffic without being restarted solely because it is not ready. Real probe implementations must not turn every temporary dependency failure into a restart loop.

Expected behavior or output

```
boot live=true ready=false
initialized startup=true ready=true
drain live=true ready=false
```

Concrete experiment

Add a dependency-degraded flag that removes readiness without automatically failing liveness. Document when a restart could actually repair that condition.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

26.2 Draining Without Losing Ownership

26.2.1 Stopping Admission Before Releasing Resources

A drain begins by preventing new work from entering the lifecycle being stopped. Then already admitted work either completes within a budget or receives a defined interruption outcome. Finally the owner joins background activity and destroys shared state.

For a connection-oriented service, removing an endpoint from routing does not instantly close existing connections or erase cached membership. The process must still handle requests that arrive during propagation according to its drain state. A readiness transition and an application admission check complement one another.

The lab server stops accepting new connections after its stop predicate becomes true and lets existing peers finish or reach their cooperative connection deadlines. Its synchronous callbacks can still consume time. This is bounded cooperative design under ordinary runtime assumptions, not an exact wall-clock shutdown guarantee.

The gateway requests dispatcher stop before its owned state is destroyed, and `std::jthread` joins. Pending durable jobs remain in SQLite for the next process. Shutdown does not need to complete every accepted job to preserve the obligation, but the next process must reopen the same authoritative state.

26.2.2 Distinguishing Request Cancellation from Service Termination

Cancelling one caller's wait should not automatically terminate shared delivery work. Conversely, stopping the service must prevent background activity from retaining destroyed objects. These two cancellation scopes need different ownership paths.

A framework such as gRPC provides graceful shutdown facilities with documented behavior. Integrations should also have a bounded forceful fallback appropriate to the service rather than waiting indefinitely for arbitrary long-running calls. The companion does not use that framework, so its local drain mechanism is documented separately. (R44)

A response can be partially written when termination occurs. The client may not know whether the logical operation committed. Preserve the idempotency and status contract through shutdown; do not promise that disconnect means no effect.

Use signal handling narrowly. The lab signal handler stores to a compile-time-verified lock-free atomic flag; ordinary execution observes it and performs cleanup. (R48) Complex allocation, database operations, and stream logging do not belong in a signal handler merely because they are convenient shutdown actions.

Drain accepted work while rejecting a late request

Keep cancellation of one request distinct from service-wide admission shutdown.

main.cpp

```
#include <map>
#include <iostream>

int main() {
    bool accepting=true;std::map<int,bool> active{{1,false},{2,false}};
    active[1]=true; // request1 cancelled
    accepting=false;
    bool late=accepting;unsigned completed=0,cancelled=0;
    for(auto[id,stop]:active){if(stop)++cancelled;else++completed;}active.clear();
    std::cout<<"late-accepted="<<std::boolalpha<<late<<" completed="<<completed<<"
    cancelled="<<cancelled<<" remaining="<<active.size()<<"\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Cancelling request 1 does not implicitly cancel request 2. Closing admission prevents a third request while the two accepted owners reach their individual terminal states.

This fixture processes terminal outcomes instantly. A real drain must wait for callbacks or threads and use a bounded deadline; erasing a request before its background work stops does not safely reclaim its resources.

Expected behavior or output

```
late-accepted=false completed=1 cancelled=1 remaining=0
```

Concrete experiment

Give request 2 a completion time beyond the drain deadline. Report it as aborted or uncertain according to its external-effect boundary, not as completed.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

26.3 Replacing Instances Without Breaking State

26.3.1 Planning Rolling Upgrades Around Compatibility and Capacity

A rolling upgrade temporarily mixes versions and reduces or changes capacity. Verify the wire, schema, and operation compatibility matrix from Chapter 15 before routing traffic among old and new instances. A successful new-instance smoke test does not prove mixed-version behavior.

Surge capacity can help maintain availability, but stateful ownership may prohibit two independent writers from using separate copies of the same authority. Do not clone the worker database, start both copies, and randomly route retries. That

violates the receipt ownership assumptions even if each process is healthy.

Kubernetes documents pod termination and lifecycle behavior, but application-specific draining and durable state semantics remain the application's responsibility. A pod replacement is not automatically a transaction handoff or a database migration. (R34)

A small single-authority service can use a planned maintenance window instead of an unsafe rolling strategy. The operational goal is a controlled supported transition, not the appearance of zero downtime at any cost.

26.3.2 Defining Rollback Conditions Before Deployment

Rollback must consider state written by the new version. If the old binary cannot read that state, replacing the executable is not enough. A forward fix, compatible reader, or data recovery procedure may be required.

Define observable abort conditions: elevated error categories, growing backlog age, unexpected schema rejection, resource exhaustion, or violated invariants. Tie them to a specific deployment revision. A rollout should not continue merely because aggregate CPU remains low.

Keep a known artifact and a tested recovery method, but do not restore old databases casually to make a new binary start. That can erase accepted work or receipts. Code rollback and state rollback have different consequences and authorization requirements.

After replacement, verify operation-specific behavior: new submission, duplicate replay, status after restart, dependency outage handling, and recovery of retained pending jobs. A PING response alone does not establish that the deployment preserved the service contract.

NEW SECTION EXAMPLE

Gate a rolling replacement on compatibility and spare capacity

Check both available capacity and protocol overlap before removing an old replica.

main.cpp

```
#include <iostream>
#include <algorithm>

int main() {
    auto rollout= [](int replicas,int per_replica,int required,int old_max,int new_min){
        bool capacity=(replicas-1)*per_replica>=required;bool compatible=new_min<=old_max;
        return replicas>1&&capacity&&compatible;};
    std::cout<<std::boolalpha<<"safe="<<rollout(3,100,180,2,1)<<"
capacity-fail="<<rollout(2,100,180,2,1)
        <<" compatibility-fail="<<rollout(3,100,180,1,2)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

Three replicas at 100 units each leave 200 units after one is removed, enough for the assumed requirement of 180. Two replicas leave only 100; protocol compatibility is checked independently.

This is a simplified admission calculation, not an availability proof. Real rollouts require measured capacity, health checks, startup delay, state compatibility, and rollback criteria. Overlapping protocol versions do not prove that an old binary can read new persistent writes.

Expected behavior or output

```
safe=true capacity-fail=false compatibility-fail=false
```

Concrete experiment

Add a state-format compatibility condition before declaring rollback possible. Keep binary deployment rollback distinct from data rollback.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

Worked example — Model admission and draining

Requirement and implementation

Admit work only after initialization, reject new work after drain begins, and report termination readiness only after in-flight work reaches zero. This is a lifecycle state model, not an orchestrator integration.

Source — source/examples/ch26.cpp

```
#include "harbor/check.hpp"
#include <iostream>
struct Lifecycle {
    bool initialized=false, draining=false;
    unsigned in_flight=0;
    bool ready() const { return initialized && !draining; }
    bool admit() { if (!ready()) return false; ++in_flight; return true; }
    void finish() { if (!in_flight) throw std::logic_error("unmatched completion");
--in_flight; }
    bool stopped() const { return draining && in_flight==0; }
};
int main() {
    Lifecycle l; harbor::check(!l.admit(),"no admission before initialization");
    l.initialized=true; harbor::check(l.admit(),"ready");
    l.draining=true; harbor::check(!l.admit() && !l.stopped(),"stop admission first");
    l.finish(); harbor::check(l.stopped(),"drain completion");
    std::cout<<"Readiness stopped before the last admitted request completed.\n";
}
```

Evidence and interpretation

The model shows why “not accepting new requests” and “safe to destroy state” are different conditions. Existing work retains ownership until its completion path records release. A real service also needs exception-safe decrement and cancellation behavior.

The integration suite separately sends termination signals to actual processes and verifies restart behavior using retained databases. It does not deploy containers or validate a production service mesh.

Failure clinic — Liveness depends on every downstream service

A worker outage makes every gateway's liveness probe fail. The orchestrator restarts them repeatedly, clearing useful in-memory diagnostics and creating new connection storms. The probe is asking whether the whole dependency graph is healthy, then applying a local restart action that cannot repair the remote cause.

Separate local process failure from dependency degradation. Use readiness or operation-specific admission policy where appropriate, and preserve enough live diagnostic capability to investigate the outage.

Engineering case — Stop a gateway while accepted work remains pending

The worker is unavailable and the gateway has accepted five jobs. An operator needs to replace the gateway binary. Waiting for every job to complete would block indefinitely; exiting without preserving state would lose obligations.

The supported lab transition stops admission, terminates the gateway through its normal signal path, joins the dispatcher, and retains the gateway database. The next process opens the same file and resumes selecting pending jobs. Completion is deferred, but acceptance remains represented durably.

A client can lose its response during this transition. On reconnect, it reuses the same key or queries status. The gateway's duplicate contract remains essential during deployment, not just during network incidents. Creating a new key after every restart would turn an operational transition into additional business work.

The worker file must also be retained. If a worker completed one of the jobs before the outage, its receipt resolves a later replay. Replacing the gateway does not authorize resetting the worker's state or copying it into several independent authorities.

Now consider a new binary that requires schema version 2. The maintenance procedure must run a supported migration and verify rollback compatibility. Simply keeping the file is not enough if the replacement code cannot interpret it. Deployment state and schema state are coupled through the accepted-work contract.

A useful smoke test after restart checks more than PING: inspect one prior completed key, one prior pending key, an identical duplicate, a conflicting duplicate, and a new key. Then restore the worker and observe pending completion. These checks target continuity of meaning across the transition.

The test does not prove zero downtime or orchestration integration. It proves a controlled local replacement path with retained state under the specified environment. That narrower evidence is valuable because an operator can reproduce it and know exactly what it supports.

Practice and implementation lab

Add a finite in-flight operation to the lifecycle model and request drain while it is active. Verify that no new work is accepted but the existing operation can finish. Add an exception path that still releases its in-flight count.

Write a deployment runbook for the actual lab binaries with a maintenance window: stop admission, terminate the gateway, preserve both database authorities, replace the binary, restart, and run duplicate/recovery checks. State why simply running two independent workers is not a supported high-availability strategy.

Review and solution reasoning

A safe lifecycle orders admission, completion, joining, and destruction. An orchestrator can trigger transitions, but the application must implement them correctly and expose meaningful readiness.

A rollback is safe only when code and state remain compatible. Deployment review should therefore include persistence and identity, not just a container image tag.

Chapter takeaway. Treat startup, drain, replacement, and rollback as explicit state transitions that preserve accepted work and resource ownership.

Chapter 27 — Operating Reliability, Capacity, and Recovery

A service reports 99.9 percent successful requests, but that percentage excludes every request rejected before reaching the application. A backup job succeeds, but nobody has restored the worker's receipt database. An incident ends when the process restarts, even though the backlog remains hours behind.

Reliability engineering connects measurements, user outcomes, operating limits, and recovery evidence. This chapter uses the job service to make those connections concrete without inventing an uptime or performance claim for the teaching environment.

27.1 Defining Service Objectives That Match User Outcomes

27.1.1 Choosing Indicators and Denominators

A service-level indicator must specify which events count as good and which events enter the denominator. For a submission API, a good event might mean a valid new request is durably accepted within a latency threshold. For the job service as a whole, it might mean an accepted job completes within a business deadline. These are different indicators.

Exclude invalid or unauthorized requests only under a documented policy. If overload rejections are omitted without explanation, a service can appear more reliable by refusing most users. Measure admission and completion separately so degradation remains visible.

Define the observation window and data source. A process-local counter resets on restart and may miss failures before the handler runs. An external probe samples only selected operations. Durable rows provide business evidence but may not record every rejected attempt. Combine sources according to the indicator's scope.

Google's SRE workbook describes implementing service-level objectives and using indicators tied to user experience. The chapter's numerical examples are hypothetical calculations illustrating that method, not measurements of HarborDispatch's production availability. (R33)

27.1.2 Interpreting Error Budgets Without False Precision

An error budget is the allowed bad-event fraction under the selected objective and window. For a request-count objective of 99.9 percent over 10,000 eligible requests, the allowance is ten bad requests. If seven are bad, 70 percent of that allowance is consumed under this simplified model.

The calculation does not determine policy automatically. Low traffic, uneven workload, measurement gaps, and severe isolated incidents can require additional judgment. An aggregate objective can also hide one tenant or operation class

receiving much worse service.

Use objectives to guide release and reliability decisions, not to relabel known incorrect behavior acceptable. A violated safety invariant such as duplicate charging is not necessarily interchangeable with one ordinary transient error in a percentage calculation.

Alerting should reflect urgency and actionability. A rapidly consumed budget can justify immediate response, while a slowly growing capacity trend may need planned work. Every alert should identify the owner, the affected contract, and an initial diagnostic path.

NEW SECTION EXAMPLE

Compute an error budget from logical service outcomes

Use a defined logical-request population and an integer error budget.

main.cpp

```
#include <cstdint>
#include <iostream>

int main() {
    constexpr std::uint64_t requests=10000,failed=7; // illustrative evaluation window
    constexpr std::uint64_t allowed_per_thousand=1;
    auto budget=requests*allowed_per_thousand/1000;
    std::cout<<"allowed-failures="<<budget<<" observed="<<failed<<"
    remaining="<<(failed<=budget?budget-failed:0)<<'\n';
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The 99.9% objective permits ten unsuccessful operations among 10,000. Seven failures consume seven units and leave three. The dataset is an illustrative closed window, not production telemetry.

Define which operations enter the denominator and what counts as success before calculating. Retries must not inflate a logical-operation denominator, and rounding policy matters for small populations.

Expected behavior or output

```
allowed-failures=10 observed=7 remaining=3
```

Concrete experiment

Set failed to 12 and report exhaustion plus two excess failures. Prevent unsigned subtraction from wrapping into a large positive allowance.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

27.2 Planning Capacity and Incident Response

27.2.1 Forecasting the Limiting Resource

Measure arrival rate, service time, backlog age, storage growth, and resource utilization under representative work. A job count without input cost can mislead capacity planning. For this lab, the square computation is trivial; persistence, connection handling, and retry behavior dominate many experiments.

Separate sustainable throughput from a short burst. A service can accept a burst into storage while completing it slowly afterward. Record backlog clearance and latency distributions before calling the burst rate sustained capacity.

Retention creates long-term capacity requirements. Completed jobs and receipts accumulate in the current implementation. Estimate growth from actual serialized and indexed storage, not merely the size of a C++ struct. Include backups, WAL behavior, logs, and temporary migration space.

Define thresholds before resources are exhausted. Disk-full behavior can affect transaction commit and diagnostics at the same time. A service should stop accepting new obligations before it loses the ability to record them safely, while preserving controlled status and recovery access where possible.

27.2.2 Responding with a Hypothesis and a Reversible Action

Begin an incident by identifying the user-visible symptom and the relevant boundary. Are submissions rejected, jobs pending, worker replies failing, or status reads inconsistent? These categories lead to different evidence and actions.

Preserve timestamps, deployment identity, error categories, and a bounded sample of correlated operations. Do not dump all credentials or database contents into a chat or ticket. Protect operational evidence according to its sensitivity.

Prefer a reversible action tied to a hypothesis. Reducing admission can protect a saturated dependency; restarting an unrelated healthy process may not. Clearing the receipt table can make the symptom disappear while destroying the duplicate-handling contract.

Track the consequences of mitigation. A lower admission rate can improve latency while increasing rejections. A stopped dispatcher can prevent repeated harmful attempts while leaving accepted jobs pending. Communicate both the immediate benefit and the remaining obligation.

NEW SECTION EXAMPLE

Estimate the limiting service capacity before choosing mitigation

Compare CPU demand, database capacity, and downstream capacity for an illustrative workload.

main.cpp

```
#include <algorithm>
#include <array>
#include <iostream>

int main() {
    constexpr double cpu_cores=4,cpu_seconds_per_request=0.002;
    double cpu_limit=cpu_cores/cpu_seconds_per_request;
    double database_limit=1200,downstream_limit=1600;
    double system_limit=std::min({cpu_limit,database_limit,downstream_limit});
    std::cout<<"cpu="<<cpu_limit<<" database="<<database_limit<<"
downstream="<<downstream_limit<<" limiting="<<system_limit<<" req/s\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The CPU model permits 2,000 requests per second, but the database's 1,200-request capacity limits this dependency chain first. Adding CPU cores alone does not remove that bottleneck.

These inputs illustrate hypothesis formation, not a forecast derived from a live system. Validate queue growth and latency near saturation, then choose a reversible mitigation instead of changing several layers at once.

Expected behavior or output

```
cpu=2000 database=1200 downstream=1600 limiting=1200 req/s
```

Concrete experiment

Raise modeled database capacity to 1,800. The downstream limit of 1,600 should become the next constraint.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

27.3 Proving Recovery Rather Than Assuming It

27.3.1 Defining Recovery Point and Recovery Time

Recovery point describes how much state loss is acceptable under a stated failure scenario. Recovery time describes how long restoration of the required service can take. A backup interval alone does not establish either value; restore procedures, dependencies, and reconciliation affect them.

For HarborDispatch, the gateway and worker have separate authoritative files. A consistent recovery plan must account for both and for any external effect a future worker performs. Restoring one file to a different point can create missing obligations or forgotten receipts.

Use a supported SQLite backup procedure and test restoration into an isolated location. Verify schema compatibility, row meaning, receipt replay, and pending-job

progress. A checksum confirms copied bytes under a known reference; it does not prove that the application can resume correctly from them. (R22)

Do not practice destructive recovery on the only copy of live state. Preserve the original evidence, establish a separate restore destination, and make any traffic cutover deliberate. The teaching runbook uses private directories and does not claim a universal production disaster-recovery procedure.

27.3.2 Closing an Incident with a Causal Review

A postmortem should distinguish trigger, contributing conditions, impact, detection, mitigation, and prevention. “The database was slow” may describe a symptom without explaining why new work was admitted without a bound or why every client retried simultaneously.

Use a timeline with event meanings and clock limitations. Correlate deployment changes, error categories, queue growth, and state transitions. Avoid declaring causality merely because two graphs changed at similar wall times.

Corrective actions should address mechanisms and have verification criteria. “Be more careful” is weak. “Reject new work when durable pending count reaches the configured limit, and add a test that preserves duplicate lookup while full” is concrete and testable.

Recovery ends when the relevant service contract is restored, not merely when the process is alive. Verify backlog clearance, status accuracy, resumed admission, and the absence of unresolved ambiguous effects. Document remaining risk and assign ownership for follow-up work.

NEW SECTION EXAMPLE

Restore through SQLite’s backup API and validate recovered records

Exercise SQLite’s backup API and reopen the copy to verify recovered records.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>
struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};
struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};
inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}
struct Transaction {

    Database& d;bool committed=false;
    explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
    void commit(){d.exec("COMMIT");committed=true;}
    ~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};
```

main.cpp

```
#include "sqlite_support.hpp"

int main(){try{
    Database source;source.exec("CREATE TABLE jobs(id INTEGER PRIMARY KEY,result
INTEGER);INSERT INTO jobs VALUES(1,144),(2,169)");
    Database backup("backup.db");sqlite3_backup*
handle=sqlite3_backup_init(backup.p,"main",source.p,"main");
    if(!handle)throw std::runtime_error("backup init");int
step=sqlite3_backup_step(handle,-1);int finish=sqlite3_backup_finish(handle);
    if(step!=SQLITE_DONE||finish!=SQLITE_OK)throw std::runtime_error("backup copy");
    Database restored("backup.db");auto count=scalar(restored,"SELECT count(*) FROM
jobs"),sum=scalar(restored,"SELECT sum(result) FROM jobs");
    if(count!=2||sum!=313)return 1;std::cout<<"restored rows=2 result-sum=313\n";
}catch(const std::exception&e){std::cerr<<e.what()<<'\n';return 1;}}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example
```


Explanation

The backup API copies a consistent database into another connection. Return codes are checked and backup_finish releases the backup handle even after an unsuccessful step.

Reopening the backup and querying count and content supplies restore evidence. It does not establish RPO or RTO without workload timestamps and measured recovery intervals.

Expected behavior or output

```
restored rows=2 result-sum=313
```

Concrete experiment

Commit a third source job after backup completes and verify that it is absent from the backup. That visible cutoff is part of the recovery-point discussion.

Validation of this new example: gcc: passed

Technical references: SQLITE-C, SQLITE-BACKUP. See the added references and revision index.

Worked example — Calculate a hypothetical request-count budget

Requirement and implementation

For an explicitly hypothetical set of 10,000 eligible requests and seven bad outcomes, compute the allowance under a 99.9 percent objective and the consumed fraction. Reject the temptation to call the result a measurement of the supplied service.

Source — source/examples/ch27.cpp

```
#include "harbor/check.hpp"
#include <cmath>
#include <iostream>
int main() {
    const long double requests=10000, bad=7, target=0.999L;
    const auto fraction=(requests-bad)/requests;
    const auto allowance=(1-target)*requests;
    harbor::check(std::abs(allowance-10)<1e-9L,"budget calculation");
    std::cout<<"good_fraction="<<static_cast<double>(fraction)
        <<" consumed_budget_fraction="<<static_cast<double>(bad/allowance)<<"\n";
    std::cout<<"The denominator must represent the actual user journey.\n";
}
```

Evidence and interpretation

The arithmetic illustrates a denominator and objective. It does not establish that 99.9 percent is the right objective for any reader's service or that all bad events have identical business severity. The application must choose indicators and policy with stakeholders who own the actual outcome.

A job-completion objective would need a different dataset: accepted job identities, their completion times, and the treatment of still-pending jobs at the observation boundary. Counting only completed jobs can hide the oldest stuck work.

Failure clinic — A successful backup contains the wrong authority

A scheduled job copies the gateway file, but the worker's receipt file is omitted. After loss of the worker host, retries are treated as new work. The backup job's exit code was true evidence about its copy operation and false evidence about complete service recoverability.

Repair the inventory of authoritative state, use a consistent backup method, and test a duplicate replay after restoration. The recovery test should target the business invariant, not only file existence.

Engineering case — Close an incident only after the backlog is reconciled

At 09:00 the worker becomes unavailable. The gateway accepts jobs until its pending limit is reached, then returns BUSY for new keys. At 09:10 the worker restarts and answers health probes. Declaring full recovery at 09:10 ignores the accepted backlog and the clients still waiting for results.

Track at least three phases: dependency reachability restored, backlog processing resumed, and the relevant completion objective recovered. Their timestamps can differ substantially. A large replay surge can also cause a second slowdown, so restoration should remain subject to capacity and retry controls.

The incident report should distinguish rejected new work from accepted delayed work. Rejected callers need an explicit retry or resubmission policy. Accepted callers should retain their existing keys and query status. Mixing those groups can produce either lost intent or unnecessary duplicate logical jobs.

After processing resumes, reconcile gateway completion and worker receipts for a bounded relevant set. An unresolved mismatch may indicate a lost response, a permanent conflict, or a damaged state file. Do not clear pending rows merely to make the dashboard green; each row represents an obligation that needs a terminal outcome or authorized repair.

The postmortem can identify a trigger, such as a worker deployment, and contributing conditions, such as insufficient capacity margin or unclear readiness. Corrective actions should be verifiable: a deployment check for schema compatibility, an alert on pending age, or a test that preserves duplicate lookup while the queue is full.

Avoid using one availability percentage to summarize every consequence. A ten-minute outage affecting only status reads differs from ten minutes of lost accepted jobs. The metric definition should preserve the outcome that matters to the service's users.

This case also demonstrates why runbooks must identify authority. A suggested mitigation that deletes worker receipts may reduce disk use while invalidating duplicate safety. An operator should be able to recognize that tradeoff from documentation before executing the command under pressure.

Practice and implementation lab

Write two indicators for the lab: submission acceptance and accepted-job completion. Define good events, eligible events, observation window, and how pending work is represented. Identify the measurements missing from the current small METRICS response.

Create an isolated restore drill for both databases. Record startup, a duplicate request, a new request, and pending recovery. State which failure scenario the drill covers and which scenarios, such as storage-controller failure or regional loss, remain untested.

Review and solution reasoning

Reliability percentages are meaningful only with a denominator and outcome contract. Recovery objectives are meaningful only with a tested procedure and state inventory. Neither should be inferred from one green dashboard or one successful file copy.

Operational maturity is the ability to explain and restore the service under failure without discarding the invariants that made its normal behavior correct.

Chapter takeaway. Measure user outcomes, preserve authoritative state, and make incident recovery and prevention verifiable engineering work.

Chapter 28 — Building and Reviewing HarborDispatch

The capstone combines a bounded wire protocol, native C++ resource ownership, a loopback reactor, local transactions, durable delivery intent, duplicate recognition, background retries, and process-level recovery tests. It is a complete instructional application, not a production distributed platform.

The goal is to make every important promise traceable to code and evidence. A reader should be able to identify where a job becomes accepted, where its effect is recorded, why a lost reply is recoverable, and which additional mechanisms would be required before remote deployment or multiple independent workers were safe.

28.1 Defining the Architecture and Its Trust Boundary

28.1.1 Assigning Responsibilities to the Client, Gateway, and Worker

The client sends one framed command over one IPv4 loopback connection and prints the response. It has a finite exchange budget and does not automatically create new job identities or retry mutations. A transport failure produces an error exit. A syntactically received application error is printed as a response; scripts must inspect that response rather than assuming every zero client exit means a job succeeded.

The gateway validates submission syntax, commits accepted intent, serves status, and runs a dispatcher for pending jobs. Its API handler and dispatcher use separate SQLite connections to the same file. Its counters are in memory and reset after restart; its jobs remain in the database.

The worker validates execution requests and commits a receipt containing key, input, and result. The domain result is the square of an integer in the closed range zero through one million. This makes the maximum result one trillion, within the chosen signed 64-bit representation, and gives the gateway an exact inexpensive validation oracle.

The process topology is deliberately fixed:

```

client -> gateway listener -> gateway SQLite jobs
                                |
                                pending dispatcher
                                |
                                v
                                worker listener -> worker SQLite receipts

```

There is one configured worker endpoint and one worker receipt authority. No TLS, authentication, broker, replication, service mesh, or consensus system is hidden behind these arrows. Any production extension must add those mechanisms explicitly and preserve the operation contract.

28.1.2 Enumerating Limits and Unsupported Promises

Payloads are bounded at 4096 bytes. Keys contain one to 64 permitted ASCII characters. Each server admits at most 64 simultaneous peer connections and accepts at most 16 new connections in one reactor iteration. A connection has a two-second cooperative lifetime; the poll loop wakes periodically to inspect state and stop requests.

The gateway accepts at most 128 currently pending unique jobs by default. Duplicate submissions with unchanged meaning remain recognizable when that limit is reached. Completed rows and receipts are not pruned, so disk growth is not fully bounded by the pending limit.

The dispatcher selects batches of at most 16 pending jobs and gives each worker exchange a 200 ms budget. These are teaching choices. A synchronous database operation or ordinary scheduler delay can extend observed wall time beyond a simple sum of nominal timers. There is no hard-real-time guarantee.

The reactor handles partial socket I/O, but handlers execute synchronously. Moving to an asynchronous database adapter would require explicit operation lifetime and completion ownership. Calling the current server “nonblocking” should refer to its socket mechanism, not imply that every dependency is asynchronous.

NEW SECTION EXAMPLE

Enforce the gateway/worker domain boundary before dispatch

Validate job keys and bounded inputs at the gateway, then check the returned result against an independent integer oracle.

main.cpp

```
#include <string_view>
#include <charconv>
#include <cstdint>
#include <iostream>
#include <optional>
#include <algorithm>

int main() {
    struct Job{std::string_view key;std::int64_t input;};
    auto parse=[](std::string_view key,std::string_view text)->std::optional<Job>{
        if(key.empty()||key.size()>64||!std::all_of(key.begin(),key.end(),[](unsigned char
c){return(c>='a'&&c<='z')||(c>='A'&&c<='Z')||(c>='0'&&c<='9')||c=='_'||c=='-';}))return{};
        std::int64_t x=0;auto[p,e]=std::from_chars(text.data(),text.data()+text.size(),x);
        if(e!=std::errc{}||p!=text.data()+text.size()||x<0||x>1000000)return{};return
Job{key,x};};
    auto good=parse("job_7","12"),bad=parse("bad key","12");
    if(!good||bad)return 1;std::int64_t worker_result=144;
    std::cout<<"gateway-valid="<<good->key<<"
worker-result-verified="<<std::boolalpha<<(worker_result==good->input*good->input)<<"\n";
}
```

Run or inspect

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -pthread main.cpp -o example
./example
```

Explanation

The gateway checks key syntax and input range before acceptance. The one-million bound keeps squaring within `int64_t`. A worker reply is compared with the accepted canonical input instead of being trusted because transport completed.

The borrowed key view remains valid only while its source input lives; durable jobs need an owned copy. This example isolates the capstone's domain boundary, while section 28.3 runs the actual three-process application.

Expected behavior or output

```
gateway-valid=job_7 worker-result-verified=true
```

Concrete experiment

Use input 1,000,000 and expect 1,000,000,000,000. Reject 1,000,001 and whitespace-bearing keys before any database or worker operation.

Validation of this new example: gcc: passed

Technical references: CPP20. See the added references and revision index.

28.2 Following a Job Through Commit and Recovery

28.2.1 Understanding the Submission and Execution Protocol

The gateway accepts `SUBMIT` key input, `STATUS` key, `PING`, and `METRICS`. The worker accepts `EXEC` key input, `COUNT`, and `PING`. Commands are contained inside the length-prefixed payload, not sent as an unframed terminal line. The supplied client handles framing.

For a valid new submission, the gateway returns `ACCEPTED` key only after the pending row commits. An identical duplicate also returns `accepted`. A changed input under an existing key returns `CONFLICT`. New work beyond the pending quota returns `BUSY`. Invalid syntax or domain values return `INVALID`; other handled application failures use a bounded error response.

Status returns `MISSING`, `PENDING` key, or `DONE` key result. Missing is an observation of this gateway's authoritative file, not a global search across hypothetical replicas. A copied or replaced database can change that observation, which is why state identity belongs in operations documentation.

The dispatcher sends the same key and canonical input to the worker. The worker returns `RESULT` key result from either a new committed receipt or an existing matching receipt. The gateway checks response structure, key, and exact result before recording completion.

28.2.2 Recovering the Lost-Reply and Restart Histories

If the gateway stops after acceptance, its pending row remains. If the worker commits and drops its reply, the dispatcher retries and retrieves the retained result. If the gateway receives a result but stops before recording completion, replay again retrieves the same receipt.

The worker's `--drop-first-reply` option is a deliberate test failpoint. It drops the response after the first valid execution is committed. It should not be enabled in ordinary use. Its placement gives the integration suite evidence for the difficult ambiguous-outcome case rather than merely a failure before state changes.

On worker restart with the same database file, the receipt survives and a repeated key returns the saved result. On gateway restart with the same gateway file, pending and completed rows survive. These are local file-backed recovery claims tested through real child processes.

Losing or restoring one authority independently requires reconciliation. The application cannot infer from the other database alone every historical promise or effect after arbitrary data loss. The supported lab recovery is process replacement with retained valid files, not automatic disaster recovery from any partial backup.

NEW SECTION EXAMPLE

Recover a pending gateway job from a committed worker receipt

Use separate gateway and worker SQLite databases to recover from a lost reply without claiming a distributed transaction.

sqlite_support.hpp

```
#pragma once
#include <sqlite3.h>
#include <stdexcept>
#include <string>
#include <iostream>

struct Database {
    sqlite3* p=nullptr;
    explicit Database(const char* path=":memory:") {
        if(sqlite3_open(path,&p)!=SQLITE_OK){std::string
e=p?sqlite3_errmsg(p):"open";if(p)sqlite3_close(p);throw std::runtime_error(e);}
    }
    ~Database(){sqlite3_close(p);}
    Database(const Database&)=delete;Database&operator=(const Database&)=delete;
    void exec(const char* sql){char* e=nullptr;int r=sqlite3_exec(p,sql,nullptr,nullptr,&e);
        if(r!=SQLITE_OK){std::string s=e?"SQL error";sqlite3_free(e);throw
std::runtime_error(s);}}
};

struct Statement {
    sqlite3_stmt* p=nullptr;
    Statement(Database& d,const char*
sql){if(sqlite3_prepare_v2(d.p,sql,-1,&p,nullptr)!=SQLITE_OK)throw
std::runtime_error(sqlite3_errmsg(d.p));}
    ~Statement(){sqlite3_finalize(p);}
    Statement(const Statement&)=delete;Statement&operator=(const Statement&)=delete;
    void text(int n,const std::string&
s){if(sqlite3_bind_text(p,n,s.data(),int(s.size()),SQLITE_TRANSIENT)!=SQLITE_OK)throw
std::runtime_error("bind");}
    void integer(int n,sqlite3_int64 v){if(sqlite3_bind_int64(p,n,v)!=SQLITE_OK)throw
std::runtime_error("bind");}
    bool row(){int r=sqlite3_step(p);if(r==SQLITE_ROW)return true;if(r==SQLITE_DONE)return
false;throw std::runtime_error(sqlite3_errstr(r));}
};

inline sqlite3_int64 scalar(Database& d,const char* sql){Statement s(d,sql);if(!s.row())throw
std::runtime_error("missing scalar");return sqlite3_column_int64(s.p,0);}

struct Transaction {
```

```

Database& d;bool committed=false;
explicit Transaction(Database&db):d(db){d.exec("BEGIN IMMEDIATE");}
void commit(){d.exec("COMMIT");committed=true;}
~Transaction(){if(!committed)(void)sqlite3_exec(d.p,"ROLLBACK",nullptr,nullptr,nullptr);}
};

```

main.cpp

```

#include "sqlite_support.hpp"

int main(){try{
    Database gateway("gateway.db"),worker("worker.db");
    gateway.exec("CREATE TABLE jobs(id TEXT PRIMARY KEY,input INTEGER,result INTEGER);INSERT
    INTO jobs VALUES('lesson28',12,NULL)");
    worker.exec("CREATE TABLE receipts(id TEXT PRIMARY KEY,input INTEGER,result INTEGER);BEGIN
    IMMEDIATE;INSERT INTO receipts VALUES('lesson28',12,144);COMMIT");
    // The worker reply is lost here; gateway remains pending.
    if(scalar(gateway,"SELECT count(*) FROM jobs WHERE result IS NULL")!=1)return 1;
    auto result=scalar(worker,"SELECT result FROM receipts WHERE id='lesson28'");
    if(result!=144)return 2;
    {Transaction tx(gateway);Statement update(gateway,"UPDATE jobs SET result=?1 WHERE
    id='lesson28' AND input=12");update.integer(1,result);update.row();tx.commit();}
    std::cout<<"DONE lesson28 "<<scalar(gateway,"SELECT result FROM jobs")<<"
    worker-receipts="<<scalar(worker,"SELECT count(*) FROM receipts")<<'\n';
    }catch(const std::exception&e){std::cerr<<e.what()<<'\n';return 1;}}

```

Run or inspect

```

g++ -std=c++20 -Wall -Wextra main.cpp -lsqlite3 -o example
./example

```

Explanation

Gateway acceptance persists a NULL result, representing pending work. Worker completion commits 144 in its own database; losing the response does not erase either record.

Recovery reads the existing receipt, checks it against the accepted input 12, then commits gateway completion. The files commit at different times, so safety depends on stable operation identity and retained receipts rather than cross-database atomicity.

Expected behavior or output

```
DONE lesson28 144 worker-receipts=1
```

Concrete experiment

Interrupt before the gateway completion transaction and rerun reconciliation. Worker receipt count must remain one; the effect need not be executed again.

Validation of this new example: gcc: passed

Technical references: [SQLITE-C](#), [SQLITE-TRANSACTION](#). See the added references and revision index.

28.3 Building, Exercising, and Reviewing the System

28.3.1 Running the Actual Binaries in a Private Lab

From the extracted project root, build and test in a separate directory:


```
cmake -S source -B build/lab -DCMAKE_BUILD_TYPE=Debug
cmake --build build/lab --parallel 4
ctest --test-dir build/lab --output-on-failure
```

Create a private temporary directory for manual state. In one terminal, start the worker with an ephemeral port:

```
LAB_DIR=$(mktemp -d)
printf '%s\n' "$LAB_DIR"
./build/lab/harbor_worker "$LAB_DIR/worker.db" 0
```

The worker prints READY followed by its selected port. Keep it running. In a second terminal, use the same printed directory and that port, replacing the explicit placeholders:

```
./build/lab/harbor_gateway \
  /REPLACE_WITH_LAB_DIRECTORY/gateway.db 0 REPLACE_WITH_WORKER_PORT
```

The gateway prints its own selected port. In another terminal, use that port with the client:

```
./build/lab/harbor_client REPLACE_WITH_GATEWAY_PORT SUBMIT lesson28 12
./build/lab/harbor_client REPLACE_WITH_GATEWAY_PORT STATUS lesson28
./build/lab/harbor_client REPLACE_WITH_GATEWAY_PORT SUBMIT lesson28 12
./build/lab/harbor_client REPLACE_WITH_GATEWAY_PORT SUBMIT lesson28 13
```

The first response should be accepted under ordinary available capacity. Status can initially be pending because delivery is asynchronous; query again to observe completion. The identical submission remains accepted, while changed input conflicts. These are expected protocol outcomes, not a promise about exact scheduling order between commands.

Stop the processes with their normal termination signal before removing the private directory. Do not point a teaching run at an existing application database or copy shell commands containing placeholders without replacing and inspecting them.

28.3.2 Reviewing the Evidence Before Extending the Design

The default test suite has three layers: chapter executables and pure model tests, file-backed storage tests, and real process integration tests. The final TEST_REPORT.md records the configurations actually completed for this edition. The source and test scripts are included so the same claims can be checked after a change.

Review the failure cases, not only the aggregate passing count. The suite tests lost replies, gateway and worker restarts, fragmented requests, invalid input, identity conflicts, overload, and interrupted transaction rollback. It does not run a live broker, a gRPC server, TLS certificate validation, Kubernetes, replicated consensus, or cross-region failover.

Before increasing scope, write the new invariant and identify which existing assumption it replaces. Adding several workers replaces the single receipt authority assumption. Public exposure replaces the controlled local trust boundary. Receipt pruning replaces indefinite replay recognition. A different result function can replace the cheap exact verification oracle.

Each change should arrive with migration, failure, and recovery tests. A larger architecture diagram is not itself an improvement. The improved design is the one whose promises remain traceable across the additional boundaries.

NEW SECTION EXAMPLE

Run the real client, gateway, and worker against a private lab

Build the original capstone's client, gateway, and worker, submit a job through loopback TCP, and test duplicates and conflicts. Unchanged capstone source is bundled under upstream/; the harness is new.

Linux-only actual loopback integration. Original supporting source is bundled under upstream/ and is not substituted for the newly written test.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(Section28Lab LANGUAGES CXX)
find_package(Threads REQUIRED)
find_package(SQLite3 REQUIRED)
add_library(support upstream/src/net.cpp upstream/src/store.cpp)
target_include_directories(support PUBLIC upstream/include)
target_compile_features(support PUBLIC cxx_std_20)
target_link_libraries(support PUBLIC Threads::Threads SQLite::SQLite3)
foreach(name client gateway worker)
  add_executable(harbor_${name} upstream/src/${name}.cpp)
  target_link_libraries(harbor_${name} PRIVATE support)
endforeach()
```

lab.py

```

import pathlib,selectors,subprocess,tempfile,time
root=pathlib.Path('build').resolve()
processes=[]
def start(name,*args):
    p=subprocess.Popen([str(root/name),*map(str,args)],stdout=subprocess.PIPE,stderr=subprocess.PIPE,text=True)
    processes.append(p)
    with selectors.DefaultSelector() as events:
        events.register(p.stdout,selectors.EVENT_READ)
        if not events.select(8):raise RuntimeError('startup timeout')
        line=p.stdout.readline().strip()
        if not line.startswith('READY '):raise RuntimeError('bad startup: '+line)
        return int(line.split()[1])
def call(port,*args):
    r=subprocess.run([str(root/'harbor_client'),str(port),*args],capture_output=True,text=True,timeout=4,check=True)
    return r.stdout.strip()
with tempfile.TemporaryDirectory(prefix='section28-') as directory:
    d=pathlib.Path(directory)
    try:
        worker=start('harbor_worker',d/'worker.db',0)
        gateway=start('harbor_gateway',d/'gateway.db',0,worker)
        assert call(gateway,'SUBMIT','lesson28','12')== 'ACCEPTED lesson28'
        deadline=time.monotonic()+8
        while call(gateway,'STATUS','lesson28')!='DONE lesson28 144':
            if time.monotonic(>deadline:raise RuntimeError('completion timeout')
            time.sleep(.02)
        assert call(gateway,'SUBMIT','lesson28','12')== 'ACCEPTED lesson28'
        assert call(gateway,'SUBMIT','lesson28','13')== 'CONFLICT'
        assert call(worker,'COUNT')== '1'
        print('real processes: DONE144 duplicate accepted conflict rejected receipts1')
    finally:
        for p in reversed(processes):
            if p.poll() is None:
                p.terminate()

                try:p.wait(timeout=4)
                except subprocess.TimeoutExpired:p.kill();p.wait(timeout=2)
            p.communicate(timeout=2)

```

run.sh

```

set -eu
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug >configure.log
cmake --build build --parallel 3 >build.log
python3 lab.py

```

Run or inspect

```
sh run.sh
```

Explanation

The harness waits for actual READY lines and uses selected ephemeral ports instead of assuming a fixed port is free. Client calls use the original executable's framing and protocol, not direct database edits.

It checks accepted intent, asynchronous DONE 144, identical retry acceptance, changed-input conflict, and one worker receipt. Cleanup stops the gateway before the worker and owns a disposable directory. This is local process/TCP/SQLite evidence, not TLS or cluster qualification.

Expected behavior or output

real processes: DONE144 duplicate accepted conflict rejected receipts1

Concrete experiment

Launch the worker with `--drop-first-reply` and rerun the scenario. Completion should still reach 144 with one receipt; retain stdout and stderr when diagnosing a failure.

Validation of this new example: gcc: passed

A script result covers only the commands actually executed by its recorded test. Separate debugger, profiler, editor, and platform instructions are not implicitly validated.

Technical references: `SQLITE-C`, `LINUX-SOCKET`. See the added references and revision index.

Worked example — Join intent and receipts across two stores

Requirement and implementation

Submit a finite set of jobs, deliberately apply each worker request twice, record matching results at the gateway, and verify that every accepted job is complete with one receipt per logical key.

Source — `source/examples/ch28.cpp`

```
#include "harbor/check.hpp"
#include "harbor/store.hpp"
#include <iostream>
int main() {
    harbor::Store api(":memory:"), worker(":memory:");
    for (int i=0; i<10; ++i) {
        harbor::Request r{"job-"+std::to_string(i), i}; api.submit(r);
    }
    for (const auto& r: api.pending()) {
        auto result=worker.apply(r);
        harbor::check(worker.apply(r)==result, "duplicate effect result");
        api.complete(r, result);
    }
    harbor::check(api.pending().empty() && worker.receipt_count()==10, "end state");
    std::cout<<"Ten jobs completed with stable identities; process/network tests are separate.\n";
}
```

Interpret the local capstone model

The chapter executable uses two in-memory stores to make the transaction protocol easy to trace. The full application sources in Appendix B implement the actual process boundary. The integration tests exercise that implementation rather than claiming that two objects in one process are equivalent to a deployment.

The model deliberately repeats the worker operation. A correct receipt count demonstrates duplicate recognition in the retained store. It does not claim the arithmetic expression was executed only once or that an arbitrary external side effect would be protected by the same transaction automatically.

Failure clinic — A convenient shortcut changes the authority

A team copies `worker.db` to a second machine and routes half the retries there. Initially the copies contain the same receipts. New writes then diverge, and a retry can reach a copy that lacks the relevant effect. The shortcut has created two independent authorities without replication or a partition protocol.

Another team deletes completed receipts to reclaim space but continues to advertise unlimited replay safety. An old retry is now new work. Both changes can pass happy-path tests. Their defects appear only when the original identity and recovery assumptions are examined.

Engineering case — Choose one extension and preserve the reference contract

Suppose the next project replaces the custom framed transport with gRPC. The goal is not to redesign every layer at once. Keep the domain request, canonicalization, store transaction, and duplicate result semantics unchanged while introducing the new adapter.

Define a schema whose fields preserve the key and integer meaning. Map application outcomes explicitly: accepted, conflict, capacity rejection, invalid input, pending, and completed. Decide which are domain response fields and which are transport status codes. Do not encode a successful business conflict as an unstructured exception that clients retry blindly.

The adapter must carry deadlines and cancellation according to the framework's documented behavior, but durable acceptance still outlives a caller's wait by design. A gRPC cancellation does not erase a committed pending row. Tests should retain the lost-response and restart histories under the new transport rather than replacing them with only a hello-world RPC.

Generated schema sources, runtime versions, and TLS credentials introduce new build and operational requirements. Keep the original loopback client as a reference path until semantic equivalence is established. The optional `.proto` file in this package is a starting contract exercise, not a prevalidated implementation of that adapter.

A second possible extension is persistent terminal failure and redrive. It changes the gateway state machine and schema rather than the transport. It needs a reason code, attempt metadata, an authorized redrive identity, and rules for preserving the original worker key. Reusing a new key during redrive could duplicate an effect that merely had an uncertain reply.

Choose one extension, write its invariant and migration plan, and add tests at the boundary it changes. Combining gRPC, sharding, new computation semantics, and receipt pruning in one revision makes failures difficult to attribute and rollback difficult to reason about.

The capstone's value is that it provides a small executable reference whose assumptions are explicit. Extend it by replacing one assumption with a stronger tested mechanism, not by accumulating frameworks around an undocumented core.

Practice and implementation lab

Choose one extension and write its acceptance review before coding. Suitable projects include persistent terminal-failure state with authorized redrive, per-tenant admission behind a verified identity adapter, a gRPC transport adapter preserving the same domain service, or a bounded metrics exporter.

For each extension, identify the unchanged domain contract, new state, new failure boundary, dependency versions, migration requirement, and negative tests. Keep the original reference implementation available so a new transport or scheduler can be compared against it.

Review and solution reasoning

The complete lab is useful because it combines real networking and persistence with a small exact domain. It exposes uncertainty and recovery without hiding them inside a vendor service. It remains intentionally narrower than a production platform.

A professional review should be able to answer: who owns this state, when does this promise commit, what happens if the reply is lost, how is duplicate meaning checked, which resources are bounded, and what evidence supports restart? When those answers are precise, adding an external framework becomes an engineering integration rather than an act of faith.

Chapter takeaway. A distributed application is understandable when every accepted obligation, effect, retry, and recovery action has an explicit owner and a verifiable boundary.

Appendix A — HarborDispatch Operations and Diagnostic Workbook

This workbook applies to the supplied local teaching service. It uses private directories, loopback connections, and the included client. It is not a runbook for an employer's infrastructure or an instruction to expose the application on a public network. Adaptation to production requires security, retention, availability, and recovery decisions that the lab intentionally does not make.

A.1 Establish the Environment Before Starting Processes

Record the operating system, compiler, CMake version, Python version, and SQLite development/runtime identity used for the build. Confirm that the binaries you are running come from the intended build directory. A stale executable left behind after a failed compilation can make a test appear to validate source that never built.

```
cmake --version
c++ --version
python3 --version
cmake -S source -B build/operations -DCMAKE_BUILD_TYPE=Debug
cmake --build build/operations --parallel 4
ctest --test-dir build/operations --output-on-failure
```

The test suite creates its own temporary state and ephemeral ports. It does not require a preexisting database. The manual procedure differs because you retain state deliberately across process restarts. Write down the private directory you create and do not confuse it with another test's temporary directory.

A successful build establishes compilation for that configuration. A successful CTest run establishes the recorded test outcomes. Neither proves that an optional gRPC schema was compiled, that TLS was configured, or that a remote cluster is healthy. Read the final verification report for the exact executed matrix.

A.2 Maintain a Small State Inventory

The gateway database contains accepted intent and completion state. The worker database contains receipt identity and recorded effect. Both are authoritative within the lab's contract. In-memory counters, active connections, decoder buffers, and retry backoff state are reconstructible or transient.

An inventory should record each file's role, owning process, schema version, backup method, and restart command. Do not include secrets in the inventory. The lab has no application credentials, but a production adaptation will have identity and storage permissions that need a separate protected mechanism.

Use distinct file names for gateway and worker even though the shared teaching store creates both tables in each database. The application uses the relevant table for

each role; sharing one file between the two processes would blur the intended independent-authority exercise.

A copied file is not automatically a current replica. If you create a backup, identify its point and method. If you restore it, keep the restore isolated until application semantics and any external effects are reconciled. Do not run two independent worker copies and call them redundant without a replication or partitioning protocol.

A.3 Start the Lab and Record Endpoints

Start the worker first with a selected or ephemeral port. Port zero asks the operating system to select an available port. The process prints the chosen value in a READY line. Use that observed value for the gateway's worker endpoint.

```
LAB_DIR=$(mktemp -d)
printf 'Private state directory: %s\n' "$LAB_DIR"
./build/operations/harbor_worker "$LAB_DIR/worker.db" 0
```

In another terminal, start the gateway with the same private state directory and the observed worker port. The gateway's own listener can also use zero. Replace placeholders deliberately rather than treating them as literal commands:

```
./build/operations/harbor_gateway \
/REPLACE_WITH_PRIVATE_DIRECTORY/gateway.db \
0 REPLACE_WITH_WORKER_PORT
```

Record both endpoint values and the process terminals. The application accepts only local IPv4 connections. A connection failure from another host is expected under this design, not evidence that firewall rules should be opened.

Use the client for protocol interaction. `echo SUBMIT ... | nc ...` does not produce the four-byte length prefix and is not a valid equivalent. A raw TCP connection establishes transport only; the application still requires a correctly framed command.

A.4 Verify the Contract with a Small Request Set

Submit key `ops1` with input 7. Query its status until completed or until a reasonable diagnostic wait ends. Then repeat the same submission and change the input under the same key. The duplicate should remain accepted, while changed meaning should conflict.

```
./build/operations/harbor_client REPLACE_WITH_GATEWAY_PORT SUBMIT ops1 7
./build/operations/harbor_client REPLACE_WITH_GATEWAY_PORT STATUS ops1
./build/operations/harbor_client REPLACE_WITH_GATEWAY_PORT SUBMIT ops1 7
./build/operations/harbor_client REPLACE_WITH_GATEWAY_PORT SUBMIT ops1 8
./build/operations/harbor_client REPLACE_WITH_WORKER_PORT COUNT
```

Do not assume the first status query must already be done. Submission and completion are asynchronous. A pending result immediately after acceptance can be correct. A permanently pending result requires diagnosis, but its mere existence does not show a defect.

The client prints an application response even when that response is `CONFLICT`, `BUSY`, or `INVALID`. Its process success means the exchange returned a response under the client implementation, not that every business operation succeeded. Automation should

interpret the response category explicitly.

A.5 Diagnose a Connection Failure by Layer

First verify the process and observed port. A restarted process using port zero may choose a different listener. The gateway's worker endpoint is fixed at startup, so restarting the worker on another port does not automatically update discovery.

Next distinguish refusal, timeout, EOF before reply, and an application error response. Refusal often indicates no listener at the selected endpoint in this local lab. Timeout indicates the local exchange budget expired. EOF before reply can be an intentional lost-response failpoint or an unexpected peer close. INVALID means framing and transport may have succeeded while command validation rejected the payload.

Inspect only the relevant process's stderr and test logs. The server deliberately closes malformed-frame connections without returning a detailed parser dump. This prevents a malformed input path from exposing internal state. Use the independent framing tests to investigate decoder behavior rather than expecting every protocol failure to produce a verbose network explanation.

A PONG response confirms one small handler path. It does not prove SQLite admission, worker delivery, or backlog clearance. Follow it with a bounded domain operation when the question concerns those capabilities.

A.6 Diagnose Pending Jobs Without Erasing Evidence

If a job remains pending, verify the worker endpoint and whether the worker can answer its own PING. Inspect gateway dispatch error counters as transient evidence, remembering that they reset after gateway restart. Check whether the worker receipt already exists through the supported application behavior or a controlled read-only database inspection in the private lab.

A receipt may exist even though the gateway is pending, because completion recording can lag a worker effect. Replaying the same key and input should recover the result. Do not change the key to "unstick" the job; that changes the logical operation.

A permanent identity conflict at the worker can cause the simple dispatcher to retry without progress. The current application has no persistent terminal-failure state or authorized redrive command. Preserve the databases and investigate the mismatch rather than deleting the receipt or pending row casually.

The pending selection is a bounded batch ordered by row identifier. It is not a fairness guarantee across tenants, a priority queue, or a broker partition ordering contract. A poison early record can affect later work. A production extension needs explicit quarantine and scheduling policies.

A.7 Exercise a Lost Reply Deliberately

Use the automated `integration_lost_reply` case for the reproducible exercise. It starts the worker with `--drop-first-reply`, places the loss after a valid commit, and checks eventual completion with one receipt. The flag exists for testing and should remain visible in any manual command that uses it.

```
ctest --test-dir build/operations \
-R '^integration_lost_reply$' --output-on-failure
```

The expected history includes a dispatch error followed by successful replay. An error counter increasing during this intentional test is evidence that the injected ambiguity was exercised, not necessarily a failed test. The oracle is the final matching result and receipt count under the test's conditions.

Do not extrapolate from this run to arbitrary network partitions, power loss, or external side effects. The failpoint closes a local response path while retaining the worker process and database. Other tests separately cover process replacement and interrupted transactions.

A.8 Perform a Controlled Restart

Stop the gateway through its normal termination signal, retaining the database. Restart it with the same state file and the intended worker port. Verify a previously completed key, a pending key if present, and an identical duplicate. Repeat the corresponding worker restart exercise with its retained receipt file.

Avoid a force kill as the first ordinary shutdown method. The service has a cooperative drain path, and using it provides clearer ownership behavior. Forceful termination remains a useful controlled fault test, but it tests a different recovery history and should be recorded as such.

A process restart does not require deleting databases. Recreating empty state can make startup easier while losing the very evidence needed to recognize retries. Treat state removal as destructive, not as a standard troubleshooting step.

The integration suite manages exact process identities and cleanup. In manual work, prefer the terminal's normal interrupt or a verified process ID over broad name-based termination commands that could affect unrelated work.

A.9 Plan Backup and Restore as an Isolated Drill

Use SQLite's supported backup facility or a consistent offline procedure. Do not copy only the main file of an active WAL database and assume it contains all committed state. Keep the backup destination separate from the live state and record the method and application version. (R20, R22)

A restore drill should verify more than file readability. Open the restored database with a compatible binary, inspect its schema version, replay a known completed key, and resume a known pending key under a controlled worker authority. Verify that changed meaning still conflicts.

If gateway and worker backups are from different points, identify the possible mismatches before resuming traffic. A newer gateway result paired with an older worker receipt set can make an old key unsafe to execute again. A newer worker with an older gateway may retain effects whose local status has been lost. Reconcile rather than assuming the two files are interchangeable snapshots of one transaction.

No local process test certifies physical storage behavior under every power failure. The operational claim should identify the tested environment and backup method, and production qualification should involve the actual storage stack and recovery requirements.

A.10 Record an Incident as a Causal Account

Use a compact record containing symptom, affected operation, first observed time, deployment identity, state-file identity, relevant endpoint, and a bounded sample of request keys handled under the data policy. Separate facts from hypotheses.

A useful hypothesis might be “the gateway is retrying a worker port that changed after restart.” Its test is to compare the configured endpoint with the worker’s observed listener. Its mitigation is a controlled gateway restart with the intended endpoint while retaining state. That is more precise than restarting every process and deleting files until the symptom disappears.

After mitigation, verify the original contract: accepted work remains represented, duplicates retain meaning, pending work progresses, and new admission behaves according to capacity. Record unresolved limitations, such as missing dead-letter handling or unbounded receipt retention, as engineering work rather than hiding them behind a successful health probe.

The workbook’s central rule is to preserve evidence while narrowing uncertainty. A command that makes a graph look better by erasing authoritative state is not a repair unless the resulting data-loss and replay consequences were explicitly accepted.

Appendix B — Complete Core and Application Source

The chapter listings show small executable uses of each mechanism. This appendix supplies the shared types and the complete client, gateway, and worker implementations used by those examples and the process tests. The assembler reads these files directly from `source/`; the book does not maintain a second hand-copied implementation that can drift from the tested code.

B.1 Read the Ownership Graph Before the Functions

A server owns its listening descriptor. Each accepted peer owns its descriptor, decoder, output buffer, offset, and deadline. A database store owns its SQLite connection. A prepared statement owns its statement handle. A transaction owns rollback responsibility until commit succeeds. The gateway owns its dispatcher thread, and the thread owns its separate database connection.

These relationships determine destruction order. The dispatcher must stop and join before the counters and state it references disappear. A peer's output bytes must remain alive across partial sends. A statement must not outlive its database connection. A handler's borrowed payload view is valid only while the peer decoder remains alive and unchanged under the call's ownership rules.

The code uses small explicit wrappers instead of a general framework so these relationships remain visible. That is an instructional choice. A larger system may use a maintained networking framework, database adapter, and observability SDK, but it still needs the same ownership decisions.

The native transport targets Linux. `accept4`, descriptor flags, and the selected close policy are not presented as a portable implementation for every operating system. Porting requires an adapter with equivalent lifecycle, partial-I/O, and error semantics, followed by platform-specific tests.

B.2 Domain and Test Assertions

The domain header defines the accepted key grammar, input range, canonical parsing, and exact result calculation. The test helper throws on a failed condition so checks remain active in Release builds. It is not a production error-reporting framework and does not replace domain result types.

The input range bounds multiplication before it occurs. This is preferable to performing an unchecked operation and inspecting the result afterward. The `Result` alias uses C++20 library types and keeps parsing failure distinct from a valid request.

Source — source/include/harbor/check.hpp

```
#pragma once
#include <stdexcept>
#include <string>
#include <string_view>
namespace harbor {
inline void check(bool condition, std::string_view message) {
    if (!condition) throw std::runtime_error(std::string(message));
}
template<class F> void rejects(F&& f, std::string_view message) {
    bool rejected = false;
    try { f(); } catch (const std::exception&) { rejected = true; }
    check(rejected, message);
}
}
```

Source — source/include/harbor/domain.hpp

```
#pragma once
#include <charconv>
#include <cstdint>
#include <optional>
#include <stdexcept>
#include <string>
#include <string_view>
#include <variant>
namespace harbor {
inline constexpr std::int64_t max_input = 1'000'000;
struct Error { std::string code; std::string detail; };
template<class T> using Result = std::variant<T, Error>;
struct Request {
    std::string key;
    std::int64_t input{};
    bool operator==(const Request&) const = default;
};
inline bool valid_key(std::string_view key) {
    if (key.empty() || key.size() > 64) return false;
    for (unsigned char c : key)
        if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ||
              (c >= '0' && c <= '9') || c == '_' || c == '-')) return false;
    return true;
}
inline Result<Request> parse_request(std::string_view key, std::string_view text) {
    if (!valid_key(key)) return Error{"INVALID", "key must be 1..64 ASCII identifier bytes"};
    std::int64_t n{};
    const auto [end, ec] = std::from_chars(text.data(), text.data()+text.size(), n);
    if (ec != std::errc{} || end != text.data()+text.size() || n < 0 || n > max_input)
        return Error{"INVALID", "input must be a whole integer in 0..1000000"};
    return Request{std::string(key), n};
}
inline std::int64_t evaluate(const Request& r) {
    if (!valid_key(r.key) || r.input < 0 || r.input > max_input)
        throw std::invalid_argument("invalid request");

    return r.input * r.input; // maximum is 10^12: within int64_t
}
}
```

B.3 Wire Representation and Incremental State

The wire header defines one four-byte big-endian length prefix followed by a bounded payload. The decoder retains incomplete input across calls, rejects oversize and trailing data under the one-frame contract, and exposes a view only after a complete frame exists.

A parser should be reviewed for both arithmetic bounds and lifetime. Here, the maximum payload is small and explicit; the decoder does not reserve arbitrary attacker-declared storage. Its returned view is borrowed from its own buffer, so callers must not retain it beyond the decoder's valid lifetime.

The client and server use the same C++ framing implementation, but tests also use independent golden bytes and Python framing. That reduces the risk of a symmetric encoding error remaining invisible in round-trip tests.

Source — `source/include/harbor/wire.hpp`

```
#pragma once
#include <cstdint>
#include <span>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>
namespace harbor {
inline constexpr std::size_t max_frame = 4096;
inline std::string frame(std::string_view payload) {
    if (payload.size() > max_frame) throw std::length_error("oversize frame");
    auto n = static_cast<std::uint32_t>(payload.size());
    std::string out(4, '\0');
    for (int i=0; i<4; ++i) out[static_cast<std::size_t>(i)] =
        static_cast<char>((n >> (24 - 8*i)) & 255U);
    out.append(payload);
    return out;
}
// One decoder instance per connection. It accepts one message, not a stream
// of unlimited messages. An error poisons the connection: close it.
class Decoder {
    std::string bytes_;
    std::size_t expected_ = 0;
    bool header_ = false;
    bool failed_ = false;
public:
    void feed(std::string_view chunk) {
        if (failed_) throw std::runtime_error("decoder already failed");
        if (chunk.size() > max_frame + 4 - bytes_.size()) {
            failed_ = true; throw std::length_error("frame buffer limit");
        }
        bytes_.append(chunk);
        if (!header_ && bytes_.size() >= 4) {
            for (std::size_t i=0; i<4; ++i)
                expected_ = (expected_ << 8) | static_cast<unsigned char>(bytes_[i]);

            header_ = true;
            if (expected_ > max_frame) {
                failed_ = true; throw std::length_error("declared frame limit");
            }
        }
        if (header_ && bytes_.size() > expected_ + 4) {
            failed_ = true; throw std::runtime_error("trailing frame bytes");
        }
    }
};
```

```

    }
    bool ready() const { return !failed_ && header_ && bytes_.size() == expected_ + 4; }
    std::string_view payload() const {
        if (!ready()) throw std::runtime_error("incomplete frame");
        return std::string_view(bytes_).substr(4);
    }
    void finish() const { if (!ready()) throw std::runtime_error("truncated frame"); }
};
}

```

B.4 Policy and Model Types

The retry, budget, breaker, token-bucket, and distributed-state model types use explicit inputs so their behavior can be tested deterministically. Unless a type contains its own synchronization, it is single-owner. Do not share a mutable breaker or cache across threads merely because its methods look small.

The stable hash is noncryptographic. The fencing register is an enforcement model without a token issuer. The replica and clock examples are not a consensus system. The cache has no bounded metadata reclamation policy. These limits are intentional and are discussed in the corresponding chapters.

A production adapter should preserve those distinctions rather than treating every model in one header as a ready-made distributed infrastructure library. The reusable value is the state transition and its tests; the deployment mechanism still requires an explicit contract.

Source — source/include/harbor/policies.hpp

```

#pragma once
#include <algorithm>
#include <cstdint>
#include <cmath>
#include <limits>
#include <optional>
#include <random>
#include <stdexcept>
namespace harbor {
using Tick = std::uint64_t; // local monotonic milliseconds in the deterministic models
inline Tick saturating_add(Tick a, Tick b) {
    return b > std::numeric_limits<Tick>::max()-a ?
        std::numeric_limits<Tick>::max() : a+b;
}
struct Budget {
    Tick end;
    Tick remaining(Tick now) const { return now < end ? end-now : 0; }
    bool expired(Tick now) const { return now >= end; }
};
struct RetryPolicy {
    unsigned max_attempts = 4; // includes initial attempt
    Tick initial = 10, cap = 1000;
    Tick ceiling(unsigned retry_index) const {
        Tick d = std::min(initial, cap);
        for (unsigned i=0; i<retry_index && d<cap; ++i)
            d = d > cap/2 ? cap : std::min(cap, d*2);
        return d;
    }
    template<class Engine> Tick delay(unsigned retry_index, Engine& rng) const {
        return std::uniform_int_distribution<Tick>(0, ceiling(retry_index))(rng);
    }
};
// Single-threaded policy model. Serialize access in a real client.
class Breaker {
public:
    enum class State { closed, open, half_open };
private:
    State state_ = State::closed;
    unsigned failures_ = 0, limit_;
    Tick reopen_at_ = 0, pause_;
    bool probe_ = false;
    std::uint64_t epoch_ = 0;
public:
    Breaker(unsigned limit, Tick pause): limit_(limit), pause_(pause) {
        if (!limit) throw std::invalid_argument("zero threshold");
    }
    std::optional<std::uint64_t> acquire(Tick now) {
        if (state_ == State::open) {
            if (now < reopen_at_) return {};
            state_ = State::half_open; probe_ = false;
        }
        if (state_ == State::half_open) {
            if (probe_) return {};
            probe_ = true;
        }
        return epoch_;
    }
    void complete(std::uint64_t ticket, bool success, Tick now) {
        if (ticket != epoch_) return; // stale completion from an older generation
        if (state_ == State::half_open) {
            probe_ = false;
            ++epoch_;
            if (success) { state_ = State::closed; failures_ = 0; }
        }
    }
};

```



```

        else { state_ = State::open; reopen_at_ = saturating_add(now, pause_); }
    } else if (state_ == State::closed) {
        if (success) failures_ = 0;
        else if (++failures_ >= limit_) {
            state_ = State::open; reopen_at_ = saturating_add(now, pause_); ++epoch_;
        }
    }
}

State state() const { return state_; }
};

class TokenBucket {
    double tokens_, capacity_, per_ms_;
    Tick last_;
public:
    TokenBucket(double capacity, double per_second, Tick now=0)
        : tokens_(capacity), capacity_(capacity), per_ms_(per_second/1000.0), last_(now) {
        if (!(capacity>0) || !(per_second>=0) || !std::isfinite(capacity) ||
!std::isfinite(per_second)) throw std::invalid_argument("bucket config");
    }
    bool admit(double cost, Tick now) {
        if (!(cost>0) || now<last_) throw std::invalid_argument("cost or clock");
        tokens_ = std::min(capacity_, tokens_+static_cast<double>(now-last_)*per_ms_);
        last_ = now;
        if (cost>tokens_) return false;
        tokens_ -= cost; return true;
    }
};
}

```

Source — source/include/harbor/models.hpp

```

#pragma once
#include "harbor/policies.hpp"
#include <algorithm>
#include <cstdint>
#include <functional>
#include <map>
#include <optional>
#include <set>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>
namespace harbor {
// Stable, non-cryptographic teaching hash; never use for credentials/signatures.
inline std::uint64_t stable_hash(std::string_view value) {
    std::uint64_t h = 14695981039346656037ULL;
    for (unsigned char c : value) { h ^= c; h *= 1099511628211ULL; }
    return h;
}
inline std::string placement(std::string_view key, const std::vector<std::string>& nodes) {
    if (nodes.empty()) throw std::invalid_argument("empty membership");
    std::string chosen; std::uint64_t best=0; bool first=true;
    for (const auto& node : nodes) {
        auto score = stable_hash(std::string(key)+"\x1f"+node);
        if (first || score>best || (score==best && node<chosen)) {
            first=false; best=score; chosen=node;
        }
    }
    return chosen;
}
class LamportClock {
    std::uint64_t value_ = 0;
public:
    std::uint64_t tick() {
        if (value_ == UINT64_MAX) throw std::overflow_error("logical clock");

        return ++value_;
    }
    std::uint64_t receive(std::uint64_t incoming) { value_=std::max(value_, incoming); return
tick(); }
    auto value() const { return value_; }
};
struct CacheEntry { std::string value; std::uint64_t version; Tick expires; };
class VersionedCache {
    std::map<std::string, CacheEntry> entries_;
    std::map<std::string, std::uint64_t> floors_;
public:
    bool put(std::string key, std::string value, std::uint64_t version, Tick expires) {
        auto floor = floors_.find(key);
        if (floor != floors_.end() && version < floor->second) return false;
        auto old = entries_.find(key);
        if (old != entries_.end() && version < old->second.version) return false;
        entries_.insert_or_assign(std::move(key),
CacheEntry{std::move(value),version,expires});
        return true;
    }
    void invalidate(const std::string& key, std::uint64_t version) {
        floors_[key]=std::max(floors_[key], version);
        auto old=entries_.find(key);
        if (old != entries_.end() && old->second.version<version) entries_.erase(old);
    }
    std::optional<std::string> get(const std::string& key, Tick now) const {
        auto it=entries_.find(key);
        if (it == entries_.end() || now >= it->second.expires) return {};
    }
};

```

```

        return it->second.value;
    }
};
class FencedRegister {
    std::uint64_t epoch_ = 0;
    std::string value_;
public:
    bool write(std::uint64_t token, std::string value) {
        if (token<epoch_) return false;
        epoch_=token; value_=std::move(value); return true;
    }
    auto epoch() const { return epoch_; }
    const auto& value() const { return value_; }
};
struct Discovery {
    std::uint64_t generation=0;
    std::vector<std::string> endpoints;
    bool update(std::uint64_t g, std::vector<std::string> next) {
        if (g<=generation) return false;
        std::sort(next.begin(),next.end());
        next.erase(std::unique(next.begin(),next.end()),next.end());
        generation=g; endpoints=std::move(next); return true;
    }
};
class Saga {
public:
    enum class State { created, reserved, charged, completed, compensating, cancelled };
private:
    State state_=State::created;
    std::map<std::string,std::string> seen_;
public:
    bool event(const std::string& id, std::string_view kind) {
        if (auto old=seen_.find(id); old!=seen_.end()) {
            if (old->second!=kind) throw std::logic_error("event identity conflict");
            return false;
        }
        State next=state_;
        if (kind=="reserve" && state_==State::created) next=State::reserved;
        else if (kind=="charge" && state_==State::reserved) next=State::charged;
        else if (kind=="finish" && state_==State::charged) next=State::completed;
        else if (kind=="cancel" && (state_==State::reserved || state_==State::charged))
            next=State::compensating;

        else if (kind=="compensated" && state_==State::compensating) next=State::cancelled;
        else throw std::logic_error("invalid saga transition");
        seen_.emplace(id,kind); state_=next; return true;
    }
    State state() const { return state_; }
};
}

```

B.5 Queue and Metrics Ownership

The queue's mutex protects its container and closed state, and the condition-variable predicate determines when consumers can progress. Closure rejects new work while allowing accepted values to drain. The owner remains responsible for joining users before destroying the queue.

The histogram is a small single-owner exposition model. Its cumulative counts and finite nonnegative observations are testable independently of an exporter. The gateway's actual metrics path uses atomic counters and does not instantiate this

histogram automatically.

Source — source/include/harbor/queue.hpp

```
#pragma once
#include <condition_variable>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <utility>
namespace harbor {
template<class T> class BoundedQueue {
    const std::size_t capacity_;
    std::deque<T> items_;
    bool closed_ = false;
    mutable std::mutex mutex_;
    std::condition_variable ready_;
public:
    explicit BoundedQueue(std::size_t n): capacity_(n) {
        if (!n) throw std::invalid_argument("queue capacity");
    }
    bool try_push(T value) {
        std::lock_guard lock(mutex_);
        if (closed_ || items_.size() >= capacity_) return false;
        items_.push_back(std::move(value)); ready_.notify_one(); return true;
    }
    std::optional<T> pop() {
        std::unique_lock lock(mutex_);
        ready_.wait(lock, [&]{ return closed_ || !items_.empty(); });
        if (items_.empty()) return {};
        T value = std::move(items_.front()); items_.pop_front(); return value;
    }
    void close() { std::lock_guard lock(mutex_); closed_ = true; ready_.notify_all(); }
    std::size_t size() const { std::lock_guard lock(mutex_); return items_.size(); }
};
}
```

Source — source/include/harbor/metrics.hpp

```

#pragma once
#include <array>
#include <stdint>
#include <cmath>
#include <sstream>
#include <stdexcept>
#include <string>
namespace harbor {
// Single-owner demonstration. Real multi-thread aggregation needs synchronization.
class Histogram {
    std::array<double,4> bounds_{0.005,0.02,0.1,1.0};
    std::array<std::uint64_t,5> counts_{};
    double sum_=0;
public:
    void observe(double seconds) {
        if (!(seconds>=0) || !std::isfinite(seconds)) throw std::invalid_argument("latency");
        sum_+=seconds;
        for (std::size_t i=0;i<bounds_.size();++i) if (seconds<=bounds_[i]) ++counts_[i];
        ++counts_.back();
    }
    std::string text() const {
        std::ostringstream out;
        for (std::size_t i=0;i<bounds_.size();++i)
            out<<"harbor_request_seconds_bucket{le=\""<<bounds_[i]<<"\"} "<<counts_[i]<<"\n";
        out<<"harbor_request_seconds_bucket{le=\""<<"+Inf\"} "<<counts_.back()<<"\n";
        out<<"harbor_request_seconds_count "<<counts_.back()<<"\n";
        out<<"harbor_request_seconds_sum "<<sum_<<"\n";
        return out.str();
    }
};
}

```

B.6 Database Interface and Implementation

The store interface makes submission outcomes explicit: accepted, duplicate, conflict, or full. Completion is a separate operation. The worker's `apply` method combines duplicate recognition and effect recording in one transaction. File-backed tests verify reopen and concurrent-connection behavior.

The implementation deliberately has a small schema and no connection pool. It requests WAL and full synchronous behavior, configures a bounded busy timeout, and rejects a newer unsupported schema version. It does not manage replica failover, online schema migration, receipt pruning, or encrypted storage.

Notice the exception boundaries. A failed connection setup releases the handle. A statement releases its compiled representation. A transaction that has not committed attempts rollback during destruction without throwing from that destructor. The outer application translates failures into bounded protocol outcomes or startup errors.

The gateway's two connections have different execution owners. They share a file but not unsynchronized transaction objects. This design keeps local concurrency understandable while relying on SQLite for the database's actual transaction serialization.

Source — source/include/harbor/store.hpp

```

#pragma once
#include "harbor/domain.hpp"
#include <sqlite3.h>
#include <optional>
#include <string>
#include <vector>
namespace harbor {
class Store {
    sqlite3* db=nullptr;
    void exec(const char* sql);
public:
    enum class Submit { accepted, duplicate, conflict, full };
    struct Job { Request request; std::optional<std::int64_t> result; };
    explicit Store(const std::string& path);
    ~Store();
    Store(const Store&)=delete;
    Store& operator=(const Store&)=delete;
    Submit submit(const Request&, std::size_t max_pending=128);
    std::optional<Job> lookup(const std::string& key);
    std::vector<Request> pending(std::size_t limit=16);
    std::int64_t apply(const Request&); // worker inbox and effect in one transaction
    void complete(const Request&, std::int64_t result);
    std::size_t receipt_count();
};
}

```

Source — source/src/store.cpp

```

#include "harbor/store.hpp"
#include <limits>
#include <stdexcept>
namespace harbor {
namespace {
class Statement {
    sqlite3_stmt* stmt=nullptr;
public:
    Statement(sqlite3* db, const char* sql) {
        if (sqlite3_prepare_v2(db, sql, -1, &stmt, nullptr) != SQLITE_OK)
            throw std::runtime_error(sqlite3_errmsg(db));
    }
    ~Statement() { sqlite3_finalize(stmt); }
    Statement(const Statement&)=delete;
    Statement& operator=(const Statement&)=delete;
    void text(int i, const std::string& s) {
        if (s.size() > static_cast<std::size_t>(std::numeric_limits<int>::max())) ||
            sqlite3_bind_text(stmt, i, s.data(), static_cast<int>(s.size()), SQLITE_TRANSIENT) != SQLITE_OK)
            throw std::runtime_error("bind text");
    }
    void number(int i, std::int64_t n) {
        if (sqlite3_bind_int64(stmt, i, n) != SQLITE_OK) throw std::runtime_error("bind integer");
    }
    bool step() {
        int rc=sqlite3_step(stmt);
        if (rc==SQLITE_ROW) return true;
        if (rc==SQLITE_DONE) return false;
        throw std::runtime_error(sqlite3_errmsg(sqlite3_db_handle(stmt)));
    }
    std::int64_t number(int i) const { return sqlite3_column_int64(stmt, i); }
    std::string text(int i) const {
        const auto* p=sqlite3_column_text(stmt, i);
        if (!p) return {};
    }
}
}

```

```

        return {reinterpret_cast<const
char*>(p),static_cast<std::size_t>(sqlite3_column_bytes(stmt_,i))};
    }
    bool is_null(int i) const { return sqlite3_column_type(stmt_,i)==SQLITE_NULL; }
};
class Transaction {
    sqlite3* db_; bool committed_=false;
public:
    explicit Transaction(sqlite3* db): db_(db) {
        if (sqlite3_exec(db_,"BEGIN IMMEDIATE",nullptr,nullptr,nullptr)!=SQLITE_OK)
            throw std::runtime_error(sqlite3_errmsg(db_));
    }
    ~Transaction() { if (!committed_) sqlite3_exec(db_,"ROLLBACK",nullptr,nullptr,nullptr); }
    void commit() {
        if (sqlite3_exec(db_,"COMMIT",nullptr,nullptr,nullptr)!=SQLITE_OK)
            throw std::runtime_error(sqlite3_errmsg(db_));
        committed_=true;
    }
};
};
}
void Store::exec(const char* sql) {
    if (sqlite3_exec(db_,sql,nullptr,nullptr,nullptr)!=SQLITE_OK)
        throw std::runtime_error(sqlite3_errmsg(db_));
}
Store::Store(const std::string& path) {
    if (sqlite3_open_v2(path.c_str(),&db_,SQLITE_OPEN_READWRITE|SQLITE_OPEN_CREATE|SQLITE_OPEN
_FULLMUTEX,nullptr)!=SQLITE_OK) {
        std::string message=db_ ? sqlite3_errmsg(db_) : "sqlite open";
        if (db_) sqlite3_close_v2(db_);
        db_=nullptr;
        throw std::runtime_error(message);
    }
    try {
        if (sqlite3_busy_timeout(db_,100)!=SQLITE_OK) throw std::runtime_error("busy
timeout");

        exec("PRAGMA foreign_keys=ON; PRAGMA journal_mode=WAL; PRAGMA synchronous=FULL;");
        Transaction tx(db_);
        int version=0;
        { Statement s(db_,"PRAGMA user_version"); if (s.step())
version=static_cast<int>(s.number(0)); }
        if (version>1) throw std::runtime_error("database schema is newer than this binary");
        if (version==0) {
            exec("CREATE TABLE jobs("
1000000),"
                "id TEXT PRIMARY KEY, input INTEGER NOT NULL CHECK(input BETWEEN 0 AND
                "result INTEGER CHECK(result IS NULL OR result>=0));"
                "CREATE TABLE receipts("
                "id TEXT PRIMARY KEY, input INTEGER NOT NULL, result INTEGER NOT NULL);"
                "PRAGMA user_version=1;");
        }
        tx.commit();
    } catch (...) { sqlite3_close_v2(db_); db_=nullptr; throw; }
}
Store::~Store() { if (db_) sqlite3_close_v2(db_); }
Store::Submit Store::submit(const Request& r,std::size_t max_pending) {
    (void)evaluate(r);
    Transaction tx(db_);
    auto old=lookup(r.key);
    if (old) {
        tx.commit(); return old->request.input==r.input ? Submit::duplicate :
Submit::conflict;
    }
    { Statement count(db_,"SELECT count(*) FROM jobs WHERE result IS NULL");
        if (!count.step()) throw std::runtime_error("count missing");
    }
}

```

```

        if (static_cast<std::uint64_t>(count.number(0))>=max_pending) { tx.commit(); return
Submit::full; }
    }
    { Statement s(db_,"INSERT INTO jobs(id,input) VALUES(?,?)");
      s.text(1,r.key); s.number(2,r.input); s.step(); }
    tx.commit(); return Submit::accepted;
}

std::optional<Store::Job> Store::lookup(const std::string& key) {
    Statement s(db_,"SELECT input,result FROM jobs WHERE id=?"); s.text(1,key);
    if (!s.step()) return {};
    Job j{Request{key,s.number(0)}, {}};
    if (!s.is_null(1)) j.result=s.number(1);
    return j;
}

std::vector<Request> Store::pending(std::size_t limit) {
    if (limit>128) throw std::invalid_argument("pending batch limit");
    Statement s(db_,"SELECT id,input FROM jobs WHERE result IS NULL ORDER BY rowid LIMIT ?");
    s.number(1,static_cast<std::int64_t>(limit)); std::vector<Request> out;
    while (s.step()) out.push_back(Request{s.text(0),s.number(1)});
    return out;
}

std::int64_t Store::apply(const Request& r) {
    auto answer=evaluate(r);
    Transaction tx(db_);
    { Statement old(db_,"SELECT input,result FROM receipts WHERE id=?"); old.text(1,r.key);
      if (old.step()) {
          if (old.number(0)!=r.input) throw std::runtime_error("CONFLICT");
          answer=old.number(1); tx.commit(); return answer;
      }
    }
    { Statement s(db_,"INSERT INTO receipts(id,input,result) VALUES(?,?,?)");
      s.text(1,r.key); s.number(2,r.input); s.number(3,answer); s.step(); }
    tx.commit(); return answer;
}

void Store::complete(const Request& r,std::int64_t result) {
    if (result!=evaluate(r)) throw std::runtime_error("incorrect worker result");
    Transaction tx(db_); auto job=lookup(r.key);
    if (!job || job->request.input!=r.input) throw std::runtime_error("completion identity
mismatch");
    if (job->result && *job->result!=result) throw std::runtime_error("completion conflict");
    { Statement s(db_,"UPDATE jobs SET result=? WHERE id=? AND input=?");

        s.number(1,result); s.text(2,r.key); s.number(3,r.input); s.step(); }
    tx.commit();
}

std::size_t Store::receipt_count() {
    Statement s(db_,"SELECT count(*) FROM receipts");
    if (!s.step()) throw std::runtime_error("count missing");
    return static_cast<std::size_t>(s.number(0));
}
}

```

B.7 Linux Transport Interface and Reactor

The transport wrapper closes descriptors once according to the documented Linux policy. (R46) The client exchange uses one elapsed deadline across connect, send, and receive. A writable nonblocking connection is checked through `SO_ERROR` before it is treated as established.

The server preserves state across readiness events. It never assumes one `recv` equals one request or one `send` equals one complete response. A handler can deliberately

return no reply for the lost-response failpoint, in which case the peer closes after the handler's effect may already have committed.

The reactor caps peers and accept work per iteration. It still invokes handlers synchronously, and those handlers can wait in SQLite. Its cooperative deadlines therefore support bounded design reasoning under the stated environment rather than a hard timing guarantee. A future asynchronous storage adapter must retain operation state safely after a peer times out.

Error handling is intentionally conservative for malformed peers: close the connection and preserve service operation for other peers. The teaching implementation does not expose a rich per-peer diagnostic API, a TLS session, or a public-network admission policy.

Source — `source/include/harbor/net.hpp`

```
#pragma once
#include <chrono>
#include <cstdint>
#include <functional>
#include <optional>
#include <string>
#include <string_view>
namespace harbor {
using Steady = std::chrono::steady_clock;
class Fd {
    int value_=-1;
public:
    explicit Fd(int value=-1): value_(value) {}
    ~Fd();
    Fd(const Fd&)=delete;
    Fd& operator=(const Fd&)=delete;
    Fd(Fd&& other) noexcept;
    Fd& operator=(Fd&& other) noexcept;
    int get() const { return value_; }
    int release() noexcept;
    void reset(int value=-1) noexcept;
};
std::uint16_t port_number(std::string_view text, bool allow_zero=false);
std::string exchange(std::uint16_t port, std::string_view request,
                    std::chrono::milliseconds timeout);
class Server {
    Fd listener_;
    std::uint16_t port_{};
public:
    using Handler = std::function<std::optional<std::string>(std::string_view)>;
    explicit Server(std::uint16_t port);
    std::uint16_t port() const { return port_; }
    // IPv4 loopback only. One bounded frame and response per connection.
    // Handler must be bounded: poll cannot preempt a blocking callback.
    void run(const Handler& handler, const std::function<bool()>& stopping);
};
}
```

Source — source/src/net.cpp

```

#include "harbor/net.hpp"
#include "harbor/wire.hpp"
#include <arpa/inet.h>
#include <sys/socket.h>
#include <poll.h>
#include <unistd.h>
#include <algorithm>
#include <array>
#include <cerrno>
#include <charconv>
#include <cstring>
#include <stdexcept>
#include <system_error>
#include <utility>
#include <vector>
namespace harbor {
namespace {
[[noreturn]] void os_error(const char* where) {
    throw std::system_error(errno, std::generic_category(), where);
}
void wait_for(int fd, short events, Steady::time_point until) {
    for (;;) {
        auto left=std::chrono::ceil<std::chrono::milliseconds>(until-Steady::now()).count();
        if (left<=0) throw std::runtime_error("RPC deadline");
        pollfd p{fd,events,0};
        int rc=:poll(&p,1,static_cast<int>(std::min<decltype(left)>(left,1000)));
        if (rc<0) { if (errno==EINTR) continue; os_error("poll"); }
        if (rc==0) continue;
        if (p.revents&POLLNVAL) throw std::runtime_error("invalid descriptor");
        // HUP/ERR must be interpreted by recv/send/getsockopt, not spun on.
        if (p.revents&(events|POLLERR|POLLHUP)) return;
    }
}
Fd connect_to(std::uint16_t port, Steady::time_point until) {
    Fd fd(::socket(AF_INET,SOCK_STREAM|SOCK_NONBLOCK|SOCK_CLOEXEC,0));

    if (fd.get()<0) os_error("socket");
    sockaddr_in address{}; address.sin_family=AF_INET;
    address.sin_addr.s_addr=htonl(INADDR_LOOPBACK); address.sin_port=htons(port);
    if (::connect(fd.get(),reinterpret_cast<sockaddr*>(&address),sizeof address)<0) {
        if (errno!=EINPROGRESS) os_error("connect");
        wait_for(fd.get(),POLLOUT,until);
        int error=0; socklen_t length=sizeof error;
        if (::getsockopt(fd.get(),SOL_SOCKET,SO_ERROR,&error,&length)<0)
os_error("getsockopt");
        if (error) throw std::system_error(error,std::generic_category(),"connect
completion");
    }
    return fd;
}
}
Fd::~Fd() { reset(); }
Fd::Fd(Fd&& other) noexcept: value_(other.release()) {}
Fd& Fd::operator=(Fd&& other) noexcept {
    if (this!=&other) reset(other.release());
    return *this;
}
int Fd::release() noexcept { return std::exchange(value_,-1); }
void Fd::reset(int value) noexcept {
    if (value_>=0) ::close(value_); // Linux close policy: do not retry EINTR
    value_=value;
}
std::uint16_t port_number(std::string_view text, bool allow_zero) {
    unsigned value=0;

```

```

    auto [end,ec]=std::from_chars(text.data(),text.data()+text.size(),value);
    if (ec!=std::errc{} || end!=text.data()+text.size() || value>65535 || (!value &&
!allow_zero))
        throw std::invalid_argument("invalid port");
    return static_cast<std::uint16_t>(value);
}
std::string exchange(std::uint16_t port, std::string_view request,
                    std::chrono::milliseconds timeout) {
    if (timeout.count()<=0) throw std::invalid_argument("timeout must be positive");
    auto until=Steady::now()+timeout;
    auto fd=connect_to(port,until);
    auto bytes=frame(request); std::size_t offset=0;
    while (offset<bytes.size()) {
        wait_for(fd.get(),POLLOUT,until);
        auto n=::send(fd.get(),bytes.data()+offset,bytes.size()-offset,MSG_NOSIGNAL);
        if (n>0) offset+=static_cast<std::size_t>(n);
        else if (n<0 && (errno==EINTR || errno==EAGAIN || errno==EWOULDBLOCK)) continue;
        else if (n<0) os_error("send");
        else throw std::runtime_error("zero send");
    }
    Decoder decoder;
    std::array<char,4100> buffer{};
    while (!decoder.ready()) {
        wait_for(fd.get(),POLLIN,until);
        auto n=::recv(fd.get(),buffer.data(),buffer.size(),0);
        if (n>0) decoder.feed({buffer.data(),static_cast<std::size_t>(n)});
        else if (n==0) throw std::runtime_error("EOF before reply");
        else if (errno==EINTR || errno==EAGAIN || errno==EWOULDBLOCK) continue;
        else os_error("recv");
    }
    return std::string(decoder.payload());
}
Server::Server(std::uint16_t port) {
    listener_.reset(::socket(AF_INET,SOCK_STREAM|SOCK_NONBLOCK|SOCK_CLOEXEC,0));
    if (listener_.get()<0) os_error("listener socket");
    int yes=1;
    if (::setsockopt(listener_.get(),SOL_SOCKET,SO_REUSEADDR,&yes,sizeof yes)<0)
os_error("reuseaddr");
    sockaddr_in a{}; a.sin_family=AF_INET; a.sin_port=htons(port);
    a.sin_addr.s_addr=htonl(INADDR_LOOPBACK);
    if (::bind(listener_.get(),reinterpret_cast<sockaddr*>(&a),sizeof a)<0) os_error("bind");
    if (::listen(listener_.get(),64)<0) os_error("listen");

    socklen_t size=sizeof a;
    if (::getsockname(listener_.get(),reinterpret_cast<sockaddr*>(&a),&size)<0)
os_error("getsockname");
    port_=ntohs(a.sin_port);
}
void Server::run(const Handler& handler, const std::function<bool()>& stopping) {
    struct Peer {
        Fd fd; Decoder decoder; std::string output; std::size_t offset=0;
        bool writing=false;
        Steady::time_point deadline=Steady::now()+std::chrono::seconds(2);
    };
    std::vector<Peer> peers;
    bool draining=false;
    for (;;) {
        draining=draining || stopping();
        if (draining && peers.empty()) return;
        std::vector<pollfd> ready{{draining ? -1 : listener_.get(),POLLIN,0}};
        for (const auto& p: peers)
ready.push_back({p.fd.get(),static_cast<short>(p.writing?POLLOUT:POLLIN),0});
        int rc=:poll(ready.data(),ready.size(),25);
        if (rc<0) { if (errno==EINTR) continue; os_error("server poll"); }

```

```

for (std::size_t i=0; i<peers.size(); ++i) {
    auto& p=peers[i]; auto re=ready[i+1].revents;
    if (Steady::now()>=p.deadline || (re&POLLNVAL)) { p.fd.reset(); continue; }
    try {
        if (!p.writing && (re&(POLLIN|POLLHUP|POLLERR))) {
            std::array<char,4100> buf{};
            auto n=::recv(p.fd.get(),buf.data(),buf.size(),0);
            if (n>0) {
                p.decoder.feed({buf.data(),static_cast<std::size_t>(n)});
                if (p.decoder.ready()) {
                    auto reply=handler(p.decoder.payload());
                    if (!reply) { p.fd.reset(); continue; }
                    p.output=frame(*reply); p.writing=true;
                }
            } else if (n==0) p.fd.reset();
            else if (errno!=EINTR && errno!=EAGAIN && errno!=EWOULDBLOCK)
p.fd.reset();
        } else if (p.writing && (re&(POLLOUT|POLLERR|POLLHUP))) {
            auto
n=::send(p.fd.get(),p.output.data()+p.offset,p.output.size()-p.offset,MSG_NOSIGNAL);
            if (n>0) {
                p.offset+=static_cast<std::size_t>(n);
                if (p.offset==p.output.size()) p.fd.reset();
            } else if (n==0 || (errno!=EINTR && errno!=EAGAIN && errno!=EWOULDBLOCK))
p.fd.reset();
        }
    } catch (const std::exception&) { p.fd.reset(); }
}
std::erase_if(peers,[](const Peer& p){ return p.fd.get()<0; });
if (!draining && (ready[0].revents&POLLIN)) {
    // At most 16 accepts per iteration avoids starving existing connections.
    for (int i=0; i<16; ++i) {
        Fd fd(::accept4(listener_.get(),nullptr,nullptr,SOCK_NONBLOCK|SOCK_CLOEXEC));
        if (fd.get()<0) {
            if (errno==EAGAIN || errno==EWOULDBLOCK || errno==EINTR) break;
            os_error("accept");
        }
        if (peers.size()<64) peers.push_back(Peer{std::move(fd),Decoder{}, {},0,false,
            Steady::now()+std::chrono::seconds(2)});
        // Otherwise RAI closes the unadmitted socket, before parsing/allocation.
    }
}
}
}
}

```

B.8 Worker Process

The worker's transaction is the durable effect boundary. `--drop-first-reply` is a test-only behavior placed after commit. It demonstrates that a client can lose an acknowledgement even when the effect exists. Ordinary use should omit the flag.

The worker does not authenticate callers. Its local-only listener is a teaching restriction, not an identity system. A remote deployment must add a reviewed transport and authorization adapter before broadening exposure.

Source — source/src/worker.cpp

```

#include "harbor/net.hpp"
#include "harbor/store.hpp"
#include <atomic>
#include <csignal>
#include <iostream>
#include <sstream>
namespace {
static_assert(std::atomic<bool>::is_always_lock_free,
              "signal notification requires a lock-free atomic bool");
constexpr std::atomic<bool> stop_requested{false};
extern "C" void request_stop(int) noexcept {
    stop_requested.store(true, std::memory_order_relaxed);
}
}
int main(int argc, char** argv) {
    try {
        if (argc<3 || argc>4) throw std::invalid_argument("usage: harbor_worker DATABASE PORT
[--drop-first-reply]");
        bool drop=argc==4;
        if (drop && std::string(argv[3])!="--drop-first-reply") throw
std::invalid_argument("unknown option");
        harbor::Store store(argv[1]);
        harbor::Server server(harbor::port_number(argv[2], true));
        if (std::signal(SIGINT, request_stop)==SIG_ERR ||
            std::signal(SIGTERM, request_stop)==SIG_ERR)
            throw std::runtime_error("install signal handler");
        std::cout<<"READY " <<server.port()<<std::endl;
        server.run([&](std::string_view message)->std::optional<std::string> {
            if (message=="PING") return "PONG";
            if (message=="COUNT") return std::to_string(store.receipt_count());
            std::istringstream in{std::string(message)};
            std::string verb, key, input, extra;
            if (!(in>>verb>>key>>input) || (in>>extra) || verb!="EXEC") return "INVALID";
            auto parsed=harbor::parse_request(key, input);
            auto* request=std::get_if<harbor::Request>(&parsed);

            if (!request) return "INVALID";
            try {
                auto result=store.apply(*request);
                if (drop) { drop=false; return {}; } // deliberate lost acknowledgment AFTER
commit
                return "RESULT "+key+" "+std::to_string(result);
            } catch (const std::exception&) { return "ERROR"; }
        }, [&]{ return stop_requested.load(std::memory_order_relaxed); });
        return 0;
    } catch (const std::exception& e) { std::cerr<<e.what()<<'\n'; return 1; }
}

```

B.9 Gateway Process

The gateway creates persistent intent before replying accepted. Its dispatcher repeatedly selects pending work, uses stable request identity, validates worker responses, and records completion. Random seeding is performed during main-thread startup so a seeding failure is handled before the background thread begins.

The dispatcher retains work across process restart through the database, not through a sleeping thread or an in-memory retry list. Its retry loop is deliberately simple: one endpoint, bounded batches, backoff after failure, and cooperative stop checks. It has no persistent terminal-failure state, multi-tenant fairness, or dead-letter

administration.

Counters describe process observations and can reset. The authoritative job state remains in SQLite. This separation is important when interpreting metrics after restart or reconciling attempts with distinct effects.

Source — source/src/gateway.cpp

```
#include "harbor/net.hpp"
#include "harbor/store.hpp"
#include "harbor/policies.hpp"
#include <atomic>
#include <csignal>
#include <iostream>
#include <memory>
#include <random>
#include <sstream>
#include <thread>
namespace {
static_assert(std::atomic<bool>::is_always_lock_free,
              "signal notification requires a lock-free atomic bool");
constexpr std::atomic<bool> stop_requested{false};
extern "C" void request_stop(int) noexcept {
    stop_requested.store(true, std::memory_order_relaxed);
}
struct Counters {
    std::atomic<std::uint64_t> accepted{0}, duplicate{0}, conflict{0}, dispatch_errors{0};
};
void pause_for(std::stop_token stop, std::chrono::milliseconds time) {
    auto until=harbor::Steady::now()+time;
    while (!stop.stop_requested() && harbor::Steady::now()<until)
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
}
int main(int argc, char** argv) {
    try {
        if (argc!=4) throw std::invalid_argument("usage: harbor_gateway DATABASE PORT
WORKER_PORT");
        auto worker_port=harbor::port_number(argv[3]);
        harbor::Store api_store(argv[1]);
        auto dispatch_store=std::make_unique<harbor::Store>(argv[1]);
        harbor::Server server(harbor::port_number(argv[2], true));
        Counters counters;

        const auto random_seed=std::random_device{}(); // failure stays in main startup catch
        if (std::signal(SIGINT, request_stop)==SIG_ERR ||
            std::signal(SIGTERM, request_stop)==SIG_ERR)
            throw std::runtime_error("install signal handler");
        std::jthread dispatcher([&, db=std::move(dispatch_store), random_seed](std::stop_token
stop) {
            harbor::RetryPolicy retry; unsigned failures=0;
            std::mt19937 rng(random_seed);
            while (!stop.stop_requested()) {
                bool failed=false;
                try {
                    for (const auto& request : db->pending()) {
                        if (stop.stop_requested()) break;
                        auto reply=harbor::exchange(worker_port, "EXEC "+request.key+" "+
                            std::to_string(request.input), std::chrono::milliseconds(200));
                        std::istringstream in(reply); std::string verb, key, extra; std::int64_t
value{};
                        if (!(in>>verb>>key>>value) || (in>>extra) || verb!="RESULT" ||
key!=request.key)
                            throw std::runtime_error("worker reply contract");
```

```

        db->complete(request,value); failures=0;
    }
} catch (const std::exception&) {
    ++counters.dispatch_errors; failed=true;
    failures=std::min(failures+1,10U);
}
// One local delivery loop owns retry. Pending rows survive restarts.
auto wait=failed ? std::max<harbor::Tick>(10,retry.delay(failures-1,rng)) :
25;
    pause_for(stop,std::chrono::milliseconds(wait));
}
});
std::cout<<"READY " <<server.port()<<std::endl;
server.run([&](std::string_view message)->std::optional<std::string> {
    if (message=="PING") return "PONG";

    if (message=="METRICS") {
        return "harbor_accepted_total " +std::to_string(counters.accepted.load())+"\\n"+
            "harbor_duplicate_total " +std::to_string(counters.duplicate.load())+"\\n"+
            "harbor_conflict_total " +std::to_string(counters.conflict.load())+"\\n"+
            "harbor_dispatch_errors_total
"+std::to_string(counters.dispatch_errors.load())+"\\n";
    }
    std::istringstream in{std::string(message)}; std::string verb,key,input,extra;
    if (!(in>>verb>>key) || !harbor::valid_key(key)) return "INVALID";
    try {
        if (verb=="STATUS") {
            if (in>>extra) return "INVALID";
            auto job=api_store.lookup(key);
            if (!job) return "MISSING";
            if (!job->result) return "PENDING "+key;
            return "DONE "+key+" "+std::to_string(*job->result);
        }
        if (verb!="SUBMIT" || !(in>>input) || (in>>extra)) return "INVALID";
        auto parsed=harbor::parse_request(key,input);
        auto* request=std::get_if<harbor::Request>(&parsed);
        if (!request) return "INVALID";
        switch (api_store.submit(*request)) {
            case harbor::Store::Submit::accepted: ++counters.accepted; break;
            case harbor::Store::Submit::duplicate: ++counters.duplicate; break;
            case harbor::Store::Submit::conflict: ++counters.conflict; return
"CONFLICT";
            case harbor::Store::Submit::full: return "BUSY";
        }
        return "ACCEPTED "+key;
    } catch (const std::exception&) { return "ERROR"; }
},[] { return stop_requested.load(std::memory_order_relaxed); });
dispatcher.request_stop(); // jthread joins before counters/stores are destroyed
return 0;
} catch (const std::exception& e) { std::cerr<<e.what()<<'\\n'; return 1; }
}

```

B.10 Client and Build Graph

The client is a protocol tool, not a full business SDK. It prints the response and reports transport failure separately. A script must interpret application outcomes explicitly. It does not retry automatically or generate job identities, which keeps ambiguous-outcome policy visible to the caller.

The CMake graph declares the actual dependencies and test targets. The default build requires installed SQLite development support, Python for integration tests, and Linux for native transport. It does not fetch remote third-party source during

configuration.

Source — source/src/client.cpp

```
#include "harbor/net.hpp"
#include <iostream>
int main(int argc, char** argv) {
    try {
        if (argc<3) throw std::invalid_argument("usage: harbor_client PORT COMMAND
[ARGUMENTS...]");
        std::string command;
        for (int i=2; i<argc; ++i) { if (i>2) command+=' '; command+=argv[i]; }
        std::cout<<harbor::exchange(harbor::port_number(argv[1]), command,
            std::chrono::milliseconds(1500))<<'\n';
        return 0;
    } catch (const std::exception& e) { std::cerr<<e.what()<<'\n'; return 2; }
}
```

Source — source/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.24)
project(HarborDispatch VERSION 1.0.0 LANGUAGES CXX)
include(CTest)
find_package(Threads REQUIRED)
find_package(SQLite3 REQUIRED)
find_package(Python3 COMPONENTS Interpreter REQUIRED)
option(HARBOR_SANITIZERS "Enable AddressSanitizer and UndefinedBehaviorSanitizer" OFF)
add_library(harbor_options INTERFACE)
target_compile_features(harbor_options INTERFACE cxx_std_20)
set(CMAKE_CXX_EXTENSIONS OFF)
if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
    target_compile_options(harbor_options INTERFACE -Wall -Wextra -Wpedantic)
    if(HARBOR_SANITIZERS)
        target_compile_options(harbor_options INTERFACE -fsanitize=address,undefined
-fno-omit-frame-pointer)
        target_link_options(harbor_options INTERFACE -fsanitize=address,undefined)
    endif()
endif()
add_library(harbor_core INTERFACE)
target_include_directories(harbor_core INTERFACE
$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>)
target_link_libraries(harbor_core INTERFACE harbor_options Threads::Threads)
add_library(harbor_store src/store.cpp)
target_link_libraries(harbor_store PUBLIC harbor_core SQLite::SQLite3)
if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
    message(FATAL_ERROR "The executable transport lab targets Linux; pure headers can be ported
separately.")
endif()
add_library(harbor_net src/net.cpp)
target_link_libraries(harbor_net PUBLIC harbor_core)
foreach(app worker gateway client)
    add_executable(harbor_${app} src/${app}.cpp)
    target_link_libraries(harbor_${app} PRIVATE harbor_net harbor_store)
    install(TARGETS harbor_${app} RUNTIME DESTINATION bin)
endforeach()

file(GLOB CHAPTER_SOURCES CONFIGURE_DEPENDS examples/ch*.cpp)
foreach(file IN LISTS CHAPTER_SOURCES)
    get_filename_component(name ${file} NAME_WE)
    add_executable(${name} ${file})
    target_link_libraries(${name} PRIVATE harbor_net harbor_store)
    add_test(NAME ${name} COMMAND ${name})
    set_tests_properties(${name} PROPERTIES TIMEOUT 15)
endforeach()
if(BUILD_TESTING)
```



```

add_executable(core_regression tests/core_tests.cpp)
target_link_libraries(core_regression PRIVATE harbor_core)
foreach(case codec policy routing cache clock breaker saga queue domain)
  add_test(NAME core_${case} COMMAND core_regression ${case})
endforeach()
add_executable(store_regression tests/store_tests.cpp)
target_link_libraries(store_regression PRIVATE harbor_store)
add_test(NAME store_transactions COMMAND store_regression)
foreach(case happy lost_reply restart worker_restart partial invalid conflict overload
crash_rollback)
  add_test(NAME integration_${case} COMMAND ${Python3_EXECUTABLE}
    ${CMAKE_CURRENT_SOURCE_DIR}/tests/integration.py
    --bin $<TARGET_FILE_DIR:harbor_gateway> --case ${case})
  set_tests_properties(integration_${case} PROPERTIES TIMEOUT 30)
endforeach()
endif()
set(CPACK_PACKAGE_NAME "harbor-dispatch-lab")
set(CPACK_PACKAGE_VERSION ${PROJECT_VERSION})
set(CPACK_GENERATOR "TGZ")
include(CPack)

```

B.11 How to Modify the Source Without Losing Evidence

Change the operative files under `source/`, not the generated manuscript listing. Run the relevant narrow test first, then the full configuration matrix appropriate to the change. Reassemble the manuscript so the included source and its hash manifest match the tested files.

When changing a contract, update its negative tests and explanatory text together. Raising the frame size affects allocation and overload assumptions. Adding worker parallelism affects ordering and receipt authority. Pruning records affects replay retention. A change can compile cleanly while invalidating the book's recovery argument.

Keep the original behavior available as a reference when experimenting. The objective is not to preserve every teaching implementation forever, but to make the consequence of replacing it explicit and verifiable.

Appendix C — Verification Atlas and Complete Test Source

A passing test is evidence for a particular input, history, oracle, and environment. This atlas explains the supplied tests so readers can decide what a change requires them to rerun and what remains outside the test boundary. The final run counts and tool versions are recorded separately in `TEST_REPORT.md` and the machine-readable reports.

C.1 Chapter Executables as Focused Checks

The 28 chapter targets each demonstrate a specific mechanism and contain checks that fail visibly. They are not interchangeable repetitions of one vector transform. Their outputs cover domain validation, repository substitution, Pimpl lifetime, configuration snapshots, coroutine stepping, actual loopback exchange, framing, budgets, discovery, clocks, replication lag, fencing, placement, persistence, migration, caching, message deduplication, outbox/inbox behavior, breaker transitions, queue closure, saga state, authorization, histograms, finite history enumeration, build identity, lifecycle, objective arithmetic, and a local capstone model.

Run one target while studying its chapter, then inspect the implementation and change a boundary input. The book includes the exact source through its assembler. A chapter's output is useful only when interpreted according to the surrounding limitation statement: the replication example is a model, while the network example opens a real local socket.

The chapter targets are individually registered CTest cases. Internal checks inside one executable are not counted as separate independently registered cases in the report. This prevents a long loop from being advertised as thousands of unrelated tests.

C.2 Core Regression Families

The codec family tests golden bytes, zero and maximum payloads, two-fragment splits, oversize declarations, incomplete input, and trailing bytes. It checks that rejection does not silently reinterpret malformed framing. The finite split enumeration is exhaustive for the chosen split model, not for every possible byte stream.

The policy family checks deadline equality, saturated arithmetic, capped jitter ranges, token refill, backward ticks, and invalid configuration. These tests use explicit time inputs, making them deterministic and fast. They do not validate a live scheduler's timing behavior.

Routing tests compare membership order, added-node movement, empty membership rejection, and stale discovery revisions. Cache tests verify invalidation floors, stale fills, and expiration. Clock tests verify logical merge behavior and

overflow, while fencing checks reject older generations.

Breaker tests include old-epoch completions and a single half-open probe. Saga tests distinguish duplicates, identity conflicts, and illegal transitions. Domain tests reject malformed numeric strings, invalid keys, negative values, and out-of-range inputs.

The queue test launches multiple producers and consumers, closes the queue, and checks exact item accounting and aggregate values after drain. It exercises a real concurrent execution under the tested runtime. It is not a formal proof of every schedule or a fairness guarantee.

Source — source/tests/core_tests.cpp

```
#include "harbor/check.hpp"
#include "harbor/domain.hpp"
#include "harbor/wire.hpp"
#include "harbor/policies.hpp"
#include "harbor/models.hpp"
#include "harbor/queue.hpp"
#include <atomic>
#include <iostream>
#include <numeric>
#include <thread>
using harbor::check;
using harbor::rejects;
void codec() {
    for (std::size_t size: {0U,1U,2U,3U,17U,63U,1024U,4096U}) {
        std::string value(size,'x'); if (size) value[size/2]='\0';
        auto bytes=harbor::frame(value);
        for (std::size_t split=0;split<=bytes.size();++split) {
            harbor::Decoder decoder;
            decoder.feed(std::string_view(bytes).substr(0,split));
            decoder.feed(std::string_view(bytes).substr(split));
            decoder.finish(); check(decoder.payload()==value,"roundtrip split");
        }
        rejects([]{harbor::frame(std::string(4097,'x'));},"oversize encode");
        rejects([]{harbor::Decoder d; d.feed(std::string("\0\0\x10\x01",4));},"oversize declared length");
        rejects([]{harbor::Decoder d;
d.feed(std::string("\0\0\0\3ab",6));d.finish();},"truncation");
        rejects([]{harbor::Decoder d; d.feed(harbor::frame("a")+ "b");},"trailing bytes");
    }
    void policy() {
        harbor::RetryPolicy p;
        check(p.ceiling(0)==10 && p.ceiling(7)==1000 && p.ceiling(1000000)==1000,"bounded exponential");
        std::mt19937 rng(19);

        for (unsigned i=0;i<1000;++i) check(p.delay(i%12,rng)<=p.ceiling(i%12),"jitter bound");
        harbor::Budget b{100}; check(b.remaining(20)==80 && b.remaining(100)==0,"budget");
        check(harbor::saturating_add(UINT64_MAX-1,9)==UINT64_MAX,"saturation");
        harbor::TokenBucket bucket(2,100);
        check(bucket.admit(2,0) && !bucket.admit(1,0) && bucket.admit(1,10),"token refill");
        rejects([&]{bucket.admit(1,9)},"backward clock");
        rejects([]{harbor::TokenBucket b(0,1)},"invalid capacity");
    }
    void routing() {
        const std::vector<std::string> before{"one","two","three"},rev{"three","two","one"};
        auto after=before; after.push_back("four");
        for (int i=0;i<1000;++i) {
            auto
            key=std::to_string(i),old=harbor::placement(key,before),next=harbor::placement(key,after);
```

```

    check(old==harbor::placement(key,rev),"membership order");
    check(old==next || next=="four","minimal reassignment");
}
rejects([]{harbor::placement("key",{ });},"no members");
harbor::Discovery d; check(d.update(5,{"a","b","a"}),"revision");
check(!d.update(4,{ }) && d.endpoints.size()==2,"old snapshot");
}
void cache() {
    harbor::VersionedCache c;
    check(c.put("a","old",1,100),"first fill");
    c.invalidate("a",2);
    check(!c.put("a","old",1,200) && !c.get("a",1),"delayed stale fill");
    check(c.put("a","new",2,200),"new fill");
    c.invalidate("a",1); check(c.get("a",199)=="new","old invalidation");
    check(!c.get("a",200),"expiry equality");
}
void clock_test() {
    harbor::LamportClock a,b;
    for (int i=0;i<100;++i) { auto t=a.tick(); check(b.receive(t)>t,"receive advances");
a.receive(b.tick()); }
    rejects([]{harbor::LamportClock c; c.receive(UINT64_MAX);},"clock wrap");

    harbor::FencedRegister r; r.write(2,"new"); check(!r.write(1,"old") &&
r.value()=="new","fence");
}
void breaker() {
    harbor::Breaker b(2,10);
    auto old=*b.acquire(0);
    b.complete(old,false,0); b.complete(old,false,1);
    check(!b.acquire(10),"cooldown");
    auto probe=*b.acquire(11); check(!b.acquire(11),"one probe");
    b.complete(old,true,11); check(b.state()==harbor::Breaker::State::half_open,"stale success
ignored");
    b.complete(probe,false,12); check(!b.acquire(21),"failed probe");
    auto probe2=*b.acquire(22); b.complete(probe2,true,23);
    check(b.state()==harbor::Breaker::State::closed,"successful recovery");
    b.complete(probe,false,24); check(b.state()==harbor::Breaker::State::closed,"late failure
ignored");
}
void saga() {
    harbor::Saga s;
    s.event("r","reserve"); s.event("c","charge"); s.event("x","cancel");
s.event("u","compensated");
    check(!s.event("u","compensated"),"duplicate ack");
    rejects([&]{s.event("u","finish");},"same identity different payload");
    rejects([&]{s.event("f","finish");},"forward after compensation");
    check(s.state()==harbor::Saga::State::cancelled,"terminal preserved");
}
void queue() {
    harbor::BoundedQueue<int> q(8);
    std::atomic<int> count{0},sum{0};
    std::vector<std::jthread> consumers;
    for(int i=0;i<3;++i) consumers.emplace_back([&]{while(auto n=q.pop()){++count;sum+=*n;}});
    std::vector<std::jthread> producers;
    for (int p=0;p<4;++p) producers.emplace_back([&,p]{
        for(int i=0;i<100;++i) { int n=p*100+i; while(!q.try_push(n))
std::this_thread::yield(); }

    });
    producers.clear(); q.close(); consumers.clear();
    check(count==400 && sum==(399*400/2),"all admitted values accounted");
    check(!q.try_push(4) && !q.pop(),"closed exhausted");
}
void domain() {
    for (const auto& s: {"-1","1000001","+1","1x","","99999999999999999999"})

```

```

        check(std::holds_alternative<harbor::Error>(harbor::parse_request("k",s)), "input
grammar");
        check(!harbor::valid_key("") && !harbor::valid_key("space key") &&
!harbor::valid_key(std::string(65,'a')), "key bounds");
        check(harbor::evaluate({"max",1000000})==1000000000000LL, "numeric max");
        rejects([]{harbor::evaluate({"max",1000001});}, "domain guard");
    }
    int main(int argc, char** argv) {
        try {
            if(argc!=2) throw std::runtime_error("case required");
            const std::string name=argv[1];
            if(name=="codec") codec(); else if(name=="policy") policy(); else if(name=="routing")
routing();
            else if(name=="cache") cache(); else if(name=="clock") clock_test(); else
if(name=="breaker") breaker();
            else if(name=="saga") saga(); else if(name=="queue") queue(); else if(name=="domain")
domain();
            else throw std::runtime_error("unknown test");
            std::cout<<name<<": passed\n";
        } catch(const std::exception& e){std::cerr<<e.what()<<'\n';return 1;}
    }

```

C.3 File-Backed Store Regression

The storage suite creates a private temporary directory and uses actual file-backed SQLite databases. It checks duplicate recognition when pending capacity is full, changed-meaning conflicts, incorrect completion rejection, persistent reopen, and two connections racing on one worker identity.

The test's exact receipt count is an independent consequence of the domain contract. Returning the right numerical answer alone would not reveal duplicate records. Likewise, preserving the pending row after a wrong completion attempt checks the failure postcondition rather than only whether an exception occurred.

The suite does not simulate a storage device lying about flush completion, torn sectors, a regional outage, or a backup restored out of coordination with external effects. Those failure models require separate qualification. The file-backed tests support local transaction and process recovery reasoning, not universal durability claims.

A temporary directory must be owned and cleaned up reliably. Test infrastructure is part of the system under review: leftover state can make a supposedly fresh test pass because a receipt from an earlier run already exists. Unique private directories avoid that accidental dependency.

Source — source/tests/store_tests.cpp

```

#include "harbor/check.hpp"
#include "harbor/store.hpp"
#include <filesystem>
#include <iostream>
#include <thread>
#include <atomic>
#include <cstdlib>
int main() {
    try {
        char pattern[]="/tmp/harbor-store-XXXXXX";
        auto* name=:mkdtemp(pattern); harbor::check(name!=nullptr,"temp directory");
        std::filesystem::path dir(name);
        struct Cleanup { std::filesystem::path path; ~Cleanup(){std::error_code
e;std::filesystem::remove_all(path,e);} } cleanup{dir};
        auto path=(dir/"test.db").string();
        {
            harbor::Store s(path);
            harbor::Request first{"a",7};
            harbor::check(s.submit(first,1)==harbor::Store::Submit::accepted,"first");
            harbor::check(s.submit({"b",8},1)==harbor::Store::Submit::full,"admission limit");
            harbor::check(s.submit(first,1)==harbor::Store::Submit::duplicate,"duplicate when
full");
            harbor::check(s.submit({"a",8})==harbor::Store::Submit::conflict,"identity
conflict");
            auto result=s.apply(first);
            harbor::check(result==49 && s.apply(first)==49 && s.receipt_count()==1,"durable
inbox");
            harbor::rejects([&]{s.apply({"a",8});},"worker conflict");
            harbor::rejects([&]{s.complete(first,50);},"wrong result");
            harbor::check(!s.lookup("a")->result,"failed completion unchanged");
            s.complete(first,49); s.complete(first,49);
            harbor::check(s.pending().empty(),"outbox drained");
        }
        {
            harbor::Store reopened(path);

            harbor::check(reopened.lookup("a")->result==49 &&
reopened.receipt_count()==1,"restart durability");
        }
        {
            harbor::Store a(path),b(path); std::atomic<int> ok{0};
            auto operation=[&](harbor::Store& store){
                try { if(store.apply({"parallel",21})==441) ++ok; } catch(...) {}
            };
            std::jthread one(operation,std::ref(a)),two(operation,std::ref(b));
            one.join(); two.join(); harbor::check(ok==2 && a.receipt_count()==2,"concurrent
deduplication");
        }
        std::cout<<"SQLite transaction, conflict, quota, restart, and concurrent dedup tests
passed.\n";
    } catch(const std::exception& e){std::cerr<<e.what()<<'\n';return 1;}
}

```

C.4 Process Integration Histories

The process suite starts the actual binaries and communicates through an independently implemented Python framing client. It obtains ephemeral ports from a startup handshake and bounds its waits. Every invocation owns a private directory and terminates its child processes during cleanup.

Happy path. Inputs zero, seven, and one million test both ordinary and boundary arithmetic. The test verifies completed status and three worker receipts. It also invokes the shipped C++ client for a basic exchange.

Lost reply. The worker commits and deliberately omits its first valid response. The gateway must retry and complete with one receipt. The dispatch-error observation confirms that the intended failure path was encountered rather than skipped.

Gateway restart. The worker is stopped, a job is accepted pending, and the gateway is replaced with the same state file. After the worker returns, the job completes. Another gateway restart verifies retained completion state.

Worker restart. A direct execution commits, the worker is replaced with its retained database, and the same request returns the same result with one receipt. This isolates worker deduplication across process lifetime.

Partial framing. One connection stalls with an incomplete header while another sends byte-fragmented valid requests. The valid peer must progress. This checks peer-state separation, not merely one parser's ability to concatenate strings.

Invalid requests. Empty commands, malformed integers, trailing tokens, unknown commands, and oversized frame declarations are rejected. A later valid ping still succeeds, and no worker effect is created by the invalid set.

Identity conflict. A completed key is repeated with both identical and changed input. Identical meaning is recognized, changed meaning conflicts, and the worker retains one effect.

Overload. With the worker unavailable, 128 unique jobs are accepted, the next new key is rejected, and an existing duplicate remains accepted. The test inspects persisted row count rather than trusting response text alone.

Interrupted transaction. A separate child process begins a write transaction and is killed before commit. Reopening the private database must show no uncommitted row. This is a process-interruption test, not a physical power-loss test.

Source — source/tests/integration.py

```
#!/usr/bin/env python3
"""Loopback-only process tests. Each invocation owns a private temporary directory."""
from __future__ import annotations
import argparse
import pathlib
import selectors
import socket
import sqlite3
import struct
import subprocess
import sys
import tempfile
import time

if not __debug__:
    raise RuntimeError("Run the integration harness without -O or PYTHONOPTIMIZE")

class Process:
    def __init__(self, executable: pathlib.Path, *args: str):
        self.p = subprocess.Popen([str(executable), *map(str,args)], stdout=subprocess.PIPE,
                                   stderr=subprocess.PIPE, text=True)

        try:
            with selectors.DefaultSelector() as selector:
                selector.register(self.p.stdout, selectors.EVENT_READ)
                if not selector.select(8):
                    raise RuntimeError('server startup timeout')
                line = self.p.stdout.readline().strip()
                if not line.startswith('READY '):
                    raise RuntimeError(f'server startup failed: {line}')
                self.port = int(line.split()[1])
        except BaseException:
            self.stop()
            raise

    def stop(self):
        if self.p.poll() is None:
            self.p.terminate()
            try: self.p.wait(timeout=5)
            except subprocess.TimeoutExpired: self.p.kill(); self.p.wait(timeout=2)
        self.out, self.err = self.p.communicate(timeout=2)
        if 'AddressSanitizer' in self.err or 'runtime error:' in self.err:
            raise AssertionError(self.err)

def exact(s: socket.socket, n: int) -> bytes:
    data = bytearray()
    while len(data)<n:
        chunk=s.recv(n-len(data))
        if not chunk: raise EOFError('incomplete reply')
        data.extend(chunk)
    return bytes(data)

def call(port: int, message: str, fragment=False) -> str:
    payload=message.encode('ascii')
    wire=struct.pack('!I',len(payload))+payload
    with socket.create_connection(('127.0.0.1',port),timeout=3) as s:
        s.settimeout(3)
        if fragment:
            for c in wire: s.sendall(bytes([c]))
        else: s.sendall(wire)
        length=struct.unpack('!I',exact(s,4))[0]
        if length>4096: raise AssertionError('unbounded response')
        return exact(s,length).decode('ascii')

def eventually(port: int, key: str, expected: int):
```



```

end=time.monotonic()+8
while time.monotonic()<end:
    status=call(port,'STATUS '+key)
    if status==f'DONE {key} {expected}': return
    if status not in {f'PENDING {key}', 'ERROR'}: raise AssertionError(status)
    time.sleep(0.03)
raise AssertionError('job did not finish')

def run(binary: pathlib.Path, case: str):
    processes=[]
    def start(exe, *args):
        p=Process(binary/exe,*args); processes.append(p); return p
    with tempfile.TemporaryDirectory(prefix='harbor-integration-') as work:
        folder=pathlib.Path(work)
        try:
            flags=['--drop-first-reply'] if case=='lost_reply' else []
            worker=start('harbor_worker', folder/'worker.db',0,*flags)
            gateway=start('harbor_gateway', folder/'gateway.db',0,worker.port)
            if case=='happy':
                for n in (0,7,1000000):
                    assert call(gateway.port,f'SUBMIT k{n} {n}')==f'ACCEPTED k{n}'
                    eventually(gateway.port,f'k{n}',n*n)
                    assert call(worker.port,'COUNT')==3
                    client=subprocess.run([str(binary/'harbor_client'),str(gateway.port),'PING'],
                                           capture_output=True,text=True,timeout=5,check=True)
                    assert client.stdout.strip()=='PONG'
            elif case=='lost_reply':
                assert call(gateway.port,'SUBMIT lost 11')==f'ACCEPTED lost'
                eventually(gateway.port,'lost',121)
                assert call(worker.port,'COUNT')==1
                metrics=call(gateway.port,'METRICS')
                errors=int([x for x in metrics.splitlines() if
x.startswith('harbor_dispatch_errors_total')][0].split()[1])
                assert errors>=1
            elif case=='restart':
                port=worker.port; worker.stop()
                assert call(gateway.port,'SUBMIT pending 13')==f'ACCEPTED pending'
                assert call(gateway.port,'STATUS pending')==f'PENDING pending'
                gateway.stop()
                worker=start('harbor_worker', folder/'worker.db',port)
                gateway=start('harbor_gateway', folder/'gateway.db',0,worker.port)
                eventually(gateway.port,'pending',169)

                gateway.stop()
                gateway=start('harbor_gateway', folder/'gateway.db',0,worker.port)
                assert call(gateway.port,'STATUS pending')==f'DONE pending 169'
            elif case=='worker_restart':
                assert call(worker.port,'EXEC direct 17')==f'RESULT direct 289'
                port=worker.port; worker.stop()
                worker=start('harbor_worker', folder/'worker.db',port)
                assert call(worker.port,'EXEC direct 17')==f'RESULT direct 289'
                assert call(worker.port,'COUNT')==1
            elif case=='partial':
                with socket.create_connection(('127.0.0.1',gateway.port),timeout=3) as
stalled:
                    stalled.sendall(b'\0\0')
                    assert call(gateway.port,'PING',fragment=True)==f'PONG'
                    assert call(gateway.port,'SUBMIT fragments 19',fragment=True)==f'ACCEPTED
fragments'
                    eventually(gateway.port,'fragments',361)
            elif case=='invalid':
                for message in ('', 'SUBMIT a -1', 'SUBMIT a 2x', 'STATUS a extra', 'SUBMIT a
1000001', 'UNKNOWN a'):
                    assert call(gateway.port,message)==f'INVALID'
                    with socket.create_connection(('127.0.0.1',gateway.port),timeout=3) as s:

```

```

        s.sendall(struct.pack('!I',4097))
        try: assert s.recv(1)==b''
        except ConnectionResetError: pass
        assert call(gateway.port,'PING')=='PONG'
        assert call(worker.port,'COUNT')==0'
    elif case=='conflict':
        assert call(gateway.port,'SUBMIT repeat 23')=='ACCEPTED repeat'
        eventually(gateway.port,'repeat',529)
        assert call(gateway.port,'SUBMIT repeat 23')=='ACCEPTED repeat'
        assert call(gateway.port,'SUBMIT repeat 24')=='CONFLICT'
        assert call(worker.port,'EXEC repeat 24')=='ERROR'
        assert call(worker.port,'COUNT')==1'
    elif case=='overload':

        worker.stop()
        for n in range(128): assert call(gateway.port,f'SUBMIT queued{n}
{n}')=='ACCEPTED queued{n}'
        assert call(gateway.port,'SUBMIT overflow 4')=='BUSY'
        assert call(gateway.port,'SUBMIT queued0 0')=='ACCEPTED queued0'
        with sqlite3.connect(folder/'gateway.db') as db:
            assert db.execute('SELECT count(*) FROM jobs').fetchone()[0]==128
    elif case=='crash_rollback':
        # Interrupt a separate uncommitted transaction in a disposable database.
        dbfile=folder/'rollback.db'
        code=''import sqlite3,sys,time
c=sqlite3.connect(sys.argv[1]);c.execute("CREATE TABLE t(x INTEGER)");c.commit()
c.execute("BEGIN IMMEDIATE");c.execute("INSERT INTO t VALUES(9)")
print("READY 0",flush=True);time.sleep(30)
'''

p=subprocess.Popen([sys.executable,'-c',code,str(dbfile)],stdout=subprocess.PIPE,text=True)
    try:
        with selectors.DefaultSelector() as selector:
            selector.register(p.stdout,selectors.EVENT_READ)
            assert selector.select(5), 'rollback child startup'
            assert p.stdout.readline().strip()=='READY 0'
            p.kill();p.wait(timeout=3)
            with sqlite3.connect(dbfile) as db: assert db.execute('SELECT count(*)
FROM t').fetchone()[0]==0
        finally:
            if p.poll() is None:p.kill();p.wait()
        else: raise ValueError(case)
        print(case+' passed')
    finally:
        failures=[]
        for p in reversed(processes):
            try:p.stop()
            except Exception as e:failures.append(str(e))
        if failures:raise AssertionError('\n'.join(failures))

if __name__=='__main__':
    parser=argparse.ArgumentParser();parser.add_argument('--bin',type=pathlib.Path,required=True)
    parser.add_argument('--case',required=True)
    args=parser.parse_args();run(args.bin.resolve(),args.case)

```

C.5 Reading Failures Without Broadening the Claim

A failing test may reveal an implementation defect, an overstrong oracle, a harness bug, or an unavailable environment. Investigate before weakening the assertion. For example, client receipt of a response and completion of a later metrics increment are distinct events; an exact counter assertion may need to wait for the event it actually claims.

Preserve stderr and the selected case name. Native sanitizer diagnostics should remain visible even when process cleanup is expected. Conversely, an intentional force kill in a rollback test is not itself a failed application recovery assertion.

Do not label a flaky test harmless because it passes on retry. Determine whether its timing assumption is unsupported. Replace fixed sleeps with observable predicates and bounded waits where possible. If the actual behavior is nondeterministic by contract, assert the permitted set of outcomes and the invariant common to them.

The process tests run over local loopback. They do not produce the delay, loss, routing changes, or trust boundaries of a remote network. Add a controlled network-emulation environment for those questions and record its configuration rather than presenting loopback success as WAN qualification.

C.6 Extending Coverage with an Explicit Hypothesis

When adding a new mechanism, write the failure history first. A persistent dead-letter state suggests tests for permanent failure, authorized redrive, and a late successful receipt. A new transport suggests compatibility, cancellation, and malformed-message tests. Sharding suggests stale placement epochs and receipt migration.

Choose the smallest test layer that can expose the issue. Pure placement arithmetic belongs in a deterministic model. Database atomicity requires a real store. TLS hostname verification requires an actual configured TLS integration. Starting a full cluster to test a pure function wastes time, while testing a mock to certify a security handshake omits the relevant boundary.

Keep independent oracles where practical. Golden wire fixtures, a separate reference computation, and direct state inspection can detect errors that two components sharing one helper would both repeat. Document any shared assumptions so a reviewer knows where correlated mistakes remain possible.

Record generated test seeds and failing histories. A reproducible small failure is more useful than a claim that a million random operations ran without a retained trace. A generated case count is not a probability of correctness unless a justified statistical model supports that interpretation.

C.7 Configuration Matrix and Release Evidence

The supplied verification driver builds named configurations and saves configure, build, and test logs. The final report identifies which runs succeeded, the compiler versions, and the source hash manifest. Do not infer unrun platforms from the existence of portable-looking code.

An AddressSanitizer/UndefinedBehaviorSanitizer run checks executed native paths under instrumentation. It is not ThreadSanitizer coverage, a formal race proof, or a performance result. Release tests retain explicit checks and exercise optimized binaries separately.

If a reader changes source, the original report no longer certifies the changed artifact. Rerun the relevant matrix and regenerate the source-listing manifest. The purpose of including scripts and logs is to make evidence renewable, not to turn one historical green result into a permanent guarantee.

A useful release decision ends with both supported evidence and limitations. The application is a tested local teaching system with a deliberately bounded domain. Remote security, live brokers, orchestration, replicated authority, and production capacity remain separate engineering projects.

Appendix D — Framework Integration Blueprints

The default companion is intentionally small enough to build with C++20, SQLite, and the Linux interfaces used by its transport. This appendix describes how to introduce common production frameworks without confusing a design blueprint with an executed integration. No gRPC server, broker cluster, TLS deployment, OpenTelemetry collector, or Kubernetes workload is started by the supplied default build.

D.1 Replace the Transport Without Replacing the Domain

A gRPC adapter should translate generated request types into the same canonical domain request used by the current service. It should call the same application/store boundary and translate outcomes deliberately. The key grammar, integer range, duplicate meaning, and accepted-versus-completed distinction should not change merely because the wire format changes.

The optional schema uses separate submission and status methods. Submission returns a domain outcome. Status uses a discriminated state so pending, completed, and missing are not represented by ambiguous default fields. Transport failure remains a separate gRPC status under the adapter's policy.

The schema's plain proto3 scalar input does not distinguish an omitted integer from explicit zero. Both decode as zero under that representation. Because zero is valid in the teaching domain, an adapter must decide whether omission is acceptable or whether explicit presence is required. Add optional or another presence representation deliberately when the business contract distinguishes the cases.

Generated code and runtime compatibility must be selected as a supported set. Record the schema revision, protoc version, gRPC code-generation plugin, runtime libraries, compiler, and CMake configuration. Do not assume an arbitrary generator downloaded during configuration matches an already installed runtime. (R14, R43)

Source — source/integrations/dispatch.proto

```
// Optional integration design. Not used or compiled by the default companion.
// Generate and test with a compatible protoc, gRPC plugin, and runtime before use.
syntax = "proto3";
package harbor.dispatch.v1;

service Dispatch {
  rpc Submit(SubmitRequest) returns (SubmitResponse);
  rpc Status(StatusRequest) returns (StatusResponse);
}

message SubmitRequest {
  string key = 1; // 1..64 ASCII letters, digits, underscore, hyphen.
  int64 input = 2; // 0..1000000. Validate presence/meaning in the adapter policy.
}

message SubmitResponse {
  enum Outcome {
    OUTCOME_UNSPECIFIED = 0;
    ACCEPTED = 1; // Durable intent, not a promise that execution has finished.
    CONFLICT = 2;
    BUSY = 3;
    INVALID = 4;
  }
  Outcome outcome = 1;
  string key = 2;
}

message StatusRequest { string key = 1; }
message Pending { string key = 1; }
message Completed { string key = 1; int64 result = 2; }
message Missing {}
message StatusResponse {
  oneof state {
    Pending pending = 1;
    Completed completed = 2;
    Missing missing = 3;
  }
}

// Transport failures remain gRPC statuses; map domain outcomes intentionally.
// Cancellation does not roll back a committed Submit. Reuse the same key.
// Authentication, tenant scoping, TLS, and server implementation are not supplied
// by this schema. Do not expose the unauthenticated loopback lab publicly.
```

D.2 Preserve Deadline and Cancellation Semantics

Use the framework’s supported deadline mechanism and document the caller’s budget. A request that expires before acceptance can fail without creating a durable obligation. A cancellation after the gateway transaction commits does not roll that transaction back. The client must retain its key to resolve an uncertain response.

The adapter should not create a fresh logical identity inside a retry callback. The generated request object can be copied or reconstructed, but the operation’s key and canonical meaning remain stable across attempts. gRPC’s retry and cancellation documentation should be read together with the application contract rather than treated as automatic business semantics. (R10, R11, R12)

For a callback or asynchronous C++ API, define which object owns call state until the framework’s completion contract is satisfied. A client disconnect can detach its interest in the result while server-side state still needs cleanup or durable completion. Destroying a callback target immediately on cancellation can produce a

native lifetime defect.

Test cancellation before parsing, before commit, after commit, and during result delivery. The expected domain state differs among these points. A single cancelled-RPC test that checks only a status code cannot verify the full recovery contract.

The framework's C++ guidance and graceful-shutdown facilities can inform the adapter's lifecycle, but the final implementation must be tested in its selected version and threading model. A callback API and a synchronous API do not necessarily have identical ownership rules. (R42, R44)

D.3 Add TLS and Identity as a Separate Reviewed Boundary

A production adapter needs trusted server identity, appropriate client authentication, and application authorization. Configure peer verification and trust roots through the chosen framework rather than disabling checks to make a demonstration connect.

The domain principal should be constructed from verified identity claims. Do not trust a tenant field merely because it appears in a protobuf message. Bind the requested resource scope and action to the authenticated caller or an explicitly authorized delegation.

Negative integration tests should cover an untrusted issuer, a wrong expected peer name, expired credentials under the selected verification policy, missing client identity where required, and a valid identity with insufficient permission. These are separate from C++ memory tests and from ordinary successful RPC calls.

Credential rotation needs an operational path. Determine whether channels reload trust or credentials, whether connections must be replaced, and how overlapping validity windows are handled. Retain enough observability to diagnose verification failure without logging private keys or bearer tokens.

The loopback teaching binaries remain unchanged by this blueprint. Opening their listener to a remote interface without implementing this boundary is not a supported shortcut. A reverse proxy also requires a clear trust rule for the hop between proxy and application.

D.4 Introduce a Broker Through the Existing Outbox

A broker adapter can publish pending intent from the gateway outbox and let workers consume it. The transaction committing acceptance still needs to preserve the outgoing obligation. A successful network publish does not replace the local record unless the protocol defines how uncertainty is resolved.

Publisher confirmation can tell the publisher something about broker acceptance under the broker's configured guarantees. Consumer acknowledgement tells the broker about delivery handling. Neither automatically proves a worker's unrelated database effect. Preserve a worker inbox transaction or another supported atomic effect mechanism. (R25)

Choose the partition or routing key according to the state authority. If jobs are independent by key, a partitioned stream can provide a placement basis, but consumer rebalance and in-flight work still need correct ownership handling. Retain domain event identity independently of a transport delivery tag or offset.

A broker integration test matrix should include publish confirmation loss, worker crash before effect, crash after effect before acknowledgement, consumer reassignment, poison-message handling, replay, and retained-schema compatibility. Use a real controlled broker for these tests; a fake publisher returning success cannot certify broker durability.

Kafka transactions can coordinate specific operations within Kafka's supported processing scope. An external SQL update still requires a corresponding integration design. State exactly which resources participate before using the phrase exactly once. (R26)

D.5 Add Observability Without Changing the Service's Meaning

An OpenTelemetry adapter can create spans around admission, database transactions, delivery attempts, and completion recording. Keep the logical job key and attempt identity distinct from trace identity, and sanitize attributes according to data policy.

A metrics adapter should preserve the dictionary established in Chapter 23. Accepted submissions, duplicate attempts, worker receipts, and completed gateway jobs are different quantities. Export failure should not block the main service indefinitely or retain an unbounded buffer of telemetry.

Define sampling and retention deliberately. A missing sampled trace cannot prove that a job never ran. Exact reconciliation should use durable records, while metrics and traces provide operational context. Avoid placing arbitrary job keys in unbounded metric labels.

Test the exporter against a controlled collector or scrape endpoint, including unavailability and slow consumption. The local histogram example verifies arithmetic and text representation only. It does not verify network export, authentication to the collector, or dashboard queries. (R30, R31)

D.6 Prepare an Orchestrated Deployment Only After State Ownership Is Solved

A stateless frontend can often be replicated more easily than an authority holding a local SQLite file. The gateway and worker in this lab are stateful authorities. Cloning their containers with independent files does not create safe shared state.

Before writing a deployment manifest, choose the persistence and ownership architecture. Options might include one controlled writer with a supported failover procedure, a managed transactional database, or a partitioned service with explicit routing and migration. Each changes the failure and operational model and must be

evaluated on its own terms.

The manifest then needs service identity, state mounts, resource requests and limits, startup/readiness/liveness probes, termination grace, network exposure, and configuration/secret delivery. Probe semantics should match the application's actual capabilities rather than merely testing a listening port. (R34, R35)

A rolling upgrade must preserve schema and operation compatibility. If two versions cannot safely share the same authority, use a controlled maintenance transition rather than hiding the conflict behind a deployment strategy. Container orchestration schedules processes; it does not infer database migration correctness or idempotency retention.

D.7 Gate Each Integration with Evidence

A useful integration gate contains the selected versions, generated artifacts, configuration, test environment, positive tests, negative tests, failure histories, and known limitations. The gate should identify the exact artifact tested, not merely the source branch name.

For a transport replacement, preserve domain and storage regression tests and add real transport compatibility cases. For a broker, add delivery and acknowledgement failure histories. For identity, add denied and invalid-credential paths. For orchestration, test drain, replacement, and retained-state recovery under the selected platform.

Do not expand the book's verification claims because a blueprint appears plausible. The default package's report remains about the default code and environments actually executed. A reader's new integration earns its own evidence through reproducible tests.

The practical objective is controlled substitution. Frameworks can provide substantial capabilities, but the service must still own its promises: accepted intent, stable meaning, bounded resources, identifiable effects, and recoverable uncertainty.

Appendix E — Architecture Review Exercises and Worked Reasoning

These exercises ask for decisions and evidence rather than memorized terminology. Each scenario changes one assumption of the teaching service. The worked reasoning identifies the invariant at risk, a defensible design direction, and the test needed before the change can be accepted. The answers are not claims that these extensions are already implemented.

E.1 A Client Wants to Retry with a New Key

Scenario. The client submitted `annual7` with input 25 and timed out. A user-interface developer proposes retrying with `annual8` so the second attempt will not conflict with anything left behind.

Reasoning. The timeout describes the client’s observation, not the worker’s effect. A new key represents a new logical job. If the original request committed, both keys can produce independent receipts. The correct recovery retains `annual7` and its canonical input, then queries status or repeats the same submission.

The interface should let the user distinguish “try to learn the outcome of the same operation” from “create a new operation.” A changed business request can receive a new identity, but an uncertain result should not silently become changed intent.

Verification. Inject a lost response after worker commit. Retry using the original key and verify one receipt. Then intentionally submit a new key and show that it is a second logical effect. The contrast teaches the API semantics rather than merely asserting that retries are safe.

E.2 A Team Wants to Delete Receipts After One Hour

Scenario. Receipt storage is growing. An operator proposes deleting every row older than one hour while leaving the API’s replay promise unchanged.

Reasoning. Receipt retention determines the temporal boundary of duplicate recognition. After deletion, a delayed request can be treated as new. Storage reclamation is therefore a contract change, not only a maintenance task.

A defensible policy could define a key validity period and reject old requests outside it, archive searchable receipts, or maintain another durable summary with a justified collision and retention model. The policy must account for queued retries, disconnected clients, backups, and replay tools.

Verification. Create a completed operation, advance a controlled retention clock, prune under the proposed rule, and replay the old key. The expected result must match the documented policy. A test that merely verifies fewer rows after cleanup says nothing about replay safety.

E.3 A Gateway Should Use Several Workers

Scenario. The gateway is changed to select a random port from three workers, each with its own SQLite file. All ordinary requests succeed, so the team claims horizontal scaling is complete.

Reasoning. A retry can reach a worker that lacks the original receipt. Local uniqueness in each file does not create a global authority. The design needs shared transactional state, partitioned ownership with migration rules, or a replicated service with the required consistency.

A fixed key-to-worker mapping helps only while membership and state placement remain coordinated. Adding or removing a node can move a key away from its receipt. The migration must carry the idempotency state and handle stale routing views.

Verification. Commit at worker A, lose the reply, route the retry to B, and inspect effects. This deliberately failing history exposes the missing authority. A corrected design must preserve the invariant under that history and during membership changes, not merely under a stable hash function.

E.4 A Database Callback Must Leave the Reactor Thread

Scenario. Database contention causes long handler pauses. A developer moves each SQLite call to a detached thread and captures the peer object by reference.

Reasoning. The change removes one synchronous wait while creating lifetime and resource problems. The peer can expire or the server can stop before the detached thread completes. Unlimited threads can also replace bounded peer state with unbounded execution resources.

Use a bounded executor with explicit operation state. The task owns the request data it needs. Completion returns to the designated state owner and checks whether the peer generation is still eligible. Shutdown stops admission, cancels or drains work under policy, joins execution, and only then releases shared state.

Verification. Delay a database completion until after peer expiry and after server drain begins. Verify no stale reference is accessed, no response is written to a reused connection identity, and accepted durable work remains recoverable even when the original peer is gone.

E.5 A Cache Should Serve Every Status Query

Scenario. A cache stores MISSING for a key, then the gateway accepts that key. A client immediately checks status and receives cached missing.

Reasoning. Negative caching conflicts with a read-your-writes expectation unless the cache is invalidated or bypassed with a revision dependency. A TTL alone may permit the stale absence for its entire duration.

Choose the status contract first. A strict post-submit read can use the authority directly or require a revision that the cache entry must satisfy. An advisory dashboard might permit bounded staleness, but it should not be used to decide whether to create a second job.

Verification. Cache absence, commit acceptance, and issue a dependent read before expiration. The result must meet the chosen contract. Include a delayed old fill after invalidation to ensure the repair does not merely delete the current entry while allowing stale repopulation.

E.6 A Broker Consumer Wants More Parallelism

Scenario. A consumer processes offsets 100 and 101 concurrently. It records checkpoint 102 when 101 finishes first.

Reasoning. If the checkpoint represents the next unprocessed offset, it has skipped unfinished 100. The consumer needs a contiguous-completion frontier or a different supported transaction protocol. Processing order and checkpoint order cannot be inferred from delivery order once concurrency is introduced.

If effects are external to the broker transaction, retain an application inbox or another effect-side deduplication mechanism. A replay after crash must not apply the completed operation again merely because its checkpoint was not retained.

Verification. Complete 101, crash before 100, restart, and verify that 100 is processed while 101's retained effect is not duplicated. A test that processes all offsets in order never exercises the gap.

E.7 A New Schema Changes Units

Scenario. A field named `timeout` previously meant milliseconds and now means seconds. The field type and tag remain unchanged, so the team calls the change wire-compatible.

Reasoning. Parsing compatibility does not preserve meaning. Old clients can now request a thousand times the intended duration, and retained messages can be reinterpreted. Introduce an explicit unit or a new versioned field, preserve old interpretation, and stage the rollout through compatible readers and writers.

Configuration and stored records need the same treatment. A migration must convert values deliberately, with bounds and overflow checks. Default and absent values should be specified separately where the format supports that distinction.

Verification. Run old and new fixtures through each supported reader, including zero, maximum, absent, and malformed values. Verify the interpreted duration, not merely successful deserialization. Include a rollback check for data written after the new version begins operating.

E.8 A Dashboard Reports Perfect Success During Overload

Scenario. The metric denominator includes only jobs that reached the worker. The gateway rejects most new work as busy, but worker success remains 100 percent.

Reasoning. The metric may accurately describe worker processing while failing to describe user submission success. Define separate indicators for valid submission acceptance and accepted-job completion. Include overload outcomes in the appropriate denominator under a documented policy.

Measure backlog age as well as count. A small number of very old pending jobs can represent a severe failure hidden by high throughput for newer work. Per-operation and tenant distributions may matter when aggregate results hide concentrated harm.

Verification. Stop the worker, fill the pending quota, and send additional valid requests. Confirm the admission metric records rejection while completion metrics retain their distinct meaning. After recovery, verify backlog clearance rather than declaring success from PING alone.

E.9 A Backup Restores Only the Gateway

Scenario. The gateway is restored from yesterday while the worker retains today's receipts. Some completed jobs no longer appear in gateway status.

Reasoning. The system has two authorities with different retained points. The worker can know effects the gateway no longer knows it accepted. Blindly creating new keys can duplicate logical work; blindly deleting worker receipts destroys evidence.

A reconciliation procedure should identify matching keys and canonical inputs, restore or classify missing gateway knowledge under authorized policy, and preserve conflict evidence. If real external effects exist, their authority must also participate in the recovery decision.

Verification. Build an isolated restore fixture with intentionally different snapshots. Verify that reconciliation detects the mismatch and does not silently overwrite changed meaning. A successful database open is only the beginning of this recovery test.

E.10 A Circuit Breaker Should Be Shared Across Threads

Scenario. Several threads call the single-owner breaker model directly because its methods are short and its fields are integers.

Reasoning. Method size and scalar fields do not remove data races or make a sequence of state changes atomic. Admission, probe ownership, epoch transition, and completion classification must be serialized consistently.

Use one executor/strand owner or add synchronization around the complete state transition. Do not protect only the failure counter while leaving state and epoch unsynchronized. The resulting policy should preserve the one-half-open-probe rule and stale-ticket rejection.

Verification. Combine a deterministic state-machine oracle with actual concurrent tests under an appropriate race-detection configuration when available. A passing AddressSanitizer run does not replace ThreadSanitizer or a concurrency argument. Record the tools actually used.

E.11 A Production Review Requests a Performance Number

Scenario. A reviewer asks how many requests per second HarborDispatch supports. The team proposes quoting the fastest local test run.

Reasoning. Functional tests are not a controlled throughput benchmark. They use small data, test-specific waits, temporary databases, and potentially instrumented builds. A performance claim needs a defined workload, hardware, build, durability settings, concurrency, warm-up, observation interval, and latency/error criteria.

Measure completed useful work, not only accepted requests. Report distributions or meaningful summaries, including rejection and backlog behavior. Run long enough to expose sustained storage and checkpoint costs relevant to the experiment, and distinguish local loopback from remote deployment.

Verification. Create a separate benchmark protocol and preserve raw results. Compare implementations under the same conditions. Do not treat a sanitizer build's timing as directly comparable to Release or present one favorable sample as a general capacity guarantee.

E.12 Write a Filled Architecture Decision Record

Decision. Use one worker receipt authority and a durable gateway pending table for the teaching service.

Context. The application must preserve a stable key's meaning across lost replies and process restarts. The domain is intentionally small, the deployment is local-only, and the instructional priority is an inspectable recovery argument.

Alternatives considered. Pure in-memory retries would lose intent on restart. Independent random workers would not share receipts. A production replicated broker/database stack would add capabilities but obscure the small example's ownership and deployment requirements.

Consequences. The service has a clear local transaction boundary and real process tests, but no high availability, global deduplication across independent stores, public authentication, or bounded historical retention. Pending capacity is bounded; total durable history is not.

Evidence. Domain, storage, framing, policy, and process tests exercise the stated invariants. The final report identifies actual compiler configurations and limitations. The source listings are assembled from the same files used by those tests.

Revisit triggers. A requirement for multiple state owners, public access, long-term retention limits, strict latency isolation, or terminal-failure workflows requires a new design and verification plan. It is not a reason to silently reinterpret the existing guarantees.

This record is short because it names the decision and its consequences directly. A useful architecture document does not need to predict every future framework. It needs to let the next engineer understand why the current boundary exists and what evidence must change before replacing it.

Appendix F — Architecture and Distributed-Systems Glossary

Definitions in this glossary follow the usage of this volume. Product-specific guarantees must still be checked against the selected implementation and version. Similar names do not make two mechanisms interchangeable.

Acceptance. The service's defined commitment to handle work. In HarborDispatch, acceptance means a matching gateway row has committed; it does not mean the worker has completed.

Acknowledgement. A message reporting a particular stage of handling. Its meaning depends on the protocol: receipt, broker persistence, consumer handling, and business-effect commitment are different acknowledgement boundaries.

Admission control. A decision about whether new work becomes the system's responsibility, based on policy and available resources. Rejection should remain distinct from failure after durable acceptance.

Applied revision. The position through which committed changes have been incorporated into a replica's serving state. Receiving or persisting a log entry does not necessarily mean queries already reflect it.

Artifact identity. A stable reference to a specific built output, commonly associated with a checksum and provenance. A branch name alone can change and is not a complete artifact identity.

Asynchronous completion. A result delivered after the initiating call has returned or suspended. Correct handling requires ownership of operation state and a rule for cancellation, stale generations, and cleanup.

Atomicity. The property that a defined group of state changes is committed together or not committed under the resource's transaction model. It does not automatically include unrelated external effects.

Authority. The component or protocol entitled to decide a particular state transition. Architecture should identify authority for uniqueness, receipts, membership, credentials, and migration rather than assuming it from network location.

Backoff. A delay policy between attempts, often increasing after repeated failure. Backoff reduces some retry pressure but does not by itself bound fleet-wide concurrency or make unsafe retries idempotent.

Backpressure. A mechanism that limits upstream production according to downstream capacity. It needs an explicit propagation and overload policy rather than merely moving work into another unbounded queue.

Binary compatibility. Compatibility among compiled components at their ABI boundary. Source compatibility, Pimpl, and matching function names do not automatically guarantee compatible object layout, runtimes, or allocation ownership.

Bulkhead. A resource partition that limits how one work class can exhaust capacity needed by another. Its protection depends on shared resources below the partition as well as visible thread pools.

Cache floor. In the teaching model, the minimum acceptable version retained after invalidation. It prevents a late older fill from repopulating the cache after the old value was removed.

Canonical request. A representation of the fields that determine an operation's meaning after validation and normalization. Duplicate recognition compares canonical meaning rather than incidental textual formatting.

Checkpoint. A recorded recovery position or state. Its semantics must specify which work is complete; advancing past unfinished work can lose processing even when later work has finished.

Circuit breaker. A stateful admission policy for a dependency, often using closed, open, and half-open states. It limits repeated failing calls but does not repair the dependency or undo remote effects.

Commit point. The point at which a resource reports that a transaction or state transition is committed under its guarantees. Client observation of that commit is a separate event.

Compensation. A new business action intended to address a previous partial effect. It may not restore the exact prior world and requires its own identity, failure handling, and verification.

Consensus. Agreement on decisions under a defined distributed failure model. Broadcasting messages or assigning a leader number does not implement the complete protocol and its safety and progress conditions.

Consistency model. The contract governing which observations and orderings clients may see. It should be expressed through operation histories, not only vague terms such as strong or current.

Consumer offset. A position in a particular log partition or stream. It is a transport coordinate and should not automatically replace a domain event identifier that must survive export or replay.

Correlation identifier. Metadata connecting observations related to work. A job key, attempt ID, and trace ID can be correlated while retaining different scopes and lifetimes.

Deadline. The latest eligible completion or waiting boundary under a defined clock and policy. A caller's expired wait does not prove that a remote mutation did not commit.

Dead-letter handling. A controlled path for work that cannot complete under ordinary retry policy. It needs ownership, retention, diagnosis, and authorized redrive rather than becoming an ignored backlog.

Deduplication receipt. Retained information binding an operation identity to its accepted meaning and effect. Its retention and atomic relationship to the effect

determine the scope of duplicate safety.

Degradation. A deliberate reduction in supported capability, such as rejecting new mutations while serving status. It should remain explicit and observable rather than being disguised as ordinary success.

Delivery attempt. One transport or execution try for a logical operation. Several attempts can correspond to one retained business effect when the idempotency contract is preserved.

Deterministic model. A controlled representation whose inputs, event order, and time can be reproduced. It supports reasoning about selected histories but does not automatically model all real-world failures.

Discovery revision. An ordering token for membership information within a defined authority domain. Restarting or replacing that authority requires an explicit generation or continuity policy.

Drain. A lifecycle phase that stops new admission while settling already admitted work under a bounded policy. Resource destruction should follow completion or cancellation ownership, not precede it.

Durable intent. Persisted state representing an accepted obligation that can be resumed after process loss. In this book, pending gateway rows serve this role and drive later delivery.

Effect boundary. The resource or protocol within which a business change is considered committed. A local database transaction cannot automatically include an email provider or another independent service.

Epoch. A generation identifying an authority or state-machine interval. It lets a component reject stale completions or stale ownership, provided generation issuance and comparison are defined correctly.

Error budget. The allowed amount of bad service outcomes under a specified objective, denominator, and window. It is a policy tool, not a substitute for correctness invariants.

Event identity. A stable identifier for a domain occurrence. It should remain associated with the same meaning across redelivery and may need to survive changes in transport position.

Eventual consistency. A convergence-oriented contract under stated conditions. The label alone does not define intermediate reads, conflict resolution, uniqueness, or the treatment of externally visible promises.

Executor. An abstraction controlling where and when work runs. A coroutine does not automatically provide an executor, and serialized execution does not necessarily imply one fixed operating-system thread.

Fencing token. An ordered ownership generation checked by the protected resource to reject stale writers. It requires a trustworthy issuer and preserved state; a client-chosen integer is not authority.

Framing. The protocol mechanism that identifies message boundaries in a byte stream. TCP's stream behavior does not preserve the boundaries of individual application writes.

Generation check. A comparison ensuring that an asynchronous result still belongs to the state that may accept it. The result can remain informationally useful even when its state-update authority is stale.

Graceful shutdown. A controlled transition that stops admission, settles or cancels work under policy, joins execution, and releases resources. It still needs a bounded fallback for uncooperative work.

Idempotency. A contract under which repeated attempts at the same defined operation do not create additional unintended effects. Identity scope, payload association, retention, and atomicity determine its actual guarantee.

Inbox. Consumer-side retained state used to recognize already handled messages or operations. For crash-safe effect deduplication, its receipt must be atomic with the relevant local effect.

Invariant. A property that must remain true across every supported state transition, including failure and recovery. Examples include stable key meaning and rejection of stale ownership generations.

Jitter. Random variation applied to timing, often retry delays, to reduce synchronization among callers. It does not replace a rate, concurrency, or total resource budget.

Lease. Time-bounded authority under a coordinator's rules. Lease expiration can permit reassignment but does not forcibly stop an old process or revoke every delayed external write.

Linearizability. A consistency contract in which each operation appears to take effect between invocation and completion while respecting real-time precedence. It is more specific than generic data validity.

Logical clock. An ordering mechanism based on events and communication rather than elapsed physical time. Increasing logical values do not by themselves prove causality between unrelated events.

Membership. The set of participants considered eligible under a service or coordination protocol. A local list is not automatically an authoritative or current view of the cluster.

Monotonic read. A session property preventing observations from moving backward through already observed versions under the service's defined conditions. Routing affinity alone may not preserve it after failure.

Outbox. Producer-side durable state tying business intent to an outgoing delivery obligation. A dispatcher can replay it after failure, while the receiver still needs appropriate duplicate handling.

Partial failure. A condition in which some components or communication paths fail while others continue. It creates uncertainty that cannot be modeled as one process

returning a single error code.

Partition key. The value used to choose a data or work authority. It should align with the invariants and ordering scope that must remain local or coordinated.

Poison message. Work that repeatedly fails for a persistent reason rather than a transient outage. It requires explicit terminal handling or repair to avoid indefinite retries and starvation.

Principal. A representation of verified identity and relevant claims used for authorization. Untrusted payload fields must not become a principal merely because they use familiar names.

Provenance. Evidence describing how an artifact was produced, including relevant inputs and build process. A checksum establishes byte identity only relative to a trusted expected value.

Quorum. A set of participants whose agreement satisfies a protocol's rule. Intersection arithmetic is necessary for some designs but does not replace voting, persistence, log, or membership-transition rules.

Readiness. Eligibility to receive a defined class of work. It differs from liveness and may change during dependency degradation, initialization, draining, or capacity exhaustion.

Reconciliation. A controlled comparison and repair process among authoritative states after uncertainty or divergence. It should preserve evidence and authorization rather than silently overwrite conflicting meaning.

Redrive. An authorized attempt to resume previously failed or quarantined work. It should preserve the original logical identity when recovering the same effect and record the reason and actor.

Rendezvous placement. A selection method that scores key-candidate pairs and chooses a deterministic winner. Stable mapping properties do not copy state or authorize a new owner during migration.

Replay. Reprocessing retained work or events from a defined point. Pure projection rebuilds and external side effects require different policies so recovery does not accidentally repeat business actions.

Retry budget. A bound on attempts, elapsed time, rate, or another resource for recovery work. Multiple nested retry owners can multiply load beyond each layer's apparent local limit.

Saga. A workflow that represents partial progress and coordinates recovery or compensation across independent effects. It is not a universal atomic transaction or automatic reversal of the world.

Schema compatibility. The ability of specified readers and writers to interpret stored or transmitted data with preserved meaning. Successful parsing is necessary but not sufficient when units or semantics change.

Service-level indicator. A measured definition of service behavior with a specified numerator, denominator, and boundary. Submission acceptance and accepted-job completion are different indicators in this volume.

Snapshot. A representation of state at a defined point. Recovery may require a matching log position, schema, receipt history, and authority generation; copying visible rows alone can be incomplete.

Strand. In asynchronous execution terminology, a mechanism for serializing associated handlers. It does not automatically imply a dedicated thread or solve object lifetime outside its execution contract.

Tail latency. Behavior in the slow portion of a latency distribution. A good average can coexist with unacceptable long waits, especially under queueing, contention, or periodic recovery work.

Token bucket. A rate-admission model with bounded burst capacity and replenishment. It controls a different resource dimension from simultaneous concurrency and queued memory.

Trace context. Metadata used to connect spans across boundaries under a tracing standard. It is not a credential and should not grant authorization merely because a caller supplies it.

Unknown outcome. A result in which the caller cannot determine whether a remote effect committed. Stable identity, status lookup, and reconciliation can manage it without inventing a new operation.

Write-ahead log. A storage mechanism recording changes before they are incorporated into other serving structures. Its durability, checkpoint, backup, and recovery semantics depend on the implementation and configuration.

Closing Perspective — Extend the System Without Losing the Argument

The progression from C++ syntax to distributed architecture is not a progression away from fundamentals. It makes ownership, lifetime, invariants, and evidence more important because each must now cross additional boundaries. A pointer can outlive an object; a callback can outlive a request; a retry can outlive a caller; a receipt can outlive the process that created it. Each lifetime needs an owner and a defined end.

The same is true of promises. A parser accepting bytes, a database committing intent, a worker recording an effect, and a client observing a result are different events. Reliable architecture connects them with recoverable state instead of collapsing them into one reassuring success flag.

HarborDispatch is deliberately limited so that this argument remains inspectable. Its value is not a claim to replace production infrastructure. It is a concrete system in which a reader can trace accepted work through actual C++ code, sockets, SQLite transactions, retries, and process tests, then identify exactly what a larger deployment would have to add.

Carry that discipline into the next design. Choose the authority, state the invariant, bound the resource, define the failure history, and retain evidence. When a framework or optimization is introduced, ask which assumption it replaces and how the replacement will be verified. That is the continuity between professional C++ programming and dependable distributed systems.

Technical reference directory

Primary documentation and research consulted on **23 September 2026**. Living documentation may change; versioned sources below name their reference release. These references support the technical mechanisms discussed, not claims that optional services were deployed or that the fictional workloads were measured in production. The authored scenarios and exercises are original teaching material.

R01 — C++ Core Guidelines

Bjarne Stroustrup, Herb Sutter, and contributors. Ownership, interfaces, resource safety, and error-handling guidance. Guidelines are recommendations, not a replacement for the selected language standard.

[Primary source](#) · Consulted 2026-09-23.

R02 — Working Draft: Coroutine definitions

ISO C++ working draft. Language-level coroutine transformation and lifetime rules. This living draft may contain revisions beyond the C++20 baseline used by the companion.

[Primary source](#) · Consulted 2026-09-23.

R03 — Asynchronous operations

Boost.Asio documentation. Asynchronous operation and completion model. Framework discussion is separate from the companion's own Linux poll reactor.

[Primary source](#) · Consulted 2026-09-23.

R04 — Strands: Use Threads Without Explicit Locking

Boost.Asio documentation. Serializing handler execution within an execution domain. A strand does not synchronize unrelated access outside that domain.

[Primary source](#) · Consulted 2026-09-23.

R05 — poll(2)

Linux man-pages project. Readiness notification, event flags, return values, and timeout interpretation for the native transport exercises.

[Primary source](#) · Consulted 2026-09-23.

R06 — `recv(2)`

Linux man-pages project. Receive operations, short reads, EOF, nonblocking behavior, and errors. TCP application framing must be implemented above this interface.

[Primary source](#) · Consulted 2026-09-23.

R07 — `send(2)`

Linux man-pages project. Partial sending, nonblocking errors, and Linux `MSG_NOSIGNAL` behavior used by the companion.

[Primary source](#) · Consulted 2026-09-23.

R08 — RFC 9110: HTTP Semantics

IETF, 2022. HTTP method semantics and idempotency. The capstone wire protocol is not HTTP; the standard informs the API comparison.

[Primary source](#) · Consulted 2026-09-23.

R09 — Core concepts, architecture and lifecycle

gRPC documentation. RPC lifecycle, metadata, method shapes, and status. These concepts do not make the optional schema an implemented gRPC service.

[Primary source](#) · Consulted 2026-09-23.

R10 — Deadlines

gRPC documentation. Client deadlines, propagation, and server-side resource considerations. A deadline is not a rollback transaction.

[Primary source](#) · Consulted 2026-09-23.

R11 — Cancellation

gRPC documentation. Cancellation notification and application responsibility for stopping ongoing work. Side effects already committed remain application concerns.

[Primary source](#) · Consulted 2026-09-23.

R12 — Retry

gRPC documentation. Retry policies, attempts, and transport commitment. Application idempotency and durable identity remain separate design obligations.

[Primary source](#) · Consulted 2026-09-23.

R13 — Custom Name Resolution

gRPC documentation. Resolver and service-configuration responsibilities when endpoints change. The supplied capstone intentionally uses one explicit worker endpoint.

[Primary source](#) · Consulted 2026-09-23.

R14 — Language Guide: proto3

Protocol Buffers documentation. Field numbering, defaults, schema evolution, and generated representation. Review selected runtime and compiler releases before integrating the optional schema.

[Primary source](#) · Consulted 2026-09-23.

R15 — Time, Clocks, and the Ordering of Events in a Distributed System

Leslie Lamport, 1978. Original treatment of happened-before and logical clocks. Logical clock ordering is not elapsed time or a complete reconstruction of causality.

[Primary source](#) · Consulted 2026-09-23.

R16 — In Search of an Understandable Consensus Algorithm: Extended Version

Diego Ongaro and John Ousterhout, 2014. Raft leader election, replicated logs, and safety reasoning. The book's fencing model is not an implementation or proof of Raft.

[Primary source](#) · Consulted 2026-09-23.

R17 — Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services

Seth Gilbert and Nancy Lynch, 2002. Formal assumptions underlying the consistency/availability limitation during partitions. Avoid interpreting it as an unconstrained product-selection slogan.

[Primary source](#) · Consulted 2026-09-23.

R18 — Transaction

SQLite documentation. BEGIN, COMMIT, ROLLBACK, transaction modes, and concurrency boundaries used by the store wrapper.

[Primary source](#) · Consulted 2026-09-23.

R19 — Isolation in SQLite

SQLite documentation. Connection isolation, serialized writes, and WAL snapshot behavior. Application-level multi-database workflows need additional coordination.

[Primary source](#) · Consulted 2026-09-23.

R20 — Write-Ahead Logging

SQLite documentation. WAL files, checkpoints, concurrency, and operational constraints. Copying only a live database's main file is not the book's backup procedure.

[Primary source](#) · Consulted 2026-09-23.

R21 — PRAGMA Statements

SQLite documentation. Synchronous settings, busy handling, journal configuration, and user_version. Settings must be checked against the installed library and storage assumptions.

[Primary source](#) · Consulted 2026-09-23.

R22 — SQLite Online Backup API

SQLite documentation. Supported snapshot-copy mechanism for a live database. Recovery evidence still requires restore and application-level reconciliation tests.

[Primary source](#) · Consulted 2026-09-23.

R23 — Transaction Isolation

PostgreSQL documentation, current branch. Comparison of isolation behavior and retry obligations in a different database. This is not a tested PostgreSQL adapter in the companion.

[Primary source](#) · Consulted 2026-09-23.

R24 — Client-side caching

Redis documentation. Invalidation and tracking mechanisms. The chapter's local versioned cache is a separate single-owner model, not a Redis client implementation.

[Primary source](#) · Consulted 2026-09-23.

R25 — Consumer Acknowledgements and Publisher Confirms

RabbitMQ documentation. Independent publication and consumer acknowledgment boundaries, including redelivery. Acknowledgment alone does not transact an external business effect.

[Primary source](#) · Consulted 2026-09-23.

R26 — Design

Apache Kafka 4.1 documentation. Versioned reference for log, delivery, and transaction design. Kafka 4.1 is the cited documentation baseline, not a claim about the newest release.

[Primary source](#) · Consulted 2026-09-23.

R27 — API guarantees

etcd 3.5 documentation. Versioned coordination-service consistency and durability guarantees. A lease or local fencing token must not be assigned guarantees beyond its authority.

[Primary source](#) · Consulted 2026-09-23.

R28 — Authentication

gRPC documentation. Channel and call credentials and authentication mechanisms. The local capstone does not implement these features.

[Primary source](#) · Consulted 2026-09-23.

R29 — RFC 8446: The Transport Layer Security Protocol Version 1.3

IETF, 2018. TLS 1.3 protocol reference. Application authorization and endpoint identity policy remain distinct from encryption.

[Primary source](#) · Consulted 2026-09-23.

R30 — Traces

OpenTelemetry documentation. Trace, span, context, and causality-oriented observability concepts. The book distinguishes conceptual instrumentation from a deployed collector.

[Primary source](#) · Consulted 2026-09-23.

R31 — Metric and label naming

Prometheus documentation. Units, naming, and label design. The capstone's compact diagnostic response is not claimed to be a complete Prometheus exporter.

[Primary source](#) · Consulted 2026-09-23.

R32 — Handling Overload

Google, Site Reliability Engineering. Admission, load shedding, queues, and overload interactions. Numerical queueing examples in this book are original stated-assumption calculations.

[Primary source](#) · Consulted 2026-09-23.

R33 — Implementing SLOs

Google, The Site Reliability Workbook. Service-level objectives, measurement definitions, and error-budget use. Fictional targets are not observed service reliability.

[Primary source](#) · Consulted 2026-09-23.

R34 — Pod Lifecycle

Kubernetes documentation. Pod termination and lifecycle behavior for deployment design. The companion is not qualified on a Kubernetes cluster.

[Primary source](#) · Consulted 2026-09-23.

R35 — Configure Liveness, Readiness and Startup Probes

Kubernetes documentation. Distinct purposes of health probes and consequences of failure. Probe syntax must match the deployed API version.

[Primary source](#) · Consulted 2026-09-23.

R36 — Importing and Exporting Guide

CMake documentation. Target interfaces, imported targets, and installed package usage. Reproducibility also depends on controlled tools and dependencies.

[Primary source](#) · Consulted 2026-09-23.

R37 — SLSA Levels

SLSA specification version 1.0. Versioned supply-chain assurance framework. Creating a checksum or source archive does not imply a certified SLSA level.

[Primary source](#) · Consulted 2026-09-23.

R38 — Making retries safe with idempotent APIs

Amazon Builders' Library. Client request identity and semantic equivalence when repeating an ambiguous operation. Retention and changed-intent policy must remain explicit.

[Primary source](#) · Consulted 2026-09-23.

R39 — FoundationDB: A Distributed Unbundled Transactional Key Value Store

FoundationDB team, SIGMOD 2021. Primary system paper including deterministic simulation. The companion's finite state-model tests are not equivalent to that system's validation infrastructure.

[Primary source](#) · Consulted 2026-09-23.

R40 — Trace Context

W3C Recommendation. Standard propagation fields and context handling. An externally supplied trace identifier is not an authorization credential.

[Primary source](#) · Consulted 2026-09-23.

R41 — SSL_set1_host

OpenSSL 3.0 documentation. Versioned hostname verification interface for TLS clients. No OpenSSL transport is silently added to the supplied loopback service.

[Primary source](#) · Consulted 2026-09-23.

R42 — C++ Best Practices

gRPC documentation. C++ callback, lifetime, and API usage recommendations. Verify behavior with the pinned gRPC release used for an integration.

[Primary source](#) · Consulted 2026-09-23.

R43 — Cross-Version Runtime Guarantee

Protocol Buffers documentation. Generated-code and runtime compatibility, including language-specific restrictions. Matching generator/runtime versions is an explicit optional-integration gate.

[Primary source](#) · Consulted 2026-09-23.

R44 — Graceful Shutdown

gRPC documentation. Graceful stop versus forced termination. An orchestration timeout does not automatically cancel every underlying application effect.

[Primary source](#) · Consulted 2026-09-23.

R45 — A Proof for the Queuing Formula: $L = \lambda W$

John D. C. Little, Operations Research 9(3), 383-387, 1961. Original relationship between long-run average occupancy, arrival rate, and time under the stated finiteness and stationarity assumptions; not a tail-latency bound.

[Primary source](#) · Consulted 2026-09-23.

R46 — close(2)

Linux man-pages project. Descriptor lifetime and Linux close-error behavior. The companion’s nonretrying destructor policy is Linux-specific, not a portable promise for every operating system.

[Primary source](#) · Consulted 2026-09-23.

R47 — clock_gettime(3)

Linux man-pages project. Linux clock domains, adjustment, and suspend distinctions. OS clock behavior and a C++ clock type must not be treated as an identical universal implementation.

[Primary source](#) · Consulted 2026-09-23.

R48 — Working Draft: Signal handlers

ISO C++ working draft. Signal-safe evaluations and plain lock-free atomic operations. The companion asserts lock freedom and uses a notification flag; this does not authorize arbitrary work in signal context.

[Primary source](#) · Consulted 2026-09-23.

Section example revision index

The original book text and original code are retained. New examples are independently authored additions for the numbered sections listed below. Their validation records do not certify the retained original manuscript or every optional platform procedure.

1. 1.1 Turning Requirements into Observable Contracts
2. 1.2 Identifying State and Responsibility
3. 1.3 Reasoning About Failure Before Choosing Technology
4. 2.1 Designing Modules Around Cohesive Responsibilities
5. 2.2 Distinguishing Logical and Deployment Boundaries
6. 2.3 Managing Coupling Across Data and Releases
7. 3.1 Making Ownership and Lifetime Visible
8. 3.2 Designing Error and Completion Semantics
9. 3.3 Managing Source, Binary, Wire, and Behavioral Compatibility
10. 4.1 Treating Configuration as Validated Input
11. 4.2 Publishing Configuration Without Breaking In-Flight Work
12. 4.3 Coordinating Startup and Shutdown
13. 5.1 Separating Initiation from Completion
14. 5.2 Understanding What a Coroutine Stores
15. 5.3 Coordinating Cancellation and Structured Completion
16. 6.1 Understanding Readiness and Completion
17. 6.2 Modeling Per-Connection State
18. 6.3 Completing Connections and Shutting Down Safely
19. 7.1 Separating Messages from Byte Streams
20. 7.2 Defining Field Representation and Meaning
21. 7.3 Evolving Protocols Across Mixed Versions
22. 8.1 Defining an RPC Contract
23. 8.2 Propagating a Finite Waiting Budget
24. 8.3 Handling Cancellation and Uncertainty
25. 9.1 Representing Membership as Versioned Information
26. 9.2 Separating Health from Routing Policy
27. 9.3 Coordinating Routing with Failure and State

- 28. 10.1 Naming the Failures a Design Assumes
- 29. 10.2 Choosing the Right Meaning of Time
- 30. 10.3 Representing Causal Relationships
- 31. 11.1 Making Read and Write Guarantees Explicit
- 32. 11.2 Understanding What Replication Acknowledges
- 33. 11.3 Reasoning About Partitions and Availability
- 34. 12.1 Identifying the State That Requires Agreement
- 35. 12.2 Interpreting Locks and Leases Correctly
- 36. 12.3 Enforcing Generations at the Resource Boundary
- 37. 13.1 Selecting the Partitioning Boundary
- 38. 13.2 Implementing Stable Placement
- 39. 13.3 Moving State Before Changing Authority
- 40. 14.1 Designing State Around Invariants
- 41. 14.2 Owning Native Database Resources in C++
- 42. 14.3 Understanding Durability and Recovery Boundaries
- 43. 15.1 Defining Compatibility in Both Directions
- 44. 15.2 Planning Expand, Backfill, Switch, and Contract
- 45. 15.3 Running Migrations as Controlled Operations
- 46. 16.1 Deciding What a Cache Is Allowed to Forget
- 47. 16.2 Coordinating Versions, Invalidations, and Fills
- 48. 16.3 Controlling Cache Load and Failure Amplification
- 49. 17.1 Separating Commands, Events, and Transport Records
- 50. 17.2 Interpreting Acknowledgement and Redelivery
- 51. 17.3 Preserving the Ordering That the Domain Needs
- 52. 18.1 Defining Idempotency as an API Contract
- 53. 18.2 Making Intent and Effect Locally Atomic
- 54. 18.3 Recovering Every Acknowledgement Gap
- 55. 19.1 Spending a Retry Budget Deliberately
- 56. 19.2 Implementing a Circuit Breaker as a State Machine
- 57. 19.3 Isolating Resource Exhaustion
- 58. 20.1 Bounding Every Waiting Location
- 59. 20.2 Coordinating Producers and Consumers Safely

- 60. 20.3 Regulating Rate and Concurrency
- 61. 21.1 Modeling the Business Process as State
- 62. 21.2 Making Each Step Recoverable
- 63. 21.3 Designing Compensation as New Work
- 64. 22.1 Establishing Identity Before Making Policy Decisions
- 65. 22.2 Protecting Transport Without Losing Application Semantics
- 66. 22.3 Limiting Damage from Inputs and Secrets
- 67. 23.1 Choosing Signals with Defined Meaning
- 68. 23.2 Connecting Work Across Processes
- 69. 23.3 Designing Diagnostics That Do Not Become the Incident
- 70. 24.1 Designing Tests Around Invariants
- 71. 24.2 Exploring Histories Deterministically
- 72. 24.3 Crossing Real Process and Persistence Boundaries
- 73. 25.1 Defining the Build as a Dependency Graph
- 74. 25.2 Recording Provenance and Supply-Chain Decisions
- 75. 25.3 Packaging the Supported Runtime
- 76. 26.1 Making Startup and Readiness Explicit
- 77. 26.2 Draining Without Losing Ownership
- 78. 26.3 Replacing Instances Without Breaking State
- 79. 27.1 Defining Service Objectives That Match User Outcomes
- 80. 27.2 Planning Capacity and Incident Response
- 81. 27.3 Proving Recovery Rather Than Assuming It
- 82. 28.1 Defining the Architecture and Its Trust Boundary
- 83. 28.2 Following a Job Through Commit and Recovery
- 84. 28.3 Building, Exercising, and Reviewing the System

Technical references for the new examples

CMAKE-BUILDSYSTEM — CMake: build targets and usage requirements

<https://cmake.org/cmake/help/latest/manual/cmake-buildsystem.7.html>

CMAKE-CONFIGURE — CMake: configure_file

https://cmake.org/cmake/help/latest/command/configure_file.html

CPP20 — C++20 working draft N4861: language and standard-library clauses

<https://timsong-cpp.github.io/cppwp/n4861/>

GCC-OUTPUT — GCC: stages and overall driver options

<https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html>

ITANIUM-ABI — Itanium C++ ABI specification

<https://itanium-cxx-abi.github.io/cxx-abi/abi.html>

LAMPORT-CLOCKS — Lamport: Time, Clocks, and the Ordering of Events in a Distributed System

<https://lamport.azurewebsites.net/pubs/time-clocks.pdf>

LINUX-FORK — Linux/POSIX manual: fork.2

<https://man7.org/linux/man-pages/man2/fork.2.html>

LINUX-LOADER — Linux/POSIX manual: ld.so.8

<https://man7.org/linux/man-pages/man8/ld.so.8.html>

LINUX-RECV — Linux/POSIX manual: recv.2

<https://man7.org/linux/man-pages/man2/recv.2.html>

LINUX-SOCKET — Linux/POSIX manual: socket.2

<https://man7.org/linux/man-pages/man2/socket.2.html>

OPENSSL-HOST — OpenSSL service identity verification

https://docs.openssl.org/3.5/man3/SSL_set1_host/

RAFT — Ongaro and Ousterhout: In Search of an Understandable Consensus Algorithm

<https://raft.github.io/raft.pdf>

REPRO-BUILDS — Reproducible Builds: SOURCE_DATE_EPOCH

<https://reproducible-builds.org/docs/source-date-epoch/>

SQLITE-BACKUP — SQLite online backup API

<https://www.sqlite.org/backup.html>

SQLITE-C — SQLite C/C++ interface specification

<https://www.sqlite.org/c3ref/intro.html>

SQLITE-TRANSACTION — SQLite transactions

https://www.sqlite.org/lang_transaction.html

SQLITE-WAL — SQLite write-ahead logging

<https://www.sqlite.org/wal.html>

TLS13 — IETF RFC8446: TLS1.3

<https://www.rfc-editor.org/rfc/rfc8446.html>